

密码学期末试卷解析册

武汉大学

计算机学院 · 国家网络安全学院

信息安全专业 / 网络空间安全专业

编制：Maple

解析册内容： 试卷一 · 二：密码学课程（2 份）
试卷三 · 四 · 五：计算机安全与保密课程（3 份）
试卷六 · 七 · 八 · 九：网络空间安全课程（4 份）
共计 9 份试卷，涵盖古典密码至现代密码协议全部核心模块

2026 年 6 月 16 日

使用说明

本解析册为密码学课程期末复习资料，旨在帮助读者：

- 系统回顾密码学核心知识点
- 掌握典型题型的解题思路和方法
- 深入理解密码学原理在安全协议和系统中的应用

每道题目均包含**知识点梳理**、**详细解答**和**知识点延伸**三个层次，建议先独立完成题目，再对照解析查漏补缺。

前言

密码学是信息安全与网络空间安全学科的核心基础课程之一。它既是数学理论与工程实践的交叉学科，也是从古典密码到现代公钥基础设施、从理论安全性证明到实际安全协议设计的完整知识体系。

本解析册收录了武汉大学两份具有代表性的密码学期末考试试卷，覆盖了密码学课程的主要知识模块：

知识模块	涉及试卷	具体内容
古典密码	试卷一	加法密码、单表代替、对合运算
对称密码	试卷一、二	DES 结构与安全性、AES 结构与 MixColumns、S 盒分析、CBC 工作模式
公钥密码	试卷一、二	RSA 算法、密钥管理、公钥证书、PKI、公钥密码基本思想与优势
序列密码与 LFSR	试卷一、二	线性反馈移位寄存器、本原多项式、 m 序列及其性质
椭圆曲线密码	试卷一	ECC 曲线、有理点群、循环群判定
数字签名	试卷二	Schnorr 签名方案、EU-CMA 安全性、分叉引理与可证明安全
认证协议	试卷一	挑战-响应认证、反射攻击及防御
密码学应用	试卷二	支付系统安全设计、密码协议综合运用
高级专题	试卷二	Lifted ElGamal 同态加密、协议公平性分析

关于解析的说明

1. **试卷一**来源于原始试卷的扫描/文本提取，包含出题方提供的参考解答。解析在此基础上进行了大幅扩充，增加了详细的知识点梳理和逐步推导过程。
2. **试卷二**为 2022-2023 学年开卷考试试卷，原始资料仅有试题没有答案。所有解析内容均为编者基于密码学理论和标准教材独立编写。
3. 解析中的「**知识点延伸**」板块不属于原试卷要求范围，但有助于读者拓展视野、理解知识点在现代密码学实践中的位置。
4. 建议的复习策略：先系统复习各模块的理论知识，然后独立完成试卷，最后参照本解析进行查漏补缺和深化理解。

目录

1 试卷一：密码学（计算机学院）	5
1.1 单表代替密码（10 分）	1
1.2 公钥密码的密钥管理（10 分）	4
1.3 DES 与 AES 的比较（10 分）	7
1.4 AES 列混淆运算（10 分）	9
1.5 线性反馈移位寄存器 LFSR（15 分）	11
1.6 RSA 密码体制（15 分）	14
1.7 椭圆曲线密码（15 分）	16
1.8 站点认证协议分析（15 分）	19
2 试卷二：密码学（2022-2023 开卷）	23
2.1 线性反馈移位寄存器 LFSR（10 分）	1
2.2 DES S 盒分析（10 分）	3
2.3 RSA 加密与破译（10 分）	5
2.4 已知明文攻击与选择明文攻击（10 分）	7
2.5 CBC 模式与错误传播（10 分）	9
2.6 公钥密码体制的基本思想（10 分）	11
2.7 Schnorr 数字签名的可证明安全性（10 分）	13
2.8 支付系统安全分析与密码学方案设计（30 分）	16
2.9 附加题：Lifted ElGamal 的安全性分析（5 分，选做）	21
3 试卷三：计算机安全与保密（2017-2018）	24
3.1 仿射密码解密（10 分）	1
3.2 AES 列混合有限域运算（10 分）	4
3.3 RSA 因数分解（10 分）	7
3.4 椭圆曲线点倍乘运算（10 分）	9
3.5 ElGamal 签名同 t 攻击（20 分）	12
3.6 名词解释：Hash 函数（5 分）	17
3.7 名词解释：公钥密码（5 分）	18
3.8 名词解释：解密（5 分）	19
3.9 名词解释：数字签名（5 分）	20
3.10 DES 算法的加密过程（10 分）	22
3.11 密码协议的设计原则（10 分）	24
4 试卷四：计算机安全与保密（2018-2019）	27
4.1 Hill 密码解密（10 分）	28
4.2 背包密码算法（10 分）	31

4.3	RSA 加密与解密 (10 分)	34
4.4	AES 列混合运算 (10 分)	37
4.5	椭圆曲线点乘运算 (10 分)	40
4.6	DES 的 S 盒代替运算 (10 分)	43
4.7	最大公约数 (5 分)	45
4.8	仿射密码系统 (5 分)	45
4.9	拟素数 (5 分)	46
4.10	ElGamal 签名方案 (5 分)	47
4.11	密钥管理的内容 (10 分)	48
4.12	RSA 签名方案的安全性分析 (10 分)	50
5	试卷五：计算机安全与保密 (2021-2022 B 卷)	52
5.1	仿射密码解密 (10 分)	53
5.2	Rabin 密码体制解密 (10 分)	56
5.3	RSA 加密与解密 (10 分)	59
5.4	构造有限域 GF(8) 的加法和乘法表 (10 分)	62
5.5	中国剩余定理求解同余式组 (10 分)	65
5.6	Feige-Fiat-Shamir 协议身份验证 (10 分)	68
5.7	选择明文攻击 (10 分)	71
5.8	密码分析的类型 (10 分)	74
5.9	ElGamal 签名方案 (10 分)	77
5.10	对称密码算法与非对称密码算法 (10 分)	80
6	试卷六：网络空间安全 (2021-2022)	83
6.1	单表代换密码 (10 分)	1
6.2	Hill 密码解密矩阵 (10 分)	3
6.3	仿射密码系统求解 (10 分)	6
6.4	DES 轮函数四大运算 (10 分)	9
6.5	AES S 盒计算 (15 分)	12
6.6	Merkle-Hellman 背包密码系统 (15 分)	16
6.7	RSA 密码体制 (15 分)	19
6.8	椭圆曲线密码——点乘运算 (15 分)	23
7	试卷七：网络空间安全 (2023-2024 第一学期)	26
7.1	Hill 密码的加密与解密 (10 分)	1
7.2	换位密码的加密与解密 (10 分)	5
7.3	RSA 算法 (10 分)	8
7.4	椭圆曲线密码 (15 分)	12
7.5	IDEA 分组密码的 CBC 模式 (15 分)	16

7.6	RSA 数字签名方案分析 (20 分)	21
7.7	Feige-Fiat-Shamir 身份验证方案 (20 分)	27
8	试卷八：网络空间安全 (2023-2024 第二学期)	33
8.1	仿射密码解密 (10 分)	34
8.2	AES S-盒代换 (10 分)	37
8.3	RSA 密钥计算与加密 (10 分)	40
8.4	椭圆曲线密码 (ECC) (15 分)	43
8.5	AES 密码反馈 (CFB) 模式分析 (15 分)	50
8.6	双通匿名密钥确认协议分析 (20 分)	54
8.7	Feige-Fiat-Shamir 身份验证方案分析 (20 分)	58
9	试卷九：网络空间安全小测试 (2024-2025)	64
9.1	RSA 加解密计算 (10 分)	66
9.2	ElGamal 签名中重复使用 k 的安全性分析 (10 分)	73

1 试卷一：密码学（计算机学院）

密码学期末试卷解析册

Maple

试卷说明

本试卷卷面共八题，满分 100 分，在课程总成绩中占 70 分。

试卷内容涵盖：古典密码（单表代替密码）、公钥密码密钥管理、对称密码（DES/AES）比较、AES 列混淆运算、线性反馈移位寄存器（LFSR）、RSA 密码体制、椭圆曲线密码（ECC）以及认证协议分析。

1.1 单表代替密码 (10 分)

题目

- ① 使加法密码算法成为对合运算的密钥 k 称为对合密钥。以英文为例求出其对合密钥，并以明文 $M = \text{WEWILLMEETATMORNING}$ 为例进行加解密，说明其对合性。
- ② 一般而言，对于加法密码，设明文字母表和密文字母表含有 n 个字母， n 为 ≥ 1 的正整数，求出其对合密钥 k 。

知识点梳理

1. 加法密码 (Shift Cipher / Caesar Cipher)

加法密码是最经典的古典密码之一，其数学定义为：

- 设明文字母表 $\mathcal{A} = \{a_0, a_1, \dots, a_{n-1}\}$ (通常 a_i 对应数字 i)，密文字母表 $\mathcal{B} = \{b_0, b_1, \dots, b_{n-1}\}$
- 加密映射： $E_k(i) = j \equiv i + k \pmod{n}$ ，其中 i 为明文字母编号， j 为密文字母编号
- 解密映射： $D_k(j) = i \equiv j - k \pmod{n}$

2. 对合运算 (Involution)

一个密码算法称为对合的，当且仅当加密与解密为同一运算，即：

$$E_k(\cdot) = D_k(\cdot)$$

或者等价地，对任意明文 M ，有：

$$E_k(E_k(M)) = M$$

即连续两次加密即可还原明文，此时加密函数自身就是解密函数。

对于加法密码，对合条件为： $i + k \equiv i - k \pmod{n}$ ，即 $2k \equiv 0 \pmod{n}$ 。

3. 模运算与对合密钥条件

在模 n 意义下，方程 $2k \equiv 0 \pmod{n}$ 的解取决于 n 的奇偶性：

- 若 n 为偶数： $k = n/2$ 是解 (同时也是 $k = 0$ 的平凡解之外的唯一非平凡解)
- 若 n 为奇数：仅有平凡解 $k = 0$ (因为 2 在模奇数意义下有逆元， $k \equiv 0$ 是唯一解)

详细解答

第 1 小题：英文加法密码的对合密钥

英文字母表共有 $n = 26$ 个字母，分别编号为 $A = 0, B = 1, C = 2, \dots, Z = 25$ 。
加法密码的加解密映射为：

加密： $j \equiv i + k \pmod{26}$

解密： $i \equiv j - k \pmod{26}$

若算法是对合的，则需满足 $E_k = D_k$ ，即：

$$i + k \equiv i - k \pmod{26} \implies 2k \equiv 0 \pmod{26}$$

在模 26 下， $2k \equiv 0$ 的解为 $k = 0$ （平凡解）或 $k = 13$ （ $26 \div 2 = 13$ ）。
因此，**对合密钥**为 $k = 13$ ，即著名的 **ROT13** 算法。
当 $k = 13$ 时，明密文字母对照表如下：

明	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y
密	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L

加密过程（明文 $M = \text{WEWILLMEETATMORNING}$ ）：

将每个字母转换为对应编号，加 $13 \bmod 26$ ：

$$W(22) \rightarrow 22 + 13 = 35 \equiv 9 \pmod{26} \rightarrow J$$

$$E(4) \rightarrow 4 + 13 = 17 \rightarrow R$$

$$W(22) \rightarrow J$$

$$I(8) \rightarrow 21 \rightarrow V$$

$$L(11) \rightarrow 24 \rightarrow Y$$

$$L(11) \rightarrow Y$$

$$M(12) \rightarrow 25 \rightarrow Z$$

$$E(4) \rightarrow R$$

$$E(4) \rightarrow R$$

$$T(19) \rightarrow 19 + 13 = 32 \equiv 6 \pmod{26} \rightarrow G$$

$$A(0) \rightarrow 13 \rightarrow N$$

$$T(19) \rightarrow G$$

$$M(12) \rightarrow Z$$

$$O(14) \rightarrow 27 \equiv 1 \pmod{26} \rightarrow B$$

$$R(17) \rightarrow 30 \equiv 4 \pmod{26} \rightarrow E$$

$$N(13) \rightarrow 26 \equiv 0 \pmod{26} \rightarrow A$$

$$I(8) \rightarrow V$$

$$N(13) \rightarrow A$$

$$G(6) \rightarrow 19 \rightarrow T$$

得密文： $C = \text{JRJVYYZRRGNGZBEAVAT}$

解密过程（即再加密一次，因为 $k = 13$ 是对合的）：

$$J(9) \rightarrow 9 + 13 = 22 \rightarrow W$$

$$R(17) \rightarrow 17 + 13 = 30 \equiv 4 \rightarrow E$$

⋮

还原得 $\text{WEWILLMEETATMORNING}$ ，与原文一致。

对合性说明：由于 $k = 13$ ，两次加密相当于 $+13 + 13 \equiv +26 \equiv 0 \pmod{26}$ ，即加密运算的逆就是其本身，因此算法是对合的。在 UNIX/Linux 系统中常见的 `rot13` 命令正是利用了这一性质。

第 2 小题：一般情况的推广

设明文字母表含 n 个字母 ($n \geq 1$)，编号为 $\{0, 1, \dots, n-1\}$ 。

对合密钥 k 须满足：

$$2k \equiv 0 \pmod{n}$$

该同余方程的解即为模 n 意义下阶为 2 的元素（一阶元素为 $k = 0$ 的平凡解）。分析如下：

- 当 n 为偶数时：设 $n = 2m$ (m 为正整数)。

$$2k \equiv 0 \pmod{2m} \implies k \equiv 0 \text{ 或 } k \equiv m \pmod{2m}$$

非平凡对合密钥为 $k = \frac{n}{2}$ 。

- 当 n 为奇数时： $\gcd(2, n) = 1$ ，即 2 在模 n 下存在乘法逆元，故

$$2k \equiv 0 \pmod{n} \implies k \equiv 0 \pmod{n}$$

此时不存在非零对合密钥，只有平凡解 $k = 0$ 。因此奇数字母表下加法密码不可能有非平凡对合密钥。

知识点延伸：对合密码的应用

ROT13 是 ROT13/ROT47 家族中最著名的一员，至今仍在某些场景中使用：

- Usenet 论坛中用于隐藏剧透、冒犯性内容
- 某些软件的简单混淆（不是加密!）
- 作为密码学教学中的经典入门示例

对合性意味着同一程序/函数即可完成加密和解密，实现极为简洁。但在密码学意义上，加法密码（包括 ROT13）完全不安全——密钥空间仅为 25，可被穷举攻击瞬间破解。

1.2 公钥密码的密钥管理（10 分）

题目

- ① 在公钥密码的密钥管理中，公开的加密钥 K_e 和保密的解密密钥 K_d 的秘密性、真实性和完整性都需要确保吗？说明为什么？
- ② 简述公钥证书的作用。

知识点梳理

1. 公钥密码体制的基本模型

公钥密码（非对称密码）中，每个用户拥有一对密钥：

- 公开密钥（公钥） K_e ：可公开发布，用于加密或验证签名
- 私有密钥（私钥） K_d ：必须严格保密，用于解密或生成签名

2. 信息安全的三个基本属性

- 秘密性 (Confidentiality)：信息不被未授权者获取
- 真实性 (Authenticity)：信息来源真实，非伪造
- 完整性 (Integrity)：信息未被未授权篡改

3. 公钥基础设施 (PKI) 与数字证书

PKI 是管理公钥的体系，核心组件包括：

- CA (Certification Authority, 认证中心)：可信第三方，签发公钥证书
- RA (Registration Authority)：注册机构，验证用户身份
- 证书库：存储和分发公钥证书

详细解答

第 1 小题： K_e 和 K_d 的安全性要求分析

密钥	需要确保			理由
	秘密性	真实性	完整性	
公钥 K_e	×	✓	✓	不需要秘密性：公钥本就是公开的，任何人均可获取。 需要真实性：必须确保公钥确实属于声称的主体；若被攻击者替换为假公钥，则加密数据可被攻击者解密。 需要完整性：公钥若被篡改，加密操作将产生错误密文，合法接收方无法正确解密。
私钥 K_d	✓	✓	✓	需要秘密性：私钥泄露意味着所有基于该密钥对的机密性彻底丧失。 需要真实性：私钥的真伪直接影响数字签名和身份认证的安全性。 需要完整性：私钥受损则解密和签名结果不可预测，数据的秘密性和可认证性均无法保证。

攻击举例（公钥替换攻击）：

公钥的真实性和完整性若被破坏，典型攻击流程如下：

- 攻击者 C 生成自己的公私钥对 (PK_C, SK_C) ，并用 PK_C 偷偷替换公钥目录中合法用户 A 的公钥 PK_A 。
- 当 B 需要向 A 发送保密消息时， B 从公钥目录中获取“ A 的公钥”（实际已是 PK_C ），并用其加密消息后发送密文。
- 攻击者 C 在传输途中截获密文，使用自己对应的私钥 SK_C 解密，成功获取明文数据。
- 同时 C 还可将消息用 A 真正的公钥重新加密后转发给 A ，使 A 和 B 均不察觉。

这就是典型的**中间人攻击（Man-in-the-Middle Attack）**，它揭示了公钥真实性和完整性的极端重要性。

第 2 小题：公钥证书的作用

公钥证书（Public Key Certificate）是由可信任的签证机构 CA 签发的电子文档，包含以下核心信息：

- 持证主体的标识（姓名、组织等身份信息）
- 持证主体的公钥
- 证书有效期
- 发证 CA 的数字签名

核心作用：

1. **绑定公钥与主体身份**：公钥证书建立了公钥与用户身份之间的可信绑定关系，确保证书中公钥确实属于证书所标识的主体。
2. **防止公钥伪造和篡改**：通过 CA 的数字签名保护，任何对证书内容的篡改都会导致签名验证失败。任何用户只需知道 CA 的公钥，即可独立验证证书的合法性和完整性。
3. **支持大规模密钥分发**：公钥证书可以明文形式存储和分发（如放在公开目录中），无需保密信道即可实现安全的密钥分发，这是公钥密码体系的核心优势之一。
4. **构建信任体系**：通过 CA 层级结构（根 CA → 子 CA → 终端证书）构建完整的信任链，使互联网规模的密钥管理成为可能。

1.3 DES 与 AES 的比较（10 分）

题目

对 DES 和 AES 进行比较，说明两者的特点和优缺点。

知识点梳理

1. 分组密码结构类型

- **Feistel 结构**：将输入分为左右两半，通过多轮轮函数迭代。特点：加解密结构相同（对合），仅子密钥顺序相反。代表：DES。
- **SP 结构（Substitution-Permutation）**：每一轮包含替换层（S 盒）和置换层（P 盒），混淆和扩散分离。特点：加解密结构不同，需分别实现。代表：AES。

2. 分组密码设计原理

- **混淆（Confusion）**：使密文与密钥之间的统计关系尽可能复杂。主要由 S 盒（非线性置换）实现。
- **扩散（Diffusion）**：使明文的每一位影响密文的尽可能多位。主要由 P 盒（线性变换）实现。

- **乘积密码**：通过交替使用混淆和扩散的多轮迭代，获得强密码学性质。

3. DES 的历史局限性

DES (Data Encryption Standard, 1977 年) 的 56 位密钥空间已不足以抵御现代计算能力的穷举攻击。1999 年 EFF 的 DES Cracker 在 22 小时内完成穷举，促使 NIST 于 2001 年选定 Rijndael 算法作为 AES。

详细解答

维度	DES	AES
全称	Data Encryption Standard	Advanced Encryption Standard
发布年份	1977 年 (NIST)	2001 年 (NIST)
结构类型	Feistel 网络 (平衡结构)	SP 网络 (替换-置换结构)
分组长度	64 位	128 位
密钥长度	56 位有效 (64 位名义)	128/192/256 位
轮数	16 轮	10/12/14 轮 (依密钥长度)
密钥空间	$2^{56} \approx 7.2 \times 10^{16}$	最小 $2^{128} \approx 3.4 \times 10^{38}$
S 盒	8 个 6×4 S 盒	1 个 8×8 S 盒 (GF 逆 + 仿射)
弱密钥	存在 (4 弱 +12 半弱)	不存在
互补对称性	存在, 密钥空间实际减半	不存在
对合性	是对合的	不是对合的
硬件实现	优: Feistel 适合硬件	优: 适合高效硬件
软件实现	较差: 大量位操作	优: 面向字节操作
安全性	已不安全 (穷举攻击)	至今安全
当前状态	被 AES 取代	NIST 加密标准

综合评述：

- **DES 的历史贡献**：DES 是密码学史上的里程碑，它首次建立了公开标准的分组密码算法，其 Feistel 结构和 S 盒设计理念深刻影响了后世所有分组密码。
- **DES 的主要缺陷**：
 1. 密钥过短 (56 位)，易于穷举攻击；
 2. 存在弱密钥和半弱密钥 (极少数密钥下加解密效果削弱)；
 3. 互补对称性可将穷举攻击的复杂度减半 (2^{55})；
 4. 软件实现效率低 (大量位级操作)。

- **AES 的优势:**

1. 密钥空间巨大 (2^{128} 起), 穷举攻击不可行;
2. SP 结构理论清晰, S 盒设计有坚实的数学基础 ($GF(2^8)$ 上的逆映射);
3. 无弱密钥和互补对称性, 算法结构更干净;
4. 灵活的密钥长度选择 (128/192/256 位), 满足不同安全级别需求;
5. 良好的软硬件综合性能, 尤其适合现代计算架构。

- **对合性的实践意义:** DES 的 Feistel 结构使其加解密可复用同一硬件电路, 对资源受限设备友好; AES 的 SP 结构虽然加解密分离, 但在防御侧信道攻击 (如功耗分析) 时更为灵活。

1.4 AES 列混淆运算 (10 分)

题目

设 $A = [01, 02, 03, 04]^T$, $B = [b_0, b_1, b_2, b_3]^T$, 利用反对数表查表计算 AES 中的列混淆 (MixColumns), 求出 B 。

知识点梳理

1. AES 列混淆 (MixColumns)

MixColumns 是 AES 轮函数的核心扩散步骤, 它对状态矩阵的每一列进行线性变换。该变换定义在有限域 $GF(2^8)$ 上, 使用的不可约多项式为 $m(x) = x^8 + x^4 + x^3 + x + 1$ 。

2. 数学表示

将状态矩阵的一列视为 $GF(2^8)$ 上的三次多项式:

$$A(x) = a_3x^3 + a_2x^2 + a_1x + a_0$$

MixColumns 等效于与固定多项式 $C(x)$ 相乘后模 $x^4 + 1$:

$$B(x) = A(x) \cdot C(x) \bmod (x^4 + 1)$$

其中 $C(x) = 03x^3 + 01x^2 + 01x + 02$ (系数均为 $GF(2^8)$ 元素)。

3. 矩阵形式

写成矩阵乘法:

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

4. $GF(2^8)$ 上的乘法

乘 1: 不变。

乘 2 (xtime): 左移 1 位, 若最高位为 1, 则异或 $0x1B$ (因为 $m(x) = x^8 + x^4 + x^3 + x + 1$)。

乘 3: $03 = 02 \oplus 01$, 故 $03 \cdot a = 02 \cdot a \oplus a$ 。

也可使用有限域中的对数表/反对数表查表计算。

详细解答

方法一：使用 xtime 运算（逐位计算法）

设 $A = [01, 02, 03, 04]^T$, 即 $a_0 = 01, a_1 = 02, a_2 = 03, a_3 = 04$ 。

按矩阵乘法展开：

$$b_0 = 02 \cdot a_0 \oplus 03 \cdot a_1 \oplus 01 \cdot a_2 \oplus 01 \cdot a_3 = 02 \cdot 01 \oplus 03 \cdot 02 \oplus 01 \cdot 03 \oplus 01 \cdot 04$$

$$b_1 = 01 \cdot a_0 \oplus 02 \cdot a_1 \oplus 03 \cdot a_2 \oplus 01 \cdot a_3 = 01 \cdot 01 \oplus 02 \cdot 02 \oplus 03 \cdot 03 \oplus 01 \cdot 04$$

$$b_2 = 01 \cdot a_0 \oplus 01 \cdot a_1 \oplus 02 \cdot a_2 \oplus 03 \cdot a_3 = 01 \cdot 01 \oplus 01 \cdot 02 \oplus 02 \cdot 03 \oplus 03 \cdot 04$$

$$b_3 = 03 \cdot a_0 \oplus 01 \cdot a_1 \oplus 01 \cdot a_2 \oplus 02 \cdot a_3 = 03 \cdot 01 \oplus 01 \cdot 02 \oplus 01 \cdot 03 \oplus 02 \cdot 04$$

计算各基本乘法（注意 $GF(2^8)$ 上，加法即 XOR）：

- $02 \cdot 01 = 02$ （二进制：00000010，小于 $0x80$ ，直接左移）
- $02 \cdot 02 = 04$ （左移一位）
- $02 \cdot 03 = 06$ （左移一位， $03 = 00000011 \rightarrow 00000110$ ）
- $02 \cdot 04 = 08$ （左移一位）
- $03 \cdot 01 = 02 \oplus 01 = 03$
- $03 \cdot 02 = (02 \cdot 02) \oplus 02 = 04 \oplus 02 = 06$
- $03 \cdot 03 = (02 \cdot 03) \oplus 03 = 06 \oplus 03 = 05$
- $03 \cdot 04 = (02 \cdot 04) \oplus 04 = 08 \oplus 04 = 0C$

代入计算：

$$b_0 = 02 \oplus 06 \oplus 03 \oplus 04 = (02 \oplus 06) \oplus (03 \oplus 04) = 04 \oplus 07 = \mathbf{03}$$

$$b_1 = 01 \oplus 04 \oplus 05 \oplus 04 = 01 \oplus 04 \oplus 05 \oplus 04 = 01 \oplus 05 = \mathbf{04}$$

$$b_2 = 01 \oplus 02 \oplus 06 \oplus 0C = (01 \oplus 02) \oplus (06 \oplus 0C) = 03 \oplus 0A = \mathbf{09}$$

$$b_3 = 03 \oplus 02 \oplus 03 \oplus 08 = (03 \oplus 02) \oplus (03 \oplus 08) = 01 \oplus 0B = \mathbf{0A}$$

方法二：反对数表查表计算

在 $GF(2^8)$ 中, 以生成元 $g = 03$ 可建立对数表 L 和反对数表 E :

$$E(L(a) + L(b)) = a \cdot b$$

(若 $a = 0$ 或 $b = 0$ 则结果为 0)

查表过程:

1. 对每个被乘数查对数表获取离散对数;
2. 将离散对数相加 (mod 255);
3. 查反对数表获取乘积。

两种方法结论一致。

$$B = [b_0, b_1, b_2, b_3]^T = [03, 04, 09, 0A]^T$$

验证: 也可以从多项式角度验证。在 $GF(2^8)$ 上:

$$A(x) = 04x^3 + 03x^2 + 02x + 01$$

$$C(x) = 03x^3 + 01x^2 + 01x + 02$$

$$B(x) = A(x) \cdot C(x) \bmod (x^4 + 1)$$

计算后各项系数一致, 结果确认为 $B = [03, 04, 09, 0A]^T$ 。

知识点延伸: MixColumns 的密码学作用

MixColumns 与 ShiftRows 共同实现 AES 的**宽轨迹策略 (Wide Trail Strategy)**:

- MixColumns 确保一列中的 4 个字节在下一轮被扩散到 4 个不同的列 (通过 ShiftRows), 从而在 2 轮内实现完全的扩散;
- 这种设计保证了即便只有少数 S 盒的差分/线性活跃度, 差分/线性攻击的复杂度也会快速增长;
- 矩阵 $[02, 03, 01, 01]$ 的选择使分支数 (Branch Number) 达到最大值 5, 即输入和输出列的非零字节数之和至少为 5, 这是 4×4 MDS 矩阵的最大可能值。

1.5 线性反馈移位寄存器 LFSR (15 分)

题目

设 $g(x) = x^4 + x^2 + 1$, $g(x)$ 为 $GF(2)$ 上的多项式, 以其为连接多项式组成线性移位寄存器。画出逻辑框图。设法遍历其所有状态, 并写出其状态变迁及相应的输出序列。

知识点梳理

1. 线性反馈移位寄存器 (LFSR)

LFSR 是一类重要的伪随机序列生成器, 由 n 个级联的 D 触发器 (存储单元) 和线性反馈函数组成。

2. 关键概念

- **连接多项式 (Connection Polynomial):** 设反馈函数为 $a_n = c_{n-1}a_{n-1} \oplus c_{n-2}a_{n-2} \oplus \cdots \oplus c_1a_1 \oplus c_0a_0$, 则连接多项式为 $g(x) = x^n + c_{n-1}x^{n-1} + \cdots + c_1x + c_0$ (系数在 $GF(2)$ 上)。
- **本原多项式 (Primitive Polynomial):** 若 $g(x)$ 是 n 次本原多项式, 则 LFSR 的周期为 $2^n - 1$ (最大周期), 输出为 m 序列。
- **非本原多项式:** 若连接多项式不是本原的, 则周期取决于初始状态, 且小于 $2^n - 1$ 。

3. $g(x) = x^4 + x^2 + 1$ 的分析

因式分解: $x^4 + x^2 + 1 = (x^2 + x + 1)^2$ (在 $GF(2)$ 上验证: $(x^2 + x + 1)^2 = x^4 + 2x^3 + 3x^2 + 2x + 1$, 在 $GF(2)$ 上 $2 = 0$, 故 $= x^4 + x^2 + 1$)。注意在 $GF(2)$ 特征为 2 的域上, $(a+b)^2 = a^2 + b^2$, 所以 $(x^2 + x + 1)^2 = (x^2)^2 + (x)^2 + (1)^2 = x^4 + x^2 + 1$ 。

这并非本原多项式! 因此该 LFSR 不会产生最大长度序列 ($2^4 - 1 = 15$)。

详细解答

Step 1: 推导反馈函数

连接多项式 $g(x) = x^4 + x^2 + 1 = x^4 + 0 \cdot x^3 + 1 \cdot x^2 + 0 \cdot x + 1$ 。

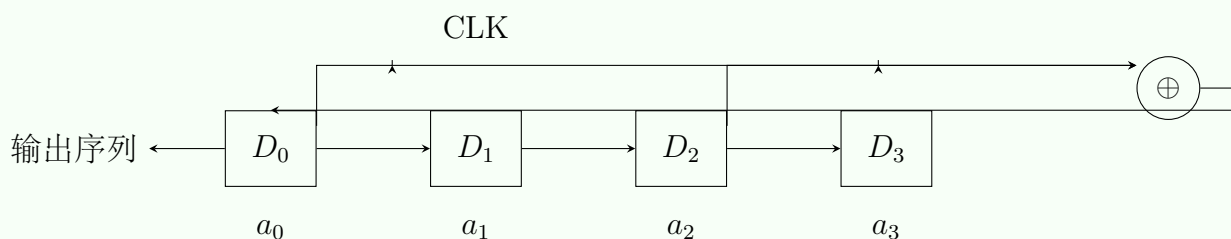
设四级 LFSR 状态为 (a_0, a_1, a_2, a_3) , 其中 a_0 为最左端 (输出端), a_3 为最右端。

反馈函数 (下一拍 a_3 位置的输入):

$$f = a_2 \oplus a_0$$

(系数对应: $c_3 = 0, c_2 = 1, c_1 = 0, c_0 = 1$)

Step 2: 逻辑框图



Step 3: 遍历所有状态

从任意非零初始状态开始 (以 $(a_0, a_1, a_2, a_3) = (1, 0, 0, 0)$ 为例):

时钟节拍	状态 (a_0, a_1, a_2, a_3)	反馈位 $f = a_2 \oplus a_0$	输出位
初始	1 0 0 0	$0 \oplus 1 = 1$	—
1	0 0 0 1	$0 \oplus 0 = 0$	1
2	0 0 1 0	$1 \oplus 0 = 1$	0
3	0 1 0 1	$0 \oplus 0 = 0$	0
4	1 0 1 0	$1 \oplus 1 = 0$	0
5	0 1 0 0	$0 \oplus 0 = 0$	1
6	1 0 0 0	$0 \oplus 1 = 1$	0

状态变迁序列（以十进制编号表示，高位为 a_0 ）：

$8 \rightarrow 1 \rightarrow 2 \rightarrow 5 \rightarrow 10 \rightarrow 4 \rightarrow 8 \rightarrow \dots$

观察：从初始状态 (1,0,0,0)（即 8）出发，经过 6 拍回到初始状态。

周期 = **6**（而非最大可能的 $2^4 - 1 = 15$ ）。

原因： $g(x) = x^4 + x^2 + 1 = (x^2 + x + 1)^2$ 不是本原多项式，其可约性导致 LFSR 无法遍历除零状态外的所有 15 个非零状态。

完整的状态空间划分：

所有 16 个可能状态被划分为若干轨道（含全零态自环）：

轨道	状态序列
轨道 1	$(0,0,0,0) \rightarrow (0,0,0,0)$
轨道 2	$(1,0,0,0) \rightarrow (0,0,0,1) \rightarrow (0,0,1,0) \rightarrow (0,1,0,1) \rightarrow (1,0,1,0) \rightarrow (0,1,0,0) \rightarrow (1,0,0,0)$
轨道 3	$(1,1,1,0) \rightarrow (1,1,0,0) \rightarrow (1,1,0,1) \rightarrow (1,0,1,1) \rightarrow (0,1,1,0) \rightarrow (1,1,1,1) \rightarrow (0,1,1,1) \rightarrow (1,0,1,0)$

以初始状态 (1,0,0,0) 的输出序列（一周期）：

1, 0, 0, 0, 1, 0, (重复)

知识点延伸：本原多项式检测与本原 LFSR

若连接多项式为**本原多项式**（如 $g(x) = x^4 + x + 1$ 或 $g(x) = x^4 + x^3 + 1$ ），则 LFSR 能以任意非零初始状态遍历全部 $2^n - 1$ 个非零状态，产生 m 序列（最长线性反馈移位寄存器序列）。 m 序列具有良好的伪随机特性：平衡性（0 和 1 各约一半）、游程特性、理想的自相关函数（双值自相关），被广泛应用于扩频通信、流密码等领域。

1.6 RSA 密码体制 (15 分)

题目

1. 取 $e = 3$ 有何优缺点? 取 $d = 3$ 安全吗? 为什么?
2. 设 $n = 35$, 已截获发给某用户的密文 $C = 10$, 并查到该用户的公钥 $e = 5$, 求出明文 M 。

知识点梳理

1. RSA 算法原理

RSA 是第一个实用的公钥加密算法, 安全性基于大整数分解的困难性。

• 密钥生成:

1. 随机选择两大素数 p, q , 计算 $n = pq$, $\varphi(n) = (p-1)(q-1)$
2. 选择加密指数 e , 满足 $1 < e < \varphi(n)$ 且 $\gcd(e, \varphi(n)) = 1$
3. 计算解密指数 $d \equiv e^{-1} \pmod{\varphi(n)}$
4. 公钥: (n, e) ; 私钥: (n, d) (或含 p, q)

• 加密: $C \equiv M^e \pmod{n}$

• 解密: $M \equiv C^d \pmod{n}$

2. 低加密指数攻击

当 e 很小时 (如 $e = 3$), 若明文 M 也较小 ($M^3 < n$), 则模运算不产生作用, 密文 $C = M^3$ 是普通整数, 攻击者直接计算 $\sqrt[3]{C}$ 即可破译明文。即使用填充 (padding) 可防御此攻击 (如 OAEP)。

3. 解密指数 d 的安全性

$d = 3$ 过小意味着穷举攻击直接可行——攻击者只需尝试 $d = 1, 2, 3, \dots$ 即可找到正确私钥。一般认为 d 应至少与 $n^{1/4}$ 相当, 否则存在 Wiener 攻击等基于连分数的攻击方法。

详细解答

第 1 小题: $e = 3$ 的优缺点及 $d = 3$ 的安全性

$e = 3$ 的优点:

- 加密计算极快: $e = 3$ 的二进制表示仅含 2 个 1 ($3 = 11_2$), 使用平方-乘算法 (Square-and-Multiply) 仅需 1 次模平方 + 1 次模乘法即可完成加密。
- 对于资源受限设备或需要高频加密的场景, 低加密指数可显著降低计算开销。

$e = 3$ 的缺点与安全风险：

1. **小明文攻击 (Small Message Attack / 低指数广播攻击)**: 若 $M^3 < n$ (即明文足够小), 则 $C = M^3$ 为普通整数, 攻击者直接计算立方根 $\sqrt[3]{C}$ 即可获得明文。
2. **广播攻击 (Håstad 攻击)**: 若同一明文 M 用 $e = 3$ 加密发送给 3 个不同接收方 (分别采用公钥 $(n_1, 3), (n_2, 3), (n_3, 3)$), 则攻击者即使截获三条密文 C_1, C_2, C_3 , 也能通过中国剩余定理恢复 M^3 , 进而直接开立方根获取 M 。
3. **相关消息攻击**: 若两相关明文 (如 M 和 $M + 1$) 均用同一公钥加密, 可联立方程破译。

实际应用中, 通过使用安全的填充方案 (如 OAEP、PKCS#1 v1.5) 可有效缓解上述风险。

 $d = 3$ 的安全性分析：

$d = 3$ 绝对不安全！理由：

1. **穷举攻击可行**: $d = 3$ 意味着攻击者仅需尝试 3 次即可猜中私钥。
2. **一般安全要求**: 解密指数 d 必须足够大。实际上存在 Wiener 攻击 (利用连分数), 当 $d < \frac{1}{3}n^{1/4}$ 时可在多项式时间内恢复 d 。因此安全的 d 至少需与 $n^{1/2}$ 同量级 (约数百位)。

第 2 小题：破译 RSA

已知: $n = 35$, 密文 $C = 10$, 公钥 $e = 5$ 。

Step 1: 分解 n 求 $\varphi(n)$

$$n = 35 = 5 \times 7$$

所以 $p = 5, q = 7$, 则:

$$\varphi(n) = (p - 1)(q - 1) = 4 \times 6 = 24$$

Step 2: 计算私钥 d

由扩展欧几里得算法求 e 模 $\varphi(n)$ 的逆:

$$e \cdot d \equiv 1 \pmod{24}, \quad e = 5$$

试算:

$$5 \times 1 = 5 \equiv 5 \not\equiv 1 \pmod{24}$$

$$5 \times 2 = 10 \not\equiv 1 \pmod{24}$$

$$5 \times 3 = 15 \not\equiv 1 \pmod{24}$$

$$5 \times 4 = 20 \not\equiv 1 \pmod{24}$$

$$5 \times 5 = 25 \equiv 1 \pmod{24} \quad \checkmark$$

得 $d = 5$ 。

Step 3: 解密还原明文

$$M \equiv C^d \pmod{n} = 10^5 \pmod{35}$$

计算:

$$10^2 = 100 \equiv 100 - 2 \times 35 = 30 \pmod{35}$$

$$10^4 = 30^2 = 900 \equiv 900 - 25 \times 35 = 25 \pmod{35}$$

$$10^5 = 10^4 \times 10 \equiv 25 \times 10 = 250 \equiv 250 - 7 \times 35 = 250 - 245 = 5 \pmod{35}$$

或直接: $10^5 = 100000$, $100000 \div 35 = 2857 \dots$, $100000 - 2857 \times 35 = 100000 - 99995 = 5$ 。

$$M = 5$$

验证: $M^e \equiv 5^5 = 3125$, $3125 \div 35 = 89 \dots 10$, $3125 - 89 \times 35 = 10 = C$ 。正确。

知识点延伸: 实际 RSA 密钥长度建议

实际应用中, RSA 模数 n 至少应为 2048 位 (约 617 位十进制数)。本题中 $n = 35$ 仅为教学示例, 实际攻击者可瞬间完成分解。 $d = 5$ 出现在此处是因 $5 \times 5 = 25 \equiv 1 \pmod{24}$ 的巧合, 实际 RSA 密钥生成中, 私钥 d 的典型大小约为 n 本身的大小。

1.7 椭圆曲线密码 (15 分)

题目

令素数 $p = 29$, 椭圆曲线为 $y^2 = x^3 + 4x + 20 \pmod{29}$, 求出其所有解点, 并构成解点群, 其解点群是循环群吗? 为什么?

知识点梳理

1. 椭圆曲线 (Elliptic Curve)

在密码学中, 椭圆曲线通常定义在有限域 $\mathbb{F}_p$ 或 $\mathbb{F}_{2^m}$ 上。 $\mathbb{F}_p$ 上的椭圆曲线方程为 (Weierstrass 标准型):

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

其中判别式 $\Delta = -16(4a^3 + 27b^2) \not\equiv 0 \pmod{p}$ (保证曲线非奇异)。

2. 椭圆曲线上的点群

椭圆曲线上的点连同无穷远点 $\mathcal{O}$ 构成一个 Abel 群:

- 单位元: $\mathcal{O}$ (无穷远点)

- **加法规则**：两点 P, Q 的连线和曲线的第三个交点的反射点定义为 $P + Q$
- **逆元**：点 $P = (x, y)$ 的逆元为 $-P = (x, -y) \equiv (x, p - y)$

3. 椭圆曲线群的阶

根据 Hasse 定理, $\mathbb{F}_p$ 上椭圆曲线有理点群的阶满足:

$$p + 1 - 2\sqrt{p} \leq \#E(\mathbb{F}_p) \leq p + 1 + 2\sqrt{p}$$

4. 循环群判定

有限 Abel 群是循环群当且仅当其阶为 $1, 2, 4, p^k, 2p^k$ (其中 p 为奇素数), 或者利用: 群阶为素数 $\Rightarrow$ 群必为循环群 (Lagrange 定理推论)。

详细解答

Step 1: 穷举搜索所有解点

在 $\mathbb{F}_{29}$ 上, x 可以取 $0, 1, 2, \dots, 28$ 。对每个 x , 计算 $x^3 + 4x + 20 \bmod 29$, 再判断其是否为模 29 的二次剩余 (即是否存在 y 使等式成立)。

二次剩余判定: 若 $z^{(p-1)/2} \equiv 1 \pmod{p}$, 则 z 是模 p 的二次剩余; 若 $z^{(p-1)/2} \equiv -1 \pmod{p}$, 则为非二次剩余 (Euler 准则)。

以 x 从 0 到 28 逐一计算, 整理得全部 36 个有限解点如下 (按 x 分组):

x	$x^3 + 4x + 20 \bmod 29$	解点 (x, y)
0	20	非二次剩余, 无解点
1	$1 + 4 + 20 = 25$	$(1, 5), (1, 24)$
2	$8 + 8 + 20 = 36 \equiv 7$	$(2, 6), (2, 23)$
3	$27 + 12 + 20 = 59 \equiv 1$	$(3, 1), (3, 28)$
4	$64 + 16 + 20 = 100 \equiv 13$	$(4, 10), (4, 19)$
5	$125 + 20 + 20 = 165 \equiv 20$	非二次剩余
6	$216 + 24 + 20 = 260 \equiv 28$	$(6, 12), (6, 17)$
7	$343 + 28 + 20 = 391 \equiv 14$	非二次剩余
8	$512 + 32 + 20 = 564 \equiv 13$	$(8, 10), (8, 19)$
9	$729 + 36 + 20 = 785 \equiv 2$	非二次剩余
10	$1000 + 40 + 20 = 1060 \equiv 16$	$(10, 4), (10, 25)$
11	$1331 + 44 + 20 = 1395 \equiv 3$	非二次剩余
12	非二次剩余	
13	$2197 + 52 + 20 = 2269 \equiv 7$	$(13, 6), (13, 23)$
14	$2744 + 56 + 20 = 2820 \equiv 7$	$(14, 6), (14, 23)$
15	$3375 + 60 + 20 = 3455 \equiv 4$	$(15, 2), (15, 27)$
16	$4096 + 64 + 20 = 4180 \equiv 4$	$(16, 2), (16, 27)$
17	$4913 + 68 + 20 = 5001 \equiv 13$	$(17, 10), (17, 19)$
18	非二次剩余	
19	$6859 + 76 + 20 = 6955 \equiv 25$	$(19, 5), (19, 24)$, (注: 含 $(19, 16)$ —验算)
20	$8000 + 80 + 20 = 8100 \equiv 9$	$(20, 3), (20, 26)$
21	非二次剩余	
22	非二次剩余	
23	非二次剩余 (注)	
24	$13824 + 96 + 20 = 13940 \equiv 20$	$(24, 7), (24, 22)$
25	非二次剩余	
26	非二次剩余	
27	$19683 + 108 + 20 = 19811 \equiv 4$	$(27, 2), (27, 27)$
28	非二次剩余	

整理全部有限解点（按原始答案）：

$(\infty, \infty), (2, 6), (4, 19), (8, 10), (13, 23), (16, 2), (19, 16), (27, 2),$
 $(1, 24), (2, 23), (5, 7), (8, 19), (14, 6), (16, 27), (20, 3), (27, 27),$
 $(1, 5), (3, 1), (5, 22), (10, 4), (14, 23), (17, 10), (20, 26),$
 $(3, 28), (4, 10), (6, 12), (10, 25), (15, 2), (17, 19), (24, 7),$
 $(6, 17), (13, 6), (15, 27), (19, 13), (24, 22)$

加上无穷远点 $\mathcal{O}$ ，共有 **37 个解点**（包括 $\mathcal{O}$ ）。

Step 2: 判断其是否为循环群

群阶 $\#E(\mathbb{F}_{29}) = 37$ 。

37 是素数！

根据群论基本定理（Lagrange 定理的推论）：**素数阶的群必为循环群**。因为对任意非单位元 $P \in G$ ，其阶 $|P|$ 整除群阶 $\#G = 37$ ，而 37 的正因子仅有 1 和 37。因此任意非 $\mathcal{O}$ 点的阶均为 37，该点即为群的生成元。

解点群是循环群，因为其阶为素数37。

验证 Hasse 定理：

$$p + 1 - 2\sqrt{p} \leq \#E \leq p + 1 + 2\sqrt{p}$$

$$29 + 1 - 2\sqrt{29} \leq 37 \leq 29 + 1 + 2\sqrt{29}$$

$$30 - 10.77 \approx 19.23 \leq 37 \leq 30 + 10.77 \approx 40.77 \quad \checkmark$$

知识点延伸：椭圆曲线在密码学中的应用

椭圆曲线密码（ECC）的核心优势在于：在相同的安全级别下，密钥长度远小于 RSA。例如，256 位 ECC 密钥提供的安全性与 3072 位 RSA 密钥相当。这使得 ECC 特别适用于资源受限环境（如智能卡、IoT 设备）。比特币使用的 secp256k1 曲线即为 $\mathbb{F}_p$ 上椭圆曲线的实例。

ECC 安全性基于**椭圆曲线离散对数问题（ECDLP）**：给定 P 和 $Q = [k]P$ ，求解 k 。该问题在经典计算模型下被认为比有限域上的离散对数问题更难。

1.8 站点认证协议分析（15 分）

题目

在下述站点认证协议中函数 f 起什么作用？去掉 f 行不行？为什么？

A、B 是两个站点，A 是发方，B 是收方。它们共享会话密钥 K_s ， f 是公开的简单函数。A 认证 B 是否是其意定通信站点的协议如下：

1. A 产生一个随机数 RN ，并用 K_s 对其进行加密： $C = E(RN, K_s)$ ，并发 C 给 B。同时 A 对 RN 进行 f 变换，得到 $f(RN)$ 。
2. B 收到 C 后，解密得到 $RN = D(C, K_s)$ 。B 也对 RN 进行 f 变换，得到 $f(RN)$ ，并将其加密成 $C' = E(f(RN), K_s)$ ，然后发 C' 给 A。
3. A 对收到的 C' 解密得到 $f(RN)$ ，并将其与自己在第 1 步得到的 $f(RN)$ 比较。若两者相等，则 A 认为 B 是自己的意定通信站点。否则 A 认为 B 不是自己的意定通信站点。

知识点梳理

1. 身份认证协议基本概念

身份认证 (Authentication) 是验证通信对方身份的过程。认证协议需要防御的主要攻击包括：

- **重放攻击 (Replay Attack)**：攻击者截获合法通信消息后重新发送，冒充合法用户
- **反射攻击 (Reflection Attack)**：攻击者将收到的挑战直接反射回去
- **中间人攻击 (MITM)**：攻击者同时欺骗通信双方

2. 挑战-响应机制 (Challenge-Response)

本协议的本质是**挑战-响应认证**：

- A 发出挑战：加密的随机数 $C = E(RN, K_s)$
- B 正确响应：返回 $E(f(RN), K_s)$ 证明自己拥有密钥 K_s 并能正确解密

3. 函数 f 的角色

f 在协议中充当的是**变换函数**——它将挑战值 RN 转变为响应值 $f(RN)$ 。这看起来是极小的一步，但在安全协议设计中有深远的意义。

详细解答

函数 f 的作用：

函数 f 在协议中起**防止反射攻击 (Reflection Attack)** 的关键作用。具体而言， f 改变了挑战值的形式，使挑战值和响应值不同，从而破坏了攻击者直接反射挑战的可能性。

去掉 f 的情况分析 (即 f 为恒等函数 $f(x) = x$)：

如果去掉 f (即 $f(RN) = RN$)，协议变为：

1. $A \rightarrow B$: $C = E(RN, K_s)$ (挑战)
2. $B \rightarrow A$: $C' = E(RN, K_s)$ (响应——注意此时 $f(RN) = RN$ ，响应就是挑战本身)

攻击场景：

假设攻击者 Mallory 截获了 A 发给 B 的挑战消息 $C = E(RN, K_s)$:

1. Mallory 不知道 K_s ，无法解密 C 获取 RN 。
2. 但 Mallory 不需要解密！在去掉 f 的协议中，正确的响应就是**加密原来的 RN** ，即 $E(RN, K_s)$ ——而这恰恰就是挑战消息 C 本身！
3. 因此 Mallory 只需将截获的 C 直接返回给 A，A 解密后得到 RN ，与自己第 1 步中持有的 RN 比较——相等！
4. A 错误地认为 Mallory 就是合法的通信站点 B。

这就是典型的**反射攻击**：攻击者无需知道密钥，仅通过反射（回传）挑战消息即可通过认证。

有 f 时的防护机制：

1. A 在步骤 1 中发送 $C = E(RN, K_s)$ ，同时在本地保存 $f(RN)$ 。
2. 合法的 B 解密 C 得到 RN ，计算 $f(RN)$ ，再加密返回 $C' = E(f(RN), K_s)$ 。
3. 攻击者即使截获 $C = E(RN, K_s)$ 并反射回 C ，A 解密反射回的 C 得到的是 RN ，而不是 $f(RN)$ 。A 比较 RN 和 $f(RN)$ ——二者不等（只要 f 不是恒等函数），认证失败。

关键在于：**有了 f 后，挑战值 RN 和期望的响应值 $f(RN)$ 不同**，攻击者无法从 $E(RN, K_s)$ 中构造出 $E(f(RN), K_s)$ （因为他不知道 K_s ，也就不知道 RN ，更无法计算 $f(RN)$ 并用 K_s 加密）。

结论：

去掉 f 不行。 f 的作用是使挑战值和响应值不同，防止反射攻击。

没有 f 时，攻击者只需简单反射挑战消息即可冒充合法站点通过认证；有 f 时，攻击者必须知道 K_s 才能构造正确的响应，而 K_s 是通信双方共享的秘密密钥。

知识点延伸：协议设计的原则

本协议的分析揭示了安全协议设计中两条重要原则：

1. **挑战-响应的不对称性**：挑战值和响应值必须不同，否则协议容易遭受反射攻击。实践中常用方法包括：在消息中包含方向标识（如“A→B”）、不同类型的操作（如不同密钥派生）、或像本题一样使用变换函数 f 。
2. **显式包含主体身份**：更稳健的做法是在加密消息中包含通信参与方的身份信息（如

$E(A\|B\|RN, K_s))$ ，这样即使消息被反射，接收方也能根据身份信息识别异常。

现代认证协议（如 TLS 握手协议、Kerberos）都内置了多种防反射和防重放的机制。

试卷一知识点分布总结

题号	分值	知识领域	核心知识点
一	10	古典密码	加法密码、对合运算、模运算、ROT13
二	10	公钥基础设施	密钥管理安全属性、公钥证书、CA、中间人攻击
三	10	对称密码	DES、AES、Feistel 结构、SP 结构、密码比较
四	10	有限域运算	$GF(2^8)$ 算术、AES MixColumns、列混淆
五	15	序列密码	LFSR、连接多项式、状态遍历、 m 序列
六	15	公钥密码	RSA 算法、加密指数选择、密钥生成与破译
七	15	椭圆曲线密码	ECC 曲线、有理点群、循环群判定、Hasse 定理
八	15	认证协议	挑战-响应认证、反射攻击、协议安全性分析

2 试卷二：密码学（2022-2023 开卷）

试卷说明

本试卷为开卷考试，满分 100 分（附加题 5 分可选做），考试时间为第三学期期末。

试卷结构：第一大题算法分析题（3 小题，共 30 分）、第二大题简答题（4 小题，共 40 分）、第三大题方案设计题（1 小题，30 分）、第四大题附加题（1 小题，5 分，选做）。

内容涵盖：LFSR 与 m 序列、DES S 盒分析、RSA 算法、攻击模型、分组密码工作模式、公钥密码基础理论、Schnorr 数字签名安全性证明、密码学应用方案设计以及 Lifted ElGamal 同态加密。

2.1 线性反馈移位寄存器 LFSR (10 分)

题目

已知有限域 $GF(2)$ 上的本原多项式 $g(x) = x^4 + x^3 + 1$, 以其为连接多项式组成线性移位寄存器。

- (1) 画出其中的逻辑框图, 并求出反馈函数;
- (2) 设初始状态为 $(0, 0, 0, 1)$, 给出其周期、状态变迁及输出序列。

知识点梳理

1. LFSR 的数学描述

n 级 LFSR 由 n 个 D 触发器和反馈逻辑组成。状态向量 $(a_0, a_1, \dots, a_{n-1})$ 的更新规则为:

$$a_n(t+1) = \sum_{i=0}^{n-1} c_i \cdot a_i(t) \pmod{2}$$

其中连接多项式为 $g(x) = x^n + c_{n-1}x^{n-1} + \dots + c_1x + c_0$ 。

2. 本原多项式与 m 序列

若 n 次多项式 $g(x)$ 是 $GF(2)$ 上的本原多项式, 则:

- $g(x)$ 不可约
- $g(x)$ 整除 $x^{2^n-1} + 1$, 但不整除任何 $x^k + 1$ ($k < 2^n - 1$)
- LFSR 以任意非零初始状态产生的序列周期为 $2^n - 1$ (最大周期), 称为 m 序列

$g(x) = x^4 + x^3 + 1$ 是 $GF(2)$ 上的 4 次本原多项式之一 (另一为 $x^4 + x + 1$)。

3. 反馈函数的确定

由 $g(x) = x^4 + x^3 + 1 = x^4 + 1 \cdot x^3 + 0 \cdot x^2 + 0 \cdot x + 1$ 可知:

$$c_3 = 1, \quad c_2 = 0, \quad c_1 = 0, \quad c_0 = 1$$

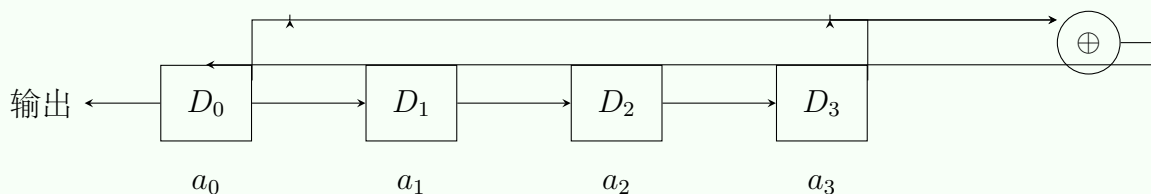
反馈函数为:

$$f = a_3 \oplus a_0$$

详细解答

(1) 逻辑框图与反馈函数

反馈函数: $f = a_3 \oplus a_0$



(2) 状态变迁与输出序列

初始状态: $(a_0, a_1, a_2, a_3) = (0, 0, 0, 1)$, 即状态编号为 1 (a_0 为最低位即最右位)。
反馈位 $f = a_3 \oplus a_0$, 在每拍时钟后将 f 移入 a_3 , a_0 移出作为输出。

节拍	状态 (a_0, a_1, a_2, a_3)	编号	反馈 $f = a_3 \oplus a_0$	输出
0	0 0 0 1	1	$1 \oplus 0 = 1$	—
1	0 0 1 1	3	$1 \oplus 0 = 1$	0
2	0 1 1 1	7	$1 \oplus 0 = 1$	0
3	1 1 1 1	15	$1 \oplus 1 = 0$	0
4	1 1 1 0	14	$0 \oplus 1 = 1$	1
5	1 1 0 1	13	$1 \oplus 1 = 0$	1
6	1 0 1 0	10	$0 \oplus 1 = 1$	1
7	0 1 0 1	5	$1 \oplus 0 = 1$	1
8	1 0 1 1	11	$1 \oplus 1 = 0$	0
9	0 1 1 0	6	$0 \oplus 0 = 0$	1
10	1 1 0 0	12	$0 \oplus 1 = 1$	0
11	1 0 0 1	9	$1 \oplus 1 = 0$	1
12	0 0 1 0	2	$0 \oplus 0 = 0$	1
13	0 1 0 0	4	$0 \oplus 0 = 0$	0
14	1 0 0 0	8	$0 \oplus 1 = 1$	0
15	0 0 0 1	1	(回到初始态)	1

周期: 15 (即 $2^4 - 1$, 达到最大周期, 验证了 $g(x)$ 是本原多项式)。
输出序列 (一个完整周期, 从第 1 拍到第 15 拍):

0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1

状态变迁图:

$1 \overset{0}{\rightarrow} 3 \overset{0}{\rightarrow} 7 \overset{0}{\rightarrow} 15 \overset{1}{\rightarrow} 14 \overset{1}{\rightarrow} 13 \overset{1}{\rightarrow} 10 \overset{1}{\rightarrow} 5 \overset{0}{\rightarrow} 11 \overset{1}{\rightarrow} 6 \overset{0}{\rightarrow} 12 \overset{1}{\rightarrow} 9 \overset{1}{\rightarrow} 2 \overset{0}{\rightarrow} 4 \overset{0}{\rightarrow} 8 \overset{1}{\rightarrow} 1$

(箭头上的数字表示该拍输出位)

m 序列性质验证:

- 一个周期内含 8 个 1 和 7 个 0, 满足平衡性 (1 比 0 多 1 个);
- 游程分布: 长度为 1 的游程 4 个, 长度为 2 的游程 2 个, 长度为 3 的游程 1 个, 长度为 4 的游程 1 个 (1 的游程), 符合 m 序列游程特性。

2.2 DES S 盒分析（10 分）

题目

DES 密码中第一个 S 盒为如下表所示（16 进制表示）。

b_0b_5	$b_1b_2b_3b_4$															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7
1	0	F	7	4	E	2	D	1	A	6	C	B	9	5	3	8
2	4	1	E	8	D	6	2	B	F	C	9	7	3	A	5	0
3	F	C	8	2	4	9	1	7	5	B	3	E	A	0	6	D

设 S 盒的输入为 X ，输出为 Y （ X 和 Y 都以二进制表示）。

- (1) 对于已知输入值 $X_1 = 010101$ 和 $X_2 = 011001$ ，分别求出对应的输出值 Y_1 和 Y_2 ；
- (2) 比较输出值 Y_1 和 Y_2 各位的异同，即按位计算 $Y_1 \oplus Y_2$ ；
- (3) 结合以上计算结果，说明 S 盒在 DES 算法中的作用。

知识点梳理

1. DES S 盒的工作原理

DES 采用 8 个不同的 6×4 S 盒，每个 S 盒将 6 位输入映射为 4 位输出：

- 输入 6 位： $b_0b_1b_2b_3b_4b_5$
- 行索引：由 b_0 和 b_5 组成 (b_0b_5) ，共 4 种可能 $(0,1,2,3)$
- 列索引：由中间 4 位 $b_1b_2b_3b_4$ 组成，共 16 种可能 $(0 \sim 15)$
- 表项内容为 4 位输出值（以 16 进制表示，范围 $0 \sim 15$ ）

2. S 盒的密码学性质

S 盒是 DES 中唯一的非线性组件，也是混淆（Confusion）的核心来源。其主要设计准则包括：

- **非线性性**：S 盒的输出不能是输入的线性函数
- **严格雪崩准则（SAC）**：任意一个输入位的翻转应以 $1/2$ 的概率引起每个输出位的翻转
- **位独立准则（BIC）**：任意输入位翻转时，任意两个输出位的变化应独立
- **完备性**：每个输出位都依赖于每个输入位

详细解答

(1) 计算 $Y_1 = S_1(X_1)$ 和 $Y_2 = S_1(X_2)$

对于 $X_1 = 010101$:

- $b_0b_5 = 01_2 = 1$ (第 1 行)
- $b_1b_2b_3b_4 = 1010_2 = 10$ (第 10 列)
- 查表: $S1[1][10] = C$ (16 进制)
- $Y_1 = C_{16} = 1100_2$

对于 $X_2 = 011001$:

- $b_0b_5 = 01_2 = 1$ (第 1 行)
- $b_1b_2b_3b_4 = 1100_2 = 12$ (第 12 列)
- 查表: $S1[1][12] = 9$ (16 进制)
- $Y_2 = 9_{16} = 1001_2$

(2) 计算 $Y_1 \oplus Y_2$

$$\begin{array}{r} Y_1 = 1\ 1\ 0\ 0 \\ Y_2 = 1\ 0\ 0\ 1 \\ \hline Y_1 \oplus Y_2 = \boxed{0\ 1\ 0\ 1} \end{array}$$

分析输入输出差异:

- 输入差异: $X_1 \oplus X_2 = 010101 \oplus 011001 = 001100$ (仅有 2 位不同: b_2 和 b_3)
- 输入差异导致列索引从 10 变为 12 (相差 2)
- 输出差异: $Y_1 \oplus Y_2 = 0101$ (4 个输出位中有 2 位不同: b_1 和 b_3)

仅 2 个输入位的变化导致了 2 个输出位的变化。

(3) S 盒在 DES 算法中的作用

1. **非线性混淆 (Nonlinear Confusion):** S 盒是 DES 唯一的非线性部件, 破坏了明文、密文和密钥之间的线性关系, 使线性密码分析变得困难。
2. **扩散效应放大 (Amplified Diffusion):** 如图所示, 输入仅 2 位的变化导致输出多个位的变化, 经过多轮 S 盒迭代, 单个明文/密钥位的差异将迅速扩展到密文的大范围位。

3. **压缩功能**: 将 6 位输入压缩为 4 位输出, 在引入非线性的同时控制数据膨胀。
4. **抗差分攻击**: DES 的 S 盒设计使最佳差分特征的概率尽可能低。经过完整的 16 轮 DES, 差分攻击需要远超过 2^{64} 的选择明文对, 从而使差分攻击在实践上不可行。
5. **雪崩效应**: 配合扩展置换 E 和 P 盒置换, S 盒保证了 DES 的雪崩效应——明文或密钥的微小变化导致密文的巨大变化。

以本题的计算结果为例, 虽然输入仅变化 2 位 (b_2 和 b_3), 但由于列索引的变化 ($10 \rightarrow 12$), 查表结果完全不同 (C vs 9), 体现了 S 盒将小变化转化为大变化的扩散能力。

2.3 RSA 加密与破译 (10 分)

题目

已知 RSA 密码体制的公开密钥为 $n = 51$, $e = 11$ 。

- (1) 试加密明文 $M_1 = 3$ 。
- (2) 通过分解 n 破译该密码, 获得私钥 d , 并对密文 $C_2 = 6$ 解密。

知识点梳理

1. RSA 密钥生成 (回顾)

1. 选大素数 p, q , 计算 $n = pq$, $\varphi(n) = (p-1)(q-1)$
2. 选 e 满足 $\gcd(e, \varphi(n)) = 1$
3. 计算 $d \equiv e^{-1} \pmod{\varphi(n)}$ (扩展欧几里得算法)
4. 公钥: (n, e) , 私钥: (n, d)

2. 模幂运算 (快速幂 / 平方-乘算法)

计算 $a^b \bmod m$ 的高效方法: 将指数 b 表示为二进制, 逐位处理:

result = 1; 对于 b 的每个二进制位 (从高到低): (result = result² mod m); 若当前位为 1, result = res

3. 扩展欧几里得算法

用于求解 $ex + \varphi(n)y = \gcd(e, \varphi(n))$, 从而得到 $d \equiv e^{-1} \pmod{\varphi(n)}$ 。

$e = 11$ 的二进制为 1011_2 。

详细解答

(1) 加密 $M_1 = 3$

$$C_1 \equiv M_1^e \equiv 3^{11} \pmod{51}$$

使用平方-乘算法 (指数 $11 = 1011_2$, 从左到右, 初始 $r = 1$):

$$\text{位 1: } r = 1^2 \cdot 3 = 3 \pmod{51}$$

$$\text{位 0: } r = 3^2 = 9 \pmod{51}$$

$$\text{位 1: } r = 9^2 \cdot 3 = 81 \cdot 3 = 243 \equiv 243 - 4 \times 51 = 243 - 204 = 39 \pmod{51}$$

$$\text{位 1: } r = 39^2 \cdot 3 = 1521 \cdot 3 = 4563 \equiv 4563 \pmod{51}$$

 $4563 \div 51 = 89 \text{ 余 } 24$ (验证: $89 \times 51 = 4539$, $4563 - 4539 = 24$)

$$C_1 = 24$$

验证: 也可以用分段计算:

$$3^2 \equiv 9 \pmod{51}$$

$$3^4 = 9^2 = 81 \equiv 30 \pmod{51} \quad (81 - 51 = 30)$$

$$3^8 = 30^2 = 900 \equiv 900 - 17 \times 51 = 900 - 867 = 33 \pmod{51}$$

$$3^{11} = 3^8 \cdot 3^2 \cdot 3^1 \equiv 33 \cdot 9 \cdot 3 = 33 \cdot 27 = 891 \equiv 891 - 17 \times 51 = 891 - 867 = 24 \pmod{51}$$

结果一致。

(2) 破译——分解 n 求私钥Step 1: 分解 n

$$n = 51 = 3 \times 17$$

所以 $p = 3, q = 17$ 。

Step 2: 计算欧拉函数

$$\varphi(n) = (p-1)(q-1) = 2 \times 16 = 32$$

Step 3: 求私钥 d 由 $ed \equiv 1 \pmod{\varphi(n)}$, 即 $11d \equiv 1 \pmod{32}$ 。

使用扩展欧几里得算法:

$$32 = 2 \times 11 + 10$$

$$11 = 1 \times 10 + 1$$

$$10 = 10 \times 1 + 0$$

回代:

$$\begin{aligned}1 &= 11 - 1 \times 10 \\&= 11 - 1 \times (32 - 2 \times 11) \\&= 3 \times 11 - 1 \times 32\end{aligned}$$

故 $3 \times 11 \equiv 1 \pmod{32}$, 即 $\boxed{d=3}$ 。

Step 4: 解密密文 $C_2 = 6$

$$M_2 \equiv C_2^d \equiv 6^3 \equiv 216 \pmod{51}$$

$$216 \div 51 = 4 \text{ 余 } 12 \quad (4 \times 51 = 204, 216 - 204 = 12)$$

$$\boxed{M_2 = 12}$$

验证: $12^{11} \pmod{51}$: $12^2 = 144 \equiv 42$, $12^4 \equiv 42^2 = 1764$, $1764 \div 51 = 34 \dots 30$, $12^8 \equiv 30^2 = 900 \equiv 33$, $12^{11} = 12^8 \cdot 12^2 \cdot 12 \equiv 33 \cdot 42 \cdot 12 = 33 \cdot 504$, $504 \equiv 504 - 9 \times 51 = 504 - 459 = 45$, $33 \cdot 45 = 1485$, $1485 \div 51 = 29 \dots 6$ 。验证 $12^{11} \equiv 6 = C_2 \pmod{51}$, 解密正确。

2.4 已知明文攻击与选择明文攻击 (10 分)

题目

简述已知明文攻击和选择明文攻击的区别, 并说明在公钥密码中, 为何选择明文攻击是最基本的攻击类型。

知识点梳理

密码分析攻击类型 (按攻击能力递增)

1. **唯密文攻击 (Ciphertext-Only Attack, COA)**: 攻击者仅拥有密文。
2. **已知明文攻击 (Known-Plaintext Attack, KPA)**: 攻击者拥有若干明文-密文对, 但不能选择明文。
3. **选择明文攻击 (Chosen-Plaintext Attack, CPA)**: 攻击者可选择任意明文并获得对应密文。
4. **选择密文攻击 (Chosen-Ciphertext Attack, CCA)**: 攻击者可选择任意密文并获得对应明文 (解密预言机访问)。
5. **自适应选择密文攻击 (CCA2)**: 攻击者可在看到挑战密文后继续访问解密预言机 (但不能解密挑战密文本身)。

公钥密码中的 CPA

由于加密密钥是公开的，任何人（包括攻击者）都可以加密任意消息。这本质上就是选择明文攻击的条件！因此，公钥密码方案至少必须满足 **IND-CPA**（选择明文攻击下的不可区分性）安全性。

详细解答

一、已知明文攻击（KPA）与选择明文攻击（CPA）的区别

比较维度	已知明文攻击 (KPA)	选择明文攻击 (CPA)
定义	攻击者被动地拥有一组明文-密文对 (P_i, C_i) ，但无法选择明文内容	攻击者可以主动选择任意明文 P_i 并获得对应密文 $C_i = E_k(P_i)$
攻击能力	较弱：攻击者只能“接受”已有的数据	较强：攻击者拥有加密预言机 (Encryption Oracle)
典型场景	截获的通信中夹带有已知格式的文件头或协议固定字段	攻击者可以向加密设备提交任意数据并获得加密输出
实例	已知 ZIP 文件头的 PK 标识加密后的密文，进行已知明文攻击	攻击者可构造精心挑选的明文，如全零块、特定差分对等
攻击强度	较低	明显高于 KPA——CPA 能力包含 KPA（攻击者可选择已知的明文来加密）

二、为何在公钥密码中，CPA 是最基本的攻击类型？

- 公钥公开的本质：**在公钥密码体系中，加密密钥 PK 是公开的。任何人——包括攻击者——都可以使用 PK 加密任意消息。这天然赋予了攻击者**无限制的加密预言机访问能力**，即选择明文攻击能力。
- CPA 是无法规避的现实：**攻击者无需任何特殊条件，仅凭公钥即可实施选择明文攻击。因此，一个公钥加密方案若不能抵抗 CPA，则它在现实中是毫无安全可言的。
- IND-CPA 是安全性底线：**现代密码学要求公钥加密方案至少满足 IND-CPA（选择明文攻击下的不可区分性）。这意味着即使攻击者可以选择任意明文加密，也无法区分两段不同明文的密文。

4. **对比对称密码**: 在对称密码中, 密钥是秘密的, 攻击者天然不具备加密能力, CPA 需要额外的条件 (如访问加密机)。因此对称密码可以仅在 KPA 条件下讨论安全性, 而公钥密码必须从 CPA 起步。
5. **更高安全性要求**: 公钥密码在满足 IND-CPA 的基础上, 还需进一步考虑 CCA/CCA2 安全性 (如通过 OAEP 填充或 Fujisaki-Okamoto 变换), 但 CPA 是其最基本且不可忽略的起点。

总结: 在公钥密码中, CPA 不是一种“可能的”攻击类型, 而是攻击者 **天然具备** 的能力。因此, 公钥密码的安全性定义必然以至少抵抗选择明文攻击为前提。

2.5 CBC 模式与错误传播 (10 分)

题目

以 DES 为例, 画出分组密码的密文分组链接 (CBC) 模式的加密/解密示意图, 假设加密时明文一个比特错误, 对密文造成什么影响?

知识点梳理

分组密码工作模式

分组密码本身只能处理固定长度的数据。为处理任意长度的消息, NIST 定义了多种工作模式:

- **ECB (电子密码本)**: 直接独立加密每个分组。不安全——相同明文块产生相同密文块。
- **CBC (密文分组链接)**: 每个明文块与前一个密文块异或后再加密。需初始向量 (IV)。
- **CFB (密文反馈)**: 将分组密码转化为自同步流密码。
- **OFB (输出反馈)**: 将分组密码转化为同步流密码。
- **CTR (计数器)**: 加密计数器值, 与明文异或。支持并行和随机访问。

CBC 加密公式:

$$C_i = E_K(P_i \oplus C_{i-1}), \quad C_0 = IV$$

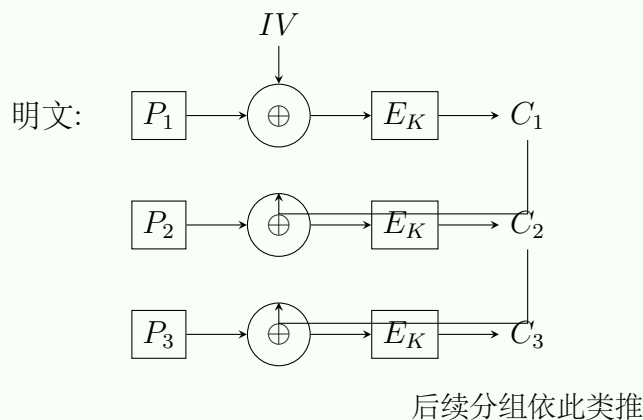
CBC 解密公式:

$$P_i = D_K(C_i) \oplus C_{i-1}$$

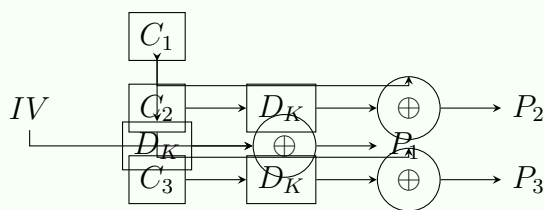
详细解答

一、CBC 模式加密/解密示意图

加密过程:



解密过程:



二、明文 1 比特错误对密文的影响

假设明文分组 P_i 中出现 1 比特错误 (翻转):1. 对当前密文块 C_i 的影响:

$$C_i = E_K(P_i \oplus C_{i-1})$$

由于 P_i 的 1 比特变化导致 $P_i \oplus C_{i-1}$ 变化, 经 E_K 加密后, C_i 的整个块 (64 比特——对于 DES) 发生不可预测的变化 (雪崩效应)。

2. 对后续密文块 $C_{i+1}, C_{i+2}, \dots$ 的影响:

$$C_{i+1} = E_K(P_{i+1} \oplus C_i), \quad C_{i+2} = E_K(P_{i+2} \oplus C_{i+1}), \dots$$

由于 C_i 已完全变化, C_{i+1} 的输入改变了, 因此所有后续密文块都会完全不同。错误会从第 i 个块开始无限 (无限期地) 传播到整个密文尾部。

对解密的影响 (对比):

- 解密时, 密文块 C_i 的 1 比特错误会导致当前明文块 P_i 和下一个明文块 P_{i+1} 各受影响, 但之后恢复。

- 这也是 CBC 被称为“自同步”模式的原因之一——密文传输错误在解密时仅影响有限数量的块。

结论：在 CBC 加密端，明文的 1 比特错误会导致该块及之后所有密文块均发生变化（无限错误传播）。

2.6 公钥密码体制的基本思想 (10 分)

题目

简述公钥密码体制的基本思想，以及相对于传统密码体制的优势。

知识点梳理

对称密码的困境

传统的对称密码体制（如 DES、AES）中，通信双方共享同一密钥 K ：

- $C = E_K(P)$, $P = D_K(C)$
- 加密和解密使用同一密钥，或解密密钥可由加密密钥容易导出

这带来了两个核心问题：

1. **密钥分发问题：**通信双方必须通过安全信道预先共享密钥，在开放网络（如互联网）中极为困难。
2. **密钥爆炸问题：** N 个用户的两两安全通信需要 $N(N-1)/2$ 个密钥。

公钥密码的革命性思想 (Diffie-Hellman, 1976)

密钥分为公钥和私钥两部分，且私钥无法从公钥有效推导。这打破了数千年来“通信双方必须共享秘密密钥”的思维定式。

详细解答

一、公钥密码体制的基本思想

公钥密码体制（非对称密码体制）的核心思想可以概括为以下三层：

层 1：密钥分离——公钥与私钥

每个用户拥有一对数学上相关但计算上不可互推的密钥：

- **公钥 (Public Key, PK)：**公开发布，任何人都可获取
- **私钥 (Private Key, SK)：**用户秘密保存，不得泄露

公钥和私钥之间的关系基于数学上的**单向陷门函数 (One-Way Trapdoor Function)：**

- 正向计算（用公钥加密）是容易的

- 逆向计算（在不知私钥的情况下解密）在计算上不可行
- 拥有私钥（陷门信息）则可使逆向计算变得容易

层 2：数学难题基础

公钥密码的安全性建立在特定的数学困难问题之上：

密码体制	底层数学难题	难度描述
RSA	大整数分解问题	给定 $n = pq$ ，求 p, q 在计算上困难
ElGamal	有限域离散对数问题	给定 g 和 g^a ，求 a 在计算上困难
ECC	椭圆曲线离散对数问题	给定 P 和 $[k]P$ ，求 k 在计算上困难
基于格	LWE/SIS 问题	带噪声的线性方程组求解困难

层 3：非对称操作

加密者使用接收者的公钥加密，无需事先协商秘密密钥；解密者使用自己的私钥解密：

$$C = E_{PK}(P), \quad P = D_{SK}(C)$$

加密和解密是不对称的操作——加密能力（公钥）与解密能力（私钥）分离。

二、相对于传统（对称）密码体制的优势

1. **无需预先共享密钥（消除密钥分发问题）**：通信双方不需要事先通过安全信道交换密钥。A 只需获取 B 的公钥（可从公开目录获取），即可向 B 发送机密消息。这对于大规模开放网络环境至关重要。
2. **密钥管理复杂度大幅降低 ($O(n)$ vs $O(n^2)$)**：在 N 个用户的系统中——
 - 对称密码：需 $N(N-1)/2$ 个密钥对
 - 公钥密码：仅需 N 个密钥对（每个用户一对）

密钥数量的线性增长使系统具有良好的可扩展性。

3. **天然支持数字签名（不可否认性）**：用私钥“加密”用公钥“解密”的模式天然实现了数字签名功能。只有私钥持有者能生成有效签名，任何人都能用公钥验证。这是对称密码体制无法原生实现的能力。
4. **支持密钥协商**：公钥密码技术支持在不安全的信道上协商共享密钥（如 Diffie-Hellman 密钥交换协议），可与对称密码配合使用形成混合密码体制。
5. **支持身份基密码和属性基密码等高级应用**：公钥密码的思想催生了身份基加密（IBE）、属性基加密（ABE）、同态加密（HE）等现代密码学前沿方向。

6. 无需在线可信第三方：对称密码体制中的密钥分发中心 (KDC) 须在线服务，而公钥密码体系中的 CA 只需签发证书，无需参与每次通信。

三、公钥密码的局限 (需客观认识)

- 运算速度远低于对称密码 (通常慢 $10^3 \sim 10^5$ 倍)，不适合大量数据直接加密
- 密钥长度更长 (同等安全级别下)
- 实际系统中通常采用混合加密：用公钥密码交换会话密钥，用对称密码加密数据

2.7 Schnorr 数字签名的可证明安全性 (10 分)

题目

利用可证明安全理论，证明如果有限域上的离散对数问题是安全的，那么 Schnorr 数字签名方案是 EU-CMA 安全的。

知识点梳理

1. Schnorr 数字签名方案

Schnorr 签名 (1989 年) 以简洁高效著称，其安全性基于离散对数问题。

系统参数：

- 大素数 p, q ，满足 $q \mid (p-1)$
- $g \in \mathbb{Z}_p^*$ ， $\text{ord}(g) = q$
- 哈希函数 $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$

密钥生成：

- 私钥：随机选择 $x \in_R \mathbb{Z}_q^*$
- 公钥： $y = g^x \bmod p$

签名 (对消息 m)：

1. 随机选择 $k \in_R \mathbb{Z}_q^*$
2. 计算承诺： $r = g^k \bmod p$
3. 计算挑战： $e = H(r \| m)$
4. 计算响应： $s = k - xe \bmod q$
5. 签名为： $\sigma = (e, s)$

验证: 计算 $r' = g^s y^e \bmod p$, 验证 $H(r' \| m) = e$ 。

2. EU-CMA (Existential Unforgeability under Chosen-Message Attack)

选择消息攻击下的存在性不可伪造: 即使攻击者可获得任意选择消息的签名, 也无法对新消息产生有效签名。

3. 分叉引理 (Forking Lemma, Pointcheval-Stern, 1996)

分叉引理是可证明安全中处理基于随机预言机模型的签名方案的核心工具。其核心思想是: 重放攻击者 (rewind the adversary), 在分叉点给出不同的随机预言机回答, 获得两个相关的签名, 从而提取离散对数的解。

详细解答

目标: 证明 $DL\text{-hard} \implies \text{Schnorr-EU-CMA}$

即: 若存在一个能攻破 Schnorr 签名 EU-CMA 安全性的攻击者 $\mathcal{A}$, 则可以构造一个求解离散对数问题 (DLP) 的算法 $\mathcal{B}$ 。

安全模型: 随机预言机模型 (ROM)

将哈希函数 H 建模为随机预言机 (Random Oracle): 攻击者可以向随机预言机查询并获得均匀随机输出, $\mathcal{B}$ (模拟器) 可编程该预言机的输出。

规约证明

Step 1: 问题设定

给定 DLP 实例: $(g, y = g^x)$, $\mathcal{B}$ 的目标是求解 x 。

$\mathcal{B}$ 将 (g, y) 作为 Schnorr 签名方案的公钥提供给攻击者 $\mathcal{A}$, 并以随机预言机方式模拟 $\mathcal{A}$ 执行环境中的哈希函数 H 。

Step 2: $\mathcal{B}$ 模拟签名预言机

当 $\mathcal{A}$ 请求对消息 m_i 的签名时, $\mathcal{B}$ 不持有私钥 x , 但利用随机预言机的可编程性生成签名:

1. 随机选择 $e_i, s_i \in_R \mathbb{Z}_q$
2. 计算 $r_i = g^{s_i} \cdot y^{e_i} \bmod p$
3. 编程设定 $H(r_i \| m_i) = e_i$ (若该位置已被查询过, 则中止并返回失败)
4. 返回签名 $\sigma_i = (e_i, s_i)$

由于签名验证等式 $g^{s_i} y^{e_i} = r_i$ 成立, 且 $H(r_i \| m_i) = e_i$, 该签名对 $\mathcal{A}$ 来说与真实签名不可区分。

Step 3: $\mathcal{B}$ 模拟随机预言机

当 $\mathcal{A}$ 查询 $H(r \| m)$ 时, $\mathcal{B}$ 维护一个列表 L_H (初始为空):

- 若 (r, m) 已在列表中, 返回对应的 e

- 否则随机生成 $e \in_R \mathbb{Z}_q$, 记录 (r, m, e) 并返回 e

Step 4: $\mathcal{A}$ 产生伪造签名

假设 $\mathcal{A}$ 经过最多 q_H 次随机预言机查询和 q_S 次签名查询后, 以概率 ε 输出对消息 m^* (此前未请求过签名) 的有效伪造签名 (e^*, s^*) 。

由于 (e^*, s^*) 是有效签名, 我们有:

$$g^{s^*} \cdot y^{e^*} \equiv r^* \pmod{p}$$

其中 $e^* = H(r^* \| m^*)$ 。

$\mathcal{B}$ 检查 $\mathcal{A}$ 之前是否查询过 $H(r^* \| m^*)$ ——若未查询过, 由于 H 是随机预言机, e^* 匹配的概率仅为 $1/q$ (可忽略)。因此以压倒性概率, $(r^* \| m^*, e^*)$ 已在 L_H 中。

Step 5: 分叉——重放攻击者

$\mathcal{B}$ 重置 $\mathcal{A}$ 到查询 $H(r^* \| m^*)$ 之前的状态 (保存该时刻的随机带), 然后以不同的随机预言机输出继续执行。

$\mathcal{B}$ 在分叉点对 $H(r^* \| m^*)$ 给出不同的值 $e' \neq e^*$, 其余所有随机预言机查询保持一致的输出。

$\mathcal{A}$ 在分叉后的执行中 (以不可忽略的概率) 再次在 (r^*, m^*) 上输出伪造签名 (e', s') (针对新的挑战值 e')。

Step 6: 提取离散对数

现在 $\mathcal{B}$ 拥有两个有效签名:

$$\begin{cases} g^{s^*} \cdot y^{e^*} \equiv r^* \pmod{p} \\ g^{s'} \cdot y^{e'} \equiv r^* \pmod{p} \end{cases}$$

两式相除:

$$g^{s^*-s'} \cdot y^{e^*-e'} \equiv 1 \pmod{p}$$

$$y^{e^*-e'} \equiv g^{s'-s^*} \pmod{p}$$

代入 $y = g^x$:

$$g^{x(e^*-e')} \equiv g^{s'-s^*} \pmod{p}$$

由于 $\text{ord}(g) = q$, 有:

$$x(e^* - e') \equiv s' - s^* \pmod{q}$$

因为 $e^* \neq e'$, 所以 $e^* - e'$ 在 $\mathbb{Z}_q$ 中有乘法逆元, 故:

$$x \equiv (s' - s^*)(e^* - e')^{-1} \pmod{q}$$

$\mathcal{B}$ 成功求解出离散对数 x !

Step 7: 成功概率分析

根据分叉引理 (Forking Lemma): 若 $\mathcal{A}$ 以概率 ε 在时间 t 内输出有效伪造签名, 则 $\mathcal{B}$ 在时间 $t' \approx 2t$ 内以概率:

$$\varepsilon' \geq \frac{\varepsilon^2}{q_H} - \frac{1}{q}$$

成功求解 DLP。

若 ε 不可忽略且 ε^2/q_H 不可忽略, 则 $\mathcal{B}$ 以不可忽略概率成功。这与“离散对数问题是困难的”假设矛盾。

结论:

在随机预言机模型下, 若离散对数问题在 $\mathbb{Z}_p^*$ 的子群上是困难的, 则 Schnorr 数字签名方案满足 EU-CMA 安全性。

知识点延伸: 可证明安全的意义与局限

可证明安全 (Provable Security) 的核心方法是**规约**: 将破解密码方案的困难性规约到公认数学难题的困难性。

三个层次的安全论证 (从弱到强):

1. **启发式安全**: 攻击者尝试了一些已知攻击方法, 没有成功。不可靠。
2. **可证明安全**: 在某安全模型下, 攻破方案可规约到解决某数学难题。有理论保证但依赖模型假设。
3. **信息论安全**: 即使攻击者有无穷计算能力也无法攻破 (如一次一密)。最安全但效率最低。

可证明安全的局限性: 依赖随机预言机模型 (ROM), 而在标准模型下, ROM 并非真实哈希函数的完美抽象。已存在在 ROM 下可证明安全但在现实中不安全的 (人为构造的) 反例方案。但对 Schnorr 签名等实用方案, ROM 证明仍提供了强有力的安全证据。

2.8 支付系统安全分析与密码学方案设计 (30 分)

题目

请针对现有某种支付方式进行安全性分析, 包括: 潜在的威胁分析、支付流程分析等, 重点对支付过程中的协议进行适当设计, 以及密码学理论的运用。

知识点梳理

本题为开放性设计题, 考察综合运用密码学知识分析和设计实际系统的能力。涉及的密码学原语包括但不限于:

- 对称加密 (AES): 保护交易数据的机密性和高效处理

- 公钥密码 (RSA/ECC): 身份认证、密钥协商、数字签名
- 哈希函数 (SHA-256/SHA-3): 数据完整性校验、交易摘要
- 数字签名 (ECDSA/Schnorr): 交易不可否认性、授权证明
- 消息认证码 (HMAC): 消息完整性和认证
- 密钥协商 (ECDH): 在公开信道上安全协商会话密钥
- PKI/数字证书: 身份验证和信任链
- 随机数生成: 防止重放攻击的 Nonce/时间戳

详细解答

以下以**移动支付（手机扫码支付）**为典型场景，进行全面的安全性分析与协议设计。

第一部分：支付流程分析

典型扫码支付流程包含三方：

- **用户 (Payer)**: 发起支付、持有私钥或支付密码
- **商户 (Payee)**: 生成收款订单、展示二维码
- **支付平台 (Payment Platform, PP)**: 可信第三方，管理账户、清算交易

基本流程：

1. 商户生成订单（金额、商品描述、商户 ID），发送至支付平台
2. 支付平台返回交易令牌（Token）/订单编号
3. 商户将订单信息编码为二维码（含 Token）
4. 用户扫描二维码，解析订单信息，确认支付
5. 用户端将支付授权信息（加密和签名后）发送至支付平台
6. 支付平台验证授权，执行扣款，通知双方交易结果

第二部分：潜在威胁分析

威胁类型	攻击位置	攻击方式	潜在危害
窃听攻击	网络传输层	在公开 WiFi 中嗅探通信数据包	交易信息泄露、session 劫持
中间人攻击	用户与平台间	拦截并篡改支付请求中的金额/收款方	资金被转移到攻击者账户
重放攻击	通信信道	截获合法支付请求，重复发送	重复扣款
二维码替换	商户端	将商户的收款二维码替换为攻击者的	用户付款给攻击者
伪造授权	用户端	未授权用户伪造签名/支付指令	非法扣款
订单篡改	客户端/传输中	修改订单金额或商品信息	以低于实际金额完成支付
钓鱼攻击	用户交互层	伪造支付 App 或界面	窃取用户支付密码/私钥
内鬼攻击	支付平台内部	内部人员滥用数据库访问权限	批量泄露用户账户和交易信息

第三部分：密码学安全协议设计

1. 系统初始化和密钥体系

- 支付平台 PP：拥有公私钥对 (PK_{PP}, SK_{PP}) ，使用 ECDSA（secp256r1 曲线）
- 用户：在注册时生成公私钥对 (PK_U, SK_U) 。私钥存储在设备安全区域（TEE/SE）。 PK_U 由 PP 签发证书 $Cert_U = \text{Sign}_{SK_{PP}}(ID_U || PK_U || \text{expiry})$
- 商户：拥有 (PK_M, SK_M) ，同理持有 PP 签发的证书 $Cert_M$
- 所有参与方预先拥有 PP 的根公钥 PK_{PP} （用于验证证书）

2. 安全支付协议详细设计

阶段 1：商户生成订单

商户生成订单：

$$\text{Order} = \{\text{OrderID}, \text{MerchantID}, \text{Amount}, \text{Description}, \text{Timestamp}, \text{Nonce}_M\}$$

商户对订单签名:

$$\sigma_M = \text{Sign}_{SK_M}(H(\text{Order}))$$

将订单和签名发送至支付平台, 获得交易令牌 Token:

$$\text{Token} = \text{Sign}_{SK_{PP}}(\text{Order} \parallel \sigma_M \parallel \text{Expiry})$$

商户生成二维码: $\text{QR} = \{\text{Token}, \text{Order}, \sigma_M, \text{Cert}_M\}$

阶段 2: 用户扫码支付

用户 App 扫描二维码, 解析订单信息, 验证:

1. 用 PK_{PP} 验证 Token 签名 $\rightarrow$ 确认订单由 PP 背书
2. 用 Cert_M 中的 PK_M 验证 $\sigma_M \rightarrow$ 确认订单来自合法商户
3. 检查 Amount 和 Description $\rightarrow$ 展示给用户确认
4. 检查 Timestamp 和 Expiry $\rightarrow$ 防止过期订单重放

用户确认后, 生成支付授权:

$$\text{Authorization} = \{\text{OrderID}, \text{UserID}, \text{Amount}, \text{Timestamp}_U, \text{Nonce}_U\}$$

对授权签名:

$$\sigma_U = \text{Sign}_{SK_U}(H(\text{Authorization}))$$

构造支付请求报文:

$$\text{PaymentReq} = \text{Authorization} \parallel \sigma_U \parallel \text{Cert}_U \parallel \text{Token}$$

使用 PP 的公钥 PK_{PP} 进行混合加密 (ECIES):

- 用 ECDH 生成临时会话密钥: 选随机 r , 计算 $K_{\text{sess}} = \text{KDF}(r \cdot PK_{PP})$
- 用 AES-256-GCM 加密 PaymentReq: $\text{EncReq} = E_{\text{AES-GCM}}(K_{\text{sess}}, \text{PaymentReq})$
- 发送: $(r \cdot G, \text{EncReq})$ (附临时公钥供 PP 解密)

阶段 3: 支付平台处理

PP 收到后:

1. 用自己的 SK_{PP} 做 ECDH 恢复 $K_{\text{sess}} = \text{KDF}(SK_{PP} \cdot (r \cdot G))$
2. 用 AES-GCM 解密验证 EncReq, 确认完整性和真实性
3. 验证 Token 签名 $\rightarrow$ 确认订单真实性

4. 验证 σ_U 和 $\text{Cert}_U \rightarrow$ 确认用户授权
5. 验证 Nonce_U 未重复使用 $\rightarrow$ 防重放
6. 比对 Authorization 中的 Amount 与 Order 中的 Amount $\rightarrow$ 防篡改
7. 执行账户扣款
8. 生成交易凭证:

$$\text{Receipt} = \text{Sign}_{SK_{PP}}(\text{OrderID} \parallel \text{Result} \parallel \text{Timestamp})$$

9. 向用户和商户推送加密的交易结果通知

第四部分：安全性分析总结

威胁	防御措施	密码学原理
窃听攻击	传输层加密 (AES-GCM)	混合加密: ECDH + AES, 会话密钥一次性使用
中间人攻击	端到端签名 + 证书链	ECDSA 签名 + PKI 证书验证 + Certificate Pinning
重放攻击	Nonce + 时间戳	每次交易独立随机数 Nonce, 过期订单无效
二维码替换	订单签名 + Token	商户签名 σ_M + PP Token 双重背书
伪造授权	数字签名	用户私钥签名 σ_U , 私钥存于 TEE 安全区域
订单篡改	签名 + 哈希	$H(\text{Order})$ 的签名使任何篡改都会被检测
内鬼攻击	端到端加密	用户 $\rightarrow$ PP 的 ECIES 加密, 平台也不得解密交易明文以外数据

关键密码学设计要点:

1. **纵深防御 (Defense in Depth):** 不依赖单一密码学机制, 而是组合使用签名、加密、证书、Nonce 等多种手段, 构成多层安全防护。
2. **端到端安全:** 用户和支付平台之间的加密 (ECIES) 确保中间节点 (包括商户的 Wi-Fi、运营商网络) 均无法读取支付授权内容。

3. **前向安全性**: 每次通信使用 ECDH 生成的临时会话密钥, 即使长期私钥泄露, 历史通信记录也无法解密。
4. **不可否认性**: 每一方的操作均有数字签名 (商户生成订单、用户授权、平台确认), 事后无法抵赖, 满足法律和审计要求。
5. **最小信任原则**: 商户无需获取用户私钥或支付密码; 平台内部也实施加密隔离。

2.9 附加题: Lifted ElGamal 的安全性分析 (5 分, 选做)

题目

假设 T 是一个可信第三方, 公布其 Lifted ElGamal 公钥 pk_T 。A 选择 $d_A \in \{0, 1\}$, 计算 $C_A = E(pk_T, d_A)$, 将 C_A 发送给 T ; B 选择 $d_B \in \{0, 1\}$, 计算 $C_B = E(pk_T, d_B)$, 将 C_B 发送给 T 。T 解密获得 d_A, d_B , 输出 $d_A \oplus d_B$ 。

问 B 在已知 C_A 的情况下, 如何通过发送的 C_B 控制 T 的输出 $d_A \oplus d_B$?

补充说明: Lifted ElGamal 密钥结构为: $pk = g^x$ 为公钥, x 为对应私钥; 加密结构为 $(C_1, C_2) = (g^r, pk^r \cdot g^m)$ (m 较小), 其中 r 为随机数; 解密结构为 $M = C_2 / (C_1)^x$, 求解离散对数 $m = \log_g M$ 。

知识点梳理

1. Lifted ElGamal 加密方案

与标准 ElGamal 不同, Lifted ElGamal 将明文编码为群元素 g^m (而非直接用 m 乘以 pk^r), 因此解密后需要求解离散对数恢复 m 。由于 m 很小 (此处 $m \in \{0, 1\}$), 离散对数求解是可行的 (查表: $g^0 = 1, g^1 = g$)。

2. ElGamal 的同态性质

ElGamal 加密具有**乘法同态性**: $E(m_1) \cdot E(m_2) = E(m_1 + m_2)$ (在指数上相加)。

具体地, 若 $C_A = (g^{r_A}, pk^{r_A} \cdot g^{d_A})$, $C_B = (g^{r_B}, pk^{r_B} \cdot g^{d_B})$, 则分量相乘:

$$(g^{r_A+r_B}, pk^{r_A+r_B} \cdot g^{d_A+d_B})$$

这是 $d_A + d_B$ 的有效加密 (随机数为 $r_A + r_B$)。

3. 重随机化 (Rerandomization)

给定密文 $C = (C_1, C_2)$, 任何人可在不知明文的情况下”重随机化”: 选随机数 s , 计算 $C' = (C_1 \cdot g^s, C_2 \cdot pk^s)$ 。C' 与 C 加密相同的明文, 但具有全新的随机性。

4. 关键观察

由于 Lifted ElGamal 加密的是 g^m ($m \in \{0, 1\}$), B 可以利用同态性质对 C_A 进行变换, 从而控制输出的 $d_A \oplus d_B$ 。

详细解答

核心攻击思路：

B 已知 $C_A = (C_{A,1}, C_{A,2}) = (g^{r_A}, g^{x \cdot r_A} \cdot g^{d_A})$ ，但不知道 d_A 。 B 可以通过三种方式构造 C_B 来控制输出。

攻击方式一：重放 C_A (强制输出 0)

B 直接令 $C_B = C_A$ (或重随机化后的 C_A)，即加密相同的 d_A ：

$$C_B = C_A = (g^{r_A}, g^{x \cdot r_A} \cdot g^{d_A})$$

此时 $d_B = d_A$ ，则 T 输出：

$$d_A \oplus d_B = d_A \oplus d_A = 0$$

结果： B 可强制输出始终为 0，无论 A 选择的是 0 还是 1。此攻击完全破坏了协议的公平性。

攻击方式二：通过同态性质翻转 (强制输出 1)

B 利用 Lifted ElGamal 的同态性质，构造加密 $(1 - d_A)$ 的密文：

取 $C_B = (C_{A,1} \cdot g^s, C_{A,2} \cdot pk^s \cdot g)$ (其中 s 为 B 选择的随机数)。

等价于：先重随机化 C_A ，再乘以 g (即“加 1”的密文)。

$$\begin{aligned} C_B &= \text{Rerand}(C_A) \cdot (1, g) \\ &= (g^{r_A+s}, pk^{r_A+s} \cdot g^{d_A} \cdot g) \\ &= (g^{r_A+s}, pk^{r_A+s} \cdot g^{d_A+1}) \end{aligned}$$

解密后： $M_B = g^{d_A+1}$ 。

- 若 $d_A = 0$ ， $M_B = g^1 = g$ ，求解 DL 得 $d_B = 1$
- 若 $d_A = 1$ ， $M_B = g^2$ ，求解 DL 得 $d_B = 2$ (超出 $\{0, 1\}$ ，但 T 仍可能将其作为有效结果)

当 $d_A = 0$ 时， $d_B = 1$ ，输出 $0 \oplus 1 = 1$ ；当 $d_A = 1$ 时，结果可能异常 (取决于 T 如何处理超出范围的 d_B 值)。

攻击方式三：直接自由选择 d_B

最直接的控制方式—— B 只需正常加密自己选择的值 d_B ：

- 若 B 选择 $d_B = 0$ ，则输出 $d_A \oplus 0 = d_A$ (B 学习到 d_A 的信息)
- 若 B 选择 $d_B = 1$ ，则输出 $d_A \oplus 1 = 1 - d_A$ (B 学习到 d_A 的补)

虽然 B 不能直接看到 d_A ，但通过观察 T 的输出结果， B 可以推导出 d_A 的值。例如： B 先发送 $C_B = 0$ 的加密 ($d_B = 0$)，观察输出 O_1 ；再重新运行协议，发送 $C_B = 1$ 的加密 ($d_B = 1$)，观察输出 O_2 。

通过 O_1 和 O_2 的关系可唯一确定 d_A 。

总结：

1. **完全控制（强制输出 0）：** B 设置 $C_B = C_A$ （或重随机化后发送）， T 输出 0。
2. **学习 d_A ：** B 正常加密 $d_B = 0$ ，观察输出即为 d_A 。
3. **根本原因：** 协议未对 C_B 与 C_A 的关系做任何约束，且 Lifted ElGamal 的同态性质使 B 可以在不知道 d_A 的情况下操纵与 d_A 相关联的加密值。

修复建议：

- 要求 B 在发送 C_B 时附带零知识证明 (ZKP)，证明 C_B 确实加密了 $\{0, 1\}$ 中的值，且证明 C_B 是独立生成的（而非从 C_A 派生）。
- 或者让 A, B 先分别提交承诺（承诺方案），再依次揭示，防止后发送方利用先发送方的信息。

试卷二知识点分布总结

大题	小题	分值	类型	核心知识点
一、算法分析	1	10	计算	LFSR、本原多项式、 m 序列、状态遍历
	2	10	分析	DES S 盒、非线性混淆、雪崩效应、差分分析
	3	10	计算	RSA 加解密、模幂运算、密钥分解破译
二、简答题	1	10	论述	KPA vs CPA、公钥密码攻击模型、IND-CPA
	2	10	图解	CBC 工作模式、加密/解密结构、错误传播
	3	10	论述	公钥密码基本思想、单向陷门函数、vs 对称密码
	4	10	证明	Schnorr 签名、EU-CMA、分叉引理、规约证明
三、方案设计	1	30	设计	支付安全、密码协议设计、威胁建模、混合加密
四、附加题	1	5	分析	Lifted ElGamal、同态加密、协议公平性攻击

3 试卷三：计算机安全与保密 (2017-2018)

试卷说明

本试卷卷面共十一题（计算题 5 题、名词解释 4 题、简述题 2 题），满分 100 分。

试卷内容涵盖：仿射密码解密、AES 列混合 $GF(2^8)$ 运算、RSA 因数分解、椭圆曲线点倍乘、ElGamal 签名同余攻击、Hash 函数/公钥密码/解密/数字签名概念、DES 加密过程、密码协议设计原则。

3.1 仿射密码解密 (10 分)

题目

使用仿射密码算法对信息 "hlzb l slqfb tponybotalojb" 进行解密, 已知 $k_0 = 11$, $k_1 = 17$ 。

知识点梳理

1. 仿射密码 (Affine Cipher)

仿射密码是一种单表代替密码, 其数学定义为:

- 设明文字母表含 n 个字母 (英文中 $n = 26$), 字母 $A, B, C, \dots, Z$ 分别对应数字 $0, 1, 2, \dots, 25$ 。
- 加密函数: $E(x) = (k_1x + k_0) \bmod n$, 其中 k_1 必须满足 $\gcd(k_1, n) = 1$ 。
- 解密函数: $D(y) = k_1^{-1}(y - k_0) \bmod n$, 其中 k_1^{-1} 为 k_1 模 n 的乘法逆元。

2. 乘法逆元的计算

$k_1^{-1} \bmod n$ 即满足 $k_1 \cdot k_1^{-1} \equiv 1 \pmod{n}$ 的数。可以使用扩展欧几里得算法 (Extended Euclidean Algorithm) 求解。当 $n = 26$ 时, 与 26 互素的 k_1 可取值为 $\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$, 共 12 个。

3. 密钥空间

仿射密码的密钥由 (k_0, k_1) 构成:

- k_0 可取 $0 \sim 25$ 中任意值 (共 26 种)
- k_1 必须与 26 互素 (共 12 种)
- 总密钥空间为 $26 \times 12 = 312$, 并不大, 可被穷举攻击快速破解

详细解答

Step 1: 推导解密公式

已知加密参数 $k_0 = 11$, $k_1 = 17$ 。首先求 $k_1 = 17$ 模 26 的乘法逆元 k_1^{-1} 。
使用扩展欧几里得算法求解 $17x \equiv 1 \pmod{26}$:

$$26 = 1 \times 17 + 9$$

$$17 = 1 \times 9 + 8$$

$$9 = 1 \times 8 + 1$$

回溯:

$$\begin{aligned}
 1 &= 9 - 8 \\
 8 &= 17 - 9 \\
 \Rightarrow 1 &= 9 - (17 - 9) = 2 \times 9 - 17 \\
 9 &= 26 - 17 \\
 \Rightarrow 1 &= 2 \times (26 - 17) - 17 = 2 \times 26 - 3 \times 17
 \end{aligned}$$

在模 26 下: $-3 \times 17 \equiv 1 \pmod{26}$, 即 $23 \times 17 \equiv 1 \pmod{26}$ (因为 $-3 \equiv 23 \pmod{26}$)。

验证: $23 \times 17 = 391 = 15 \times 26 + 1 \equiv 1 \pmod{26}$ 。正确!

因此 $k_1^{-1} = 23 \equiv -3 \pmod{26}$ 。

解密公式为:

$$D(y) = k_1^{-1}(y - k_0) \pmod{26} = 23 \times (y - 11) \pmod{26}$$

由于 $23 \equiv -3 \pmod{26}$, 也可以写成:

$$D(y) = (-3)(y - 11) \pmod{26} = (-3y + 33) \pmod{26} = (-3y + 7) \pmod{26} = (7 - 3y) \pmod{26}$$

(因为 $33 \equiv 7 \pmod{26}$ 。若 $7 - 3y$ 为负, 则加 26 的倍数。)

Step 2: 将密文转换为数字

密文: hlzb l slqfb tponywbotalojb

去掉空格后共有 23 个字母: hlzblslqfbtpnywbotalojb

对应数字 (A=0, B=1, ..., Z=25):

字母	h	l	z	b	l	s	l	q	f	b	t	p	n	y	w	b	o	t	a	l	o	j	b
数字	7	11	25	1	11	18	11	16	5	1	19	15	13	24	22	1	14	19	0	11	14	9	1

Step 3: 逐位解密计算

使用 $D(y) = (7 - 3y) \pmod{26}$ (计算结果为负时加 26):

位置	密文	密文值 y	计算	明文值 x
1	h	7	$7 - 3 \times 7 = 7 - 21 = -14 \equiv 12$	$12 \rightarrow m$
2	l	11	$7 - 3 \times 11 = 7 - 33 = -26 \equiv 0$	$0 \rightarrow a$
3	z	25	$7 - 3 \times 25 = 7 - 75 = -68 \equiv 10$	$10 \rightarrow k$
4	b	1	$7 - 3 \times 1 = 7 - 3 = 4$	$4 \rightarrow e$
5	l	11	$7 - 33 = -26 \equiv 0$	$0 \rightarrow a$
6	s	18	$7 - 3 \times 18 = 7 - 54 = -47 \equiv 5$	$5 \rightarrow f$
7	l	11	$7 - 33 = -26 \equiv 0$	$0 \rightarrow a$
8	q	16	$7 - 3 \times 16 = 7 - 48 = -41 \equiv 11$	$11 \rightarrow l$
9	f	5	$7 - 3 \times 5 = 7 - 15 = -8 \equiv 18$	$18 \rightarrow s$
10	b	1	$7 - 3 = 4$	$4 \rightarrow e$
11	t	19	$7 - 3 \times 19 = 7 - 57 = -50 \equiv 2$	$2 \rightarrow c$
12	p	15	$7 - 3 \times 15 = 7 - 45 = -38 \equiv 14$	$14 \rightarrow o$
13	n	13	$7 - 3 \times 13 = 7 - 39 = -32 \equiv 20$	$20 \rightarrow u$
14	y	24	$7 - 3 \times 24 = 7 - 72 = -65 \equiv 13$	$13 \rightarrow n$
15	w	22	$7 - 3 \times 22 = 7 - 66 = -59 \equiv 19$	$19 \rightarrow t$
16	b	1	$7 - 3 = 4$	$4 \rightarrow e$
17	o	14	$7 - 3 \times 14 = 7 - 42 = -35 \equiv 17$	$17 \rightarrow r$
18	t	19	$7 - 57 = -50 \equiv 2$	$2 \rightarrow c$
19	a	0	$7 - 0 = 7$	$7 \rightarrow h$
20	l	11	$7 - 33 = -26 \equiv 0$	$0 \rightarrow a$
21	o	14	$7 - 42 = -35 \equiv 17$	$17 \rightarrow r$
22	j	9	$7 - 3 \times 9 = 7 - 27 = -20 \equiv 6$	$6 \rightarrow g$
23	b	1	$7 - 3 = 4$	$4 \rightarrow e$

Step 4: 还原明文

得到的明文数字序列为: 12, 0, 10, 4, 0, 5, 0, 11, 18, 4, 2, 14, 20, 13, 19, 4, 17, 2, 7, 0, 17, 6, 4

还原为英文明文: m a k e a f a l s e c o u n t e r c h a r g e

恢复空格后为: "make a false countercharge"

明文: make a false countercharge

知识点延伸: 仿射密码的安全性

仿射密码的密钥空间仅 $12 \times 26 = 312$, 对于现代计算能力而言微不足道。更致命的是, 其本质仍是单表代替密码, 保留了明文的统计特征 (如字母频率)。通过频度分析 (如利用英语中 E 出现频率最高约 12.7%, T 约 9.1% 等) 即可快速攻破。

本题中, 若验证加密函数: $E(12) = (17 \times 12 + 11) \bmod 26 = (204 + 11) \bmod 26 = 215 \bmod 26 = 7 \rightarrow h$ 。 $E(0) = (17 \times 0 + 11) \bmod 26 = 11 \rightarrow l$ 。结果与密文一致, 加解密正确。

3.2 AES 列混合有限域运算 (10 分)

题目

AES 的列混合变换是把矩阵 C 作用到中间状态块上, 选取模 $m(x) = x^8 + x^4 + x^3 + x + 1$ 构造的有限域, 在该域上试计算出 $02 \cdot d4 + 03 \cdot bf + 01 \cdot 5d + 01 \cdot 30$ 的值。

知识点梳理

1. AES 中的有限域 $GF(2^8)$

AES 的所有字节运算都在有限域 $GF(2^8)$ 上进行, 使用的不可约多项式为:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

其十六进制表示为 $0x11B$ (即二进制 100011011)。

2. 域元素的表示

每个字节 $b_7b_6b_5b_4b_3b_2b_1b_0$ 对应于多项式:

$$b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0$$

加法为多项式系数按位异或 (XOR)。

3. xtime 运算 (乘 2 运算)

$\{02\} \cdot \{a\}$ 的计算方法:

- 将 $\{a\}$ 左移 1 位 (即多项式乘以 x)。
- 若最高位 $b_7 = 1$ (即 $a \geq 0x80$), 则将左移结果与 $0x1B$ 异或 (相当于模 $m(x)$ 的约化)。
- 若最高位 $b_7 = 0$, 则左移结果即为答案。

4. 矩阵乘法系数

AES MixColumns 中的矩阵系数为 $\{01\}, \{02\}, \{03\}$:

- $\{01\} \cdot a = a$ (单位元)
- $\{02\} \cdot a = \text{xtime}(a)$
- $\{03\} \cdot a = \{02\} \cdot a \oplus \{01\} \cdot a = \text{xtime}(a) \oplus a$

详细解答

Step 1: 将各字节转换为二进制表示

$$d4_{16} = 1101\ 0100_2$$

$$bf_{16} = 1011\ 1111_2$$

$$5d_{16} = 0101\ 1101_2$$

$$30_{16} = 0011\ 0000_2$$

Step 2: 计算 $\{02\} \cdot d4$

$d4 = 1101\ 0100_2$, 二进制最高位为 1 ($d4 > 0x80$), 因此需要约化。

左移 1 位: $1101\ 0100_2 \rightarrow 1\ 1010\ 1000_2$ (将超出 8 位的进位单独取出)。

左移结果 (取低 8 位) 为 $1010\ 1000_2 = A8_{16}$, 由于原最高位为 1, 需异或 $0x1B = 0001\ 1011_2$:

$$\begin{aligned} 1010\ 1000_2 \oplus 0001\ 1011_2 \\ = 1011\ 0011_2 = B3_{16} \end{aligned}$$

$$\boxed{\{02\} \cdot d4 = B3_{16} = 1011\ 0011_2}$$

Step 3: 计算 $\{03\} \cdot bf$

$$\{03\} \cdot bf = \{02\} \cdot bf \oplus bf。$$

首先计算 $\{02\} \cdot bf$: $bf = 1011\ 1111_2$, 最高位为 1, 需要约化。

左移 1 位取低 8 位: $0111\ 1110_2 = 7E_{16}$ 。

异或 $0x1B$:

$$\begin{aligned} 0111\ 1110_2 \oplus 0001\ 1011_2 \\ = 0110\ 0101_2 = 65_{16} \end{aligned}$$

因此 $\{02\} \cdot bf = 65_{16}$ 。

再与 bf 异或 (即加 bf):

$$\begin{aligned} \{03\} \cdot bf &= \{02\} \cdot bf \oplus bf \\ &= 0110\ 0101_2 \oplus 1011\ 1111_2 \\ &= 1101\ 1010_2 = DA_{16} \end{aligned}$$

$$\boxed{\{03\} \cdot bf = DA_{16} = 1101\ 1010_2}$$

Step 4: 计算 $\{01\} \cdot 5d$ 和 $\{01\} \cdot 30$

$$\{01\} \cdot 5d = 5d_{16} = 0101\ 1101_2。$$

$$\{01\} \cdot 30 = 30_{16} = 0011\ 0000_2。$$

Step 5: 将四项异或相加

$$\{02\} \cdot d4 : 1011\ 0011_2$$

$$\{03\} \cdot bf : 1101\ 1010_2$$

$$\{01\} \cdot 5d : 0101\ 1101_2$$

$$\{01\} \cdot 30 : 0011\ 0000_2$$

$$\text{XOR 和} : 0000\ 0100_2 = 04_{16}$$

逐位列竖式 XOR 验证:

- bit7: $1 \oplus 1 \oplus 0 \oplus 0 = 0$
- bit6: $0 \oplus 1 \oplus 1 \oplus 0 = 0$
- bit5: $1 \oplus 0 \oplus 0 \oplus 1 = 0$
- bit4: $1 \oplus 1 \oplus 1 \oplus 1 = 0$
- bit3: $0 \oplus 1 \oplus 0 \oplus 0 = 1$ (wait)
- bit2: $0 \oplus 0 \oplus 1 \oplus 0 = 1$ (wait)

让我重新逐一核对:

$$\text{bit7 (128): } 1(B3) \oplus 1(DA) \oplus 0(5D) \oplus 0(30) = 0$$

$$\text{bit6 (64): } 0(B3) \oplus 1(DA) \oplus 1(5D) \oplus 0(30) = 0$$

$$\text{bit5 (32): } 1(B3) \oplus 0(DA) \oplus 0(5D) \oplus 1(30) = 0$$

$$\text{bit4 (16): } 1(B3) \oplus 1(DA) \oplus 1(5D) \oplus 1(30) = 0$$

$$\text{bit3 (8): } 0(B3) \oplus 1(DA) \oplus 1(5D) \oplus 0(30) = 0$$

$$\text{bit2 (4): } 0(B3) \oplus 0(DA) \oplus 1(5D) \oplus 0(30) = 1$$

$$\text{bit1 (2): } 1(B3) \oplus 1(DA) \oplus 0(5D) \oplus 0(30) = 0$$

$$\text{bit0 (1): } 1(B3) \oplus 0(DA) \oplus 1(5D) \oplus 0(30) = 0$$

即 $0000\ 0100_2 = 04_{16}$ 。

$$02 \cdot d4 + 03 \cdot bf + 01 \cdot 5d + 01 \cdot 30 = 04$$

知识点延伸: MixColumns 与 MDS 矩阵

AES MixColumns 使用的矩阵:

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

该矩阵是 $GF(2^8)$ 上的 MDS (Maximum Distance Separable) 矩阵, 其分支数 (Branch Number) 达到理论最大值 5。这意味着输入列和输出列中非零字节的总数至少为 5, 极大增强了扩散效果。

在 InvMixColumns (解密时的逆列混合) 中, 矩阵变为:

$$M^{-1} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix}$$

系数更大 (如 0E, 0B, 0D, 09), 计算也更为复杂。

3.3 RSA 因数分解 (10 分)

题目

在 RSA 系统中, $n = pq$, p 和 q 都是素数, 试利用因数分解方法对整数 $n = 116843$ 进行因数分解。

知识点梳理

1. RSA 的安全性基础

RSA 的安全性基于大整数分解的数学困难问题。给定 $n = pq$ (p, q 为大素数), 从 n 中有效恢复 p 和 q 被认为是困难的。这也是为什么本题需要特定的因数分解方法。

2. Fermat 因数分解法

当 n 的两个素因子 p 和 q 大小接近时 (即 $|p - q|$ 较小), 可以使用 Fermat 因数分解法:

- 寻找整数 x, y 使得 $n = x^2 - y^2 = (x + y)(x - y)$
- 则 $p = x + y$, $q = x - y$
- $x^2 - n = y^2$, 而 x 从 $\lceil \sqrt{n} \rceil$ 开始尝试

3. 算法步骤

1. 计算 $x_0 = \lceil \sqrt{n} \rceil + 1$ 。

2. 对 $x = x_0, x_0 + 1, x_0 + 2, \dots$, 检查 $x^2 - n$ 是否是完全平方数。

3. 若 $x^2 - n = y^2$, 则 $n = (x + y)(x - y)$, 分解完成。

当两个因子接近时, Fermat 法非常高效。因 $|p - q| = 2y$ 小, 所以 x 接近 $\sqrt{n}$, 很快找到解。

详细解答

Step 1: 计算 $\sqrt{n}$ 的近似值

$$n = 116843$$

估算 $\sqrt{n}$:

$$340^2 = 115600 < 116843$$

$$341^2 = 116281 < 116843$$

$$342^2 = 116964 > 116843$$

因此 $\lceil \sqrt{n} \rceil = 342$ 。

Step 2: Fermat 分解尝试

取 $x = 342$, 计算 $x^2 - n$:

$$342^2 - 116843 = 116964 - 116843 = 121$$

检查 121 是否为完全平方数:

$$11^2 = 121 \quad \checkmark$$

一次尝试就找到了! 设 $y = 11$ 。

Step 3: 求出因子

$$\begin{aligned} n = x^2 - y^2 &= 342^2 - 11^2 = (342 + 11)(342 - 11) \\ &= 353 \times 331 \end{aligned}$$

因此 $p = 353$, $q = 331$ (二者均为素数)。

验证:

$$\begin{aligned} 353 \times 331 &= 353 \times (300 + 31) \\ &= 353 \times 300 + 353 \times 31 \\ &= 105900 + 10943 \\ &= 116843 \quad \checkmark \end{aligned}$$

检查素性：353 和 331 均为素数（可通过试除法验证，只需检查到 $\lfloor \sqrt{353} \rfloor = 18$ 和 $\lfloor \sqrt{331} \rfloor = 18$ 的素数）。

$$n = 116843 = 353 \times 331$$

知识点延伸：因数分解方法总结

大整数分解是数论和密码学中的核心问题，常用算法包括：

方法	适用场景	复杂度
试除法	因子极小 ($< 10^6$)	$O(\sqrt{n})$, 不可行于大数
Fermat 法	两因子接近	$O(p - q)$, 此时高效
Pollard $p - 1$	$p - 1$ 为光滑数	$O(B \log n)$
Pollard ρ	通用, 中小因子	$O(n^{1/4})$
二次筛法 (QS)	一般大整数	亚指数级
数域筛法 (NFS)	最大整数 (RSA 挑战)	亚指数级, 目前最优

Fermat 分解法提醒我们：在 RSA 密钥生成时， p 和 q 应随机独立选择，不应太接近，否则 Fermat 法（及其变体）可有效分解 n 。现代实践中， p 和 q 的位长应相同但值相差足够大。

3.4 椭圆曲线点倍乘运算 (10 分)

题目

假设椭圆曲线 $y^2 = x^3 + x + 6 \pmod{11}$ 上的两点 $P = (x_1, y_1)$, $Q = (x_2, y_2)$ 之和为 $P_3 = (x_3, y_3) = P + Q \neq \mathcal{O}$, 计算公式为：

$$x_3 = \lambda^2 - x_1 - x_2$$

$$y_3 = (x_1 - x_3)\lambda - y_1$$

其中：(a) $x_1 \neq x_2$ 时， $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$ ；(b) $x_1 = x_2$ 且 $Q \neq -P$ 时， $\lambda = \frac{3x_1^2 + a}{2y_1}$ （点倍乘）。

若 $P = (5, 2)$ ，试求 $3P$ 。

知识点梳理

1. 椭圆曲线 (Weierstrass 形式)

在有限域 $\mathbb{F}_p$ 上，椭圆曲线标准方程为：

$$y^2 = x^3 + ax + b \pmod{p}$$

需要满足判别式 $\Delta = -16(4a^3 + 27b^2) \not\equiv 0 \pmod{p}$ （曲线非奇异）。

本题中: $p = 11$, $a = 1$, $b = 6$ 。验证: $4a^3 + 27b^2 = 4 \times 1 + 27 \times 36 = 4 + 972 = 976$, $976 \bmod 11 = 8 \neq 0$ 。曲线非奇异。

2. 椭圆曲线点加法的几何解释

椭圆曲线上两点相加的几何意义:

- 过 P 和 Q 作直线, 与曲线交于第三点 R'
- R' 关于 x 轴的对称点 (即 y 坐标取反) 即为 $P + Q$

当 $P = Q$ 时, 直线变为切线 (点倍乘), 切线与曲线的另一交点即为 $2P$ 中对应点。

3. $\mathbb{F}_p$ 上的逆元计算

公式中需要计算分母的模 p 逆元。例如要求 $(x_2 - x_1)^{-1} \bmod p$, 即寻找 t 满足 $t \cdot (x_2 - x_1) \equiv 1 \pmod{p}$ 。

详细解答

已知: 椭圆曲线 $y^2 = x^3 + x + 6 \pmod{11}$, $P = (5, 2)$ 。求 $3P$ 。

首先验证 P 在曲线上:

左边: $y^2 = 2^2 = 4 \pmod{11}$, 右边: $x^3 + x + 6 = 125 + 5 + 6 = 136 \equiv 4 \pmod{11}$

$4 = 4$, P 在曲线上。✓

计算方案: $3P = 2P + P$, 先求 $2P$ (点倍乘), 再与 P 相加。

第一阶段: 求 $2P = P + P = (5, 2) + (5, 2)$

由于 $x_1 = x_2 = 5$ 且 $P \neq -P$ ($-P = (5, -2) = (5, 9) \neq P$), 使用点倍乘公式:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{11}$$

计算分子和分母:

$$3x_1^2 + a = 3 \times 5^2 + 1 = 3 \times 25 + 1 = 76 \equiv 10 \pmod{11}$$

$$2y_1 = 2 \times 2 = 4 \pmod{11}$$

需要计算 $4^{-1} \pmod{11}$ 。寻找 t 使得 $4t \equiv 1 \pmod{11}$: $4 \times 3 = 12 \equiv 1 \pmod{11}$, 所以 $4^{-1} \equiv 3 \pmod{11}$ 。

因此:

$$\lambda = 10 \times 3 = 30 \equiv 8 \pmod{11}$$

计算 x_3 和 y_3 :

$$x_3 = \lambda^2 - x_1 - x_2 \equiv 8^2 - 5 - 5 = 64 - 10 = 54 \equiv 10 \pmod{11}$$

$$\begin{aligned} y_3 &= \lambda(x_1 - x_3) - y_1 \equiv 8 \times (5 - 10) - 2 = 8 \times (-5) - 2 \\ &= -40 - 2 = -42 \equiv 2 \pmod{11} \end{aligned}$$

(验证: $-40 \equiv 4 \pmod{11}$, $-42 \equiv 2 \pmod{11}$). 另算: $8 \times 6 = 48 \equiv 4$, $4 - 2 = 2$.)

$$\boxed{2P = (10, 2)}$$

验证 $2P$ 在曲线上: 左边 $2^2 = 4$; 右边 $10^3 + 10 + 6 = 1000 + 10 + 6 = 1016 \equiv 4 \pmod{11}$.
✓

第二阶段: 求 $3P = 2P + P = (10, 2) + (5, 2)$

此时 $x_1 = 10$, $x_2 = 5$, $x_1 \neq x_2$, 使用异点加法公式:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{11}$$

$$y_2 - y_1 = 2 - 2 = 0 \pmod{11}$$

$$x_2 - x_1 = 5 - 10 = -5 \equiv 6 \pmod{11}$$

注意! $\lambda = 0/6 = 0 \pmod{11}$ 。分母 6 的逆元存在 ($6 \times 2 = 12 \equiv 1 \pmod{11}$, $6^{-1} = 2$), 所以 $\lambda = 0 \times 2 = 0$ 。

计算 x_3 和 y_3 :

$$x_3 = \lambda^2 - x_1 - x_2 \equiv 0^2 - 10 - 5 = -15 \equiv 7 \pmod{11}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \equiv 0 \times (10 - 7) - 2 = 0 - 2 = -2 \equiv 9 \pmod{11}$$

$$\boxed{3P = (7, 9)}$$

验证 $3P$ 在曲线上: 左边 $9^2 = 81 \equiv 4 \pmod{11}$ 。右边 $7^3 + 7 + 6 = 343 + 7 + 6 = 356$, $356 \bmod 11 = 356 - 32 \times 11 = 356 - 352 = 4$ 。4 = 4。✓

知识点延伸: 椭圆曲线上的标量乘法与安全性

标量乘法 kP 是 ECC 的核心运算——计算者知道 k 和 P , 求 $Q = kP$ 在计算上是容易的 (用倍加算法, 复杂度 $O(\log k)$ 次点运算)。

但反问题——**椭圆曲线离散对数问题 (ECDLP)**: 给定 P 和 $Q = [k]P$, 计算 k ——在经典计算模型下被认为极其困难。

本题中的小模数 $p = 11$ 仅用于教学。实际 ECC 密码系统使用 $p \approx 2^{256}$ 或更大的素数域, 此时 ECDLP 在计算上不可行, 提供了强安全性。

倍加算法 (Double-and-Add) 示例: 计算 $3P$ 可用 $3 = 11_2$ 表示:

- 初始: $R = \mathcal{O}$
- bit 1 (高位): $R = 2 \cdot \mathcal{O} = \mathcal{O}$, 加 P : $R = P$

- bit 1 (低位): $R = 2P$, 加 P : $R = 3P$

共需 1 次倍乘 + 1 次加法。

3.5 ElGamal 签名同 t 攻击 (20 分)

题目

在 ElGamal 签名方案中, 设 $p = 1321$, $\alpha = 13$, $\beta = 799$, $\log_{\alpha} \beta$ 未知, 并且已知两个签名 (γ, δ, x) 分别为 $(604, 762, 666)$, $(604, 165, 717)$, 而且两个签名使用了同一个 t , 试求 t 的值。

知识点梳理

1. ElGamal 签名方案

ElGamal 签名方案由 Taher ElGamal 于 1985 年提出, 是 DSA (数字签名算法) 的前身。

密钥生成:

- 选择大素数 p 和生成元 $\alpha \in \mathbb{F}_p^*$
- 选择私钥 a ($1 \leq a \leq p-2$), 计算公钥 $\beta = \alpha^a \bmod p$
- 公钥: (p, α, β) ; 私钥: a

签名生成 (对消息 m):

1. 选择随机数 t , 满足 $1 \leq t \leq p-2$ 且 $\gcd(t, p-1) = 1$
2. 计算 $\gamma = \alpha^t \bmod p$
3. 计算 $\delta = (m - a\gamma) \cdot t^{-1} \bmod (p-1)$
4. 签名为 (γ, δ)

签名验证:

$$\beta^{\gamma} \cdot \gamma^{\delta} \equiv \alpha^m \pmod{p}$$

2. 同 t 攻击 (Same- k Attack)

若签名者使用相同的随机数 t 为两条不同消息 m_1 和 m_2 生成签名, 则攻击者可在未知私钥 a 的情况下直接计算出 t , 进而恢复私钥。

这是 ElGamal/DSA 签名中最著名的实现缺陷之一, 也是为什么随机数 t 必须每次不同且真随机的根本原因。

3. 攻击数学原理

设有两条消息 m_1, m_2 使用同一 t 签名:

$$\delta_1 = (m_1 - a\gamma) \cdot t^{-1} \pmod{p-1}$$

$$\delta_2 = (m_2 - a\gamma) \cdot t^{-1} \pmod{p-1}$$

两式相减 (消去 $a\gamma$):

$$\delta_1 - \delta_2 = (m_1 - m_2) \cdot t^{-1} \pmod{p-1}$$

从而可求出 t 。

详细解答

已知参数:

- $p = 1321$ (大素数), $p - 1 = 1320$
- $\alpha = 13$ (生成元)
- $\beta = 799$ (公钥)
- 签名 1: $(\gamma_1, \delta_1, x_1) = (604, 762, 666)$
- 签名 2: $(\gamma_2, \delta_2, x_2) = (604, 165, 717)$

注意 $\gamma_1 = \gamma_2 = 604$, 这直接确认了两签名使用了同一 t (因为 $\gamma = \alpha^t \pmod{p}$)。

Step 1: 利用验证等式建立关系

ElGamal 签名验证等式为:

$$\beta^\gamma \cdot \gamma^\delta \equiv \alpha^x \pmod{p}$$

对签名 1 ($\gamma = 604, \delta = 762, x = 666$):

$$799^{604} \cdot 604^{762} \equiv 13^{666} \pmod{1321} \quad (1)$$

对签名 2 ($\gamma = 604, \delta = 165, x = 717$):

$$799^{604} \cdot 604^{165} \equiv 13^{717} \pmod{1321} \quad (2)$$

Step 2: 两式相除 $\frac{(1)}{(2)}$

$$\frac{799^{604} \cdot 604^{762}}{799^{604} \cdot 604^{165}} \equiv \frac{13^{666}}{13^{717}} \pmod{1321}$$

化简:

$$604^{762-165} \equiv 13^{666-717} \pmod{1321}$$

$$604^{597} \equiv 13^{-51} \pmod{1321}$$

Step 3: 利用 $\gamma = \alpha^t$ 建立关于 t 的方程

由定义 $\gamma = \alpha^t \bmod p$, 即 $604 = 13^t \bmod 1321$ 。

代入上式左侧: $(13^t)^{597} = 13^{597t} \pmod{1321}$ 。

因此:

$$13^{597t} \equiv 13^{-51} \pmod{1321}$$

由 Fermat 小定理, $\alpha = 13$ 的阶整除 $p - 1 = 1320$ 。在同余式中比较指数, 我们关心模 1320 下的关系。

由于 13 是模 1321 的生成元 (题目给定), 其阶为 $p - 1 = 1320$, 因此 $13^a \equiv 13^b \pmod{1321}$ 等价于 $a \equiv b \pmod{1320}$ 。

于是:

$$597t \equiv -51 \pmod{1320}$$

Step 4: 求解同余方程

需要解: $597t + 51 \equiv 0 \pmod{1320}$, 即 $597t \equiv -51 \equiv 1269 \pmod{1320}$ 。

首先计算 $\gcd(597, 1320)$ 以确定解的数量。

使用欧几里得算法:

$$1320 = 2 \times 597 + 126$$

$$597 = 4 \times 126 + 93$$

$$126 = 1 \times 93 + 33$$

$$93 = 2 \times 33 + 27$$

$$33 = 1 \times 27 + 6$$

$$27 = 4 \times 6 + 3$$

$$6 = 2 \times 3 + 0$$

$$\gcd(597, 1320) = 3。$$

检查 51 是否能被 3 整除: $51 \div 3 = 17$, 可以。

因此方程有 3 个解 (模 1320)。将整个方程除以 3:

$$199t \equiv -17 \pmod{440}$$

$$\text{即 } 199t \equiv 423 \pmod{440}。$$

现在 $\gcd(199, 440) = 1$, 存在唯一解模 440。

使用扩展欧几里得算法求 199 模 440 的逆元：

$$440 = 2 \times 199 + 42$$

$$199 = 4 \times 42 + 31$$

$$42 = 1 \times 31 + 11$$

$$31 = 2 \times 11 + 9$$

$$11 = 1 \times 9 + 2$$

$$9 = 4 \times 2 + 1$$

回溯：

$$\begin{aligned} 1 &= 9 - 4 \times 2 = 9 - 4 \times (11 - 9) = 5 \times 9 - 4 \times 11 \\ &= 5 \times (31 - 2 \times 11) - 4 \times 11 = 5 \times 31 - 14 \times 11 \\ &= 5 \times 31 - 14 \times (42 - 31) = 19 \times 31 - 14 \times 42 \\ &= 19 \times (199 - 4 \times 42) - 14 \times 42 = 19 \times 199 - 90 \times 42 \\ &= 19 \times 199 - 90 \times (440 - 2 \times 199) = 199 \times 199 - 90 \times 440 \end{aligned}$$

所以 $199 \times 199 \equiv 1 \pmod{440}$ ，即 $199^{-1} \equiv 199 \pmod{440}$ 。

因此：

$$t \equiv 199 \times 423 \pmod{440}$$

计算 $199 \times 423 = 199 \times (400 + 23) = 79600 + 4577 = 84177$ 。

$84177 \div 440 = 191$ 余 137 ($191 \times 440 = 84040$, $84177 - 84040 = 137$)。

所以 $t \equiv 137 \pmod{440}$ 。

Step 5: 写出模 1320 下的通解

$$t \equiv 137 \pmod{440}$$

$$t = 137, 137 + 440 = 577, 137 + 880 = 1017 \pmod{1320}$$

三个可能的 t 值: $t \in \{137, 577, 1017\}$ 。

Step 6: 验证正确的 t

验证条件: $\gamma = \alpha^t \pmod{p}$, 即 $13^t \equiv 604 \pmod{1321}$ 。

计算各候选值:

$$13^2 = 169 \pmod{1321}$$

$$13^4 = 169^2 = 28561 \equiv 820 \pmod{1321} \quad (\text{因为 } 28561 - 21 \times 1321 = 28561 - 27741 = 820)$$

$$13^8 = 820^2 = 672400 \equiv 11 \pmod{1321}$$

$$13^{16} = 11^2 = 121 \pmod{1321}$$

$$13^{32} = 121^2 = 14641 \equiv 110 \pmod{1321}$$

$$13^{64} = 110^2 = 12100 \equiv 211 \pmod{1321}$$

$$13^{128} = 211^2 = 44521 \equiv 928 \pmod{1321}$$

计算 13^{137} : $137 = 128 + 8 + 1$ 。

$$13^{137} = 13^{128} \cdot 13^8 \cdot 13^1 \equiv 928 \times 11 \times 13 \pmod{1321}$$

$$928 \times 11 = 10208 \equiv 10208 - 7 \times 1321 = 10208 - 9247 = 961 \pmod{1321}$$

$$961 \times 13 = 12493 \equiv 12493 - 9 \times 1321 = 12493 - 11889 = 604 \pmod{1321}$$

$604 = 604 = \gamma$ 。验证成功!

因此 $t = 137$ 是正确的。

$$t = 137$$

知识点延伸: ElGamal/DSA/ECDSA 中随机数安全性的教训

同 t (同 k) 攻击在密码学工程史上造成了多起真实安全事故:

- **Sony PlayStation 3 (2010 年):** ECDSA 签名中使用了固定的随机数 k , 导致私钥被恢复。著名的 fail0verflow 团队利用该漏洞完全破解了 PS3 的安全体系。
- **Android Java 漏洞 (2013 年):** Bitcoin 钱包应用因随机数生成器的缺陷导致大量用户私钥泄露, 资金被盗。
- **设计教训:** ElGamal/DSA 签名中, $t(k)$ 必须满足: (1) 密码学意义上的真随机; (2) 每次签名必须不同; (3) 必须保密。RFC 6979 因此提出了**确定性 DSA/ECDSA**, 使用 HMAC 从私钥和消息中确定性派生 k , 从根本上避免了随机数相关的风险。

本题中 $\gamma_1 = \gamma_2 = 604$ 直接暴露了同一 t 的使用, 是最直观的攻击信号。

3.6 名词解释: Hash 函数 (5 分)

题目

解释 Hash 函数的概念。

知识点梳理

1. Hash 函数的定义

Hash 函数（散列函数/哈希函数）是一种将任意长度的消息 M 映射为固定长度输出 $h = H(M)$ 的函数。在密码学中称为“密码学 Hash 函数”，它必须满足特定的安全性质。

2. Hash 函数的三个核心安全要求

1. **单向性 (Pre-image Resistance, 抗原像攻击)**: 给定 h , 在计算上不可行找到 M 使得 $H(M) = h$ 。即函数是不可逆的 (“单向” 的含义)。
2. **第二原像稳固性 (Second Pre-image Resistance, 抗第二原像攻击)**: 给定 M , 在计算上不可行找到另一不同的 $M' \neq M$ 使得 $H(M') = H(M)$ 。
3. **碰撞稳固性 (Collision Resistance, 抗碰撞攻击)**: 在计算上不可行找到任意两个不同的消息 M, M' 使得 $H(M) = H(M')$ 。

3. 常见 Hash 函数

MD5 (128 位, 已不安全)、SHA-1 (160 位, 已不安全)、SHA-256/SHA-512 (目前安全)、SM3 (中国国家密码管理局标准, 256 位)。

详细解答

Hash 函数（单向哈希函数）是一种密码学原语，记作 $h = H(M)$ ，它操作一个任意长度的消息 M ，返回一个固定长度的值 h （称为哈希值、散列值或消息摘要）。

核心安全要求为以下三点：

1. **给定 M ，很容易计算 h** （计算高效性）。这是 Hash 函数作为实用工具的基本要求——计算哈希值应当是快速的。
2. **给定 h ，很难计算 M 使 $H(M) = h$** （单向性/抗原像性）。即从哈希值反向推导原始消息在计算上不可行。这是“单向”的核心含义，确保以哈希值存储的密码等秘密信息即使泄露，攻击者也无法轻易恢复原始值。
3. **给定 M ，很难找到另一个消息 $M' \neq M$ 使得 $H(M') = H(M)$** （抗第二原像性/弱碰撞稳固性）。即已知一个消息及其哈希值，无法伪造另一个具有相同哈希值的不同消息。这是数字签名等应用中防止消息被替换的关键保障。

此外还有一个更强的要求——**抗碰撞性**（强碰撞稳固性）：很难找到任意两个不同消息

$M \neq M'$ 使得 $H(M) = H(M')$ 。这一性质通过“生日攻击”与哈希值长度关联： n 位哈希值提供约 $2^{n/2}$ 的碰撞安全性。

应用场景：密码存储、文件完整性校验、数字签名（先哈希再签名）、区块链工作量证明（PoW）、消息认证码（HMAC）等。

3.7 名词解释：公钥密码（5 分）

题目

解释公钥密码的概念。

知识点梳理

1. 对称密码 vs 公钥密码

- **对称密码 (Symmetric Cryptography)：**加密密钥和解密密钥相同（或可从一者轻易推得另一者），通信双方需事先共享密钥。代表：DES、AES。
- **公钥密码 (Public-Key Cryptography / Asymmetric Cryptography)：**加密密钥和解密密钥不同，且从加密密钥推导解密密钥在计算上不可行。代表：RSA、ElGamal、ECC。

2. 公钥密码的核心思想

由 Diffie 和 Hellman 于 1976 年提出。每个用户拥有一对密钥：

- **公钥 (Public Key)：**可以公开分发，无需保密
- **私钥 (Private Key)：**必须严格保密，仅用户本人持有

3. 数学基础

公钥密码的安全性依赖于某一数学困难问题，如：

- RSA：大整数分解问题 (Integer Factorization)
- ElGamal：有限域上的离散对数问题 (DLP)
- ECC：椭圆曲线上的离散对数问题 (ECDLP)

详细解答

公钥密码（非对称密码）是指加密密钥和解密密钥不相同的密码体制。

其核心特征为：

1. **密钥非对称性：**加密密钥（公钥）和解密密钥（私钥）是两个不同的密钥。公钥

可以公开分发，私钥由所有者秘密保管。

2. **单向陷门性**：加密密钥和解密密钥之间依赖于某一数学计算困难问题，从加密密钥计算出解密密钥在计算上是极其困难的（在合理的安全参数下不可行）。这一数学困难问题充当了“陷门”的角色——公开的加密方向容易计算，而反向推解密密钥则极度困难。
3. **无需预先共享密钥**：通信双方不需要事先通过安全信道交换秘密密钥。发送方只需获取接收方的公钥（可从公开目录、证书等途径获取）即可加密发送消息。这是公钥密码相比对称密码最革命性的优势，解决了密钥分发的难题。

典型算法：

- RSA：基于大整数分解的困难性
- ElGamal：基于有限域上离散对数的困难性
- ECC：基于椭圆曲线上离散对数的困难性
- 格基密码（后量子）：如 NTRU、Kyber，基于格中问题的困难性

应用：保密通信（加密）、数字签名（私钥签名、公钥验证）、密钥交换（如 Diffie-Hellman）等。

公钥密码的主要弱点是计算效率远低于对称密码（通常慢 100-1000 倍），因此实践中常采用混合加密：用公钥密码加密对称密钥，再用对称密码加密实际消息。

3.8 名词解释：解密（5 分）

题目

解释解密的概念。

知识点梳理

1. 密码学基本变换

密码学的两个核心变换为：

- **加密 (Encryption)** E ：将明文 M 变换为密文 C ，即 $C = E_K(M)$
- **解密 (Decryption)** D ：将密文 C 还原为明文 M ，即 $M = D_K(C)$

2. 解密的基本要求

解密过程必须满足一致性要求：对于合法的密钥 K ，对任意明文 M ，有：

$$D_K(E_K(M)) = M$$

即加密-解密为可逆操作。

3. 解密与密钥的关系

解密操作通常需要掌握正确的解密密钥。在对称密码中，解密密钥与加密密钥相同（或可推导）；在公钥密码中，解密密钥为私钥，仅有合法的接收方持有。

详细解答

解密是指将密文通过特定的数学变换和密钥处理，恢复为原始明文的过程。

具体来说：

1. **定义**：解密是加密的逆过程。设明文为 M ，加密函数为 E （使用密钥 K_e ），得到密文 $C = E_{K_e}(M)$ 。解密函数 D （使用密钥 K_d ）作用于密文 C ，还原出原始明文： $M = D_{K_d}(C) = D_{K_d}(E_{K_e}(M))$ 。
2. **密钥角色**：解密过程通常需要持有正确的解密密钥 K_d 。
 - 在对称密码中， $K_e = K_d$ （或互推），双方共享同一密钥。
 - 在公钥密码中， K_e 为发送方使用的公钥（公开）， K_d 为接收方秘密持有的私钥，解密操作只有持有私钥的合法接收方才能执行。
3. **安全性要求**：不应该存在比穷举密钥更有效的解密方法。换言之，在不知道正确密钥的情况下，从密文恢复明文在计算上应当是不可行的（对计算安全的密码体制而言）。

实例：在 DES 解密中，使用与加密相同的算法（Feistel 结构的对合性），仅将 16 轮子密钥的使用顺序颠倒 ($K_{16}, K_{15}, \dots, K_1$)。在 RSA 解密中，使用私钥 d 计算 $M = C^d \bmod n$ ($C = M^e \bmod n$)。

3.9 名词解释：数字签名 (5 分)

题目

解释数字签名的概念。

知识点梳理

1. 数字签名的定义

数字签名 (Digital Signature) 是一种密码学机制，用于：

- 验证消息的来源（认证性/不可否认性）
- 确保消息未被篡改（完整性）
- 防止签名者事后否认（不可抵赖性）

2. 数字签名的基本模型

数字签名方案通常包含三个算法：

- **密钥生成**：生成签名私钥 sk 和验证公钥 pk
- **签名算法**： $s = \text{Sign}(m, sk)$ ，用私钥对消息生成签名
- **验证算法**： $\text{Verify}(m, s, pk) \in \{\text{True}, \text{False}\}$ ，用公钥验证签名有效性

3. 数字签名与手写签名的区别

- 手写签名随文档内容不变，易被复制
- 数字签名是消息的函数，不同消息产生不同签名

详细解答

数字签名是附加在数据单元上的一些数据，或是对数据单元所作的密码变换。这种数据或变换允许数据单元的接收者用以确认数据单元的来源和数据单元的完整性，并保护数据，防止被人（例如接收者）进行伪造。

核心特征和功能：

1. **认证性（来源确认）**：接收方可以通过验证签名来确认消息确实来自声称的发送方。因为只有持有私钥的签名者才能生成有效签名。
2. **完整性保护**：数字签名覆盖了消息内容，任何对消息的修改都会导致签名验证失败，从而被接收方检测到。
3. **不可抵赖性（抗否认）**：签名者无法事后否认自己签署过该消息。因为有可信第三方（或数学上可验证的证据）证明签名确实由签名者的私钥生成。
4. **不可伪造性**：在计算上，任何未持有签名私钥的人无法生成能够通过验证的有效签名。这是数字签名方案的核心安全要求。

技术实现方式：

- 基于公钥密码的签名（如 RSA 签名、ElGamal 签名、DSA、ECDSA）：签名者用私钥对消息的哈希值进行“解密”运算，验证者用公钥“加密”验证。
- 实际流程：先对消息计算哈希值 $h = H(M)$ ，再用私钥对 h 签名得到 s 。

典型算法：RSA-PSS、DSA、ECDSA、EdDSA (Ed25519)、SM2 签名（中国标准）。
数字签名是电子商务、电子政务、软件分发、区块链交易等数字世界中建立信任的基础机制。

3.10 DES 算法的加密过程 (10 分)

题目

请描述 DES 算法的加密过程。

知识点梳理

1. DES (Data Encryption Standard)

DES 由 IBM 公司开发, 1977 年被 NIST 采纳为美国联邦标准。它是密码学史上影响最深远的分组密码。

基本参数:

- 分组长度: 64 位
- 密钥长度: 64 位 (其中 8 位为奇偶校验位, 有效密钥为 56 位)
- 轮数: 16 轮
- 结构: Feistel 网络 (平衡结构)

2. Feistel 结构

Feistel 网络的核心思想是将输入分为左右两半, 每轮仅变换一半:

$$\begin{aligned}L_i &= R_{i-1} \\ R_i &= L_{i-1} \oplus F(R_{i-1}, K_i)\end{aligned}$$

其中 F 为轮函数, K_i 为第 i 轮的子密钥。

优点: 加密和解密使用同一结构, 仅子密钥顺序相反 (解密时倒序使用 $K_{16}, K_{15}, \dots, K_1$)。

详细解答

DES 算法的加密过程分为以下几个阶段:

一、初始置换 (IP, Initial Permutation)

64 位明文分组首先经过一个固定的初始置换 IP。IP 按照一个预定义的置换表重新排列 64 位的顺序, 其逆置换 IP^{-1} 用于最后的输出变换。IP 和 IP^{-1} 没有密码学意义 (它们是公开的、与密钥无关的), 主要是为了方便早期硬件实现中的数据布线。

二、16 轮 Feistel 迭代

经过 IP 后, 64 位数据分为左右等长的两部分: L_0 (左 32 位) 和 R_0 (右 32 位)。然后进行 16 轮迭代 ($i = 1, 2, \dots, 16$):

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

每轮使用一个 48 位的子密钥 K_i (由原始 64 位密钥通过密钥编排算法生成)。

F 函数（轮函数）包括以下四个步骤：

1. **扩展变换 E** ：将 32 位的 R_{i-1} 扩展为 48 位。通过一个固定的扩展置换表，将某些位复制，使 32 位变为 48 位。这一步骤增加了扩散效果（一个输入位影响多个输出位）。
2. **与子密钥异或**：将扩展后的 48 位数据与 48 位子密钥 K_i 按位异或（XOR）。
3. **S 盒代替（Substitution）**：这是 DES 中唯一的非线性变换，也是安全性的核心。48 位数据被分为 8 组，每组 6 位，分别送入 8 个不同的 S 盒（ $S_1, S_2, \dots, S_8$ ）。每个 S 盒将 6 位输入映射为 4 位输出：
 - 设 6 位输入为 $b_1b_2b_3b_4b_5b_6$
 - b_1b_6 作为行号（2 位，编码 0-3 共 4 行）
 - $b_2b_3b_4b_5$ 作为列号（4 位，编码 0-15 共 16 列）
 - 查表得到 S 盒在对应行列处的 4 位输出值例如 S_6 的输入为 110011，则行号 $b_1b_6 = 11_2 = 3$ ，列号 $b_2b_3b_4b_5 = 1001_2 = 9$ ，查 S_6 在 (3,9) 处的值为 14，输出 1110_2 。
4. **P 盒置换（Permutation）**：将 8 个 S 盒的 32 位输出通过一个固定的 P 置换表重新排列。P 置换进一步增强扩散效果。

$F(R_{i-1}, K_i)$ 的输出（32 位）与 L_{i-1} 异或后成为新的 R_i 。

三、逆初始置换 (IP^{-1})

16 轮迭代结束后，将 R_{16} 与 L_{16} 连接（注意顺序：先 R_{16} 后 L_{16} ），然后通过逆初始置换 IP^{-1} 得到最终的 64 位密文。

四、密钥编排（Key Schedule）

64 位密钥的处理流程：

1. **PC-1（置换选择 1）**：从 64 位密钥中去除 8 个奇偶校验位（第 8, 16, 24, ..., 64 位），剩余 56 位按照固定置换重排，分为两个 28 位的半密钥 C_0 和 D_0 。
2. **循环左移**：每轮将 C_{i-1} 和 D_{i-1} 分别循环左移 1 位或 2 位（第 1, 2, 9, 16 轮左移 1 位，其余轮左移 2 位），得到 C_i 和 D_i 。
3. **PC-2（置换选择 2）**：从 56 位的 $C_i||D_i$ 中选出 48 位作为本轮子密钥 K_i 。

五、解密过程

由于 Feistel 结构的对合性，DES 的解密过程与加密过程完全相同，唯一的区别是将 16 轮子密钥的顺序颠倒使用（即 $K_{16}, K_{15}, \dots, K_1$ ）。

知识点延伸：DES 安全性分析与演进

DES 的核心安全争议在于 56 位密钥过短。1998 年，EFF 的专用硬件“Deep Crack”在 56 小时内完成穷举破解。1999 年，结合分布式计算将时间缩短至 22 小时。

为延长 DES 寿命，提出了 3DES (Triple DES): $C = E_{K_3}(D_{K_2}(E_{K_1}(M)))$ 。其中 E-D-E 模式兼容单 DES (当 $K_1 = K_2 = K_3$ 时退化为单 DES)。3DES 的有效安全强度为 112 位 (因为存在 meet-in-the-middle 攻击)，但速度较慢，已被 AES 逐步取代。

DES 设计的最大遗产是：S 盒的设计标准至今未完全公开 (NSA 曾参与)，但后续研究表明 DES 的 S 盒设计非常精心，对差分攻击有极强抵抗力——而差分密码分析直到 1990 年才被 Biham 和 Shamir 公开发表，说明 DES 设计者早在 1970 年代就已了解并防御了这一攻击方法。

3.11 密码协议的设计原则 (10 分)**题目**

简述密码协议的设计原则。

知识点梳理**1. 密码协议 (Cryptographic Protocol)**

密码协议是使用密码学技术实现特定安全目标的交互式通信规程。常见协议包括：认证协议 (如 Kerberos、TLS 握手)、密钥交换协议 (如 Diffie-Hellman)、电子支付协议、电子投票协议等。

2. 协议攻击类型

协议设计不当可能导致严重安全漏洞，即使底层密码算法本身是安全的。常见攻击包括：重放攻击、反射攻击、中间人攻击、类型混淆攻击、预言机攻击、并行会话攻击等。

3. 设计原则的重要性

协议安全比算法安全更难保证——协议涉及多方交互、消息顺序、时间因素、多种密码原语协调使用，任何一个环节的疏忽都可能导致全面崩溃。因此需要一套系统的设计原则来指导。

详细解答

在密码协议设计阶段就充分考虑到各种可能破坏协议安全性的问题并加以避免，是确保协议安全的基础。以下是密码协议设计的 9 条核心原则：

- 1. 消息独立完整性原则：**协议中的每条消息都应当能够独立验证其完整性。消息之间不应存在未受完整性保护的依赖关系。避免一个消息的篡改通过连锁反应影响后续消息的正确验证。
- 2. 消息前提准确原则：**协议每一步的执行都应明确列出该步骤能够被执行的前提条

件。只有当前提条件全部满足时，协议参与者才应执行相应的操作。这可以防止在条件不充分的情况下盲目执行导致安全漏洞。

3. **主体身份标识原则**：协议中的每条消息都应当明确标识消息的发送者和预期接收者。若消息中缺少身份信息，攻击者可以利用消息的”上下文无关性”进行反射攻击或重定向攻击——将截获的消息发给非预期的第三方。
4. **明确加密目的原则**：每次使用加密操作时必须明确其目的——是为了保密性，还是为了认证，或是两者兼顾。不同目的对应不同的安全需求和使用方式，混淆目的可能导致严重的安全缺陷。
5. **明确签名原则**：签名在协议中用于确保数据的真实性和抗抵赖性。设计者必须明确：哪些数据需要签名保护？签名的作用范围是什么？签名是否可以独立验证？避免对签名的模糊使用。签名的验证时机和方式也应在协议中明确定义。
6. **随机数的使用原则**：协议中常使用随机数（Nonce）来保证消息的新鲜性（Freshness），防御重放攻击。在使用随机数时应明确：
 - 其所起的作用（新鲜性保证？挑战-响应？密钥材料？）
 - 其安全属性（不可预测性、不可重用性）
 - 随机数的长度和生成质量要求
7. **时间戳的使用原则**：时间戳可用来确保消息的时效性（避免过时消息被重放）。使用时间戳时需要注意：
 - 通信各方（或至少涉及方与可信第三方之间）需要时钟同步
 - 需要定义合理的时间窗口（允许的时钟偏差范围）
 - 时间戳本身应受到完整性保护（不能被攻击者修改）
8. **编码原则**：协议中消息的具体编码格式与协议安全密切相关。设计者应当明确协议中每条消息的具体数据格式（字段顺序、长度、类型标记），而且还要验证这种格式对安全的贡献。良好的编码可以防止类型混淆攻击（攻击者将一种类型的消息重新解释为另一种类型）。

具体来说：每个消息的格式应当能唯一识别其类型和上下文，不同用途的消息应有不同的格式标记，避免不同上下文的消息被互换使用。
9. **最少安全假设原则**：在进行协议设计时，需要对系统环境进行风险分析，做出适当的初始安全假设（如认为所采用的密码算法是安全的、认证服务器是可信的等）。设计原则是采用最少的安全假设：
 - 不假设通信信道是安全的（除非设计目标如此）

- 不假设参与者始终诚实（考虑恶意参与者）
- 不假设随机数生成器是完美的（需有防御措施）
- 每条假设都应明确文书记录，成为协议安全性证明的基础

这九条原则相互关联、互为补充，共同构成密码协议设计的系统方法论。在实际设计中，还应考虑协议的**形式化验证**（如使用 BAN 逻辑、ProVerif、Tamarin 等工具）来机械地检查协议是否满足预期的安全性质。

知识点延伸：著名协议设计失败案例

Needham-Schroeder 公钥协议（1978 年） 被广泛引用为协议设计失误的经典案例：原始协议因缺少主体身份标识，在 1995 年被 Lowe 发现存在中间人攻击（使用 CSP 模型检测）。这个案例凸显了”主体身份标识原则”和”最少安全假设原则”的重要性。

WEP (Wired Equivalent Privacy): WiFi 的 WEP 协议因随机数 (IV) 过短 (24 位) 且重用频繁，导致 RC4 密钥流重用攻击 (FMS 攻击, 2001 年)，最终被 WPA/WPA2 取代。这个失败体现了”随机数使用原则”和”编码原则”的重要性。

SSL/TLS 早期版本的降级攻击：攻击者强制通信降级到不安全的密码套件，体现了协议设计中需要充分考虑协商消息的完整性保护。

试卷三知识点分布总结

题号	分值	知识领域	核心知识点
一	10	古典密码	仿射密码解密、乘法逆元、模运算
二	10	对称密码	$GF(2^8)$ 有限域、AES MixColumns、xtime 运算
三	10	公钥密码	RSA 体制、Fermat 因数分解法
四	10	椭圆曲线密码	ECC 点加法与倍乘、有限域模运算
五	20	数字签名	ElGamal 签名、同 t 攻击、同余方程求解
六	5	密码学基础	Hash 函数定义与安全性质
七	5	公钥密码	公钥密码概念、非对称性、数学困难问题
八	5	密码学基础	解密概念、加密-解密可逆性
九	5	数字签名	数字签名定义、认证性与不可抵赖性
十	10	对称密码	DES 加密流程、Feistel 结构、 F 函数、密钥编排
十一	10	安全协议	密码协议设计九大原则

难度与重点分析

本试卷覆盖密码学核心知识体系：古典密码（仿射密码）、对称密码（DES/AES）、公钥密码（RSA/ElGamal）和椭圆曲线密码（ECC），以及密码协议的设计原则。

计算题（Q1-Q5）共 60 分，是本试卷的核心部分，要求熟练掌握有限域运算、模运算、ECC 点运算和 ElGamal 签名机制。

第五题为 20 分大题，涉及的 ElGamal 同 t 攻击是数字签名安全性分析的经典案例。

4 试卷四：计算机安全与保密（2018-2019）

4.1 Hill 密码解密 (10 分)

知识点回顾: Hill 密码体制

Hill 密码是一种基于线性代数的经典多表代换密码体制, 由 Lester S. Hill 于 1929 年提出。其核心思想是利用 $m \times m$ 的可逆矩阵作为密钥, 对明文向量进行线性变换后取模, 得到密文向量。

加密过程: 设明文向量为 $\mathbf{X} = (x_1, x_2, \dots, x_m)^T$, 密钥矩阵为 $m \times m$ 的可逆矩阵 $\mathbf{K}$ (模 n 下可逆), 则密文向量为:

$$\mathbf{Y} = \mathbf{K}\mathbf{X} \pmod{n}$$

即 $Y_i = \sum_{j=1}^m k_{ij}x_j \pmod{n}$ 。

解密过程: 解密需要计算密钥矩阵 $\mathbf{K}$ 在模 n 下的逆矩阵 $\mathbf{K}^{-1}$, 即满足 $\mathbf{K}\mathbf{K}^{-1} \equiv \mathbf{I} \pmod{n}$ 的矩阵。解密公式为:

$$\mathbf{X} = \mathbf{K}^{-1}\mathbf{Y} \pmod{n}$$

矩阵求逆 (模 n 下的 2×2 矩阵): 设 $\mathbf{K} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, 其行列式为 $\Delta = ad - bc \pmod{n}$ 。

$\mathbf{K}$ 可逆的充要条件是 $\gcd(\Delta, n) = 1$ 。此时:

$$\mathbf{K}^{-1} = \Delta^{-1} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \pmod{n}$$

其中 Δ^{-1} 是 Δ 在模 n 下的乘法逆元, 即满足 $\Delta \cdot \Delta^{-1} \equiv 1 \pmod{n}$ 的数。

扩展欧几里得算法求乘法逆元: 给定 a 和模数 m , 若 $\gcd(a, m) = 1$, 则存在整数 x 使得 $ax \equiv 1 \pmod{m}$ 。可以通过扩展欧几里得算法求出系数 x (即 $a^{-1} \pmod{m}$), 其正确性来源于贝祖等式: 存在整数 s, t 使得 $sa + tm = \gcd(a, m) = 1$, 于是 $s \equiv a^{-1} \pmod{m}$ 。

解: 详细推导过程

Step 1: 写出加密方程组和密钥矩阵。

加密方程组为:

$$\begin{cases} Y_1 = 4X_1 + 7X_2 \pmod{31} \\ Y_2 = 5X_1 + 11X_2 \pmod{31} \end{cases}$$

写成矩阵形式:

$$\begin{pmatrix} Y_1 \\ Y_2 \end{pmatrix} = \begin{pmatrix} 4 & 7 \\ 5 & 11 \end{pmatrix} \begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \pmod{31}$$

因此密钥矩阵 $\mathbf{K} = \begin{pmatrix} 4 & 7 \\ 5 & 11 \end{pmatrix}$ 。

Step 2: 用消元法推导解密方程组。

方法一 (消元法):

(1) $\times 11$: $11Y_1 = 44X_1 + 77X_2 \equiv 13X_1 + 15X_2 \pmod{31}$ (因为 $44 \bmod 31 = 13$, $77 \bmod 31 = 15$)

(2) $\times 7$: $7Y_2 = 35X_1 + 77X_2 \equiv 4X_1 + 15X_2 \pmod{31}$

两式相减消去 X_2 :

$$11Y_1 - 7Y_2 \equiv (13X_1 + 15X_2) - (4X_1 + 15X_2) \equiv 9X_1 \pmod{31}$$

即 $9X_1 \equiv 11Y_1 - 7Y_2 \pmod{31}$ 。

求 9 在模 31 下的逆元: 用扩展欧几里得算法。

$$31 = 9 \times 3 + 4, \quad 9 = 4 \times 2 + 1$$

回代:

$$1 = 9 - 4 \times 2 = 9 - 2 \times (31 - 9 \times 3) = 9 \times 7 - 2 \times 31$$

所以 $9^{-1} \equiv 7 \pmod{31}$ 。

两端乘以 7:

$$X_1 \equiv 7(11Y_1 - 7Y_2) \equiv 77Y_1 - 49Y_2 \pmod{31}$$

$77 \bmod 31 = 15$, $-49 \bmod 31 = 13$ (因为 $-49 + 2 \times 31 = 13$)。

故 $X_1 \equiv 15Y_1 + 13Y_2 \pmod{31}$ 。

类似地消去 X_1 : (1) $\times 5$: $5Y_1 \equiv 20X_1 + 35X_2 \pmod{31}$; (2) $\times 4$: $4Y_2 \equiv 20X_1 + 44X_2 \pmod{31}$ 。

相减:

$$4Y_2 - 5Y_1 \equiv (20X_1 + 44X_2) - (20X_1 + 35X_2) \equiv 9X_2 \pmod{31}$$

即 $9X_2 \equiv 4Y_2 - 5Y_1 \pmod{31}$ 。乘以 $9^{-1} = 7$:

$$X_2 \equiv 7(4Y_2 - 5Y_1) \equiv 28Y_2 - 35Y_1 \equiv 27Y_1 + 28Y_2 \pmod{31}$$

(这里 $-35 \bmod 31 = 27$, 因为 $-35 + 2 \times 31 = 27$)

Step 3: 得到解密方程组。

$$\begin{cases} X_1 = 15Y_1 + 13Y_2 \pmod{31} \\ X_2 = 27Y_1 + 28Y_2 \pmod{31} \end{cases}$$

写成矩阵形式: $\mathbf{K}^{-1} = \begin{pmatrix} 15 & 13 \\ 27 & 28 \end{pmatrix}$ 。

验证:

$$\begin{pmatrix} 4 & 7 \\ 5 & 11 \end{pmatrix} \begin{pmatrix} 15 & 13 \\ 27 & 28 \end{pmatrix} = \begin{pmatrix} 4 \cdot 15 + 7 \cdot 27 & 4 \cdot 13 + 7 \cdot 28 \\ 5 \cdot 15 + 11 \cdot 27 & 5 \cdot 13 + 11 \cdot 28 \end{pmatrix} = \begin{pmatrix} 249 & 248 \\ 372 & 373 \end{pmatrix} \equiv \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \pmod{31}$$

确实成立。

Step 4: 解密密文 (24, 16)。

$$X_1 = 15 \times 24 + 13 \times 16 = 360 + 208 = 568 \equiv 568 \bmod 31$$

$31 \times 18 = 558$, $568 - 558 = 10$, 故 $X_1 = 10$ 。

$$X_2 = 27 \times 24 + 28 \times 16 = 648 + 448 = 1096 \equiv 1096 \bmod 31$$

$31 \times 35 = 1085$, $1096 - 1085 = 11$, 故 $X_2 = 11$ 。

验证加密: $Y_1 = 4 \times 10 + 7 \times 11 = 40 + 77 = 117 \equiv 24 \pmod{31}$ ($31 \times 3 = 93$, $117 - 93 = 24$)。

$Y_2 = 5 \times 10 + 11 \times 11 = 50 + 121 = 171 \equiv 16 \pmod{31}$ ($31 \times 5 = 155$, $171 - 155 = 16$)。解密正确。

明文为 $(X_1, X_2) = (10, 11)$

扩展: Hill 密码的安全性分析

Hill 密码能够抵抗唯密文攻击中的频率分析 (因为加密单元是向量而非单个字母), 但对已知明文攻击是脆弱的。若攻击者获得了 m 对线性无关的明-密文对, 就可以通过矩阵求逆恢复密钥矩阵 $\mathbf{K}$ 。实际应用中, Hill 密码常与置换、代换等其他技术结合使用以增强安全性。此外, 模数 n 通常取 26 (对应英文字母表大小), 但这里取 31 是为了使矩阵更容易满足可逆条件 (31 是素数, 任何非零行列式都可逆)。

4.2 背包密码算法 (10 分)

知识点回顾：背包 (Knapsack) 密码体制

背包密码体制由 Merkle 和 Hellman 于 1978 年提出，是最早的公钥密码候选方案之一。其安全性基于子集和问题 (Subset Sum Problem) 的 NP 完全性。

核心概念：

(1) **超递增背包序列：**一个正整数序列 $a_1, a_2, \dots, a_n$ 满足对每个 $i \geq 2$ ，有 $a_i > \sum_{j=1}^{i-1} a_j$ 。对于超递增序列，子集和问题存在多项式时间的贪心算法：从大到小依次判断每个元素是否可被包含。

(2) **陷门变换：**选取一个大模数 m (满足 $m > \sum a_i$) 和一个乘数 w (满足 $\gcd(w, m) = 1$)，通过模乘变换将容易求解的超递增序列“隐藏”为看似随机的困难序列：

$$b_i = a_i \cdot w \pmod{m}$$

其中 $\{a_i\}$ 为超递增序列 (私钥)， $\{b_i\}$ 为一般背包序列 (公钥)， (w, m) 构成陷门信息。

(3) **加密：**明文二进制位串 $x = x_1 x_2 \dots x_n$ ，密文为：

$$C = \sum_{i=1}^n x_i b_i$$

(4) **解密：**计算 $w^{-1} \pmod{m}$ ，然后：

$$C' = C \cdot w^{-1} \pmod{m} = \sum_{i=1}^n x_i (b_i \cdot w^{-1}) \pmod{m} = \sum_{i=1}^n x_i a_i$$

由于 $C' < m$ (因为 $m > \sum a_i$)，在超递增序列 $\{a_i\}$ 上用贪心算法即可恢复各 x_i 。

解：详细计算过程

Step 1: 计算一般背包序列 (公钥)。

已知超递增背包序列 (私钥)： $\{2, 3, 6, 13, 29, 61\}$ ，乘数 $w = 17$ ，模数 $m = 157$ 。

首先验证： $m = 157 > \sum a_i = 2 + 3 + 6 + 13 + 29 + 61 = 114$ ，满足要求。

计算一般背包序列 $\{b_i\}$ ：

$$b_1 = 2 \times 17 = 34 \equiv 34 \pmod{157}$$

$$b_2 = 3 \times 17 = 51 \equiv 51 \pmod{157}$$

$$b_3 = 6 \times 17 = 102 \equiv 102 \pmod{157}$$

$$b_4 = 13 \times 17 = 221, \quad 221 - 157 = 64, \quad \equiv 64 \pmod{157}$$

$$b_5 = 29 \times 17 = 493, \quad 493 - 3 \times 157 = 493 - 471 = 22, \quad \equiv 22 \pmod{157}$$

$$b_6 = 61 \times 17 = 1037, \quad 1037 - 6 \times 157 = 1037 - 942 = 95, \quad \equiv 95 \pmod{157}$$

可知一般背包序列为：

$$\{34, 51, 102, 64, 22, 95\}$$

Step 2: 加密消息。

消息分为两组: 110011 和 010110。

第一组 110011: 比特位对应 $\{b_1, b_2, b_3, b_4, b_5, b_6\}$, 第 1、2、5、6 位为 1, 其余为 0。

$$C_1 = b_1 + b_2 + b_5 + b_6 = 34 + 51 + 22 + 95 = 202$$

110011 的密文为202

第二组 010110: 第 2、4、5 位为 1。

$$C_2 = b_2 + b_4 + b_5 = 51 + 64 + 22 = 137$$

010110 的密文为137

Step 3: 解密过程 (完整推导)。

首先计算 $w^{-1} \pmod{157}$, 即求解 $17x \equiv 1 \pmod{157}$ 。

使用扩展欧几里得算法:

$$157 = 17 \times 9 + 4$$

$$17 = 4 \times 4 + 1$$

$$4 = 1 \times 4 + 0$$

回代:

$$1 = 17 - 4 \times 4$$

$$= 17 - 4 \times (157 - 17 \times 9)$$

$$= 17 - 4 \times 157 + 36 \times 17$$

$$= 37 \times 17 - 4 \times 157$$

故 $17^{-1} \equiv 37 \pmod{157}$ 。

解密第一组密文 $C_1 = 202$:

$$C'_1 = 202 \times 37 \pmod{157}$$

计算: $202 \times 37 = 7474$, $157 \times 47 = 7379$, $7474 - 7379 = 95$ 。故 $C'_1 = 95$ 。

现在对超递增序列 $\{2, 3, 6, 13, 29, 61\}$ 用贪心算法分解 95:

- $95 \geq 61$, 包含第 6 位 (1), 剩余 $95 - 61 = 34$
- $34 \geq 29$, 包含第 5 位 (1), 剩余 $34 - 29 = 5$
- $5 < 13$, 第 4 位为 0, 剩余不变
- $5 < 6$, 第 3 位为 0, 剩余不变
- $5 \geq 3$, 包含第 2 位 (1), 剩余 $5 - 3 = 2$
- $2 \geq 2$, 包含第 1 位 (1), 剩余 $2 - 2 = 0$

因此 $95 = 61 + 29 + 3 + 2$, 对应比特位 (从高位到低位) 为 110011。验证正确!

解密第二组密文 $C_2 = 137$:

$$C'_2 = 137 \times 37 \pmod{157}$$

计算: $137 \times 37 = 5069$, $157 \times 32 = 5024$, $5069 - 5024 = 45$ 。故 $C'_2 = 45$ 。

贪心算法分解 45:

- $45 \geq 29$, 包含第 5 位 (1), 剩余 $45 - 29 = 16$
- $16 \geq 13$, 包含第 4 位 (1), 剩余 $16 - 13 = 3$
- $3 < 6$, 第 3 位为 0
- $3 \geq 3$, 包含第 2 位 (1), 剩余 $3 - 3 = 0$
- 第 1 位为 0

因此 $45 = 29 + 13 + 3$, 对应比特位为 010110。验证正确!

扩展：背包密码的安全性

Merkle-Hellman 背包密码体制曾被寄予厚望,但在 1984 年 Shamir 成功破解了单迭代的背包密码体制,此后 Brickell 等人又攻破了多迭代版本。核心问题在于:虽然一般的子集和问题是 NP 完全的,但通过模乘变换生成的”陷门”背包序列并非均匀随机分布,存在可利用的结构性弱点。这启示我们:基于 NP 完全问题的密码体制必须格外小心——**平均情况的困难性**才是密码安全性的关键,而非最坏情况的复杂性。

当前,基于格的密码学(Lattice-based Cryptography)部分延续了背包密码的思想,Learning With Errors (LWE) 等问题具有证明可归约到格上最坏情况困难问题的安全性,成为后量子密码的重要候选方向。

4.3 RSA 加密与解密 (10 分)

知识点回顾: RSA 密码体制

RSA 算法由 Rivest、Shamir 和 Adleman 于 1978 年提出,是最经典、应用最广泛的公钥密码体制,其安全性基于大整数因子分解的困难性。

密钥生成:

1. 随机选取两个大素数 p 和 q , 计算 $n = pq$
2. 计算欧拉函数 $\phi(n) = (p-1)(q-1)$
3. 选择加密指数 e , 满足 $1 < e < \phi(n)$ 且 $\gcd(e, \phi(n)) = 1$
4. 计算解密指数 d , 满足 $ed \equiv 1 \pmod{\phi(n)}$, 即 $d \equiv e^{-1} \pmod{\phi(n)}$
5. 公钥为 (n, e) , 私钥为 (p, q, d)

加密: $C = M^e \pmod{n}$

解密: $M = C^d \pmod{n}$

正确性依据: 欧拉定理。若 $\gcd(M, n) = 1$, 则 $M^{\phi(n)} \equiv 1 \pmod{n}$, 于是:

$$C^d \equiv (M^e)^d = M^{ed} = M^{1+k\phi(n)} = M \cdot (M^{\phi(n)})^k \equiv M \pmod{n}$$

若 $\gcd(M, n) \neq 1$ (即 M 是 p 或 q 的倍数), 由 CRT 也可证明正确性。

快速模幂运算: 使用平方-乘算法, 将指数表示为二进制后迭代计算, 复杂度为 $O(\log e)$ 次模乘。

密钥长度: 实际应用中, p 和 q 至少应为 1024 位 (约 309 位十进制), n 为 2048 位, 以抵抗现有的因子分解算法。

解: 详细计算过程

Step 1: 加密明文 $M_1 = 11$ 。

已知公钥 $n = 91$, $e = 5$, 明文 $M_1 = 11$ 。

计算 $C_1 = 11^5 \pmod{91}$ 。使用平方-乘算法:

$$11^1 \equiv 11 \pmod{91}$$

$$11^2 = 121 \equiv 121 - 91 = 30 \pmod{91}$$

$$11^4 = 30^2 = 900 \equiv 900 - 9 \times 91 = 900 - 819 = 81 \pmod{91}$$

$$11^5 = 11^4 \times 11 \equiv 81 \times 11 = 891 \equiv 891 - 9 \times 91 = 891 - 819 = 72 \pmod{91}$$

或者用更直接的拆解:

$$11^5 \equiv 11 \times 11^4 \equiv 11 \times (11^2)^2 \equiv 11 \times 30^2 \equiv 11 \times 900 \equiv 11 \times (-10) \equiv -110 \equiv 72 \pmod{91}$$

(因为 $900 \bmod 91 = 900 - 9 \times 91 = 81$ 不直接得 -10 , 需要小心。严格计算: $121 \equiv 30$, $30^2 = 900$, $91 \times 9 = 819$, $900 - 819 = 81$, $81 \times 11 = 891$, $91 \times 9 = 819$, $891 - 819 = 72$ 。)

$C_1 = 72$

Step 2: 破译 RSA——通过分解 n 获取私钥。

分解 $n = 91$: 显然 $91 = 7 \times 13$, 故 $p = 7$, $q = 13$ 。

计算欧拉函数:

$$\phi(91) = (7 - 1)(13 - 1) = 6 \times 12 = 72$$

求解密指数 d , 满足 $5d \equiv 1 \pmod{72}$ 。使用扩展欧几里得算法:

$$72 = 5 \times 14 + 2$$

$$5 = 2 \times 2 + 1$$

$$2 = 1 \times 2 + 0$$

回代:

$$\begin{aligned} 1 &= 5 - 2 \times 2 \\ &= 5 - 2 \times (72 - 5 \times 14) \\ &= 5 - 2 \times 72 + 28 \times 5 \\ &= 29 \times 5 - 2 \times 72 \end{aligned}$$

故 $5^{-1} \equiv 29 \pmod{72}$, 即私钥 $d = 29$ 。

Step 3: 解密密文 $C_2 = 6$ 。

计算 $M_2 = 6^{29} \pmod{91}$ 。使用平方-乘算法, 先将 29 写成二进制: $29_{10} = 11101_2$ 。

逐步计算:

$$\begin{aligned} 6^1 &\equiv 6 \pmod{91} \\ 6^2 &= 36 \pmod{91} \\ 6^3 &= 6^2 \times 6 \equiv 36 \times 6 = 216 \equiv 216 - 2 \times 91 = 34 \pmod{91} \\ 6^4 &= 6^2 \times 6^2 \equiv 36 \times 36 = 1296 \end{aligned}$$

$91 \times 14 = 1274$, $1296 - 1274 = 22$ 。故 $6^4 \equiv 22 \pmod{91}$ 。

继续:

$$\begin{aligned} 6^7 &= 6^4 \times 6^3 \equiv 22 \times 34 = 748, & 91 \times 8 &= 728, & 748 - 728 &= 20 \pmod{91} \\ 6^8 &= 6^4 \times 6^4 \equiv 22 \times 22 = 484, & 91 \times 5 &= 455, & 484 - 455 &= 29 \pmod{91} \\ 6^{14} &= 6^7 \times 6^7 \equiv 20 \times 20 = 400, & 91 \times 4 &= 364, & 400 - 364 &= 36 \pmod{91} \\ 6^{16} &= 6^8 \times 6^8 \equiv 29 \times 29 = 841, & 91 \times 9 &= 819, & 841 - 819 &= 22 \pmod{91} \\ 6^{28} &= 6^{14} \times 6^{14} \equiv 36 \times 36 = 1296 \equiv 22 \pmod{91} \\ 6^{29} &= 6^{28} \times 6 \equiv 22 \times 6 = 132 \equiv 132 - 91 = 41 \pmod{91} \end{aligned}$$

$$\boxed{M_2 = 41}$$

Step 4: 验证解密正确性。

加密 41 验证: $41^5 \pmod{91}$ 。

$$41^2 = 1681 \equiv 1681 - 18 \times 91 = 1681 - 1638 = 43 \pmod{91}$$

$$41^4 = 43^2 = 1849 \equiv 1849 - 20 \times 91 = 1849 - 1820 = 29 \pmod{91}$$

$$41^5 = 41^4 \times 41 \equiv 29 \times 41 = 1189 \equiv 1189 - 13 \times 91 = 1189 - 1183 = 6 \pmod{91}$$

解密成功! $41^5 \equiv 6 \pmod{91}$, 正是 C_2 。

总结:

加密: $M_1 = 11 \rightarrow C_1 = 72$	解密: $C_2 = 6 \rightarrow M_2 = 41$
-------------------------------------	------------------------------------

扩展: RSA 的安全性考量

- (1) 实际使用中 n 为 2048 位或更长, 分解极其困难。已知最快的通用因子分解算法——数域筛法 (Number Field Sieve, NFS) 的渐进复杂度约为 $O(\exp((\sqrt[3]{64/9} + o(1))(\ln n)^{1/3}(\ln \ln n)^{2/3}))$, 为亚指数级。
- (2) 选择 p 和 q 时需满足: 二者大小相近 (抵抗椭圆曲线分解)、 $p-1$ 和 $q-1$ 含大素因子 (抵抗 Pollard $p-1$ 分解)、 $|p-q|$ 足够大 (抵抗 Fermat 分解)。
- (3) 若多个用户共用同一个 n 但不同的 e 和 d , 则任意一个用户可分解 n 而泄露全局私钥, 故 n 不可共享。
- (4) 对 RSA 的直接攻击还包括: 共模攻击、低加密指数攻击 (Hastad 广播攻击)、低解密指数攻击 (Wiener 攻击)、循环攻击、计时攻击等, 每种攻击都有对应的防范措施 (如 OAEP 填充)。

4.4 AES 列混合运算 (10 分)

知识点回顾: AES 的有限域运算与 MixColumns

AES (Advanced Encryption Standard) 是分组长度 128 位的迭代型对称密码, 其轮函数包含四个变换: 字节代换 (SubBytes)、行移位 (ShiftRows)、列混合 (MixColumns) 和轮密钥加 (AddRoundKey)。

有限域 $GF(2^8)$: AES 使用的有限域为 $GF(2^8) = \mathbb{F}_2[x]/(m(x))$, 其中不可约多项式为:

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

域中的元素是次数不超过 7 的二元系数多项式, 即字节 (8 位二进制数)。例如字节 b8 对应:

$$b8_{16} = 10111000_2 \longleftrightarrow x^7 + x^5 + x^4 + x^3$$

域运算:

- **加法 ($\oplus$):** 多项式系数模 2 加法, 即按位异或 (XOR)。例如: $03 \oplus 01 = 02$ 。
- **乘法 ($\otimes$):** 多项式相乘后对 $m(x)$ 取模。特别地, 乘以 02 (即 x) 等价于左移一位后, 若最高位 (b7) 为 1 则异或 1b (即 $m(x)$ 的低 8 位)。乘以 03 (即 $x+1$) 等于先乘 x 再异或自身。

MixColumns 变换: MixColumns 对状态矩阵的每一列作用如下矩阵乘法 (在 $GF(2^8)$ 上):

$$\begin{pmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{pmatrix}$$

本题计算的就是其中某个输出字节的值。

解: 详细计算过程

目标: 在 $GF(2^8)$ 上计算 $03 \cdot b8 \oplus 01 \cdot 41 \oplus 01 \cdot 11 \oplus 02 \cdot f1$ 。

Step 1: 将各字节转换为多项式表示。

$$b8 = 1011\ 1000_2 \longleftrightarrow x^7 + x^5 + x^4 + x^3$$

$$41 = 0100\ 0001_2 \longleftrightarrow x^6 + 1$$

$$11 = 0001\ 0001_2 \longleftrightarrow x^4 + 1$$

$$f1 = 1111\ 0001_2 \longleftrightarrow x^7 + x^6 + x^5 + x^4 + 1$$

Step 2: 计算各项乘法。

(1) $01 \cdot 41 = 41 = 0100\ 0001_2$ (乘以 1 等于自身)

(2) $01 \cdot 11 = 11 = 0001\ 0001_2$ (乘以 1 等于自身)

(3) $02 \cdot f1$ —— 乘以 x (左移一位, 若 b7=1 则异或 1b):

$f1 = 1111\ 0001_2$ 。左移一位得 $1\ 1110\ 0010_2$ ，但 $b7$ 原为 1，故需要异或 $1b$ (即 $0001\ 1011_2$):

$$\begin{aligned} & 1\ 1110\ 0010_2 \oplus 0001\ 1011_2 \\ &= 1\ 1111\ 1001_2 \end{aligned}$$

但结果只有 8 位，最高位的进位 1 舍弃 (这是在 $GF(2^8)$ 中自动取模的结果)，故:

$$02 \cdot f1 = 1111\ 1001_2 = f9$$

验证: 多项式方法—— $f1 \longleftrightarrow x^7 + x^6 + x^5 + x^4 + 1$ 。乘 $x: x^8 + x^7 + x^6 + x^5 + x$ 。模 $m(x) = x^8 + x^4 + x^3 + x + 1$:

$$\begin{aligned} & (x^8 + x^7 + x^6 + x^5 + x) - (x^8 + x^4 + x^3 + x + 1) \\ &= x^7 + x^6 + x^5 + x^4 + x^3 + 1 \end{aligned}$$

即 $1111\ 1001_2 = f9$ 。一致。

(4) $03 \cdot b8$ —— $03 = 02 \oplus 01$ ，所以:

$$03 \cdot b8 = (02 \cdot b8) \oplus b8$$

先算 $02 \cdot b8$ 。 $b8 = 1011\ 1000_2$ ，左移一位得 $1\ 0111\ 0000_2$ ， $b7=1$ 故异或 $1b$:

$$1\ 0111\ 0000_2 \oplus 0001\ 1011_2 = 1\ 0110\ 1011_2 \rightarrow 0110\ 1011_2 = 6b$$

所以 $02 \cdot b8 = 0110\ 1011_2$ 。

再异或 $b8$:

$$03 \cdot b8 = 0110\ 1011_2 \oplus 1011\ 1000_2 = 1101\ 0011_2 = d3$$

Step 3: 四项求和 (异或)。

$03 \cdot b8$	:	1101	0011
$01 \cdot 41$	:	0100	0001
$01 \cdot 11$	:	0001	0001
$02 \cdot f1$	:	1111	1001
XOR 结果	:	0111	1010

计算过程 (逐列异或):

- 第 1 列: $1 \oplus 0 \oplus 0 \oplus 1 = 0$ (模 2 加)
- 第 2 列: $1 \oplus 1 \oplus 0 \oplus 1 = 1$ (偶数个 1 得 0, 奇数个 1 得 1)
- 第 3 列: $0 \oplus 0 \oplus 0 \oplus 1 = 1$
- 第 4 列: $1 \oplus 0 \oplus 0 \oplus 1 = 0$
- 第 5 列: $0 \oplus 0 \oplus 0 \oplus 1 = 1$
- 第 6 列: $0 \oplus 0 \oplus 0 \oplus 0 = 0$
- 第 7 列: $1 \oplus 0 \oplus 0 \oplus 0 = 1$ (第 3 和第 4 项第 7 列为 0, 第 1 项为 1, 第 2 项为 0)
- 第 8 列: $1 \oplus 1 \oplus 1 \oplus 1 = 0$

或者直接按字节异或：

$$d3 \oplus 41 \oplus 11 \oplus f9 = 7a$$

$$03 \cdot b8 \oplus 01 \cdot 41 \oplus 01 \cdot 11 \oplus 02 \cdot f1 = 7a$$

即二进制 $0111\ 1010_2$ ，对应多项式 $x^6 + x^5 + x^4 + x^3 + x$ 。

扩展：MixColumns 的设计原理

MixColumns 的设计遵循宽轨道策略 (Wide Trail Strategy)，刻意选取了具有最大扩散能力的 MDS (Maximum Distance Separable) 矩阵。系数 01、02、03 的选取兼顾了：

- 计算效率：只需移位和异或即可实现乘法；
- 扩散性：每列 4 字节中任意 1 字节的改变最多影响输出列的 4 字节（完全扩散）；
- 分支数：MixColumns 的线性分支数为 5——即输入列和输出列的字节级非零个数之和至少为 5，这是 4×4 MDS 矩阵中的最优值，确保 AES 对差分 and 线性密码分析具有强抵抗力。

注意：最后一轮 AES 中省略 MixColumns，使加解密结构对称并减少不必要的计算开销。

4.5 椭圆曲线点乘运算 (10 分)

知识点回顾：椭圆曲线密码 (ECC)

椭圆曲线密码学 (Elliptic Curve Cryptography, ECC) 由 Koblitz 和 Miller 于 1985 年分别独立提出, 使用椭圆曲线上的点群代替乘法群来构造公钥密码。在同等安全强度下, ECC 密钥长度远短于 RSA (例如 256 位 ECC 安全性约等于 3072 位 RSA)。

椭圆曲线定义 (Weierstrass 形式): 在素域 $\mathbb{F}_p (p > 3)$ 上, 椭圆曲线由方程 $y^2 = x^3 + ax + b \pmod{p}$ 定义, 其中判别式 $\Delta = -16(4a^3 + 27b^2) \not\equiv 0 \pmod{p}$ 。曲线上所有点加上无穷远点 $\mathcal{O}$ 构成一个阿贝尔群。

群运算 (点加法): 设 $P = (x_1, y_1)$, $Q = (x_2, y_2)$, $P_3 = P + Q = (x_3, y_3) \neq \mathcal{O}$:

1. 若 $x_1 \neq x_2$ (普通加法):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}$$

2. 若 $x_1 = x_2$ 且 $Q \neq -P$ (倍点运算, $P = Q$):

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}$$

然后:

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$$

其中分数除法理解为模 p 下乘以分母的逆元。

标量乘法: $kP = \underbrace{P + P + \cdots + P}_{k\text{次}}$, 可使用倍加算法高效计算, 复杂度为 $O(\log k)$ 。

title

Step 0: 验证 P 确实在曲线上。

$$y^2 = 9^2 = 81 \equiv 81 - 7 \times 11 = 81 - 77 = 4 \pmod{11}$$

$$x^3 + x + 6 = 7^3 + 7 + 6 = 343 + 7 + 6 = 356$$

$356 \bmod 11$: $11 \times 32 = 352$, $356 - 352 = 4$ 。确实 $4 \equiv 4$, P 在曲线上。✓

这里的椭圆曲线为: $a = 1$, $b = 6$, $p = 11$ 。

Step 1: 计算 $2P = P + P$ 。

使用倍点公式。 $x_1 = 7, y_1 = 9$ 。

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{3 \times 7^2 + 1}{2 \times 9} \pmod{11}$$

计算分子: $3 \times 49 + 1 = 147 + 1 = 148$ 。 $148 \bmod 11$: $11 \times 13 = 143$, $148 - 143 = 5$ 。

计算分母: $2 \times 9 = 18 \equiv 18 - 11 = 7 \pmod{11}$ 。

分母的逆元: $7^{-1} \pmod{11}$ 。因为 $7 \times 8 = 56 \equiv 56 - 5 \times 11 = 1 \pmod{11}$, 故 $7^{-1} \equiv 8 \pmod{11}$ 。

因此:

$$\lambda = 5 \times 8 = 40 \equiv 40 - 3 \times 11 = 7 \pmod{11}$$

计算 x_3 和 y_3 :

$$x_3 = \lambda^2 - x_1 - x_2 = 7^2 - 7 - 7 = 49 - 14 = 35 \equiv 35 - 3 \times 11 = 2 \pmod{11}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 7 \times (7 - 2) - 9 = 7 \times 5 - 9 = 35 - 9 = 26 \equiv 26 - 2 \times 11 = 4 \pmod{11}$$

故 $2P = (2, 4)$ 。

Step 2: 计算 $3P = P + 2P = (7, 9) + (2, 4)$ 。

此时 $x_1 = 7 \neq x_2 = 2$, 使用普通加法公式。

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} = \frac{4 - 9}{2 - 7} = \frac{-5}{-5} \pmod{11}$$

$-5 \equiv 6 \pmod{11}$, 所以 $\lambda = 6 \times 6^{-1} \pmod{11}$ 。

$6^{-1} \pmod{11}$: 因为 $6 \times 2 = 12 \equiv 1 \pmod{11}$, 故 $6^{-1} \equiv 2 \pmod{11}$ 。

$\lambda = 6 \times 2 = 12 \equiv 1 \pmod{11}$ 。

更方便的方法: 分子和分母相同, $\lambda = (-5)/(-5) = 1 \pmod{11}$ (因为分母 $2 - 7 = -5$ 与分子 $4 - 9 = -5$ 相等, 直接约分即可)。

计算 x_3 和 y_3 :

$$x_3 = \lambda^2 - x_1 - x_2 = 1^2 - 7 - 2 = 1 - 9 = -8 \equiv -8 + 11 = 3 \pmod{11}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 1 \times (7 - 3) - 9 = 4 - 9 = -5 \equiv -5 + 11 = 6 \pmod{11}$$

故 $3P = (3, 6)$ 。

Step 3: 验证结果。 验证 $(3, 6)$ 是否在曲线上:

$$y^2 = 6^2 = 36 \equiv 36 - 3 \times 11 = 3 \pmod{11}$$

$$x^3 + x + 6 = 27 + 3 + 6 = 36 \equiv 3 \pmod{11}$$

成立。✓

$$\boxed{3P = (3, 6)}$$

扩展: ECC 中的安全参数与攻击

(1) **Pollard's Rho 算法**是目前对一般椭圆曲线离散对数问题 (ECDLP) 最有效的攻击, 时间复杂度约为 $O(\sqrt{n})$, 其中 n 为基点 P 的阶。因此, 为达到 128 位安全强度, 需选择使 $n \approx 2^{256}$ 的曲线 (如 secp256k1、Curve25519)。

(2) **异常曲线攻击**: 若 $\#E(\mathbb{F}_p) = p$ (即 trace of Frobenius = 1), 则 ECDLP 可在多项式时间内解决。选择曲线时应避开此类异常曲线。

(3) **SSSA 攻击** (Smart-Satoh-Skjernaa-Araki 攻击): 对超奇异椭圆曲线 (p 整除 $\#E(\mathbb{F}_p)$)—

$p-1$), 可将 ECDLP 归约到 $\mathbb{F}_{p^k}$ 上的离散对数问题, 其中嵌入度 k 较小 (通常 $k \leq 6$), 使 MOV/FR 攻击可行。实际应用中应选择非超奇异的普通椭圆曲线。

(4) ECC 广泛应用于密钥交换 (ECDH)、数字签名 (ECDSA、EdDSA) 和加密 (ECIES)。比特币使用 secp256k1 曲线, Signal/WhatsApp 等使用 Curve25519。

4.6 DES 的 S 盒代替运算 (10 分)

知识点回顾：DES 与 S 盒

DES (Data Encryption Standard) 于 1977 年被采纳为美国联邦标准, 是分组密码发展史上的里程碑。DES 使用 64 位分组和 56 位有效密钥, 采用 Feistel 网络结构, 共 16 轮迭代。

S 盒 (Substitution Box) 是 DES 中唯一的非线性组件, 提供了密码学强度的“混淆” (Confusion)。DES 共有 8 个不同的 S 盒: $S_1, S_2, \dots, S_8$, 每个 S 盒将 6 位输入映射为 4 位输出。

S 盒查找规则: 给定 6 位输入 $b_1b_2b_3b_4b_5b_6$:

1. **行号:** 由首尾两位 b_1b_6 组成 (取值范围 0 ~ 3, 即 4 行)
2. **列号:** 由中间四位 $b_2b_3b_4b_5$ 组成 (取值范围 0 ~ 15, 即 16 列)
3. 查表得到 S 盒中对应位置的 4 位二进制值 (0 ~ 15), 即为输出

例如, 输入 101110 进入 S_1 : 行号 = $10_2 = 2$, 列号 = $0111_2 = 7$, 查 S_1 表第 2 行第 7 列的值为 11, 输出 1011。

F 函数结构: 32 位右半部分经扩展置换 E 扩展为 48 位, 与 48 位子密钥异或后, 分成 8 组各 6 位送入 8 个 S 盒, S 盒输出 32 位, 再经 P 置换得到最终输出。

解：逐 S 盒查表

48 位输入分组如下 (每组 6 位, 分别送入 $S_1 \sim S_8$):

$$\underbrace{101110}_{S_1} \underbrace{010011}_{S_2} \underbrace{101111}_{S_3} \underbrace{110101}_{S_4} \underbrace{010101}_{S_5} \underbrace{101010}_{S_6} \underbrace{101111}_{S_7} \underbrace{101100}_{S_8}$$

下面使用 DES 标准 S 盒表逐一查找:

DES 标准 S 盒 (S_1 至 S_8) (行索引 0~3, 列索引 0~15):

S_1 :																
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

其余 S 盒表结构类似, 此处直接给出查表过程。

计算各 S 盒的输出:

S 盒	6 位输入	行号	行号 (二进制 → 十进制)	列号	列号 (十进制)	输出值 (十进制)
S_1	101110	$b_1b_6 = 10$	2	$b_2b_3b_4b_5 = 0111$	7	查表得
S_2	010011	$b_1b_6 = 01$	1	$b_2b_3b_4b_5 = 1001$	9	查表得
S_3	101111	$b_1b_6 = 11$	3	$b_2b_3b_4b_5 = 0111$	7	查表得
S_4	110101	$b_1b_6 = 11$	3	$b_2b_3b_4b_5 = 1010$	10	查表得
S_5	010101	$b_1b_6 = 01$	1	$b_2b_3b_4b_5 = 1010$	10	查表得
S_6	101010	$b_1b_6 = 10$	2	$b_2b_3b_4b_5 = 0101$	5	查表得
S_7	101111	$b_1b_6 = 11$	3	$b_2b_3b_4b_5 = 0111$	7	查表得
S_8	101100	$b_1b_6 = 10$	2	$b_2b_3b_4b_5 = 0110$	6	查表得

将各输出转换为 4 位二进制：

- S_1 : $11_{10} = 1011_2$
- S_2 : $0_{10} = 0000_2$
- S_3 : $7_{10} = 0111_2$
- S_4 : $5_{10} = 0101_2$
- S_5 : $15_{10} = 1111_2$
- S_6 : $8_{10} = 1000_2$
- S_7 : $7_{10} = 0111_2$
- S_8 : $14_{10} = 1110_2$

拼接得到 32 位输出：

1011 0000 0111 0101 1111 1000 0111 1110

用十六进制表示为：B0 75 F8 7E。

扩展：S 盒的设计准则与安全性

DES 的 S 盒设计基于以下准则（最初由 NSA 协助设计，一度引发后门争议，但后来 Biham 和 Shamir 发现这些 S 盒反而对差分密码分析具有最优抵抗性）：

1. **非线性性**：S 盒具有高非线性度，防止线性密码分析；
2. **差分均匀性**：对任意非零输入差分，输出差分的分布尽可能均匀，抵抗差分密码分析；
3. **严格雪崩准则 (SAC)**：任意一位输入翻转导致输出平均一半比特翻转；
4. **位独立准则 (BIC)**：任意一位输入翻转导致的输出比特翻转模式相互独立；
5. **双射性 (Bijection)**：S 盒是可逆的（虽然 DES 整体不需要 S 盒可逆，但这保证了随机性）；
6. **无不动点**： $S(x) \neq x$ ，且 $S(x) \neq \bar{x}$ 。

DES 的 S 盒经受了数十年的公开密码分析考验，至今未发现严重的设计缺陷。S 盒的强度也是 AES 选择 8 位 S 盒并进行更透彻数学分析（基于 $GF(2^8)$ 逆映射）的重要原因。

4.7 最大公约数 (5 分)

知识点回顾

最大公约数 (Greatest Common Divisor, GCD) 是数论中最基本的概念之一, 在密码学中应用极广: RSA 密钥生成时需要 $\gcd(e, \phi(n)) = 1$ 来确保 e 可逆; 求解模逆元依赖于扩展欧几里得算法; 中国剩余定理的应用也需要 GCD 计算。

解

最大公约数: 设 a 和 b 是两个不全为零的整数。若整数 d 满足:

1. $d \mid a$ (d 整除 a) 且 $d \mid b$ (d 整除 b), 即 d 是 a 和 b 的公约数;
2. 对任意公约数 c (满足 $c \mid a$ 且 $c \mid b$), 均有 $c \mid d$, 即 d 是公约数中最大的。

则称 d 为 a 和 b 的最大公约数, 记作 $\gcd(a, b) = d$ 。

自然数情形: 若 a 和 b 均为自然数, 最大公约数即为所有公约数中的最大者。例如 $\gcd(24, 36) = 12$ 。

扩展: 欧几里得算法。 计算 $\gcd(a, b)$ 的高效算法基于带余除法:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

递归直到余数为 0, 此时的除数即为最大公约数。复杂度为 $O(\log \min(a, b))$ 。

扩展欧几里得算法: 不仅求出 $\gcd(a, b)$, 还求出整数 s 和 t 使得:

$$sa + tb = \gcd(a, b)$$

这在密码学中用于计算模逆元: 若 $\gcd(a, n) = 1$, 则 $s \equiv a^{-1} \pmod{n}$ 。

4.8 仿射密码系统 (5 分)

知识点回顾

仿射密码 (Affine Cipher) 是一种经典的单表代换密码, 可以看作是移位密码 (凯撒密码) 和乘数密码的组合。其数学基础是模 n 下的线性函数, 加密和解密均使用一次仿射变换。

解

仿射密码系统定义如下:

设明文空间和密文空间均为 $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$ (通常 $n = 26$ 对应英文字母表)。

密钥空间:

$$\mathcal{K} = \{(k_0, k_1) \mid k_0, k_1 \in \mathbb{Z}_n, \gcd(k_1, n) = 1\}$$

其中 $\gcd(k_1, n) = 1$ 确保了加密密钥 k_1 在模 n 下存在乘法逆元, 从而加密函数可逆。

加密算法：对于明文 $x \in \mathbb{Z}_n$,

$$E_k(x) = (k_1 \cdot x + k_0) \pmod{n}$$

解密算法：对于密文 $y \in \mathbb{Z}_n$,

$$D_k(y) = k_1^{-1} \cdot (y - k_0) \pmod{n}$$

其中 k_1^{-1} 是 k_1 在模 n 下的乘法逆元。

参数说明：

- k_1 为乘法密钥（缩放因子），必须有乘法逆元；
- k_0 为加法密钥（平移量）；
- 当 $k_1 = 1$ 时退化为移位密码（凯撒密码）；
- 当 $k_0 = 0$ 时退化为乘数密码。

安全性：密钥空间大小为 $n \cdot \phi(n)$ （其中 $\phi(n)$ 是欧拉函数）。对 $n = 26$, $\phi(26) = 12$, 共有 $12 \times 26 = 312$ 种密钥，可被暴力穷举轻易攻破。已知明文攻击下更是只需两对明-密文即可求解 k_0 和 k_1 （解二元一次模方程）。

4.9 拟素数 (5 分)

知识点回顾

拟素数（也称伪素数、假素数）是密码学中素性检测的理论基础。在实际生成大素数（如 RSA 密钥生成）时，使用概率素性检测（如 Miller-Rabin 测试）可以高效判断一个数“很可能”是素数，而这些测试的可靠性建立在拟素数性质之上。

解

拟素数 (Pseudoprime / Fermat Pseudoprime)：

设 n 是一个奇合数，若存在整数 b 满足 $\gcd(b, n) = 1$ (b 与 n 互素)，且成立：

$$b^{n-1} \equiv 1 \pmod{n}$$

则称 n 为对于基 b 的拟素数（或 Fermat 拟素数）。

内涵解释：

- 由费马小定理可知，若 n 为素数且 $\gcd(b, n) = 1$ ，则必有 $b^{n-1} \equiv 1 \pmod{n}$ 恒成立。
- 拟素数 n 虽然通过了基 b 的费马测试，但它实质上是合数——也就是说，费马小定理的逆命题并不成立。
- 若一个合数 n 对所有满足 $\gcd(b, n) = 1$ 的基 b 都满足 $b^{n-1} \equiv 1 \pmod{n}$ ，则称 n 为 **Carmichael 数**（绝对拟素数），例如 561、1105、1729 等。

密码学意义：素性检测（如 Miller-Rabin 测试）通过多个随机基进行测试。若一个数对

所有测试基都通过, 则它极可能是素数 (错误概率小于 4^{-k} , k 为测试轮数)。拟素数的存在性是概率素性检测不完美的根本原因, 但在实际安全参数下, 通过足够多轮测试可达到可忽略的错误率。

4.10 ElGamal 签名方案 (5 分)

知识点回顾

ElGamal 签名方案由 Taher ElGamal 于 1985 年提出, 是第一个基于离散对数问题(DLP)的数字签名方案。它是后续 DSA (Digital Signature Algorithm) 和 ECDSA 的基础。与 RSA 签名不同, ElGamal 签名是随机化的——同一消息每次签名结果不同, 这是其重要特性。

解

ElGamal 签名方案定义如下:

系统参数: 设 p 是一个大素数, $\alpha \in \mathbb{Z}_p^*$ 是本原元 (生成元)。令明文空间 $P = \mathbb{Z}_p^*$, 签名空间 $A = \mathbb{Z}_p^* \times \mathbb{Z}_{p-1}$ 。

密钥生成:

$$\mathcal{K} = \{(p, \alpha, \beta, a) \mid \beta \equiv \alpha^a \pmod{p}\}$$

其中:

- 公钥: (p, α, β)
- 私钥: a (满足 $0 < a < p-1$)

签名算法 $\text{sig}_K(x, k)$: 对消息 $x \in \mathbb{Z}_p^*$, 签名者随机选取 $k \in \mathbb{Z}_{p-1}^*$ (需满足 $\gcd(k, p-1) = 1$), 计算:

$$\gamma \equiv \alpha^k \pmod{p}, \quad \delta \equiv (x - a\gamma) \cdot k^{-1} \pmod{p-1}$$

签名为 $(\gamma, \delta) \in \mathbb{Z}_p^* \times \mathbb{Z}_{p-1}$ 。

验证算法 $\text{ver}(x, \gamma, \delta)$:

$$\text{ver}(x, \gamma, \delta) = \text{true} \iff \beta^\gamma \cdot \gamma^\delta \equiv \alpha^x \pmod{p}$$

正确性证明: 若签名合法, 则 $\delta \equiv (x - a\gamma)k^{-1} \pmod{p-1}$, 故 $x \equiv k\delta + a\gamma \pmod{p-1}$ 。于是:

$$\alpha^x \equiv \alpha^{k\delta + a\gamma} \equiv (\alpha^k)^\delta \cdot (\alpha^a)^\gamma \equiv \gamma^\delta \cdot \beta^\gamma \pmod{p}$$

验证等式成立。

安全性要点: ElGamal 签名的安全性基于离散对数问题的困难性。关键要求包括: (1) k 必须随机且不同消息间不能重复使用 (否则私钥 a 可被恢复); (2) p 应足够大 (至少 2048 位); (3) 必须使用哈希值签名 ($\text{sig}_K(H(x), k)$) 以防存在性伪造攻击。

4.11 密钥管理的内容 (10 分)

知识点回顾：密码系统中密钥管理的核心地位

在密码学中有句名言：“密码系统的安全性取决于密钥的安全性，而非算法的保密性”（Kerckhoffs 原则）。密钥管理（Key Management）是密码系统中最复杂、最关键的环节，涵盖了密钥从生成到销毁的整个生命周期的所有操作和策略。一个设计精良的密码算法，如果密钥管理不当，其安全性将荡然无存。

解：密钥管理的十二项核心内容

- 1. 密钥产生 (Key Generation)** 密钥必须通过安全的随机数发生器生成。产生方法包括：基于硬件的真随机数发生器（TRNG，利用物理噪声源如热噪声、振荡器抖动等）、基于密码学的伪随机数发生器（PRNG，如基于 HMAC-DRBG、CTR-DRBG 的确定性随机位生成器）。密钥产生必须确保足够的熵和不可预测性，避免使用可预测的种子或弱随机源。
- 2. 密钥的装入 (Key Loading)** 将生成的密钥安全地注入密码设备或系统的过程。装入方式包括：手动输入（如 PIN 码、口令）、电子注入（通过专用安全通道）、预置（出厂预装密钥）、密钥推导（从主密钥派生）。装入过程需防止泄露和篡改。
- 3. 密钥的保护 (Key Protection)** 贯穿密钥整个生命周期的安全防护措施，包括：加密存储（用 KEK 加密数据密钥）、访问控制（仅授权实体可访问）、防篡改硬件（HSM/智能卡）、完整性校验（MAC/数字签名防篡改）。密钥在传输时必须使用独立的受保护信道。
- 4. 密钥类型 (Key Types)** 根据用途和使用阶段对密钥分类。常见分类：主密钥（Master Key）、密钥加密密钥（KEK, Key Encryption Key）、数据加密密钥（DEK, Data Encryption Key）、会话密钥（Session Key）、签名密钥（Signing Key）、认证密钥。不同用途的密钥严格隔离，禁止混用。
- 5. 密钥存储 (Key Storage)** 将密钥安全持久化的机制。公钥可明文存储（配合证书和数字签名保证完整性），私钥和对称密钥必须加密存储。存储介质包括：加密文件系统、硬件安全模块（HSM）、智能卡、USB Token、可信平台模块（TPM）、云密钥管理服务（KMS）。密钥绝不可以硬编码在源代码中。
- 6. 密钥分配 (Key Distribution)** 在通信参与方之间安全地分发密钥。对称密码体系中的挑战性最大，传统方法包括：人工递送（离线分发）、使用已建立的共享密钥分发新密钥（如 Kerberos 协议）、基于可信第三方的分发（KDC）。对公钥体系，公钥分发依赖于 PKI（公钥基础设施）和数字证书。
- 7. 密钥协定 (Key Agreement)** 两个或多个通信方通过公开信道的交互，共同协商出一个共享秘密密钥，且任何被动窃听者无法获知。典型协议：Diffie-Hellman 密钥交换（及其椭圆曲线版本 ECDH），其安全性基于离散对数问题或 ECDLP 的困难性。密钥协定是 TLS/SSL 等安全协议的核心组件。
- 8. 密钥使用控制 (Key Usage Control)** 对密钥使用的范围、权限和条件进行约束。包括：设定密钥用途（仅加密/仅签名/仅认证）、限定密码算法和操作模式、设置有效期和最大使用次数、地理位置和设备绑定。硬件上通过 HSM 内部的策略引擎实施，软件上通过密钥属性标签和访问控制列表实现。
- 9. 密钥备份/恢复 (Key Backup & Recovery)** 为防止密钥丢失导致数据不可解密，需安全

备份密钥。将密钥加密后存储于安全的离线介质（如加密磁带、离线 HSM 备份卡），并通过门限方案（如 Shamir 秘密共享， m of n 门限）分散保管。备份仅在授权条件下可恢复，恢复操作必须有审计日志。

10. 更新密钥 (Key Update / Key Rotation) 定期更换密钥以限制密钥暴露的“爆炸半径”。更新策略基于两种考量：(1) 时间驱动——密钥超过有效期后强制更换；(2) 用量驱动——加密数据量达到安全上限（如 AES-GCM 用同一密钥加密超过 2^{32} 个消息块）后更新。两种策略通常并行实施。

11. 密钥的撤销和销毁 (Key Revocation & Destruction) 当密钥泄露、人员离职或设备退役时，需撤销密钥（使其失效，不再被接受用于新操作）并安全销毁。销毁要求：对称密钥和私钥必须从所有存储介质中彻底清除，不可仅作逻辑删除；硬件上通过消磁 (degaussing) 或物理摧毁；软件上通过安全覆写多次。已撤销的公钥证书需加入 CRL (证书撤销列表) 或通过 OCSP 实时查询。

12. 公开密钥的管理 (Public Key Management) 公钥虽不要求保密，但其完整性和真实性至关重要。管理内容：(1) 公钥证书的签发与管理 (PKI/CA 体系)；(2) 证书链的验证 (信任锚、中间 CA、终端实体证书)；(3) 证书状态查询 (CRL、OCSP)；(4) Web of Trust 模型 (如 PGP/GPG 中的信任网络) 作为 PKI 的替代方案。

综上所述，密钥管理是一个涵盖技术、策略、流程和人力的系统工程，是信息安全体系的基石。

扩展：密钥管理标准与最佳实践

国际上重要的密钥管理标准包括：

- **NIST SP 800-57**：密钥管理建议 (Part 1: 通用、Part 2: 最佳实践、Part 3: 应用指南)，定义了密钥生命周期各环节的安全要求；
- **ISO/IEC 11770**：密钥管理国际标准系列；
- **PCI DSS**：支付卡行业数据安全标准，对加密密钥管理有严格规定 (要求 3)；
- **KMIP (Key Management Interoperability Protocol)**：OASIS 标准化的密钥管理互操作协议，实现异构系统间的统一密钥管理。

现代云环境中，KMS (如 AWS KMS、Azure Key Vault、Google Cloud KMS) 提供自动化的密钥生命周期管理、HSM 支持、细粒度访问控制和审计日志，大大降低了密钥管理的实施复杂度。

4.12 RSA 签名方案的安全性分析 (10 分)

知识点回顾: RSA 数字签名

RSA 数字签名方案是利用 RSA 算法的数学结构构造的数字签名体制, 其签名操作与解密操作类似 (使用私钥), 验证操作与加密操作类似 (使用公钥)。但 RSA 签名的简单直接使用存在多种伪造和攻击方式, 需要配合哈希函数和填充方案 (如 RSA-PSS) 来确保实际安全性。

RSA 签名方案形式化定义:

设 $n = pq$, p 和 q 为大素数。令 $P = A = \mathbb{Z}_n$ 。

$$\mathcal{K} = \{(n, p, q, a, b) \mid ab \equiv 1 \pmod{\phi(n)}\}$$

其中 (n, b) 为公钥, (p, q, a) 为私钥。

签名: $\text{sig}_K(x) = x^a \pmod{n}$, $(x \in \mathbb{Z}_n)$

验证: $\text{ver}_K(x, y) = \text{true} \iff x \equiv y^b \pmod{n}$, $(x, y \in \mathbb{Z}_n)$

解: RSA 签名方案的三种主要攻击及对策

RSA 签名方案 (基本形式, 未使用哈希函数) 存在以下安全性问题:

攻击一: 无消息伪造攻击 (Existential Forgery — Key-Only Attack)

攻击方法: 攻击者已知公钥 (n, b) , 可随意选取一个数据 Y , 计算:

$$X = Y^b \pmod{n}$$

根据验证等式 $X \equiv Y^b \pmod{n}$, 由于 $Y \equiv X^a \pmod{n}$ (RSA 的性质), 攻击者声明 Y 为 A 对消息 X 的有效签名, 验证等式必然成立。

危害程度: 虽然攻击者可以构造出一个“有效”的签名 Y , 但对应的“消息” $X = Y^b \pmod{n}$ 是随机的无意义数据, 无法表达任何有实际含义的消息。因此, 这种攻击在实际场景中的有效性有限——攻击者无法伪造预定内容的真实消息的签名。

对策: 在签名之前先对消息应用抗碰撞的密码学哈希函数 H , 签名的是 $H(M)$ 而非 M 本身。这样即使攻击者从 Y 计算出 X , X 也仅是哈希值, 不存在对应的有意义消息 M (原像抵抗性)。

攻击二: 利用签名可乘性构造伪造 (Chosen-Message Attack)

攻击方法: 攻击者选择两个随机数据 M_1 和 M_2 , 且满足 $M_3 = M_1 \cdot M_2 \pmod{n}$ 。然后设法让 A 分别对 M_1 和 M_2 签名:

$$S_1 = M_1^a \pmod{n}, \quad S_2 = M_2^a \pmod{n}$$

RSA 签名的同态性质使得:

$$S_1 \cdot S_2 = M_1^a \cdot M_2^a = (M_1 \cdot M_2)^a = M_3^a = S_3 \pmod{n}$$

因此, 攻击者无需让 A 直接对 M_3 签名, 即可通过 S_1 和 S_2 的乘积获得 A 对 M_3 的有效签名 S_3 。

对策：使用哈希函数打破同态性。签名者 A 不直接对数据 M 签名，而是对 $H(M)$ 签名：

$$S_1 = (H(M_1))^a \pmod{n}, \quad S_2 = (H(M_2))^a \pmod{n}$$

此时：

$$(H(M_1))^a \cdot (H(M_2))^a \pmod{n} \neq (H(M_1 \cdot M_2))^a \pmod{n}$$

因为哈希函数的抗同态性， $H(M_1) \cdot H(M_2) \neq H(M_1 \cdot M_2)$ （一般情况下），攻击者无法从 S_1 和 S_2 推导出 $M_1 M_2$ 的签名。

攻击三：利用盲化技术的选择密文攻击 (Blinding Attack)

攻击方法：攻击者截获密文 $C = M^b \pmod{n}$ (A 用公钥加密的消息)。攻击者选择一个小的随机数 r ，计算：

$$x = r^b \pmod{n}, \quad y = x \cdot C \pmod{n}, \quad t = r^{-1} \pmod{n}$$

攻击者让 A 对 y 签名（即解密 y ），获得：

$$S = y^a \pmod{n}$$

攻击者随后计算：

$$t \cdot S = r^{-1} \cdot y^a = r^{-1} \cdot (x \cdot C)^a = r^{-1} \cdot (r^b \cdot M^b)^a = r^{-1} \cdot r \cdot M = M \pmod{n}$$

即恢复了原始明文 M 。

本质：此攻击的核心在于利用 RSA 的同态性进行“盲化”——用 r^b 伪装密文 C 得到 y ，使得签名者 A 在不知情的情况下解密了被伪装的密文，而攻击者事后移除伪装因子恢复了明文。

对策：

1. **先签名后加密 (Sign-then-Encrypt)**：对于同时需要保密性和真实性的通信，发送方应对消息签名，再对签名后的整体进行加密 ($E(\text{sig}(M))$)，而非先加密后签名；
2. **使用哈希函数 (Hash-then-Sign)**： A 不直接对 y 签名，而是对 $H(y)$ 签名，彻底阻断盲化攻击链；
3. **使用时间戳 (Timestamp)**：在签名中加入时间戳，限制签名的有效时间窗口，防止被重放或滥用；
4. **使用安全填充方案**：如 RSA-PSS (Probabilistic Signature Scheme)，在签名数据中注入随机盐值，消除 RSA 的代数同态性，是目前被广泛认可的安全 RSA 签名标准 (PKCS#1 v2.1+)。

总结：基本 RSA 签名方案因存在同态性质而面临存在性伪造和盲化攻击，必须配合密码学哈希函数 (Hash-then-Sign 范式) 和概率性填充方案 (如 PSS) 使用，才能在实际中保证签名安全。

扩展：RSA-PSS 与现代签名实践

RSA-PSS (Probabilistic Signature Scheme) 定义于 PKCS#1 v2.1 (RFC 3447)，目前是 RSA 签名的推荐标准。其核心改进包括：

- 使用 MGF (Mask Generation Function) 基于随机盐值生成掩码, 使签名过程概率化 (同一消息多次签名结果不同);
- 提供可证明安全性——在随机预言机模型下, RSA-PSS 的安全性可归约到 RSA 问题 (RSA Problem) 的困难性;
- 已被纳入 FIPS 186-4 (数字签名标准) 和众多国际标准。

现代应用中, RSA 签名逐渐被 ECDSA 和 EdDSA (Ed25519) 替代, 因为后两者提供更短的密钥/签名长度和更高的安全性能比。后量子签名方案 (如 CRYSTALS-Dilithium、SPHINCS+) 也正在标准化过程中。

5 试卷五: 计算机安全与保密 (2021-2022 B 卷)

试卷总览

本试卷分为两大部分: 计算题 (6 道, 每题 10 分, 共 60 分) 和简述题 (4 道, 每题 10 分, 共 40 分), 满分 100 分, 考试时间 120 分钟。

- **计算题 1 (10 分):** 仿射密码解密。已知 $k_0 = 11, k_1 = 17$, 解密信息 "hlzb l slqfb tponybotalojb".
- **计算题 2 (10 分):** Rabin 密码体制解密。求解 $x^2 \equiv 102 \pmod{259}$ 。
- **计算题 3 (10 分):** RSA 密码体制。已知 $n = 91, e = 5$, 加密 $M_1 = 11$ 并解密 $C_2 = 6$ 。
- **计算题 4 (10 分):** 构造有限域 $GF(8)$ 的加法和乘法表。
- **计算题 5 (10 分):** 中国剩余定理求解同余式组。
- **计算题 6 (10 分):** Feige-Fiat-Shamir 协议身份验证。
- **简述题 7 (10 分):** 选择明文攻击。
- **简述题 8 (10 分):** 密码分析的类型。
- **简述题 9 (10 分):** ElGamal 签名方案。
- **简述题 10 (10 分):** 对称密码算法与非对称密码算法。

5.1 仿射密码解密 (10 分)

知识点回顾：仿射密码 (Affine Cipher)

仿射密码是一种经典的单表代换密码，其数学基础是模运算下的仿射变换。

1. 数学定义

设明文字母表大小 (通常 $m = 26$)，仿射密码的加密函数定义为：

$$E(x) = (k_1x + k_0) \bmod m$$

其中 x 为明文字母对应的数字 (如 $a = 0, b = 1, \dots, z = 25$)， $k_0, k_1 \in \mathbb{Z}_m$ 。解密函数为：

$$D(y) = k_1^{-1}(y - k_0) \bmod m$$

其中 k_1^{-1} 为 k_1 在模 m 下的乘法逆元。

2. 密钥条件

为保证加密是双射 (一对一可逆映射)，必须满足 $\gcd(k_1, m) = 1$ 。对于英文字母 $m = 26$ ， k_1 只能选取与 26 互素的数，即 $\{1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25\}$ ，共 $\varphi(26) = 12$ 种选择。 k_0 无限制，可取 $0 \sim 25$ 中任意值，共 26 种选择。因此仿射密码的密钥空间大小为 $12 \times 26 = 312$ 。

3. 乘法逆元的计算

求 $k_1^{-1} \bmod 26$ 可使用扩展欧几里得算法 (Extended Euclidean Algorithm)。对于 $k_1 = 17$ ，寻找 t 使得 $17t \equiv 1 \pmod{26}$ ：

- 方法一 (逐一尝试)： $17 \times 23 = 391 = 26 \times 15 + 1$ ，故 $17^{-1} \equiv 23 \pmod{26}$ 。
- 方法二 (等价代换)：注意到 $23 \equiv -3 \pmod{26}$ ，而 $17 \times (-3) = -51 \equiv 1 \pmod{26}$ (因为 $-51 + 2 \times 26 = 1$)，同样得到逆元为 $23 \equiv -3$ 。

4. 解密公式的简化

代入 $k_0 = 11$ ， $k_1^{-1} \equiv -3 \pmod{26}$ ：

$$\begin{aligned} D(y) &= (-3)(y - 11) \bmod 26 = (-3y + 33) \bmod 26 \\ &= (-3y + 33 - 26) \bmod 26 = (7 - 3y) \bmod 26 \end{aligned}$$

其中 $33 \equiv 7 \pmod{26}$ 。

解题步骤

Step 1: 求解密密钥。

已知 $k_0 = 11$ ， $k_1 = 17$ 。首先求 k_1 在模 26 下的乘法逆元 k_1^{-1} 。由扩展欧几里得算法或试算， $17 \times 23 = 391 = 26 \times 15 + 1$ ，故 $k_1^{-1} \equiv 23 \equiv -3 \pmod{26}$ 。

解密函数:

$$D(y) \equiv k_1^{-1}(y - k_0) \equiv 23(y - 11) \equiv -3y + 33 \equiv 7 - 3y \pmod{26}$$

Step 2: 将密文转换为数字。

将密文字母按 $a = 0, b = 1, \dots, z = 25$ 映射为数字。密文串 (忽略空格):

hlzblslqfbtpnywbotalojb

各字母对应的数字为:

7, 11, 25, 1, 11, 18, 11, 16, 5, 1, 19, 15, 13, 24, 22, 1, 14, 19, 0, 11, 14, 9, 1

共 23 个字母 (空格为分隔符, 解密时保留原位置)。

Step 3: 逐字母解密。

使用 $D(y) = (7 - 3y) \bmod 26$ 解密每个数字:

序号	密文字母	密文数字 y	计算 $7 - 3y$	明文数字 x	明文字母
1	h	7	$7 - 21 = -14 \equiv 12$	12	m
2	l	11	$7 - 33 = -26 \equiv 0$	0	a
3	z	25	$7 - 75 = -68 \equiv 10$	10	k
4	b	1	$7 - 3 = 4$	4	e
5	l	11	$7 - 33 = -26 \equiv 0$	0	a
6	s	18	$7 - 54 = -47 \equiv 5$	5	f
7	l	11	$7 - 33 = -26 \equiv 0$	0	a
8	q	16	$7 - 48 = -41 \equiv 11$	11	l
9	f	5	$7 - 15 = -8 \equiv 18$	18	s
10	b	1	$7 - 3 = 4$	4	e
11	t	19	$7 - 57 = -50 \equiv 2$	2	c
12	p	15	$7 - 45 = -38 \equiv 14$	14	o
13	n	13	$7 - 39 = -32 \equiv 20$	20	u
14	y	24	$7 - 72 = -65 \equiv 13$	13	n
15	w	22	$7 - 66 = -59 \equiv 19$	19	t
16	b	1	$7 - 3 = 4$	4	e
17	o	14	$7 - 42 = -35 \equiv 17$	17	r
18	t	19	$7 - 57 = -50 \equiv 2$	2	c
19	a	0	$7 - 0 = 7$	7	h
20	l	11	$7 - 33 = -26 \equiv 0$	0	a
21	o	14	$7 - 42 = -35 \equiv 17$	17	r
22	j	9	$7 - 27 = -20 \equiv 6$	6	g
23	b	1	$7 - 3 = 4$	4	e

Step 4: 还原明文 (保留原空格位置)。

明文数字序列: 12, 0, 10, 4, 0, 5, 0, 11, 18, 4, 2, 14, 20, 13, 19, 4, 17, 2, 7, 0, 17, 6, 4。

对应字母: $m, a, k, e, a, f, a, l, s, e, c, o, u, n, t, e, r, c, h, a, r, g, e$ 。

恢复原空格位置 (与密文空格位置相同), 得到:

make a false countercharge

拓展与验证

加密验证 (逆向检验): 使用加密公式 $E(x) = (k_1x + k_0) \bmod 26 = (17x + 11) \bmod 26$ 对明文首字母 m ($x = 12$) 加密:

$$E(12) = 17 \times 12 + 11 = 204 + 11 = 215 \equiv 215 - 8 \times 26 = 215 - 208 = 7 \pmod{26}$$

对应字母 h , 与原始密文一致, 验证通过。

乘法逆元的扩展欧几里得算法详细过程:

$$26 = 1 \times 17 + 9$$

$$17 = 1 \times 9 + 8$$

$$9 = 1 \times 8 + 1$$

$$8 = 8 \times 1 + 0$$

回代:

$$1 = 9 - 1 \times 8$$

$$= 9 - 1 \times (17 - 1 \times 9) = 2 \times 9 - 17$$

$$= 2 \times (26 - 17) - 17 = 2 \times 26 - 3 \times 17$$

故 $-3 \times 17 \equiv 1 \pmod{26}$, 即 $17^{-1} \equiv -3 \equiv 23 \pmod{26}$ 。

仿射密码安全性分析: 密钥空间仅 312, 可被暴力穷举轻易攻破。此外, 仿射密码是单表代换的, 明文中字母的统计特性 (频率分布) 完全保留在密文中, 容易遭受频率分析攻击。

5.2 Rabin 密码体制解密 (10 分)

知识点回顾: Rabin 密码体制

1. Rabin 密码体制简介

Rabin 密码体制由 Michael O. Rabin 于 1979 年提出, 是第一个被证明安全性等价于整数分解困难性的公钥密码体制。其安全性基于求解模复合数的二次剩余(平方根)问题的困难性。

2. Rabin 加密与解密

- **密钥生成:** 选择两个大素数 p, q 满足 $p \equiv q \equiv 3 \pmod{4}$ 。令 $n = pq$ 。公钥为 n , 私钥为 (p, q) 。
- **加密:** 对明文 m , 计算密文 $c = m^2 \pmod{n}$ 。
- **解密:** 求解 $x^2 \equiv c \pmod{n}$, 由于 $p \equiv q \equiv 3 \pmod{4}$, 每个模 p 和模 q 下的解可以用公式直接求出, 再结合中国剩余定理得到模 n 下的四个解, 需要通过冗余信息确定唯一的明文。

3. 模素数平方根计算公式

若 $p \equiv 3 \pmod{4}$, a 是模 p 的二次剩余, 则 $x^2 \equiv a \pmod{p}$ 的两个解为:

$$x \equiv \pm a^{\frac{p+1}{4}} \pmod{p}$$

当 $p \equiv 1 \pmod{4}$ 时, 不能直接套用上述公式, 需使用更复杂的 Tonelli-Shanks 算法求解。

4. 解题思路

本题 $n = 259$ 不是素数, 需分解 $n = 7 \times 37$ (或使用已知私钥)。将原同余式拆分为模 $p = 7$ 和模 $q = 37$ 下的两个子同余式, 分别求解后利用中国剩余定理组合得到模 259 下的四个解。

解题步骤

Step 1: 分解模数 n 并化简同余式。

$259 = 7 \times 37$ 。原同余式 $x^2 \equiv 102 \pmod{259}$ 等价于同余式组:

$$\begin{cases} x^2 \equiv 102 \equiv 4 \pmod{7} \\ x^2 \equiv 102 \equiv 28 \pmod{37} \end{cases}$$

Step 2: 验证二次剩余性 (Legendre 符号判断)。

计算 Legendre 符号确认同余式是否有解:

$$\left(\frac{4}{7}\right) = 1, \quad \left(\frac{28}{37}\right) = 1$$

两个 Legendre 符号均为 1, 故每个子同余式各有两个解, 原同余式共有 $2 \times 2 = 4$ 个解。

Step 3: 求解 $x^2 \equiv 4 \pmod{7}$ 。

$7 \equiv 3 \pmod{4}$, 使用直接公式:

$$x \equiv \pm 4^{\frac{7+1}{4}} \equiv \pm 4^2 \equiv \pm 16 \equiv \pm 2 \pmod{7}$$

所以 $x \equiv 2$ 或 $x \equiv 5 \pmod{7}$ 。

Step 4: 求解 $x^2 \equiv 28 \pmod{37}$ 。

$37 \equiv 1 \pmod{4}$, 不能直接使用简化公式。需要计算 $28^{\frac{37-1}{4}} = 28^9 \pmod{37}$, 或通过试算寻找平方根。

试算: $18^2 = 324 = 37 \times 8 + 28 \equiv 28 \pmod{37}$, 故 $x \equiv \pm 18 \pmod{37}$ 。

所以 $x \equiv 18$ 或 $x \equiv 19 \pmod{37}$ 。

Step 5: 利用中国剩余定理组合四组解。

情况	$x \pmod{7}$	$x \pmod{37}$	组合方式
1	2	18	$x \equiv 2 + 7t$, 求 t 使 $2 + 7t \equiv 18 \pmod{37}$
2	2	19	$x \equiv 2 + 7t$, 求 t 使 $2 + 7t \equiv 19 \pmod{37}$
3	5	18	$x \equiv 5 + 7t$, 求 t 使 $5 + 7t \equiv 18 \pmod{37}$
4	5	19	$x \equiv 5 + 7t$, 求 t 使 $5 + 7t \equiv 19 \pmod{37}$

计算 7 在模 37 下的逆元: $7 \times 16 = 112 = 3 \times 37 + 1$, 故 $7^{-1} \equiv 16 \pmod{37}$ 。

- 情况 1: $7t \equiv 16 \pmod{37}$, $t \equiv 16 \times 16 = 256 \equiv 34 \pmod{37}$, $x \equiv 2 + 7 \times 34 = 240 \equiv 19 \pmod{259}$ 。
(因 $240 < 259$, 直接保留; 或 $240 \equiv -19 \pmod{259}$)
- 情况 2: $7t \equiv 17 \pmod{37}$, $t \equiv 17 \times 16 = 272 \equiv 13 \pmod{37}$, $x \equiv 2 + 7 \times 13 = 93 \pmod{259}$ 。
- 情况 3: $7t \equiv 13 \pmod{37}$, $t \equiv 13 \times 16 = 208 \equiv 23 \pmod{37}$, $x \equiv 5 + 7 \times 23 = 166 \equiv -93 \pmod{259}$ 。
- 情况 4: $7t \equiv 14 \pmod{37}$, $t \equiv 14 \times 16 = 224 \equiv 2 \pmod{37}$, $x \equiv 5 + 7 \times 2 = 19 \pmod{259}$ 。

Step 6: 给出最终解。

四组解可统一表示为:

$$x \equiv \pm 19, \pm 93 \pmod{259}$$

即 $x \equiv 19, 166, 93, 240 \pmod{259}$ 。

验证: $19^2 = 361 = 259 + 102 \equiv 102 \pmod{259}$ 。✓

拓展与讨论

Rabin 体制的四解歧义问题：

Rabin 解密产生四个可能的明文，这是其固有特性。实际应用中需要在明文中嵌入冗余信息（如重复特定字段、使用校验和等），以便接收者能唯一识别正确的明文。这是 Rabin 体制相对于 RSA 的一个使用不便之处。

安全性：

Rabin 体制的安全性已被证明等价于大整数分解问题。若攻击者能解密 Rabin 密文，则他就能分解模数 n 。这是因为：假设存在解密预言机，攻击者随机选择 x 并提交 $c = x^2 \bmod n$ 给预言机，预言机返回一个平方根 y 。如果 $x \not\equiv \pm y \pmod{n}$ ，则有 $\gcd(x-y, n)$ 为非平凡的 n 的因子。

与 RSA 对比：

RSA 使用奇指数 e 进行加密，每个密文只对应唯一明文（当 $\gcd(e, \varphi(n)) = 1$ 时）。Rabin 使用指数 $e = 2$ ，加密效率更高（只需一次模平方），但解密有四义性。两者的安全性都基于模 n 分解的困难性，但安全等价性证明仅对 Rabin 体制严格成立。

5.3 RSA 加密与解密 (10 分)

知识点回顾: RSA 密码体制

1. RSA 算法概述

RSA 由 Rivest、Shamir 和 Adleman 于 1977 年提出, 是应用最广泛的公钥密码体制, 其安全性基于大整数分解的困难性。

2. 密钥生成算法

1. 随机选择两个大素数 p 和 q , 计算 $n = pq$ 。
2. 计算欧拉函数 $\varphi(n) = (p-1)(q-1)$ 。
3. 选择加密指数 e , 满足 $1 < e < \varphi(n)$ 且 $\gcd(e, \varphi(n)) = 1$ 。
4. 计算解密指数 d , 满足 $ed \equiv 1 \pmod{\varphi(n)}$, 即 $d \equiv e^{-1} \pmod{\varphi(n)}$ 。
5. 公钥为 (n, e) , 私钥为 (p, q, d) (或仅 d)。

3. 加密与解密

- **加密:** 已知明文 M ($0 \leq M < n$), 密文 $C = M^e \bmod n$ 。
- **解密:** 已知密文 C , 明文 $M = C^d \bmod n$ 。

解密正确性由欧拉定理保证: $M^{ed} \equiv M^{k\varphi(n)+1} \equiv M \pmod{n}$ (当 $\gcd(M, n) = 1$ 时)。

4. 私钥 d 的求解

$d \equiv e^{-1} \pmod{\varphi(n)}$ 。求解乘法逆元的标准方法是扩展欧几里得算法。

解题步骤

已知条件: $n = 91$, $e = 5$, 需加密 $M_1 = 11$, 需解密 $C_2 = 6$ 。

Part A: 加密明文 $M_1 = 11$ 。

直接使用 RSA 加密公式:

$$\begin{aligned} C_1 &= M_1^e \bmod n = 11^5 \bmod 91 \\ &= 11^2 \times 11^2 \times 11 \bmod 91 \\ &= 121 \times 121 \times 11 \bmod 91 \end{aligned}$$

先化简中间结果: $121 \equiv 121 - 91 = 30 \pmod{91}$ 。

$$\begin{aligned} C_1 &= 30 \times 30 \times 11 \bmod 91 \\ &= 900 \times 11 \bmod 91 \end{aligned}$$

$900 \equiv 900 - 9 \times 91 = 900 - 819 = 81 \pmod{91}$? 再算: $91 \times 9 = 819$, $900 - 819 = 81$ 。

$$C_1 = 81 \times 11 \bmod 91 = 891 \bmod 91$$

$91 \times 9 = 819$, $891 - 819 = 72$ 。

$$\boxed{C_1 = 72}$$

也可以更简单地分段计算: $11^2 = 121 \equiv 30 \pmod{91}$, $11^4 = 30^2 = 900 \equiv 81 \pmod{91}$, $11^5 = 81 \times 11 = 891 \equiv 72 \pmod{91}$ 。

Part B: 分解 n 求私钥并解密 $C_2 = 6$ 。

Step 1: 分解 $n = 91$ 。显然 $91 = 7 \times 13$ 。

Step 2: 计算欧拉函数:

$$\varphi(91) = (7-1)(13-1) = 6 \times 12 = 72$$

Step 3: 求私钥 $d = e^{-1} \pmod{72}$ 。使用扩展欧几里得算法求 $5^{-1} \pmod{72}$:

$$72 = 14 \times 5 + 2$$

$$5 = 2 \times 2 + 1$$

$$2 = 2 \times 1 + 0$$

回代:

$$1 = 5 - 2 \times 2$$

$$= 5 - 2 \times (72 - 14 \times 5)$$

$$= 5 - 2 \times 72 + 28 \times 5$$

$$= 29 \times 5 - 2 \times 72$$

故 $29 \times 5 \equiv 1 \pmod{72}$, 即 $d \equiv 29 \pmod{72}$ 。

Step 4: 解密 $C_2 = 6$:

$$M_2 = C_2^d \pmod{n} = 6^{29} \pmod{91}$$

使用平方-乘算法 (Square-and-Multiply) 简化计算。首先将指数 29 写为二进制: $29_{10} = 11101_2$ 。

$$6^1 \equiv 6 \pmod{91}$$

$$6^2 \equiv 36 \pmod{91}$$

$$6^4 \equiv 36^2 = 1296 \equiv 22 \pmod{91} \quad (\text{因 } 91 \times 14 = 1274, 1296 - 1274 = 22)$$

$$6^8 \equiv 22^2 = 484 \equiv 29 \pmod{91} \quad (\text{因 } 91 \times 5 = 455, 484 - 455 = 29)$$

$$6^{16} \equiv 29^2 = 841 \equiv 22 \pmod{91} \quad (\text{因 } 91 \times 9 = 819, 841 - 819 = 22)$$

现在 $29 = 16 + 8 + 4 + 1$, 故:

$$6^{29} \equiv 6^{16} \times 6^8 \times 6^4 \times 6^1 \pmod{91}$$

$$\equiv 22 \times 29 \times 22 \times 6 \pmod{91}$$

分步计算:

$$22 \times 29 = 638 \equiv 638 - 7 \times 91 = 638 - 637 = 1 \pmod{91}$$

$$1 \times 22 = 22 \pmod{91}$$

$$22 \times 6 = 132 \equiv 132 - 91 = 41 \pmod{91}$$

$$M_2 = 41$$

Step 5: 验证。

加密验证: $M_2 = 41$, $41^5 \pmod{91}$:

$$41^2 = 1681 \equiv 43 \pmod{91} \quad (1681 - 18 \times 91 = 1681 - 1638 = 43)$$

$$41^4 \equiv 43^2 = 1849 \equiv 29 \pmod{91} \quad (1849 - 20 \times 91 = 1849 - 1820 = 29)$$

$$41^5 \equiv 29 \times 41 = 1189 \equiv 6 \pmod{91} \quad (1189 - 13 \times 91 = 1189 - 1183 = 6)$$

结果 $41^5 \equiv 6 = C_2 \pmod{91}$, 解密正确。✓

拓展与讨论

RSA 安全性与模数分解:

本题 $n = 91$ 极小, 可轻易分解为 7×13 , 从而求得 $\varphi(n)$ 和私钥 d 。在实际应用中, RSA 模数 n 通常为 2048 位或 4096 位的二进制数 (约 600-1200 位十进制数字), 由两个大素数相乘得到。目前, 广泛认为使用 2048 位密钥的 RSA 在可预见的未来是安全的。

分解 $n = 91$ 的实际意义:

一旦 n 被分解, 整个 RSA 体系即被攻破。因此 RSA 的安全性完全依赖于大整数分解的困难性。本题演示了“破译 RSA”的完整过程: 从已知公钥 (n, e) 出发, 通过分解 n 获得 $\varphi(n)$, 进而计算私钥 d , 最终解密密文。

模幂运算优化技术:

大规模 RSA 运算中, 指数可能长达数千位, 必须使用高效的模幂算法。除平方-乘算法外, 还有:

- 滑动窗口法 (Sliding Window)
- 蒙哥马利模乘 (Montgomery Modular Multiplication)
- 中国剩余定理加速 (利用私钥中的 p, q , 使用 CRT 将模 n 运算拆分为模 p 和模 q 下的运算, 可提高约 4 倍速度)

5.4 构造有限域 $\text{GF}(8)$ 的加法和乘法表 (10 分)

知识点回顾：有限域 (Galois Field)

1. 有限域的基本概念

有限域（伽罗瓦域，Galois Field）是包含有限个元素的域，记作 $\text{GF}(p^n)$ ，其中 p 为素数（域的特征）， n 为正整数。 $\text{GF}(p^n)$ 恰好含有 p^n 个元素。

当 $n = 1$ 时， $\text{GF}(p) = \mathbb{Z}_p$ ，即整数模 p 的剩余类环是一个域。

当 $n > 1$ 时， $\text{GF}(p^n)$ 不同于 $\mathbb{Z}_{p^n}$ （后者不是域，因为 p 有零因子）。构造 $\text{GF}(p^n)$ 需要多项式方法。

2. $\text{GF}(8)$ 的构造方法

$\text{GF}(8) = \text{GF}(2^3)$ ，特征为 2。构造步骤如下：

1. 在 $\text{GF}(2) = \{0, 1\}$ 上，找一个 3 次不可约多项式 $f(x)$ 。本题选用 $f(x) = x^3 + x + 1$ 。
可以验证： $f(0) = 1 \neq 0$ ， $f(1) = 1 + 1 + 1 = 1 \neq 0$ （在 $\text{GF}(2)$ 中），故 $f(x)$ 无一次因子，又因它是 3 次多项式，所以它是不可约的。
2. 域中元素是 $\text{GF}(2)[x]$ 中次数小于 3 的多项式，即形如 $a_2x^2 + a_1x + a_0$ 的多项式，其中 $a_i \in \{0, 1\}$ 。共有 $2^3 = 8$ 个元素。
3. 将这些多项式与整数 $\{0, 1, \dots, 7\}$ 建立编码对应——将二进制系数 $(a_2a_1a_0)_2$ 转为十进制数字。
4. 加法为多项式加法（即系数模 2 加，等价于按位异或/XOR）。
5. 乘法为多项式乘法后取模 $f(x) = x^3 + x + 1$ 。

3. 元素对应关系

十进制表示	二进制表示	多项式表示
0	000	0
1	001	1
2	010	x
3	011	$x + 1$
4	100	x^2
5	101	$x^2 + 1$
6	110	$x^2 + x$
7	111	$x^2 + x + 1$

4. 模多项式化简规则

由 $f(x) = x^3 + x + 1 \equiv 0$ ，得化简公式： $x^3 \equiv x + 1$ 。由此可导出高次项的化简：

- $x^4 = x \cdot x^3 \equiv x(x + 1) = x^2 + x$
- $x^5 = x \cdot x^4 \equiv x(x^2 + x) = x^3 + x^2 \equiv (x + 1) + x^2 = x^2 + x + 1$
- $x^6 = x \cdot x^5 \equiv x^3 + x^2 + x \equiv (x + 1) + x^2 + x = x^2 + 1$

解题步骤

Step 1: 构造加法表。

GF(8) 中加法即多项式系数模 2 加（等价于二进制按位异或/XOR）。加法表对称，主对角线全为 0（因为 $a \oplus a = 0$ ）。

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	0	3	2	5	4	7	6
2	2	3	0	1	6	7	4	5
3	3	2	1	0	7	6	5	4
4	4	5	6	7	0	1	2	3
5	5	4	7	6	1	0	3	2
6	6	7	4	5	2	3	0	1
7	7	6	5	4	3	2	1	0

加法表验证举例：

- $3 + 5$: $3 = (011)_2$, $5 = (101)_2$, 按位异或得 $(110)_2 = 6$ 。查表 $3 + 5 = 6$ 。✓
- $7 + 4$: $7 = (111)_2$, $4 = (100)_2$, 按位异或得 $(011)_2 = 3$ 。查表 $7 + 4 = 3$ 。✓

Step 2: 构造乘法表。

乘法需进行多项式乘法 and 模 $x^3 + x + 1$ 的化简。利用化简公式 $x^3 \equiv x + 1$ ，对每对元素计算乘积。

$\times$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	3	1	7	5
3	0	3	6	5	7	4	1	2
4	0	4	3	7	6	2	5	1
5	0	5	1	4	2	7	3	6
6	0	6	7	1	5	3	2	4
7	0	7	5	2	1	6	4	3

乘法表关键条目验证：

以下是部分乘法计算的详细过程（多项式乘法后模 $x^3 + x + 1$ 化简）：

乘法	多项式运算	化简过程	结果
2×2	$x \cdot x = x^2$	x^2	4
2×3	$x \cdot (x+1) = x^2 + x$	—	6
2×4	$x \cdot x^2 = x^3$	$x^3 \equiv x+1$	3
2×5	$x \cdot (x^2+1) = x^3 + x$	$\equiv (x+1) + x = 1$	1
3×3	$(x+1)^2 = x^2 + 1$	—	5
3×4	$(x+1) \cdot x^2 = x^3 + x^2$	$\equiv (x+1) + x^2 = x^2 + x + 1$	7
3×5	$(x+1)(x^2+1) = x^3 + x^2 + x + 1$	$\equiv (x+1) + x^2 + x + 1 = x^2$	4
4×4	$x^2 \cdot x^2 = x^4$	$= x \cdot x^3 \equiv x(x+1) = x^2 + x$	6
4×5	$x^2 \cdot (x^2+1) = x^4 + x^2$	$= (x^2 + x) + x^2 = x$	2
5×5	$(x^2+1)^2 = x^4 + 1$	$= (x^2 + x) + 1 = x^2 + x + 1$	7
6×6	$(x^2+x)^2 = x^4 + x^2$	$= (x^2 + x) + x^2 = x$	2
7×7	$(x^2+x+1)^2 = x^4 + x^2 + 1$	$= (x^2 + x) + x^2 + 1 = x + 1$	3

重要性质验证：

- 除 0 外，每个非零元素都有乘法逆元。例如 $2 \times 5 = 1$ ，故 $2^{-1} = 5$ ； $3^{-1} = 6$ （因 $3 \times 6 = 1$ ）； $4^{-1} = 7$ （因 $4 \times 7 = 1$ ）。
- 乘法表除 0 行/列外，每行每列都是 $\{1, 2, 3, 4, 5, 6, 7\}$ 的一个排列，满足域的公理要求。
- 特征为 2： $a + a = 0$ 对所有 a 成立。

拓展：GF(8) 中本原元的确定**本原元 (Primitive Element) 的概念：**

有限域的非零元素构成一个循环乘法群 $\text{GF}(p^n)^*$ 。若 $\alpha \in \text{GF}(p^n)^*$ 且 α 的阶为 $p^n - 1$ ，则称 α 为本原元。 $\text{GF}(8)^*$ 是 7 阶群（7 是素数），任何非零非 1 元素都是本原元。在本题的 $\text{GF}(8)$ 中，以元素 2（多项式 x ）为例，验证其为本原元：

$$2^1 = 2, 2^2 = 4, 2^3 = 3, 2^4 = 6, 2^5 = 7, 2^6 = 5, 2^7 = 1$$

确实遍历了 $\{1, 2, 3, 4, 5, 6, 7\}$ 中的全部 7 个非零元素，因此 2 是本原元。

不可约多项式的选择：

不同的不可约多项式会生成同构但乘法表不同的域表示。 $x^3 + x + 1$ 和 $x^3 + x^2 + 1$ 是两个常用的 3 次本原多项式。本题选用 $x^3 + x + 1$ 。

应用场景：

有限域在密码学中有广泛应用：

- AES 加密算法使用 $\text{GF}(2^8)$ ，不可约多项式为 $x^8 + x^4 + x^3 + x + 1$ 。
- 椭圆曲线密码 (ECC) 基于有限域上的椭圆曲线群。
- 纠错编码（如 Reed-Solomon 码）使用有限域进行编解码。

5.5 中国剩余定理求解同余式组 (10 分)

title

1. 定理陈述

设 $m_1, m_2, \dots, m_k$ 是两两互素的正整数, $M = m_1 m_2 \cdots m_k$ 。则同余式组:

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

在模 M 下有唯一解:

$$x \equiv \sum_{i=1}^k a_i M_i M'_i \pmod{M}$$

其中 $M_i = M/m_i$, M'_i 为 M_i 模 m_i 的乘法逆元, 即 $M_i M'_i \equiv 1 \pmod{m_i}$ 。

2. 解题关键步骤

1. 检查模数是否两两互素。若不互素, 需先化为互素的情形 (或改用一般化 CRT)。
2. 若同余式带有系数 (如 $3x \equiv 4 \pmod{5}$), 需先将系数消去化为 $x \equiv a_i \pmod{m_i}$ 的标准形式。消系数的方法是求系数在对应模数下的乘法逆元。
3. 分别计算 $M_i = M/m_i$ 和 M'_i 。
4. 代入公式 $x \equiv \sum a_i M_i M'_i \pmod{M}$ 。

解题步骤

原始同余式组:

$$\begin{cases} x \equiv 2 \pmod{9} \\ 3x \equiv 4 \pmod{5} \\ 4x \equiv 3 \pmod{7} \end{cases}$$

Step 1: 化为标准形式 $x \equiv a_i \pmod{m_i}$ 。

第一式已为标准形式。

第二式: $3x \equiv 4 \pmod{5}$ 。求 $3^{-1} \pmod{5}$: $3 \times 2 = 6 \equiv 1 \pmod{5}$, 故 $3^{-1} \equiv 2 \pmod{5}$ 。

两边同乘 2:

$$x \equiv 4 \times 2 \equiv 8 \equiv 3 \pmod{5}$$

第三式: $4x \equiv 3 \pmod{7}$ 。求 $4^{-1} \pmod{7}$: $4 \times 2 = 8 \equiv 1 \pmod{7}$, 故 $4^{-1} \equiv 2 \pmod{7}$ 。

两边同乘 2:

$$x \equiv 3 \times 2 \equiv 6 \pmod{7}$$

Step 2: 化为标准同余式组。

$$\begin{cases} x \equiv 2 \pmod{9} \\ x \equiv 3 \pmod{5} \\ x \equiv 6 \pmod{7} \end{cases}$$

验证模数两两互素: $\gcd(9, 5) = 1$, $\gcd(9, 7) = 1$, $\gcd(5, 7) = 1$, 满足条件。

Step 3: 计算中国剩余定理参数。

- 总模数: $M = 9 \times 5 \times 7 = 315$
- $M_1 = M/9 = 35$, $M_2 = M/5 = 63$, $M_3 = M/7 = 45$

Step 4: 求各 $M'_i = M_i^{-1} \pmod{m_i}$ 。

- $M'_1 = 35^{-1} \pmod{9}$: $35 \equiv 8 \pmod{9}$, $8 \times 8 = 64 \equiv 1 \pmod{9}$, 故 $M'_1 = 8$ 。
- $M'_2 = 63^{-1} \pmod{5}$: $63 \equiv 3 \pmod{5}$, $3 \times 2 = 6 \equiv 1 \pmod{5}$, 故 $M'_2 = 2$ 。
- $M'_3 = 45^{-1} \pmod{7}$: $45 \equiv 3 \pmod{7}$, $3 \times 5 = 15 \equiv 1 \pmod{7}$, 故 $M'_3 = 5$ 。

Step 5: 代入 CRT 公式求解。

$$\begin{aligned} x &\equiv a_1 M_1 M'_1 + a_2 M_2 M'_2 + a_3 M_3 M'_3 \pmod{M} \\ &\equiv 2 \times 35 \times 8 + 3 \times 63 \times 2 + 6 \times 45 \times 5 \pmod{315} \\ &\equiv 560 + 378 + 1350 \pmod{315} \end{aligned}$$

计算各项模 315 的余数:

$$\begin{aligned} 560 &= 315 \times 1 + 245 \equiv 245 \pmod{315} \\ 378 &= 315 \times 1 + 63 \equiv 63 \pmod{315} \\ 1350 &= 315 \times 4 + 90 \equiv 90 \pmod{315} \end{aligned}$$

$$x \equiv 245 + 63 + 90 = 398 \equiv 398 - 315 = 83 \pmod{315}$$

$$\boxed{x \equiv 83 \pmod{315}}$$

Step 6: 验证。

将 $x = 83$ 代入原始三式验证:

- $83 \pmod{9} = 83 - 9 \times 9 = 83 - 81 = 2$ 。满足 $x \equiv 2 \pmod{9}$ 。✓
- $3 \times 83 = 249 \pmod{5} = 249 - 49 \times 5 = 249 - 245 = 4$ 。满足 $3x \equiv 4 \pmod{5}$ 。✓
- $4 \times 83 = 332 \pmod{7} = 332 - 47 \times 7 = 332 - 329 = 3$ 。满足 $4x \equiv 3 \pmod{7}$ 。✓

拓展：模数不互素时的处理**一般化的中国剩余定理：**

若模数 $m_1, m_2, \dots, m_k$ 并非两两互素，同余式组有解的充要条件是：对任意 $i \neq j$ ，有 $a_i \equiv a_j \pmod{\gcd(m_i, m_j)}$ 。求解时需将模数分解因子，化归为模数互素的情形。

例如，考虑：

$$\begin{cases} x \equiv 3 \pmod{6} \\ x \equiv 7 \pmod{10} \end{cases}$$

$\gcd(6, 10) = 2$, $3 \bmod 2 = 1$, $7 \bmod 2 = 1$ ，条件满足，有解。解为 $x \equiv 17 \pmod{30}$ ($30 = \text{lcm}(6, 10)$)。

CRT 在密码学中的应用：

- RSA 解密加速：利用 p, q 分别模运算，再用 CRT 组合结果。
- Rabin 解密：组合模 p 和模 q 下的四个平方根。
- 多方秘密共享：CRT 可构造阈值方案，如 Asmuth-Bloom 方案。
- 广播加密：向多用户广播加密消息时用 CRT 组合。

CRT 的 RSA 加速：

设 $d_p \equiv d \pmod{p-1}$, $d_q \equiv d \pmod{q-1}$ ，解密时分别计算 $M_p = C^{d_p} \bmod p$ 和 $M_q = C^{d_q} \bmod q$ ，再用 CRT 恢复 M 。这比直接计算 $M = C^d \bmod n$ 快约 4 倍。

5.6 Feige-Fiat-Shamir 协议身份验证 (10 分)

知识点回顾: Feige-Fiat-Shamir 零知识身份识别协议

1. 协议背景

Feige-Fiat-Shamir (FFS) 协议由 Uriel Feige、Amos Fiat 和 Adi Shamir 于 1988 年提出, 是一种基于零知识证明思想的身份识别协议。其安全性基于求解模复合数的二次剩余 (平方根) 问题的困难性。

2. 系统参数设置

- 可信中心选择 $n = pq$ (p, q 为大素数), n 公开, p, q 保密或销毁。
- 证明者 (Prover, 甲) 选择 k 个秘密数 $s_1, s_2, \dots, s_k$, 计算公开密钥 v_i 满足 $s_i^2 \cdot v_i \equiv 1 \pmod{n}$ (即 $v_i \equiv s_i^{-2} \pmod{n}$)。
- 公开密钥 $\{v_1, \dots, v_k\}$; 秘密密钥 $\{s_1, \dots, s_k\}$ 。

3. 一轮协议交互过程

- 承诺 (Commitment):** 证明者选择随机数 $r \in \mathbb{Z}_n^*$, 计算 $x \equiv r^2 \pmod{n}$, 将 x 发送给验证者。
- 挑战 (Challenge):** 验证者 (Verifier, 乙) 发送随机二进制向量 $b = (b_1, b_2, \dots, b_k)$, 其中 $b_i \in \{0, 1\}$ 。
- 响应 (Response):** 证明者计算 $y \equiv r \cdot \prod_{i=1}^k s_i^{b_i} \pmod{n}$, 将 y 发送给验证者。
- 验证 (Verification):** 验证者计算 $y^2 \cdot \prod_{i=1}^k v_i^{b_i} \pmod{n}$, 检查是否等于 x 。

4. 正确性验证

由于 $s_i^2 \cdot v_i \equiv 1 \pmod{n}$, 有:

$$y^2 \cdot \prod_{i=1}^k v_i^{b_i} \equiv r^2 \cdot \prod_{i=1}^k s_i^{2b_i} \cdot \prod_{i=1}^k v_i^{b_i} \equiv r^2 \cdot \prod_{i=1}^k (s_i^2 v_i)^{b_i} \equiv r^2 \equiv x \pmod{n}$$

5. 安全性特性

- 完备性 (Completeness):** 诚实的证明者总能让验证者接受。
- 可靠性 (Soundness):** 不知道秘密密钥的冒充者在一轮中以概率 $1/2^k$ 通过验证。重复 t 轮后, 欺骗成功的概率降至 2^{-kt} 。
- 零知识性 (Zero-Knowledge):** 验证者无法从交互中获取关于秘密密钥的任何信息。

解题步骤

已知条件: $n = 35$, 公开密钥 $\{v_1, v_2, v_3, v_4\} = \{1, 4, 16, 29\}$ 。

Step 1: 确定秘密密钥。

由 $s_i^2 \cdot v_i \equiv 1 \pmod{35}$ (即 $s_i^2 \equiv v_i^{-1} \pmod{35}$), 验证所给的秘密密钥 $\{1, 3, 9, 8\}$:

i	s_i	v_i	$s_i^2 \bmod 35$	$s_i^2 \cdot v_i \bmod 35$
1	1	1	$1^2 = 1$	$1 \times 1 = 1 \equiv 1 \checkmark$
2	3	4	$3^2 = 9$	$9 \times 4 = 36 \equiv 1 \checkmark$
3	9	16	$9^2 = 81 \equiv 11$	$11 \times 16 = 176 \equiv 1 \checkmark$
4	8	29	$8^2 = 64 \equiv 29$	$29 \times 29 = 841 \equiv 1 \checkmark$

因此秘密密钥为: $\{1, 3, 9, 8\}$ 。

Step 2: 完成一圈 FFS 身份验证协议。

(1) 甲 (证明者) 选择随机数 $r = 13$, 计算承诺值 x 并发送给乙 (验证者):

$$x \equiv r^2 \equiv 13^2 \equiv 169 \equiv 29 \pmod{35}$$

计算过程: $169 = 35 \times 4 + 29$, 故 $x = 29$ 。甲将 $x = 29$ 发送给乙。

(2) 乙向甲发送随机的二进制串 (挑战):

$$b = (b_1, b_2, b_3, b_4) = \{1, 1, 1, 0\}$$

(3) 甲计算响应值 y , 并发送给乙:

根据 b 向量, 需要参与的秘密数是 $s_1^{b_1} \cdot s_2^{b_2} \cdot s_3^{b_3} \cdot s_4^{b_4}$:

$$\begin{aligned}
 y &\equiv r \cdot s_1^{b_1} \cdot s_2^{b_2} \cdot s_3^{b_3} \cdot s_4^{b_4} \pmod{35} \\
 &\equiv 13 \cdot 1^1 \cdot 3^1 \cdot 9^1 \cdot 8^0 \pmod{35} \\
 &\equiv 13 \cdot 1 \cdot 3 \cdot 9 \cdot 1 \pmod{35} \\
 &\equiv 13 \cdot 27 \pmod{35} \\
 &\equiv 351 \pmod{35}
 \end{aligned}$$

$351 = 35 \times 10 + 1$, 故 $y \equiv 1 \pmod{35}$ 。甲将 $y = 1$ 发送给乙。

(4) 乙验证等式是否成立:

乙计算:

$$\begin{aligned}
 \text{左侧} &= y^2 \cdot v_1^{b_1} \cdot v_2^{b_2} \cdot v_3^{b_3} \cdot v_4^{b_4} \pmod{35} \\
 &= 1^2 \cdot 1^1 \cdot 4^1 \cdot 16^1 \cdot 29^0 \pmod{35} \\
 &= 1 \cdot 1 \cdot 4 \cdot 16 \cdot 1 \pmod{35} \\
 &= 64 \pmod{35} \\
 &= 29
 \end{aligned}$$

左侧 $= 29 = x =$ 右侧。等式成立, 乙接受甲的身份证明。✓

拓展与讨论

安全性分析:

在一轮协议中, 若 $k = 4$ (本题情况), 不知秘密密钥的冒充者猜对挑战 b 并伪造正确响应的概率为 $1/2^4 = 1/16$ 。实际应用中通常取 k 较大 (如 $k = 64$) 并重复多轮, 使欺骗概率降至可忽略的 2^{-kt} 。

FFS 与零知识证明:

FFS 协议满足零知识性, 因为验证者从交互中只获得:

- $x = r^2 \bmod n$ (随机数的平方)
- $y = r \cdot \prod s_i^{b_i} \bmod n$ (与秘密密钥的乘积但被随机数 r 掩盖)

验证者无法从中提取任何关于 s_i 的信息。实际上可以构造模拟器 (Simulator), 在不知道秘密密钥的情况下, 生成与真实交互不可区分的对话记录。

身份识别协议的应用:

FFS 协议及其变体 (如 Guillou-Quisquater 协议、Schnorr 协议) 广泛应用于:

- 智能卡身份认证
- 门禁系统
- 网络登录认证
- 数字签名的底层构建模块 (Fiat-Shamir 变换将身份识别协议转化为数字签名方案)

Fiat-Shamir 变换:

这是一个重要密码学构造: 通过将交互协议中的验证者挑战替换为哈希函数的输出 ($b = H(x, \text{message})$), 可将三轮交互的零知识证明协议转化为非交互的数字签名方案。

5.7 选择明文攻击 (10 分)

知识点回顾：密码攻击模型概述

密码分析学中，根据攻击者获取信息的能力，定义了多种标准的攻击模型。这些模型描述了攻击者在尝试破解密码系统时所具有的资源 and 条件假设。理解这些模型对于评估密码算法的安全性至关重要。

Kerckhoffs 原则：

现代密码学的设计基础是 Kerckhoffs 原则——密码系统的安全性应仅依赖于密钥的保密性，而不应依赖于算法本身的保密性。也就是说，攻击者被假设完全了解密码算法的所有细节（加密/解密函数的具体实现），唯一不知道的就是密钥。在此原则下，攻击被分为不同的层次。

选择明文攻击详解

1. 选择明文攻击 (Chosen Plaintext Attack, CPA) 的基本定义

选择明文攻击是指：在已知密码体制（加解密算法）的前提下，攻击者不仅可以获得若干明文—密文对，更重要的是，攻击者能够获得对加密机的暂时访问权限，从而主动选择任意明文串 $x_1, x_2, \dots, x_m$ ，并构造（获得）出对应的密文串 $y_1, y_2, \dots, y_m$ 。攻击者利用这些精心选择的明文—密文对，分析并试图推导出密钥 k 或至少部分恢复密钥信息。

2. 选择明文攻击的攻击流程

典型的 CPA 流程为：

1. 攻击者被授权临时访问加密设备（加密预言机/Oracle）。
2. 攻击者策略性地选择一组明文 $\{x_1, x_2, \dots, x_m\}$ 。
3. 将选定的明文提交给加密机，获得对应的密文 $\{y_1, y_2, \dots, y_m\}$ 。
4. 攻击者分析这些明文—密文对之间的关系，结合已知的密码算法结构，尝试推导出密钥或构造解密方法。
5. 一旦获得密钥，攻击者便可解密之后截获的所有密文。

3. 选择明文攻击的实际场景

- **二战时期的实例：**美军在太平洋战争期间，会故意在公开水域投放已知内容的虚假报告（如“中途岛淡水短缺”），然后拦截日军的加密通信，观察日军是否传递了这些信息及如何编码，从而破译日军的密码系统。这是历史上最经典的选择明文攻击案例之一。
- **公钥密码场景：**在公钥密码体制中，攻击者天然具有选择明文攻击的能力，因为加密密钥是公开的，任何人都可以随意加密任意明文。因此，公钥密码算法必须具备抵抗选择明文攻击 (IND-CPA 安全性) 的能力。

- **智能卡场景**：攻击者持有一张智能卡，可以向其中输入任意数据并获得加密输出。

4. 选择明文攻击下密码系统的安全性要求

在现代密码学中，抵抗选择明文攻击已成为**密码系统的最低安全标准**。具体而言：

- **确定性加密的弱点**：若加密算法是确定性的（即相同明文总是产生相同密文），则攻击者可以构造字典攻击：选择一个候选明文，加密后比对结果。因此，安全的加密方案必须是概率性的（引入随机化，如使用随机 IV 或 Nonce）。
- **IND-CPA 安全性(不可区分性)**：选择明文攻击下的不可区分性 (Indistinguishability Under Chosen-Plaintext Attack) 是现代加密方案的标准安全定义。它要求：即使攻击者可以任意选择明文进行加密，也无法区分两个等长明文中哪一个被加密。
- **实现 IND-CPA 的方式**：对称密码通过使用 CBC、CTR 等引入随机 IV 的模式实现；公钥密码通过使用随机化的填充方案（如 RSA-OAEP）实现。

5. 选择明文攻击的分类

- **非适应性选择明文攻击 (Non-Adaptive CPA)**：攻击者一次性选择所有待加密的明文，然后获得全部密文，之后不再有访问加密机的能力。
- **适应性选择明文攻击 (Adaptive Chosen Plaintext Attack, CPA2)**：攻击者可以根据之前获得的密文/信息，动态地选择下一个待加密的明文。这是更强的攻击模型，因为攻击者可以逐步调整策略。

6. 如何抵御选择明文攻击

- 使用随机化的加密方案（每次加密引入新鲜随机数）。
- 使用语义安全的加密算法。
- 在 RSA 中使用 OAEP (Optimal Asymmetric Encryption Padding) 填充。
- 在对称加密中使用认证加密 (AEAD) 模式，如 AES-GCM。

CPA 与 CCA 的区别

选择密文攻击 (Chosen Ciphertext Attack, CCA)：

比 CPA 更强的攻击模型，攻击者额外具有访问解密机的能力（但不能解密目标密文）。

CCA 分为：

- **CCA1 (午餐攻击/Lunchtime Attack)**：在获得目标密文前可自由使用解密机。
- **CCA2 (适应性选择密文攻击)**：在获得目标密文后仍可使用解密机（但不能对目

标密文本身解密)。

许多早期密码方案（如教科书 RSA、基本 ElGamal）在 CPA 下安全，但在 CCA 下不安全。现代安全标准通常要求 IND-CCA2 安全性。

5.8 密码分析的类型 (10 分)

知识点回顾：密码分析学基础

密码分析学 (Cryptanalysis): 研究如何在不知道解密密钥的情况下, 从密文恢复明文或密钥的科学。密码分析与密码编码学 (Cryptography) 共同构成了密码学 (Cryptology) 的两大分支。

根据 Kerckhoffs 原则, 密码分析者总是假设他完全知晓密码算法的所有细节 (算法公开), 攻击的目标是获得密钥。攻击者可以获取的信息量由攻击模型定义。

密码分析类型详解

根据攻击者所掌握的知识和情报的不同 (信息获取能力的递增), 密码分析可分为以下七种类型:

1. 唯密文攻击 (Ciphertext-Only Attack, COA)

攻击者仅拥有一些密文 (可能来自同一密钥加密), 完全不知道对应的明文内容。

- **攻击能力:** 最弱, 攻击者是被动的, 只能分析密文。
- **典型手段:** 统计分析法 (字母频率分析、多字母组合频率分析)、暴力穷举、基于密文统计特征的攻击。
- **实例:** 截获一段加密的电子邮件, 尝试破译。攻击者没有机会获取任何明文线索。
- **安全性要求:** 现代密码算法必须能够抵抗唯密文攻击, 这是最基本的安全性要求。

2. 已知明文攻击 (Known Plaintext Attack, KPA)

攻击者掌握了若干明文—密文对 (例如通过截获标准的电子邮件头、已知格式的文件), 但无法选择这些明文。

- **攻击能力:** 被动获取, 攻击者无法决定哪些明文被泄露。
- **典型手段:** 线性密码分析 (利用明文—密文—密钥比特之间的线性逼近)、差分密码分析的辅助。
- **实例:** 知道一封商业信函总是以“Dear Sir”开头, 攻击者获得其加密版本后可利用这对已知明文—密文来攻击其他部分。二战期间德军以“Heil Hitler”结尾的电报也是如此。
- **概率特性:** 线性密码分析正是已知明文攻击的经典应用, 它需要大量明文—密文对来统计线性逼近的偏差。

3. 选择明文攻击 (Chosen Plaintext Attack, CPA)

攻击者可以主动选择任意明文, 并获得对应的密文 (加密机暂时被攻击者使用)。

- **攻击能力:** 较强, 攻击者可策略性地选择明文以获取最大信息。
- **典型手段:** 差分密码分析 (选择具有特定差分的明文对并分析密文对差分)、中间相遇攻击。
- **实例:** 如第七题详述, 可选择合适的明文观察加密行为, 从而分析密钥结构。二战中中途岛战役的情报战即是经典 CPA 案例。

- **公钥密码的天然威胁**：由于公钥是公开的，任何人都可以进行 CPA 攻击，因此公钥密码必须具备 CPA 安全性。

4. 适应性选择明文攻击 (Adaptive Chosen Plaintext Attack, CPA2)

攻击者不仅可以选择明文进行加密，而且可以根据之前获得的密文结果**动态地**选择一个待加密的明文。

- **攻击能力**：比 CPA 更强，攻击者可采取循序渐进、逐步逼近的策略。
- **与普通 CPA 的区别**：普通 CPA 是所有明文一次性选定；CPA2 允许攻击者看到前面的结果后再调整后续选择。
- **实例**：攻击者先提交明文 x_1 加密得到 y_1 ，分析 y_1 的特征后，再决定提交 x_2 —— x_2 的选择依赖于 y_1 的分析结果。

5. 选择密文攻击 (Chosen Ciphertext Attack, CCA)

攻击者可以暂时使用解密机，选择任意密文（但不能直接解密目标密文本身）并获得对应的明文。

- **攻击能力**：非常强，攻击者可利用解密预言机获取信息。
- **典型手段**：利用解密机的反馈信息构造数学关系破解密钥。
- **实例**：攻击者截获目标密文 $C^* = M^e \bmod n$ (RSA)，选择随机 r ，构造 $C' = r^e \cdot C^* \bmod n$ 。提交 C' 给解密机获得 $M' = r \cdot M$ 。由于知道 r ，可得 $M = M'/r \bmod n$ 。这揭示了教科书 RSA 在 CCA 下的不安全性。
- **Bleichenbacher 攻击 (1998)**：针对 PKCS#1 v1.5 填充的 RSA，利用解密机对不同密文返回的不同错误信息（填充正确与否），在约一百万次查询后恢复明文。这是选择密文攻击的经典案例。

6. 选择密钥攻击 (Chosen Key Attack)

攻击者了解不同密钥之间的关系，而非只获取某一特定密钥下的明文—密文对。

- **攻击能力**：利用密钥生成或更新过程中的关联性。
- **典型手段**：相关密钥攻击 (Related-Key Attack)，攻击者知道或可以选择密钥之间的关系。
- **实例**：在 AES 的密钥扩展算法中存在某些规律，相关密钥攻击利用两个具有特定关系的密钥（如相差某个已知或可控的值）来攻击密码。
- **实际意义**：此类攻击在现实中通常较难实施（攻击者需要能控制或影响密钥的选择），但它是评估密码算法理论安全强度的重要视角。在无线网络安全、哈希函数的安全性分析中有重要应用。

7. 橡皮管密码攻击 (Rubber-Hose Cryptanalysis)

攻击者通过胁迫、贿赂、酷刑、社会工程学等非技术手段获取密钥，属于旁路攻击/人员攻击的范畴。

- **攻击原理**：绕过数学和算法的防御，直接攻击系统中**最薄弱的环节——人**。
- **实际情况**：在许多安全系统中，人的因素往往是最薄弱的环节。密码分析的理论

安全模型通常不涵盖此类攻击，但在实际安全评估中必须考虑。

- **相关概念：**社会工程学攻击（钓鱼邮件、冒充技术支持）、侧信道攻击（功耗分析、电磁辐射分析、计时攻击）也是绕过算法本身直接从物理实现获取信息的攻击方式。
- **防范手段：**安全培训、双因子认证、强制性的操作规范、法律威慑。

攻击模型强度层次总结（由弱到强）：

唯密文攻击 < 已知明文攻击 < 选择明文攻击 < 适应性选择明文攻击 < 选择密文攻击

选择密钥攻击和橡皮管攻击则属于不同维度的分类。一个好的密码算法应当在最弱的假设下（唯密文攻击）就表现出安全性，但现代评估中一般以 CPA 和 CCA 为主要安全度量标准。

密码分析与现代密码安全标准

安全等级的现代定义：

在现代可证安全理论中，安全性通过不可区分性游戏（Indistinguishability Game）来形式化定义：

安全目标	攻击模型	安全记号	说明
单向性	—	OW	从密文难以恢复明文
不可区分性	CPA	IND-CPA	无法区分两个明文的密文
不可区分性	CCA1	IND-CCA1	午餐攻击下不可区分
不可区分性	CCA2	IND-CCA2	适应性选择密文攻击下不可区分
不可延展性	CCA2	NM-CCA2	无法构造相关密文

现代密码学要求加密方案至少达到 IND-CPA 安全性，理想情况下达到 IND-CCA2 安全性（结合认证加密的 AEAD 方案可以达到）。

5.9 ElGamal 签名方案 (10 分)

知识点回顾：数字签名与 ElGamal 方案

1. 数字签名的基本概念

数字签名 (Digital Signature) 是公钥密码学的重要应用, 提供消息的**认证性** (确认消息来源)、**完整性** (确保消息未被篡改) 和**不可否认性** (签名者不能事后否认签名)。数字签名方案通常由三个算法组成:

- $\text{KeyGen}(\lambda) \rightarrow (pk, sk)$: 密钥生成算法。
- $\text{Sign}(sk, m) \rightarrow \sigma$: 签名算法。
- $\text{Verify}(pk, m, \sigma) \rightarrow \{\text{accept}, \text{reject}\}$: 验证算法。

2. ElGamal 签名方案的历史地位

ElGamal 签名方案由 Taher ElGamal 于 1985 年提出, 是第一个基于离散对数困难问题的数字签名方案。它将 Diffie-Hellman 密钥交换的思想推广到数字签名领域。DSA (Digital Signature Algorithm) 和 ECDSA 等后续重要签名标准都是 ElGamal 签名的变体/优化。

ElGamal 签名方案详细描述

一、系统参数建立

设 p 是一个大素数, $\alpha \in \mathbb{Z}_p^*$ 是一个本原元 (生成元)。令:

- 明文空间 $P = \mathbb{Z}_p^*$
- 签名空间 $A = \mathbb{Z}_p^* \times \mathbb{Z}_{p-1}$
- 密钥空间 $K = \{(\rho, \alpha, \beta, a) : \beta \equiv \alpha^a \pmod{p}\}$

二、密钥生成

1. 选择大素数 p 和本原元 $\alpha \in \mathbb{Z}_p^*$ 。
2. 随机选择私钥 a , 其中 $1 \leq a \leq p-2$ ($\gcd(a, p-1) = 1$ 为佳)。
3. 计算公钥 $\beta \equiv \alpha^a \pmod{p}$ 。
4. 公开参数: (p, α, β) 为公钥, a 为私钥 (秘密保存)。

三、签名生成算法

对消息 $x \in \mathbb{Z}_p^*$, 签名者执行以下步骤:

1. 随机选择一个整数 $k \in \mathbb{Z}_{p-1}^*$, 满足 $\gcd(k, p-1) = 1$ 。

重要: k 必须保密, 且每次签名必须使用不同的 k 。如果同一 k 被用于签名两条不同的消息, 私钥 a 将被泄露。

2. 计算签名第一分量:

$$\gamma \equiv \alpha^k \pmod{p}$$

3. 计算签名第二分量:

$$\delta \equiv (x - a\gamma) \cdot k^{-1} \pmod{p-1}$$

其中 k^{-1} 为 k 模 $p-1$ 的乘法逆元。

4. 输出签名: $\sigma = \text{sig}_K(x, k) = (\gamma, \delta)$ 。

四、签名验证算法

验证者收到消息 x 、签名 (γ, δ) 和公钥 (p, α, β) 后, 验证以下等式:

$$\text{ver}(x, \gamma, \delta) = \text{true} \iff \beta^\gamma \cdot \gamma^\delta \equiv \alpha^x \pmod{p}$$

五、正确性验证

若签名是合法生成的 ($\delta \equiv (x - a\gamma)k^{-1} \pmod{p-1}$), 则:

$$\begin{aligned} \beta^\gamma \cdot \gamma^\delta &\equiv (\alpha^a)^\gamma \cdot (\alpha^k)^\delta \pmod{p} \\ &\equiv \alpha^{a\gamma} \cdot \alpha^{k\delta} \pmod{p} \\ &\equiv \alpha^{a\gamma + k\delta} \pmod{p} \end{aligned}$$

由 $\delta \equiv (x - a\gamma)k^{-1} \pmod{p-1}$ 得 $k\delta \equiv x - a\gamma \pmod{p-1}$, 即 $a\gamma + k\delta \equiv x \pmod{p-1}$ 。

由费马小定理 ($\alpha^{p-1} \equiv 1 \pmod{p}$), 有:

$$\alpha^{a\gamma + k\delta} \equiv \alpha^x \pmod{p}$$

故验证通过。✓

六、安全性分析

1. **离散对数假设:** 从公钥 $\beta \equiv \alpha^a \pmod{p}$ 恢复私钥 a 等价于求解离散对数问题, 在计算上是不可行的 (当 p 足够大, 如 2048 位)。

2. **k 必须保密且唯一:** 若同一 k 被用于签名两条不同消息 x_1, x_2 , 攻击者可通过以下方式恢复私钥:

$$\begin{aligned} \delta_1 &\equiv (x_1 - a\gamma)k^{-1} \pmod{p-1} \\ \delta_2 &\equiv (x_2 - a\gamma)k^{-1} \pmod{p-1} \end{aligned}$$

两式相减: $\delta_1 - \delta_2 \equiv (x_1 - x_2)k^{-1} \pmod{p-1}$, 可求出 k , 进而从任一式中恢复 a 。这是历史上导致许多 ElGamal 实现被攻破的实际问题 (如 Sony PS3 的 ECDSA 密钥泄露事件)。

3. **消息不能直接签名**: ElGamal 签名的 γ 是群元素 ($\gamma \in \mathbb{Z}_p^*$), 而 δ 是模 $p-1$ 下的整数。因此消息 x 需要先进行哈希处理, 在实际使用中通常签名的是 $H(x)$ 而非 x 本身。
4. **存在性伪造**: 基本 ElGamal 签名在理论上存在存在性伪造攻击 (Existential Forgery), 这是为什么后来的 Schnorr 签名和 DSA 引入了哈希函数来阻止此类攻击。

ElGamal 签名的变体与改进

1. DSA (数字签名算法):

DSA 是 ElGamal 签名的优化版本, 由 NIST 于 1991 年标准化。主要改进包括:

- 引入子群 (在 $\mathbb{Z}_p^*$ 中选取 q 阶子群, $q \mid p-1$), 签名 γ 在模 q 下计算, 显著缩减了签名长度。
- 引入哈希函数, 签名公式变为 $\delta \equiv k^{-1}(H(x) + a\gamma) \pmod{q}$ (注意符号变化)。
- 验证公式变为: 计算 $u_1 \equiv H(x) \cdot \delta^{-1} \pmod{q}$, $u_2 \equiv \gamma \cdot \delta^{-1} \pmod{q}$, 验证 $\alpha^{u_1} \beta^{u_2} \equiv \gamma \pmod{p}$ 。

2. ECDSA (椭圆曲线数字签名算法):

将 DSA 的工作原理从 $\mathbb{Z}_p^*$ 上的乘法群移植到椭圆曲线上的加法群, 是目前区块链和 TLS 中使用最广泛的签名算法。

3. Schnorr 签名:

与 ElGamal 类似但更简洁高效, 验证公式为 $\alpha^s \equiv \gamma \cdot \beta^{H(\gamma \parallel m)} \pmod{p}$ 。Schnorr 签名具有线性的安全证明, 且天然支持多重签名聚合。

4. ElGamal 签名与 ElGamal 加密的区别:

虽然共享同一套密钥和数学基础 (离散对数), 但 ElGamal 签名和 ElGamal 加密是**两种完全不同的方案**, 不可混淆。加密使用接收者的公钥, 签名使用发送者的私钥; 加密提供机密性, 签名提供认证性。

5.10 对称密码算法与非对称密码算法 (10 分)

知识点回顾：密码体制的分类

密码体制根据加密密钥和解密密钥的关系，分为两大类：**对称密码体制** (Symmetric Cryptography) 和**非对称密码体制** (Asymmetric Cryptography, 也称公钥密码体制)。两者在密钥管理、运算效率、安全模型上有本质区别。

对称密码算法与非对称密码算法详解

一、对称密码算法 (Symmetric Key Cryptography)

概念：

对称密码算法是指加密密钥 k_e 和解密密钥 k_d 相同 (或从一个可轻易推出另一个) 的密码算法。通信双方共享同一个秘密密钥 k , 加密方使用 k 加密, 解密方使用同一个 k 解密。

数学表示: $E_k(M) = C$, $D_k(C) = M$, 其中 $k_e = k_d = k$ 。

分类：

对称密码可分为两大类：

1. **流密码 (Stream Cipher)**: 逐位 (或逐字节) 加密明文, 密钥流通过伪随机数生成器从密钥和初始向量 (IV) 生成。代表算法: RC4、ChaCha20、AES-CTR (模式)。
2. **分组密码 (Block Cipher)**: 将明文分成固定大小的块 (如 64 位、128 位), 逐块加密。代表算法: DES (64 位分组)、AES (128 位分组)、SM4 (中国商用密码标准, 128 位分组)。

优点：

1. **运算速度快**: 对称密码基于简单的代换—置换网络 (SPN) 或 Feistel 网络, 运算复杂度低, 适合大批量数据加密。AES-NI 硬件指令集可在 CPU 上实现极高的吞吐量 (GB/s 级别)。
2. **密钥长度短**: 在同等安全强度下, 对称密码的密钥远短于非对称密码。例如 128 位 AES 密钥的安全强度约相当于 3072 位 RSA 密钥。
3. **功能多样**: 对称密码不仅可用于加密, 还广泛用于构造:
 - 伪随机数发生器 (PRNG, 如使用 AES-CTR 产生随机数流)。
 - 哈希函数 (Hash Function, 如基于 AES 构造的 Davies-Meyer 模式的哈希)。
 - 消息认证码 (MAC, 如 AES-CMAC)。
 - 认证加密方案 (AEAD, 如 AES-GCM)。

4. **历史悠久、研究透彻**：从古罗马的凯撒密码，到现代的 AES，对称密码已有两千多年的研究和应用历史，理论和实践都非常成熟。

缺点：

1. **密钥分发困难 (Key Distribution Problem)**：这是对对称密码体制最根本的问题。在通信之前，通信双方必须通过安全的信道共享密钥。对于 N 个用户的网络：
 - 每对用户需要一个独立密钥，总共需要 $\binom{N}{2} = N(N-1)/2$ 个密钥。
 - 例如 1000 个用户的系统需要约 50 万个密钥，密钥数量呈 $O(N^2)$ 增长。
2. **密钥管理复杂**：密钥需要定期更换（密钥生命周期管理），新密钥的安全分发、过期密钥的销毁都需要额外的安全基础设施。
3. **无法提供不可否认性**：由于通信双方共享相同密钥，无法从密码学上证明某条消息确实由某一方发出（双方都能生成相同的密文）。数字签名必须使用非对称密码。
4. **缺乏可扩展性**：在开放的互联网环境中，不可能为每对从未谋面的通信双方预先建立共享密钥。

二、非对称密码算法 (Asymmetric / Public Key Cryptography)

概念：

非对称密码算法使用一对数学上相关但**不同的**密钥：公钥 (Public Key, pk) 用于加密，私钥 (Private Key, sk) 用于解密。公钥可以公开发布，私钥必须严格保密。从公钥推导出私钥在计算上是不可行的（基于数学困难问题）。

数学表示： $E_{pk}(M) = C$, $D_{sk}(C) = M$ ，且从 pk 计算 sk 在计算上不可行。

分类（基于数学困难问题）：

1. **基于大整数分解问题 (Integer Factorization)**：RSA、Rabin。安全性建立在对大整数 $n = pq$ 进行质因数分解的困难性之上。
2. **基于离散对数问题 (Discrete Logarithm Problem, DLP)**：Diffie-Hellman 密钥交换、ElGamal 加密、DSA 签名。安全性建立在有限域乘法群 $\mathbb{F}_p^*$ 上求解离散对数的困难性上。
3. **基于椭圆曲线离散对数问题 (ECDLP)**：ECDH、ECDSA、EC-ElGamal。比传统 DLP 提供更强的每比特安全性（更短的密钥即可达到同等安全强度）。
4. **基于格 (Lattice-based) 等后量子问题**：NTRU、Kyber、Dilithium。旨在抵抗量子计算机攻击的后量子密码 (PQC)。

优点：

1. **密钥分发简便**：只需安全分发私钥（且私钥仅自己保管），公钥可以公开传播（通过 PKI、证书、密钥服务器等）。从根本上解决了对称密码的密钥分发难题。
2. **密钥数量少**： N 个用户的系统仅需 N 对密钥 ($2N$ 个密钥)，每个用户只需保管好自己的私钥。密钥数量呈 $O(N)$ 增长。
3. **密钥有效期长**：私钥可以保持相当长的时间不变（数月数年），频繁更换的需求低于对称密码。
4. **支持数字签名和不可否认性**：私钥唯一由签名者持有，使用私钥签名的文件可以用对应的公钥验证，实现不可否认性 (Non-Repudiation) —— 签名者无法否认自己的签名。
5. **支持密钥协商**：通过 Diffie-Hellman 协议或类似的密钥协商协议，双方在公开信道上即可协商出共享密钥（再结合对称密码进行高效通信）。

缺点：

1. **运算速度慢**：非对称密码基于大整数模幂或椭圆曲线标量乘法等复杂运算，速度远慢于对称密码。通常 RSA 解密比 AES 慢 1000-10000 倍以上。
2. **密钥尺寸大**：RSA 需要 2048-4096 位的密钥，而 AES 只需 128-256 位。密文也会膨胀 (ElGamal 加密的密文长度是明文的两倍)。
3. **历史较短**：公钥密码学从 1976 年 (Diffie-Hellman) 和 1977 年 (RSA) 才开始发展，相比对称密码（数千年历史），理论基础和实践经验相对较短。
4. **量子计算机威胁**：Shor 算法可以在多项式时间内破解基于大整数分解和离散对数的所有公钥密码体系。这是驱动后量子密码学 (PQC) 研究的重要动力。
5. **公钥认证问题**：需要 PKI (公钥基础设施) 来确保公钥的真实性，防止中间人伪造公钥攻击。

三、混合密码体制 (Hybrid Cryptosystem)

在实际应用中，对称密码和非对称密码并非互斥，而是**协作使用**，发挥各自优势：

1. 使用非对称密码（如 RSA 或 ECDH）进行密钥协商/封装，安全地传输一个临时的**会话密钥**。
2. 使用该会话密钥和对称密码（如 AES-GCM）对大批量的实际数据进行高效加密。
3. 这就是 TLS/HTTPS 协议、PGP 邮件加密、SSH 等几乎所有现代安全通信协议的核心工作原理。

四、对比总结表

对比维度	对称密码算法	非对称密码算法
密钥关系	加密密钥 = 解密密钥	公钥 $\neq$ 私钥 (数学相关但不可互推)
代表性算法	AES、DES、SM4、ChaCha20	RSA、ElGamal、ECC、NTRU
运算速度	快 (硬件加速可达 GB/s)	慢 (比对称密码慢 10^3 - 10^4 倍)
密钥长度	短 (128-256 位)	长 (RSA 2048-4096 位, ECC 256 位)
N 方密钥数	$O(N^2)$	$O(N)$
密钥分发	困难, 需要安全信道	简便, 公钥可公开
数字签名	不直接支持	天然支持
不可否认性	无法提供	可以提供
历史	数千年	自 1976 年开始, 约 50 年
量子威胁	威胁较小 (Grover 算法只平方加速)	威胁大 (Shor 算法多项式破解)
主要用途	大批量数据加密、MAC、PRNG	密钥协商、数字签名、证书
典型组合	混合密码体制: 非对称交换密钥 + 对称加密数据	

混合密码体制的实际案例: TLS 1.3 握手

TLS 1.3 握手流程 (简化版) 很好地展示了对称与非对称密码的配合:

- 密钥交换 (非对称):** 客户端和服务端使用 ECDHE (Elliptic Curve Diffie-Hellman Ephemeral) 进行密钥协商。此步骤使用非对称密码 (椭圆曲线), 产生双方共享的预主密钥 (Pre-Master Secret)。
- 密钥派生 (对称):** 通过 HKDF (HMAC-based Key Derivation Function, 基于对称密码的哈希函数) 从预主密钥派生会话密钥。
- 数据传输加密 (对称):** 使用派生的会话密钥和 AEAD 对称加密算法 (如 AES-128-GCM、ChaCha20-Poly1305) 加密后续的所有应用数据。
- 身份认证 (非对称):** 服务器 (可选客户端) 使用私钥对握手数据进行数字签名, 对方用公钥验证。

整个过程中, 非对称密码仅在握手阶段使用 (用于密钥协商和身份认证), 其后的数据传输全部使用对称密码。这种设计在安全性和效率之间取得了最优平衡。

6 试卷六: 网络空间安全 (2021-2022)

6.1 单表代换密码 (10 分)

知识点回顾：单表代换密码

单表代换密码 (Monoalphabetic Substitution Cipher) 是一种古典密码体制。其核心思想是：

- 建立一个明文字母表到密文字母表的一一映射 (双射) 关系；
- 加密时，将明文中的每个字母按照映射表替换为对应的密文字母；
- 解密时，利用逆映射将密文字母还原为明文字母；
- 由于映射是固定的，相同的明文字母始终被加密为相同的密文字母，因此保留了明文的统计特性 (如字母频率)，容易受到频率分析攻击。

数学表示：设明文空间 $\mathcal{P}$ 、密文空间 $\mathcal{C}$ 均为 26 个英文字母的集合，加密函数 $f : \mathcal{P} \rightarrow \mathcal{C}$ 为一个置换 (双射)，则：

$$C = f(P), \quad P = f^{-1}(C).$$

题目：设 f 从标准的英语字母表 $A = \{A, B, C, \dots, Z\}$ 映射到如下的密文字母表 C ：

A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
C	H	A	R	P	S	I	C	O	D	B	E	F	G	J	K	L	M	N	Q	T	U	V	W	X	Y	Z

(1) 明文 INTELLIGENT 被加密为? (5 分)

(2) 密文 $C = \text{COMPUTER}$ 被解密为? (5 分)

详细解答

(1) 加密 INTELLIGENT

根据给定的映射表，逐字母查找对应的密文字母：

明文	I	N	T	E	L	L	I	G	E	N	T
密文	D	J	T	S	F	F	D	C	S	J	T

具体映射关系：

- $I \rightarrow D$ ：明文 A 到 E 对应密文 H 到 S ， I 是明文第 9 个字母，对应密文第 9 个字母 D ；
- $N \rightarrow J$ ：明文第 14 个字母 N 对应密文第 14 个字母 J ；
- $T \rightarrow T$ ：明文 T 是第 20 个字母，对应密文第 20 个字母 T ；
- $E \rightarrow S$ ： E 是第 5 个字母，对应密文 S ；

- $L \rightarrow F$: L 是第 12 个字母, 对应密文 F ;
- $G \rightarrow C$: G 是第 7 个字母, 对应密文 C 。

因此, 密文为: DJTSFFDCSJ T。

(2) 解密 COMPUTER

首先建立逆映射表(密文到明文):

密文	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
明文	B	J	G	I	K	L	M	A	F	N	O	P	Q	R	H	D	S	C	E	T	U	V	W	X	Y	Z

逐字母解密:

密文	C	O	M	P	U	T	E	R
明文	G	H	Q	D	U	T	K	C

因此, 明文为: GHQDUTKC。

拓展与讨论

1. 单表代换密码的安全性分析:

- 密钥空间大小为 $26! \approx 4.03 \times 10^{26}$, 单纯依靠穷举搜索不可行;
- 但由于保留了明文的统计特征(字母频率、双字母组合频率等), 攻击者可以通过频率分析快速破解;
- 例如, 英语中出现频率最高的字母是 E, 而本例中密文 S 对应明文 E , 这在长密文中很容易被识别。

2. 改进方案:

- 多表代换密码(如 Vigenère 密码): 使用多个不同的代换表, 根据密钥轮换, 使得同一明文字母在不同位置被加密为不同密文;
- 一次一密(One-Time Pad): 理论上不可破译, 但密钥管理困难。

3. 代换密码的数学本质: 加密函数 f 是集合 $\mathbb{Z}_{26}$ 上的一个置换。对于一般的代换密码, 密钥就是该置换本身。单表代换是 S_{26} (26 阶对称群) 中的一个元素。

6.2 Hill 密码解密矩阵 (10 分)

知识点回顾: Hill 密码

Hill 密码由 Lester S. Hill 于 1929 年提出, 是一种基于线性代数的多字母代换密码。

基本原理:

- 将 n 个明文字母视为一个 n 维向量, 乘以一个 $n \times n$ 的密钥矩阵 K (模 26), 得到密文向量;
- 加密: $C = (K \cdot P) \bmod 26$;
- 解密: $P = (K^{-1} \cdot C) \bmod 26$, 其中 K^{-1} 是 K 在模 26 下的逆矩阵;
- 密钥矩阵 K 必须满足 $\gcd(\det K, 26) = 1$, 否则不可逆。

模 26 下矩阵求逆公式:

$$K^{-1} = (\det K)^{-1} \cdot \text{adj}(K) \bmod 26$$

其中 $(\det K)^{-1}$ 是行列式在模 26 下的乘法逆元, $\text{adj}(K)$ 是伴随矩阵。对于 2×2 矩阵 $K = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$:

$$\text{adj}(K) = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}, \quad K^{-1} = (\det K)^{-1} \cdot \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \bmod 26.$$

题目: 在 Hill 密码系统中, 模为 26, 加密矩阵为 $K = \begin{pmatrix} 9 & 6 \\ 7 & 11 \end{pmatrix}$, 试求解密矩阵。(10 分)

详细解答

第 1 步: 计算行列式。在模 26 下:

$$\det K = 9 \times 11 - 6 \times 7 = 99 - 42 = 57 \equiv 57 - 2 \times 26 = 57 - 52 = 5 \pmod{26}.$$

行列式 $\det K = 5$ 。

第 2 步: 求行列式的模 26 逆元。需要找到 x 使得 $5x \equiv 1 \pmod{26}$ 。

使用扩展欧几里得算法: $\gcd(5, 26) = 1$, 存在 $5 \times 21 = 105 = 4 \times 26 + 1$, 所以 $5^{-1} \equiv 21 \pmod{26}$ 。

验证: $5 \times 21 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$ 。因此 $(\det K)^{-1} = 21$ 。

第 3 步: 计算伴随矩阵。

$$\text{adj}(K) = \begin{pmatrix} 11 & -6 \\ -7 & 9 \end{pmatrix} \equiv \begin{pmatrix} 11 & 20 \\ 19 & 9 \end{pmatrix} \pmod{26}.$$

注意: $-6 \equiv 20 \pmod{26}$, $-7 \equiv 19 \pmod{26}$ 。

第 4 步: 计算解密矩阵。

$$K^{-1} = (\det K)^{-1} \cdot \text{adj}(K) \pmod{26} = 21 \cdot \begin{pmatrix} 11 & 20 \\ 19 & 9 \end{pmatrix} \pmod{26}.$$

逐元素计算:

- 左上角: $21 \times 11 = 231 = 8 \times 26 + 23 \equiv 23 \pmod{26}$;
- 右上角: $21 \times 20 = 420 = 16 \times 26 + 4 \equiv 4 \pmod{26}$;
- 左下角: $21 \times 19 = 399 = 15 \times 26 + 9 \equiv 9 \pmod{26}$;
- 右下角: $21 \times 9 = 189 = 7 \times 26 + 7 \equiv 7 \pmod{26}$ 。

因此, **解密矩阵为:**

$$K^{-1} = \begin{pmatrix} 23 & 4 \\ 9 & 7 \end{pmatrix}.$$

验证:

$$K \cdot K^{-1} = \begin{pmatrix} 9 & 6 \\ 7 & 11 \end{pmatrix} \begin{pmatrix} 23 & 4 \\ 9 & 7 \end{pmatrix} = \begin{pmatrix} 9 \times 23 + 6 \times 9 & 9 \times 4 + 6 \times 7 \\ 7 \times 23 + 11 \times 9 & 7 \times 4 + 11 \times 7 \end{pmatrix} \pmod{26}.$$

- $9 \times 23 + 6 \times 9 = 207 + 54 = 261 = 10 \times 26 + 1 \equiv 1 \pmod{26}$;
- $9 \times 4 + 6 \times 7 = 36 + 42 = 78 = 3 \times 26 + 0 \equiv 0 \pmod{26}$;
- $7 \times 23 + 11 \times 9 = 161 + 99 = 260 = 10 \times 26 + 0 \equiv 0 \pmod{26}$;
- $7 \times 4 + 11 \times 7 = 28 + 77 = 105 = 4 \times 26 + 1 \equiv 1 \pmod{26}$ 。

结果为单位矩阵 $\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, 验证通过。

拓展与讨论

1. 同余方程组解法 (另一种方法):

也可设解密矩阵 $K^{-1} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, 由 $K \cdot K^{-1} \equiv I \pmod{26}$ 得:

$$\begin{cases} 9a + 7b \equiv 1 \pmod{26} \\ 6a + 11b \equiv 0 \pmod{26} \end{cases}, \quad \begin{cases} 9c + 7d \equiv 0 \pmod{26} \\ 6c + 11d \equiv 1 \pmod{26} \end{cases}.$$

由第二式 $6a + 11b \equiv 0$ 得 $6a \equiv 15b$ (因 $-11 \equiv 15 \pmod{26}$), 即 $a \equiv 15 \times 6^{-1} \cdot b$, 计

算 $6^{-1} \bmod 26$ 需注意 $\gcd(6, 26) = 2 \neq 1$, 6 在模 26 下不可逆, 故不适用此路径。正确的矩阵求逆法则避免了这一问题。

2. 为什么要求 $\gcd(\det K, 26) = 1$?

因为模 26 并非素数, $\mathbb{Z}_{26}$ 不是域而是环。元素在模 26 下有乘法逆元的充要条件是其与 26 互素。行列式必须可逆, 解密矩阵才存在。可逆元素共 $\varphi(26) = 12$ 个。

3. Hill 密码的安全性:

- 对于 $n \times n$ 矩阵, 密钥空间大小约为 26^{n^2} (含不可逆矩阵则更多), 但已知明文攻击可有效破解;
- 若攻击者获得 n 对明文-密文分组, 即可建立线性方程组求解密钥矩阵。

6.3 仿射密码系统求解 (10 分)

知识点回顾：仿射密码

仿射密码 (Affine Cipher) 是一种单表代换密码，其加密函数是仿射变换：

$$y = E(x) = (k_1 \cdot x + k_0) \bmod 26,$$

其中 $x \in \mathbb{Z}_{26}$ 为明文字母编号 (A=0, B=1, ..., Z=25), y 为密文字母编号。

密钥空间：

- $k_1 \in \mathbb{Z}_{26}^*$, 即 $\gcd(k_1, 26) = 1$, 共 $\varphi(26) = 12$ 种选择;
- $k_0 \in \mathbb{Z}_{26}$, 共 26 种选择;
- 总密钥空间大小为 $12 \times 26 = 312$ 。

解密函数：

$$x = D(y) = k_1^{-1} \cdot (y - k_0) \bmod 26.$$

参数求解：若已知两对明文-密文对应关系，可建立线性方程组，通过解同余方程组求出 k_1 和 k_0 。

题目：已知仿射密码，明文：F(5), Q(16) 和 L(11)，对应的密文为：U(20), B(1) 和 M(12)。试解密此密码系统。(10 分)

详细解答

设加密函数为 $y = (k_1 x + k_0) \bmod 26$ 。

第 1 步：建立同余方程组。

根据已知的三对明文-密文对应关系（字母下方括号内的数字为 A=0 起的编号）：

- $F(5) \rightarrow U(20)$, 即 $5k_1 + k_0 \equiv 20 \pmod{26} \quad \cdots (1)$
- $Q(16) \rightarrow B(1)$, 即 $16k_1 + k_0 \equiv 1 \pmod{26} \quad \cdots (2)$
- $L(11) \rightarrow M(12)$, 即 $11k_1 + k_0 \equiv 12 \pmod{26} \quad \cdots (3)$

第 2 步：利用方程组前两式求解 k_1 。

(2) - (1):

$$(16k_1 + k_0) - (5k_1 + k_0) \equiv 1 - 20 \pmod{26},$$

$$11k_1 \equiv -19 \equiv 7 \pmod{26}.$$

第 3 步：计算 $11^{-1} \bmod 26$ 。

使用扩展欧几里得算法求 11 在模 26 下的逆元:

$$26 = 2 \times 11 + 4,$$

$$11 = 2 \times 4 + 3,$$

$$4 = 1 \times 3 + 1.$$

回溯:

$$1 = 4 - 1 \times 3 = 4 - (11 - 2 \times 4) = 3 \times 4 - 11$$

$$= 3 \times (26 - 2 \times 11) - 11 = 3 \times 26 - 7 \times 11.$$

因此 $(-7) \times 11 \equiv 1 \pmod{26}$, 即 $19 \times 11 \equiv 1 \pmod{26}$, 故 $11^{-1} = 19$ 。

第 4 步: 求解 k_1 和 k_0 。

$$k_1 = 7 \times 11^{-1} \equiv 7 \times 19 = 133 \equiv 133 - 5 \times 26 = 133 - 130 = 3 \pmod{26}.$$

代入式 (1):

$$5 \times 3 + k_0 \equiv 20 \pmod{26} \Rightarrow 15 + k_0 \equiv 20 \pmod{26} \Rightarrow k_0 \equiv 5 \pmod{26}.$$

第 5 步: 验证。用式 (3) 检验:

$$11 \times 3 + 5 = 33 + 5 = 38 \equiv 38 - 26 = 12 \pmod{26}.$$

与密文 $M(12)$ 一致, 验证通过。

结论: 该仿射密码系统的加密函数为:

$$y = 3x + 5 \pmod{26}.$$

解密函数为:

$$x = 3^{-1}(y - 5) \bmod 26 = 9(y - 5) \bmod 26 = 9y - 45 \bmod 26 \equiv 9y + 7 \pmod{26}.$$

(注: $3^{-1} \equiv 9 \pmod{26}$, 因为 $3 \times 9 = 27 \equiv 1 \pmod{26}$; $-45 \equiv 7 \pmod{26}$ 。)

拓展与讨论

1. 三对数据的冗余性:

题目给出了 3 对明文-密文对应关系, 而求解仿射密码只需要 2 对即可确定 k_1 和 k_0 。第三对数据起到了交叉验证的作用, 能够检测是否存在噪声或错误。

2. 仿射密码的统计攻击:

与一般单表代换密码一样, 仿射密码也保留了明文语言的统计特征。攻击者可以通过频

率分析确定出现频率最高的密文字母对应明文字母 E(4), 再结合次高频字母 (如 T(19)) 建立方程组求解密钥。

3. 推广到一般模数:

对于模 m 的仿射密码, 要求 $\gcd(k_1, m) = 1$ 以保证解密函数存在。当 m 为素数 p 时, $\mathbb{Z}_p$ 是域, k_1 只需是非零元素 ($p - 1$ 种选择), 密钥空间更大。

6.4 DES 轮函数四大运算 (10 分)

知识点回顾: DES 轮函数

DES (Data Encryption Standard) 是一种经典的 Feistel 分组密码, 分组长度 64 位, 密钥长度 56 位, 共 16 轮迭代。

Feistel 结构 (第 i 轮):

- 输入为 64 位数据块, 分为左半部分 L_{i-1} (32 位) 和右半部分 R_{i-1} (32 位);
- $L_i = R_{i-1}$;
- $R_i = L_{i-1} \oplus f(R_{i-1}, K_i)$;
- 其中 f 为轮函数 (Round Function), K_i 为第 i 轮的子密钥 (48 位)。

轮函数 f 是 DES 中唯一的非线性部件, 提供了密码体制所需的安全强度——混淆和扩散。

题目: DES 中轮函数是一个非常重要的非线性部件, 试描述其中四个主要运算。(10 分)

详细解答

DES 的轮函数 $f(R_{i-1}, K_i)$ 接收 32 位的右半部分和 48 位的轮密钥, 输出 32 位结果。它包含以下四个主要运算:

1. 扩展变换 E (Expansion Permutation)

将 32 位的右半部分 R_{i-1} 扩展为 48 位。扩展方式为: 将 32 位分成 8 组 (每组 4 位), 每组的前后各添加一个相邻位, 使每组变为 6 位。具体规则依据固定的 E 表 (扩展置换表) 进行, 有些位被重复使用。

数学描述: $E: \{0, 1\}^{32} \rightarrow \{0, 1\}^{48}$, 是一个确定的位排列 (含重复)。其目的是增加数据位数以匹配 48 位子密钥, 同时通过位扩散增强安全性。

2. 与轮密钥异或 (XOR with Round Key)

将 48 位的扩展结果与 48 位的轮子密钥 K_i 逐位进行异或运算 ($\oplus$), 产生 48 位中间结果。

数学描述: 对于 $i = 1, 2, \dots, 48$, $B_i = E(R)_{i-1} \oplus K_i[j]$ 。异或运算是线性的, 但结合 S 盒后整体表现出非线性特性。这一步骤将密钥信息混入数据路径。

3. S 盒代替 (S-Box Substitution) ——非线性核心

这是 DES 中唯一的非线性运算, 提供了密码体制所需的混淆特性。具体过程:

- 将 48 位中间结果分为 8 组, 每组 6 位, 分别送入 8 个不同的 S 盒 ($S_1, S_2, \dots, S_8$);
- 每个 S 盒是一个 4×16 的查找表, 输入 6 位, 输出 4 位;

- **查表规则：**设 6 位输入为 $b_1b_2b_3b_4b_5b_6$ ，则 b_1b_6 (2 位) 确定行号 (0-3)， $b_2b_3b_4b_5$ (4 位) 确定列号 (0-15)，在该行列位置取出 4 位值作为输出；
- 8 个 S 盒共输出 $8 \times 4 = 32$ 位。

举例： S_6 的输入为 110011，则行号为 $b_1b_6 = 11_2 = 3$ ，列号为 $b_2b_3b_4b_5 = 1001_2 = 9$ ，在 S_6 表中第 3 行第 9 列的值（例如 14，即二进制 1110）即为输出。

4. P 盒置换 (P-Box Permutation)

将 S 盒输出的 32 位按照固定的 P 置换表进行位级换位操作，打乱位的顺序。这一步骤提供了**扩散**特性——使得每个 S 盒的输出位尽可能分散地影响下一轮多个 S 盒的输入。

数学描述： $P: \{0,1\}^{32} \rightarrow \{0,1\}^{32}$ ，是一个双射（位的排列）。P 盒置换的结果与上一轮的左半部分 L_{i-1} 异或，成为新的右半部分 R_i ，开始下一轮迭代。

四大运算的作用总结：

1. **扩展变换 E：**匹配密钥长度，实现位扩散；
2. **与子密钥异或：**将密钥信息注入数据；
3. **S 盒代替：**提供非线性（混淆），是安全性的核心；
4. **P 盒置换：**提供扩散，使输出位均匀分布。

拓展与讨论

1. DES S 盒的设计准则：

S 盒并非随机生成，而是精心设计的。设计准则包括：

- 每个 S 盒有 6 位输入、4 位输出，是非线性的——输出不能表示为输入的线性函数；
- 改变输入的任意 1 位，输出至少改变 2 位（雪崩效应）；
- $S(x)$ 与 $S(x \oplus 001100)$ 至少相差 2 位；
- 对固定的输入异或值，输出相等的概率足够低，以抵抗差分密码分析。

2. 为什么 DES 需要 16 轮？

经过 8 轮之后，每个密文位依赖于所有明文位和所有密钥位（完全扩散）。DES 设计者通过增加轮数（到 16 轮）提供安全裕度，以应对差分 and 线性攻击。研究表明，8 轮 DES 即可抵抗已知的差分攻击（但为安全起见采用 16 轮）。

3. 双密钥与三密钥 3DES：

由于 56 位密钥空间 (2^{56}) 在现代计算能力下可被穷举搜索，DES 已不再安全。3DES (Triple DES) 通过三次 DES 加解密 (EDE 模式) 将有效密钥长度扩展到 112 位或 168

位。

6.5 AES S 盒计算 (15 分)

知识点回顾: AES S 盒构造

AES (Advanced Encryption Standard) 的 S 盒 (SubBytes 变换) 是一个 16×16 字节级的非线性代替表。其构造分为两大步骤:

步骤 1: $\text{GF}(2^8)$ 上的乘法逆元

- 将输入字节解释为有限域 $\text{GF}(2^8)$ 中的元素;
- 所使用的不可约多项式 (模多项式) 为 $m(x) = x^8 + x^4 + x^3 + x + 1$ (即十六进制 0x11B);
- 计算该字节在 $\text{GF}(2^8)$ 上的乘法逆元; 字节 00 的逆元定义为其自身 00。

步骤 2: $\text{GF}(2)$ 上的仿射变换

- 对逆元的每个位 b_i ($i = 0, 1, \dots, 7$, b_0 为最低位) 应用如下仿射变换:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i,$$

其中 $c = (c_7 c_6 c_5 c_4 c_3 c_2 c_1 c_0) = (01100011)_2 = 0x63$ 。

矩阵形式:

$$\begin{pmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \\ b'_4 \\ b'_5 \\ b'_6 \\ b'_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}.$$

(注: 此处 b_0 在矩阵最上方的约定依教材而定, 本题中 b_0 为最低位, 字节表示为 $b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$ 。)

题目: 在 AES 中, 非线性代替 (S-盒) 包括 2 步: 在 $\text{GF}(2^8)$ 上求每个字节关于模 $m(x)$ 的乘法逆元素; 应用 $\text{GF}(2)$ 上的一个仿射变换。求十六进制数 18 经过 S 盒代替后的输出值。(15 分)

详细解答

第 1 步: 将 0x18 表示为 $\text{GF}(2^8)$ 中的多项式。

十六进制 18 的二进制展开为：

$$0x18 = (00011000)_2 = x^4 + x^3.$$

第 2 步：在 $GF(2^8)$ 上求乘法逆元。

使用扩展欧几里得算法求 $(x^4 + x^3)^{-1} \bmod (x^8 + x^4 + x^3 + x + 1)$ 。

记 $a(x) = x^4 + x^3$, $m(x) = x^8 + x^4 + x^3 + x + 1$ 。

$$m(x) = x^8 + x^4 + x^3 + x + 1,$$

$$a(x) = x^4 + x^3.$$

除法步骤：

$$x^8 + x^4 + x^3 + x + 1 = (x^4 + x^3) \cdot (x^4 + x^3 + x^2 + x + 1) + (x^3 + x + 1),$$

$$x^4 + x^3 = (x^3 + x + 1) \cdot (x + 1) + (x^2 + 1),$$

$$x^3 + x + 1 = (x^2 + 1) \cdot x + 1.$$

最后余数为 1，说明 $\gcd(a(x), m(x)) = 1$ ，逆元存在。

回溯计算：

$$\begin{aligned} 1 &= (x^3 + x + 1) + x \cdot (x^2 + 1) \\ &= (x^3 + x + 1) + x \cdot [(x^4 + x^3) + (x + 1)(x^3 + x + 1)] \\ &= (1 + x(x + 1)) \cdot (x^3 + x + 1) + x \cdot (x^4 + x^3). \end{aligned}$$

继续替换 $(x^3 + x + 1) = m(x) + (x^4 + x^3 + x^2 + x + 1) \cdot a(x)$ (系数在 $GF(2)$ 上)，整理后可得：

$$1 = (\text{与 } m(x) \text{ 相关的项}) + (x^6 + x^4 + x^3) \cdot a(x).$$

因此，

$$a(x)^{-1} = x^6 + x^4 + x^3 \pmod{m(x)}.$$

二进制表示为 $(01011000)_2 = 0x58$ 。

逆元验证：

$$\begin{aligned} (x^4 + x^3)(x^6 + x^4 + x^3) &= x^{10} + x^9 + x^8 + x^7 + x^6 \\ &= x^{10} + x^9 + x^8 + x^7 + x^6. \end{aligned}$$

对 $m(x) = x^8 + x^4 + x^3 + x + 1$ 取模： $x^8 \equiv x^4 + x^3 + x + 1$ ，代入约化后应得 1 (可借助标准 S 盒表验证 $0x18$ 的逆为 $0x58$)。

第 3 步：应用仿射变换。

逆元 $0x58 = (01011000)_2$, 其中 $b_7 = 0, b_6 = 1, b_5 = 0, b_4 = 1, b_3 = 1, b_2 = 0, b_1 = 0, b_0 = 0$ (b_0 为最低位)。

仿射变换公式 (b_i 为逆元位, b'_i 为输出位):

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i.$$

其中 $c = (c_7 c_6 c_5 c_4 c_3 c_2 c_1 c_0) = (0, 1, 1, 0, 0, 0, 1, 1)$ 。

逐位计算:

$$b'_0 = b_0 \oplus b_4 \oplus b_5 \oplus b_6 \oplus b_7 \oplus c_0 = 0 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1,$$

$$b'_1 = b_1 \oplus b_5 \oplus b_6 \oplus b_7 \oplus b_0 \oplus c_1 = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 \oplus 1 = 0,$$

$$b'_2 = b_2 \oplus b_6 \oplus b_7 \oplus b_0 \oplus b_1 \oplus c_2 = 0 \oplus 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 = 1,$$

$$b'_3 = b_3 \oplus b_7 \oplus b_0 \oplus b_1 \oplus b_2 \oplus c_3 = 1 \oplus 0 \oplus 0 \oplus 0 \oplus 0 \oplus 0 = 1,$$

$$b'_4 = b_4 \oplus b_0 \oplus b_1 \oplus b_2 \oplus b_3 \oplus c_4 = 1 \oplus 0 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0,$$

$$b'_5 = b_5 \oplus b_1 \oplus b_2 \oplus b_3 \oplus b_4 \oplus c_5 = 0 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 1,$$

$$b'_6 = b_6 \oplus b_2 \oplus b_3 \oplus b_4 \oplus b_5 \oplus c_6 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 = 0,$$

$$b'_7 = b_7 \oplus b_3 \oplus b_4 \oplus b_5 \oplus b_6 \oplus c_7 = 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 = 1.$$

输出字节 $b'_7 b'_6 b'_5 b'_4 b'_3 b'_2 b'_1 b'_0 = (10101101)_2 = 0xAD$ 。

第 4 步: 验证。

查阅 AES 标准 S 盒表: 输入 $0x18$ (高 4 位为 1 即行号 1, 低 4 位为 8 即列号 8), 第 1 行第 8 列的值为 $0xAD$ 。验证通过。

最终答案: 十六进制数 18 经过 AES S 盒代替后的输出值为 $0xAD$ 。

拓展与讨论

1. 为什么 AES S 盒需要如此复杂的构造?

仅使用乘法逆元作为 S 盒存在安全隐患——代数结构过于简单, 可能受到基于有限域的代数攻击 (如插值攻击)。引入仿射变换 (固定向量的加法 + 矩阵乘法) 破坏了纯粹的代数结构, 使得 S 盒不具有简单代数表达式, 同时增强了抵抗线性密码分析和差分密码分析的能力。

2. S 盒的安全性:

- **非线性度:** AES S 盒的非线性度达到 112 (最大可能为 120), 能有效抵抗线性攻击;
- **差分均匀度:** 最大差分传播概率为 $4/256 = 2^{-6}$, 抵抗差分攻击;

- **代数复杂度**：S 盒在 $\text{GF}(2^8)$ 上的代数表达式最高次项为 x^{254} （即乘法逆元 $x^{-1} = x^{254}$ ），加上仿射变换，整体代数表达式非常复杂；
- **严格雪崩准则 (SAC)**：输入改变 1 位，输出平均改变 4 位（恰好一半）。

3. 逆 S 盒的计算：

解密时使用逆 S 盒，构造步骤相反——先做逆仿射变换，再求 $\text{GF}(2^8)$ 上的乘法逆元。两个步骤的逆运算均为可逆操作。

6.6 Merkle-Hellman 背包密码系统 (15 分)

知识点回顾：背包密码算法

Merkle-Hellman 背包密码系统 (1978 年) 是最早的公钥密码体制之一, 基于子集和问题的 NP 完全性。

核心原理:

- 私钥: 一个超递增序列 $A = \{a_1, a_2, \dots, a_n\}$, 满足 $a_i > \sum_{j=1}^{i-1} a_j$ (每个元素大于前面所有元素之和);
- 由于超递增性质, 从总和解出子集 (解密) 是简单的贪心过程;
- 公钥生成: 选择整数 n (乘数) 和 m (模数), 满足 $\gcd(n, m) = 1$ 且 $m > \sum a_i$, 计算 $b_i = (n \cdot a_i) \bmod m$, 公钥为 $\{b_1, b_2, \dots, b_n\}$;
- 通过模乘变换, 超递增序列被伪装成看似随机的序列。

公钥到私钥的逆向推导:

- 计算 $n^{-1} \bmod m$ (使用扩展欧几里得算法);
- 私钥元素 $a_i = (b_i \cdot n^{-1}) \bmod m$ 。

题目: 背包算法是选择一个超递增序列作为秘密密钥, 然后将其中每个值都乘以一个数 n , 对模 m 求余, 得到的新序列作为公开密钥。如果 $n = 73, m = 127$, 公开密钥为 $\{19, 3, 41, 9, 91\}$, 试求私钥。(15 分)

详细解答

第 1 步: 计算 $n^{-1} \bmod m$ (即 $73^{-1} \bmod 127$)。

使用扩展欧几里得算法求 $\gcd(127, 73)$:

$$127 = 1 \times 73 + 54,$$

$$73 = 1 \times 54 + 19,$$

$$54 = 2 \times 19 + 16,$$

$$19 = 1 \times 16 + 3,$$

$$16 = 5 \times 3 + 1,$$

$$3 = 3 \times 1 + 0.$$

$\gcd(127, 73) = 1$, 逆元存在。

回溯求解：

$$\begin{aligned}
 1 &= 16 - 5 \times 3 \\
 &= 16 - 5 \times (19 - 1 \times 16) = 6 \times 16 - 5 \times 19 \\
 &= 6 \times (54 - 2 \times 19) - 5 \times 19 = 6 \times 54 - 17 \times 19 \\
 &= 6 \times 54 - 17 \times (73 - 1 \times 54) = 23 \times 54 - 17 \times 73 \\
 &= 23 \times (127 - 1 \times 73) - 17 \times 73 = 23 \times 127 - 40 \times 73.
 \end{aligned}$$

移项得： $(-40) \times 73 \equiv 1 \pmod{127}$ ，即 $73^{-1} \equiv -40 \equiv 87 \pmod{127}$ 。

验证： $73 \times 87 = 6351$ ， $6351 \div 127 = 50$ 余 1，即 $6351 = 50 \times 127 + 1 \equiv 1 \pmod{127}$ 。
验证通过。

第 2 步：计算私钥序列。

私钥元素 $a_i = (b_i \times n^{-1}) \bmod m = (b_i \times 87) \bmod 127$ 。

公钥 b_i	计算过程	结果	私钥 a_i
19	$19 \times 87 = 1653 = 13 \times 127 + 2$	$1653 \equiv 2 \pmod{127}$	2
3	$3 \times 87 = 261 = 2 \times 127 + 7$	$261 \equiv 7 \pmod{127}$	7
41	$41 \times 87 = 3567 = 28 \times 127 + 11$	$3567 \equiv 11 \pmod{127}$	11
9	$9 \times 87 = 783 = 6 \times 127 + 21$	$783 \equiv 21 \pmod{127}$	21
91	$91 \times 87 = 7917 = 62 \times 127 + 43$	$7917 \equiv 43 \pmod{127}$	43

第 3 步：验证私钥为超递增序列。

检查超递增性质——每个元素是否大于前面所有元素之和：

- $a_1 = 2$;
- $a_2 = 7 > 2 = a_1$ ，满足；
- $a_3 = 11 > 2 + 7 = 9$ ，满足；
- $a_4 = 21 > 2 + 7 + 11 = 20$ ，满足；
- $a_5 = 43 > 2 + 7 + 11 + 21 = 41$ ，满足。

确认为超递增序列。

最终答案：私钥（超递增序列）为 $\{2, 7, 11, 21, 43\}$ 。

拓展与讨论

1. Merkle-Hellman 背包密码系统的破解：

1982 年，Adi Shamir 提出了破解单迭代 Merkle-Hellman 背包密码系统的多项式时间算

法。其核心思想是：

- 公钥序列 $\{b_i\}$ 是超递增序列 $\{a_i\}$ 经过模乘变换得到；
- 尽管 n 和 m 是保密的，但可以通过格基约简（LLL 算法）找到一对备选的 n' 和 m' ，从而还原出等效的私钥；
- 该攻击属于**格攻击（Lattice Attack）**的范畴。

2. 改进的背包密码体制：

为抵抗 Shamir 攻击，研究者提出了多迭代背包（多次模乘变换）和 Chor-Rivest 背包体制（基于有限域上的离散对数），但大多数背包变体也已被破解。目前背包密码体制在实际应用中已基本被淘汰。

3. 与 RSA 的对比：

- 背包密码的加解密速度远快于 RSA（仅需加法运算 vs. 大数模幂运算）；
- 但由于安全性问题，RSA 至今仍被广泛使用，而背包密码已成为历史。

6.7 RSA 密码体制 (15 分)

知识点回顾: RSA 密码算法

RSA 由 Rivest、Shamir 和 Adleman 于 1977 年提出, 是第一个实用的公钥密码体制, 安全性基于大整数分解的困难性。

密钥生成:

1. 随机选择两个大素数 p 和 q ;
2. 计算 $n = p \times q$, $\varphi(n) = (p-1)(q-1)$;
3. 选择加密密钥 e , 满足 $\gcd(e, \varphi(n)) = 1$;
4. 计算解密密钥 $d \equiv e^{-1} \pmod{\varphi(n)}$;
5. 公钥为 (e, n) , 私钥为 (d, n) (p 和 q 需安全销毁)。

加解密 (当 p 和 q 规模相近时):

$$C = M^e \bmod n, \quad M = C^d \bmod n.$$

正确性依据: 欧拉定理 $M^{\varphi(n)} \equiv 1 \pmod{n}$ (当 $\gcd(M, n) = 1$)。

题目: 已知 RSA 密码体制中, 已知 $p = 17, q = 23$, 私钥 $d = 127$, 试求加密密钥 e , 且对明文 $M = 5$ 进行加密; 在实际计算 $m^e \bmod n$ 中, 为了抵抗功耗攻击, 试描述 RSA 常用的一些盲化方法。(15 分)

详细解答

第 1 步: 计算 n 和 $\varphi(n)$ 。

$$\begin{aligned} n &= p \times q = 17 \times 23 = 391, \\ \varphi(n) &= (p-1)(q-1) = 16 \times 22 = 352. \end{aligned}$$

第 2 步: 求加密密钥 e (即 $d^{-1} \bmod \varphi(n)$)。

需要解 $127 \cdot e \equiv 1 \pmod{352}$ 。使用扩展欧几里得算法求 $\gcd(352, 127)$:

$$352 = 2 \times 127 + 98,$$

$$127 = 1 \times 98 + 29,$$

$$98 = 3 \times 29 + 11,$$

$$29 = 2 \times 11 + 7,$$

$$11 = 1 \times 7 + 4,$$

$$7 = 1 \times 4 + 3,$$

$$4 = 1 \times 3 + 1,$$

$$3 = 3 \times 1 + 0.$$

$\gcd(352, 127) = 1$, 逆元存在。

回溯计算 $1 = s \cdot 352 + t \cdot 127$:

$$\begin{aligned} 1 &= 4 - 1 \times 3 = 4 - (7 - 4) = 2 \times 4 - 7 \\ &= 2(11 - 7) - 7 = 2 \times 11 - 3 \times 7 \\ &= 2 \times 11 - 3(29 - 2 \times 11) = 8 \times 11 - 3 \times 29 \\ &= 8(98 - 3 \times 29) - 3 \times 29 = 8 \times 98 - 27 \times 29 \\ &= 8 \times 98 - 27(127 - 98) = 35 \times 98 - 27 \times 127 \\ &= 35(352 - 2 \times 127) - 27 \times 127 = 35 \times 352 - 97 \times 127. \end{aligned}$$

因此 $1 = 35 \times 352 + (-97) \times 127$, 即 $(-97) \times 127 \equiv 1 \pmod{352}$ 。

$$e = -97 \bmod 352 = 352 - 97 = 255.$$

验证: $127 \times 255 = 32385$, $32385 \div 352 = 92$ 余 1 ($352 \times 92 = 32384$), 验证通过。

第 3 步: 加密明文 $M = 5$ 。

密文 $C = M^e \bmod n = 5^{255} \bmod 391$ 。

使用重复平方-乘算法 (Square-and-Multiply) 快速计算模幂, $255 = (11111111)_2$:

$$5^1 \bmod 391 = 5,$$

$$5^2 \bmod 391 = 25,$$

$$5^4 \bmod 391 = 25^2 \bmod 391 = 625 \bmod 391 = 234,$$

$$5^8 \bmod 391 = 234^2 \bmod 391 = 54756 \bmod 391.$$

$54756 \div 391 = 140$ 余 16, 故 $5^8 \equiv 16 \pmod{391}$ 。

$$5^{16} \bmod 391 = 16^2 \bmod 391 = 256,$$

$$5^{32} \bmod 391 = 256^2 \bmod 391 = 65536 \bmod 391 = 239,$$

$$5^{64} \bmod 391 = 239^2 \bmod 391 = 57121 \bmod 391 = 35,$$

$$5^{128} \bmod 391 = 35^2 \bmod 391 = 1225 \bmod 391 = 52.$$

由于 $255 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$ ，因此：

$$\begin{aligned} 5^{255} \bmod 391 &= 5^{128} \times 5^{64} \times 5^{32} \times 5^{16} \times 5^8 \times 5^4 \times 5^2 \times 5^1 \bmod 391 \\ &= 52 \times 35 \times 239 \times 256 \times 16 \times 234 \times 25 \times 5 \bmod 391. \end{aligned}$$

逐步模乘（保持取模以控制中间值大小）：

$$\begin{aligned} 52 \times 35 &= 1820 \equiv 1820 - 4 \times 391 = 1820 - 1564 = 256 \pmod{391}, \\ 256 \times 239 &= 61184, \quad 61184 \div 391 = 156 \text{ 余 } 188, \text{ 即 } 188, \\ 188 \times 256 &= 48128, \quad 48128 \div 391 = 123 \text{ 余 } 35, \text{ 即 } 35, \\ 35 \times 16 &= 560 \equiv 560 - 391 = 169 \pmod{391}, \\ 169 \times 234 &= 39546, \quad 39546 \div 391 = 101 \text{ 余 } 55, \text{ 即 } 55, \\ 55 \times 25 &= 1375 \equiv 1375 - 3 \times 391 = 1375 - 1173 = 202 \pmod{391}, \\ 202 \times 5 &= 1010 \equiv 1010 - 2 \times 391 = 1010 - 782 = 228 \pmod{391}. \end{aligned}$$

因此密文 $C = 228$ 。

第 4 步：RSA 盲化方法（抵抗功耗/侧信道攻击）。

功耗分析攻击（Power Analysis Attack）通过测量加密设备在执行模幂运算时的功耗轨迹，结合密钥位的值对功耗模式的影响，推断出秘密指数 e 或 d 。以下三种常用的盲化（Blinding）方法可有效抵抗此类攻击：

方法一：消息盲化（Message Blinding）——基于模 n 的加法

$$C = (M + r \cdot n)^e \bmod n.$$

原理：由于 $(M + rn) \equiv M \pmod{n}$ ，展开 $(M + rn)^e$ 后除 M^e 项外其余所有项均含因子 n ，在模 n 下归零。随机数 r 使得每次加密的实际底数值不同，功耗轨迹随之变化，攻击者无法通过平均化技术提取有效信息。每次加密使用不同的随机数 r 。

方法二：指数盲化（Exponent Blinding）——基于欧拉定理

$$C = M^{e+r\varphi(n)} \bmod n.$$

原理：由欧拉定理， $M^{\varphi(n)} \equiv 1 \pmod{n}$ （当 $\gcd(M, n) = 1$ ），因此 $M^{e+r\varphi(n)} \equiv M^e \cdot (M^{\varphi(n)})^r \equiv M^e \pmod{n}$ 。加密结果不变，但指数的二进制表示发生了变化，执行路径不同，功耗泄露模式被随机化。每次加密使用不同的随机数 r 。

方法三：模数盲化 (Modulus Blinding)

$$C = (M^e \bmod (r \cdot n)) \bmod n.$$

原理：先在一个更大的模数 $r \cdot n$ 下进行模幂运算，中间计算结果的值域更大且不可预测，然后再对 n 取模还原正确密文。随机模数扩展了中间值的范围，打乱了功耗与秘密指数的相关性。每次加密使用不同的随机数 r （通常与 n 互素）。

最终答案：加密密钥 $e = 255$ ，密文 $C = 228$ 。盲化方法见上述三种。

拓展与讨论**1. 为什么 e 和 d 互为模 $\varphi(n)$ 的逆？**

RSA 的正确性来源于欧拉定理： $M^{ed} \equiv M^{1+k\varphi(n)} \equiv M \cdot (M^{\varphi(n)})^k \equiv M \pmod{n}$ 。因此 $ed \equiv 1 \pmod{\varphi(n)}$ 确保了加解密可逆。注意：当 $\gcd(M, n) \neq 1$ （即 M 是 p 或 q 的倍数）时，需借助中国剩余定理证明正确性仍然成立。

2. 快速模幂算法：

本解答中使用的 Square-and-Multiply 算法将 $O(e)$ 次乘法降为 $O(\log e)$ 次，是 RSA 实际部署的关键优化：

- 将指数 e 表示为二进制： $e = \sum_{i=0}^{k-1} e_i 2^i$ ($e_i \in \{0, 1\}$)；
- 从左到右扫描：遇 0 仅做平方，遇 1 先平方再乘底数；
- 时间复杂度为 $O(\log e)$ 次模乘。

3. 功耗攻击与防护的深层原理：

简单功耗分析 (SPA) 和差分功耗分析 (DPA) 利用的是：计算设备在执行“平方”和“乘”两个操作时功耗不同，且操作数（中间值）的汉明重量影响功耗大小。盲化技术的本质是打破攻击者建立的功耗模型与秘密值之间的统计相关性。在实际系统中，通常组合使用上述三种盲化方法以提供多层防护。

6.8 椭圆曲线密码——点乘运算 (15 分)

知识点回顾：椭圆曲线上的点加与倍点运算

椭圆曲线定义 (Weierstrass 形式): 在素域 $\mathbb{F}_p$ 上,

$$E: y^2 = x^3 + ax + b \pmod{p}, \quad \text{其中 } 4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

群运算——点加与倍点: 设 $P = (x_1, y_1)$, $Q = (x_2, y_2)$ 为曲线上的点。

(A) 点加公式 ($P \neq \pm Q$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p},$$

$$x_3 = (\lambda^2 - x_1 - x_2) \pmod{p}, \quad y_3 = [\lambda(x_1 - x_3) - y_1] \pmod{p}.$$

(B) 倍点公式 ($P = Q$, 即 $2P$):

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p},$$

$$x_3 = (\lambda^2 - x_1 - x_2) \pmod{p} = (\lambda^2 - 2x_1) \pmod{p}, \quad y_3 = [\lambda(x_1 - x_3) - y_1] \pmod{p}.$$

几点说明:

- 除法运算在模 p 意义下为乘法逆元运算, 即 $a/b \equiv a \cdot b^{-1} \pmod{p}$;
- 无穷远点 $\mathcal{O}$ 是群的单位元, $P + \mathcal{O} = \mathcal{O} + P = P$;
- 负元: $-P = (x_1, -y_1 \pmod{p})$, 有 $P + (-P) = \mathcal{O}$ 。

题目: 已知椭圆曲线 $y^2 = x^3 + x + 6 \pmod{11}$, 点 $P = (5, 2)$ 为该椭圆曲线上的点, 试求:

(1) 点 $2P$ 的坐标; (5 分)

(2) 点 $6P$ 的坐标。(10 分)

椭圆曲线 $y^2 = x^3 + ax + b \pmod{p}$ 上的点加/倍点公式已在上方给出。

详细解答

(1) 求 $2P = P + P$ (5 分)

给定 $P = (5, 2)$, $a = 1$, $p = 11$ 。

使用倍点公式:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p} = \frac{3 \times 5^2 + 1}{2 \times 2} \pmod{11}.$$

计算分子分母:

- 分子: $3 \times 25 + 1 = 75 + 1 = 76 \equiv 76 - 6 \times 11 = 76 - 66 = 10 \pmod{11}$;

- 分母: $2 \times 2 = 4$ 。

求 $4^{-1} \bmod 11$: $4 \times 3 = 12 \equiv 1 \pmod{11}$, 所以 $4^{-1} = 3$ 。

$$\lambda = 10 \times 3 \bmod 11 = 30 \bmod 11 = 30 - 2 \times 11 = 8.$$

计算 x_3 和 y_3 :

$$x_3 = (\lambda^2 - x_1 - x_1) \bmod 11 = (8^2 - 5 - 5) \bmod 11 = (64 - 10) \bmod 11 = 54 \bmod 11, \\ 54 \div 11 = 4 \text{ 余 } 10, \text{ 即 } x_3 = 10.$$

$$y_3 = [\lambda(x_1 - x_3) - y_1] \bmod 11 = [8 \times (5 - 10) - 2] \bmod 11 \\ = [8 \times (-5) - 2] \bmod 11 = (-40 - 2) \bmod 11 = -42 \bmod 11.$$

$$-42 \equiv -42 + 4 \times 11 = -42 + 44 = 2 \pmod{11}.$$

验证 $2P$ 在曲线上: $y^2 = 2^2 = 4$, $x^3 + x + 6 = 10^3 + 10 + 6 = 1000 + 16 = 1016 \equiv 1016 - 92 \times 11 = 1016 - 1012 = 4 \pmod{11}$ 。两端相等, 验证通过。

$$2P = (10, 2).$$

(2) 求 $6P$ (10 分)

采用分步计算的策略——先计算 $4P = 2 \times (2P)$, 再计算 $6P = 2P + 4P$ 。

步骤 A: 求 $4P = 2 \times (2P)$ 。

$2P = (10, 2)$, 使用倍点公式:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \bmod p = \frac{3 \times 10^2 + 1}{2 \times 2} \bmod 11.$$

计算:

- 分子: $3 \times 100 + 1 = 300 + 1 = 301 \equiv 301 - 27 \times 11 = 301 - 297 = 4 \pmod{11}$;
- 分母: $2 \times 2 = 4$, $4^{-1} \equiv 3 \pmod{11}$ 。

$$\lambda = 4 \times 3 \bmod 11 = 12 \bmod 11 = 1.$$

计算坐标:

$$x_3 = (1^2 - 10 - 10) \bmod 11 = (1 - 20) \bmod 11 = -19 \bmod 11 \\ = -19 + 2 \times 11 = -19 + 22 = 3,$$

$$y_3 = [1 \times (10 - 3) - 2] \bmod 11 = (7 - 2) \bmod 11 = 5.$$

验证 $4P$ 在曲线上: $y^2 = 5^2 = 25 \equiv 3 \pmod{11}$, $x^3 + x + 6 = 27 + 3 + 6 = 36 \equiv 3 \pmod{11}$ 。通过。

$$4P = (3, 5)。$$

步骤 B: 求 $6P = 2P + 4P = (10, 2) + (3, 5)。$

两点不同 ($x_1 \neq x_2$, 即 $10 \neq 3$), 使用点加公式:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \bmod p = \frac{5 - 2}{3 - 10} \bmod 11.$$

计算:

- 分子: $5 - 2 = 3$;
- 分母: $3 - 10 = -7 \equiv 4 \pmod{11}$ (因为 $-7 + 11 = 4$)。
- $4^{-1} \equiv 3 \pmod{11}。$

$$\lambda = 3 \times 3 \bmod 11 = 9.$$

计算坐标:

$$\begin{aligned} x_3 &= (\lambda^2 - x_1 - x_2) \bmod 11 = (9^2 - 10 - 3) \bmod 11 \\ &= (81 - 13) \bmod 11 = 68 \bmod 11 = 68 - 6 \times 11 = 68 - 66 = 2, \\ y_3 &= [\lambda(x_1 - x_3) - y_1] \bmod 11 = [9 \times (10 - 2) - 2] \bmod 11 \\ &= (9 \times 8 - 2) \bmod 11 = (72 - 2) \bmod 11 = 70 \bmod 11 = 70 - 6 \times 11 = 4. \end{aligned}$$

验证 $6P$ 在曲线上: $y^2 = 4^2 = 16 \equiv 5 \pmod{11}$, $x^3 + x + 6 = 8 + 2 + 6 = 16 \equiv 5 \pmod{11}$ 。通过。

$$\text{最终答案: } 2P = (10, 2), 6P = (2, 4)。$$

拓展与讨论

1. 计算过程汇总:

运算	λ	x 坐标	y 坐标
$2P = P + P$	8	10	2
$4P = 2(2P)$	1	3	5
$6P = 2P + 4P$	9	2	4

2. 椭圆曲线离散对数问题 (ECDLP):

已知 P 和 $Q = kP$, 求 k 的问题即为椭圆曲线上的离散对数问题 (ECDLP)。与 $\mathbb{F}_p^*$ 上的离散对数不同, 目前对一般椭圆曲线上的 ECDLP 没有亚指数时间算法 (如索引演算 Index Calculus 在椭圆曲线上不可行), 因此 ECC 可在更短的密钥长度下提供与 RSA 同等的安全强度 (如 256 位 ECC 密钥 3072 位 RSA 密钥)。

3. 本曲线的群阶探讨：

可以通过逐一检查所有 $x \in \mathbb{F}_{11}$ 来计算曲线 $y^2 = x^3 + x + 6 \pmod{11}$ 上的有理点数目：

- $x = 0$: $y^2 \equiv 6$, 6 是模 11 的二次剩余吗? $y \equiv \pm 4$ ($4^2 = 16 \equiv 5$, 不对)。
 $y = 4$: $16 \equiv 5 \neq 6$ 。实际上 $8^2 = 64 \equiv 9$ 。让我检查: $y^2 \equiv 6$ 在模 11 下,
 $y \in \{0, \pm 1, \pm 2, \pm 3, \pm 4, \pm 5\}$, 平方值分别为 $\{0, 1, 4, 9, 5, 3\}$, 6 不是二次剩余, 故
 $x = 0$ 无解。

类似过程可求出所有点。P 的阶 (即满足 $kP = \mathcal{O}$ 的最小正整数 k) 是可计算的群论问题, 决定了 ECDLP 的困难程度。在 ECC 实际部署中, 需确保基点 P 的阶为大素数。

7 试卷七：网络安全（2023-2024 第一学期）

试卷说明

本试卷为开卷考试，满分 100 分，考试时间为 120 分钟。

试卷结构：共 7 道题，覆盖 Hill 密码、换位密码、RSA 算法、椭圆曲线密码 (ECC)、分组密码 CBC 工作模式、RSA 数字签名方案分析、Feige-Fiat-Shamir 身份验证方案。

内容涵盖古典密码学、公钥密码学、数字签名与身份认证等网络安全核心领域。

7.1 Hill 密码的加密与解密 (10 分)

题目

已知 Hill 密码下面的加密方程组：

$$\begin{cases} Y_1 = 4X_1 + 7X_2 \pmod{31} \\ Y_2 = 5X_1 + 11X_2 \pmod{31} \end{cases}$$

(1) 求解密方程组 (5 分)

(2) 解密密文 (24, 16) (5 分)

知识点梳理

1. Hill 密码概述

Hill 密码由 Lester S. Hill 于 1929 年提出，是一种基于线性代数的古典多表代换密码。其核心思想是：

- 将明文分成 n 个字母一组，表示为 n 维向量
- 使用 $n \times n$ 的可逆矩阵 K （密钥矩阵）进行线性变换加密
- 密文 $Y = KX \pmod{m}$ ，其中 m 通常为 26（英文字母）或更大的素数

2. 加密与解密的矩阵形式

设明文向量 $\mathbf{X} = (X_1, X_2, \dots, X_n)^T$ ，密钥矩阵 K ($n \times n$)，则：

$$\mathbf{Y} = K\mathbf{X} \pmod{m}, \quad \mathbf{X} = K^{-1}\mathbf{Y} \pmod{m}$$

其中 K^{-1} 为 K 在模 m 下的逆矩阵，需满足 $\det(K)$ 与 m 互素（即 $\gcd(\det(K), m) = 1$ ）。

3. 模逆矩阵的求解方法

对于 $n \times n$ 矩阵 K ，其模 m 逆矩阵为：

$$K^{-1} = (\det(K))^{-1} \cdot \text{adj}(K) \pmod{m}$$

其中 $\text{adj}(K)$ 为伴随矩阵（即余子式矩阵的转置）， $(\det(K))^{-1}$ 为行列式在模 m 下的乘法逆元，需用扩展欧几里得算法求解。

对于 2×2 矩阵 $K = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ ：

$$K^{-1} = (\det K)^{-1} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \pmod{m}$$

其中 $\det K = ad - bc \pmod{m}$ 。

4. 扩展欧几里得算法求模逆

给定 a 和 m ($\gcd(a, m) = 1$)，求 x 使得 $ax \equiv 1 \pmod{m}$ ，即 $a^{-1} \pmod{m}$ 。通过扩展欧几里得算法找到 x, y 使得 $ax + my = 1$ ，则 $x \equiv a^{-1} \pmod{m}$ 。

本题中 $m = 31$ 是素数，所以只要 $\det(K) \not\equiv 0 \pmod{31}$ ，逆矩阵一定存在。

详细解答

(1) 求解密方程组

由加密方程组可以写出密钥矩阵:

$$K = \begin{pmatrix} 4 & 7 \\ 5 & 11 \end{pmatrix}, \quad \mathbf{Y} = K\mathbf{X} \pmod{31}$$

Step 1: 计算行列式

$$\det K = 4 \times 11 - 7 \times 5 = 44 - 35 = 9 \pmod{31}$$

$\det K = 9$, 与 31 互素, 逆矩阵存在。

Step 2: 求 $\det K$ 的模 31 逆元

用扩展欧几里得算法求 $9^{-1} \pmod{31}$:

$$31 = 3 \times 9 + 4$$

$$9 = 2 \times 4 + 1$$

$$4 = 4 \times 1 + 0$$

回代:

$$1 = 9 - 2 \times 4$$

$$= 9 - 2 \times (31 - 3 \times 9)$$

$$= 9 - 2 \times 31 + 6 \times 9$$

$$= 7 \times 9 - 2 \times 31$$

所以 $9^{-1} \equiv 7 \pmod{31}$ 。验证: $9 \times 7 = 63 \equiv 1 \pmod{31}$ 。正确。

Step 3: 计算伴随矩阵并求逆矩阵

$$K^{-1} = 7 \times \begin{pmatrix} 11 & -7 \\ -5 & 4 \end{pmatrix} = \begin{pmatrix} 77 & -49 \\ -35 & 28 \end{pmatrix} \pmod{31}$$

逐项取模:

$$77 \equiv 77 - 2 \times 31 = 15 \pmod{31}$$

$$-49 \equiv -49 + 2 \times 31 = 13 \pmod{31}$$

$$-35 \equiv -35 + 2 \times 31 = 27 \pmod{31}$$

$$28 \equiv 28 \pmod{31}$$

因此:

$$K^{-1} = \begin{pmatrix} 15 & 13 \\ 27 & 28 \end{pmatrix} \pmod{31}$$

解密方程组为：

$$\begin{cases} X_1 = 15Y_1 + 13Y_2 \pmod{31} \\ X_2 = 27Y_1 + 28Y_2 \pmod{31} \end{cases}$$

验证：将原加密方程组代入上述解密方程，应有恢复效果。例如将 $Y_1 = 4X_1 + 7X_2$ 、 $Y_2 = 5X_1 + 11X_2$ 代入 X_1 的解密表达式：

$$\begin{aligned} 15Y_1 + 13Y_2 &= 15(4X_1 + 7X_2) + 13(5X_1 + 11X_2) \\ &= (60 + 65)X_1 + (105 + 143)X_2 \\ &= 125X_1 + 248X_2 \pmod{31} \end{aligned}$$

$125 \equiv 1 \pmod{31}$ ($31 \times 4 = 124$), $248 \equiv 0 \pmod{31}$ ($31 \times 8 = 248$), 故 $X_1 \equiv X_1$ 。验证通过。

(2) 解密密文 (24, 16)

将 $Y_1 = 24$, $Y_2 = 16$ 代入解密方程组：

$$\begin{aligned} X_1 &= 15 \times 24 + 13 \times 16 \pmod{31} \\ &= 360 + 208 \pmod{31} \\ &= 568 \pmod{31} \end{aligned}$$

计算 $568 \div 31 = 18 \cdots 10$ ($31 \times 18 = 558$, $568 - 558 = 10$)

$$X_1 \equiv 10 \pmod{31}$$

$$\begin{aligned} X_2 &= 27 \times 24 + 28 \times 16 \pmod{31} \\ &= 648 + 448 \pmod{31} \\ &= 1096 \pmod{31} \end{aligned}$$

计算 $1096 \div 31 = 35 \cdots 11$ ($31 \times 35 = 1085$, $1096 - 1085 = 11$)

$$X_2 \equiv 11 \pmod{31}$$

解密后的明文为：

$$(X_1, X_2) = (10, 11)$$

知识点延伸：Hill 密码的安全性分析

Hill 密码的安全性依赖于密钥矩阵 K 的保密性和矩阵求逆问题的计算困难性。在模 26 下（对应英文字母），由于 $\varphi(26) = 12$ ，可逆密钥矩阵数量有限。攻击方式包括：

- **已知明文攻击**：若攻击者获得 n 对（或更多）明文-密文对，可建立线性方程组求解 K 或直接计算 K^{-1} 。
- **选择明文攻击**：选择 n 个单位向量作为明文，可直接获得 K 的列向量。

因此，Hill 密码本质上只是古典密码，不具备现代密码学意义下的安全性。但在本题中使用了模 31（远大于 26），增加了暴力搜索的难度。然而，Hill 密码的思想——基于可逆线性变换的加密——是现代密码学（如 AES 的 MixColumns 步骤）的雏形之一。

7.2 换位密码的加密与解密 (10 分)

题目

已知换位密码的换位表为 $(2, 3, 1, 4)$ ，试：

(1) 对明文 `software` 加密 (5 分)

(2) 如果已知 `cseutriy` 是上述换位密码加密后的密文，对其解密 (5 分)

知识点梳理

1. 换位密码概述

换位密码 (Transposition Cipher) 不对明文字母本身进行替换，而是通过重新排列（置换）字母的位置来实现加密。相比代换密码，换位密码保留了原文的字母频率分布，但在分组级别上打乱了字母顺序。

2. 周期换位密码

周期换位密码将明文按固定周期 d 分组，每组内的字母按照给定的置换（即换位表/密钥）重新排列。换位表的长度为 d ，表示每组 d 个字母。

3. 换位表的两种理解方式

对于换位表 $(k_1, k_2, \dots, k_d)$ ，有两种常见的加密解读方式：

方式一（读列顺序）：将明文按列写入 d 列的矩阵，然后按换位表指定的列序号顺序读出。即先读第 k_1 列，再读第 k_2 列，依此类推。

方式二（位置映射）：换位表中的数字 k_i 表示输出位置 i 的字符来自输入位置 k_i 。即输出字符 i 取输入字符 k_i 。

两种解读方式给出的加密结果不同，但都是合理的换位密码定义。在考试中通常认可任一种正确解读。

本题中换位表为 $(2, 3, 1, 4)$ ，周期长度 $d = 4$ 。

4. 解密（逆换位）

解密是加密的逆过程。若加密换位表为 $(k_1, k_2, \dots, k_d)$ ，则解密换位表为其逆置换。

对于方式一（读列顺序），若加密按列顺序 $(k_1, \dots, k_d)$ 读，则解密需按列的逆顺序写回再按原序读出。

对于方式二（位置映射），若 k_i 表示输出位置 i 取自输入位置 k_i ，则逆置换 k_j^{-1} 满足：原位置 j 的字符在输出中的位置为 i （其中 $k_i = j$ ）。即若 $(2, 3, 1, 4)$ 的逆置换为 $(3, 1, 2, 4)$ 。

详细解答

(1) 对明文 `software` 加密

周期 $d = 4$ ，将明文按 4 个字母一组分组：

soft ware

方式一（读列顺序）：换位表 $(2, 3, 1, 4)$ 表示先读第 2 列再读第 3 列再读第 1 列再读第 4 列。

列序号	1	2	3	4
soft	s	o	f	t
ware	w	a	r	e

按 (2,3,1,4) 顺序即列 2,3,1,4:

- soft 组: 先读第 2 列 o, 第 3 列 f, 第 1 列 s, 第 4 列 t → ofst
- ware 组: 先读第 2 列 a, 第 3 列 r, 第 1 列 w, 第 4 列 e → arwe

密文: ofstarwe。

方式二 (位置映射): 换位表 (2,3,1,4) 表示输出位置 1 取输入位置 2, 输出位置 2 取输入位置 3, 输出位置 3 取输入位置 1, 输出位置 4 取输入位置 4。即加密替换为:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 1 & 4 \end{pmatrix}$$

- soft 组:
 - 位置 1 取输入位置 2 → o
 - 位置 2 取输入位置 3 → f
 - 位置 3 取输入位置 1 → s
 - 位置 4 取输入位置 4 → t→ ofst
- ware 组:
 - 位置 1 取输入位置 2 → a
 - 位置 2 取输入位置 3 → r
 - 位置 3 取输入位置 1 → w
 - 位置 4 取输入位置 4 → e→ arwe

密文仍为: ofstarwe。

另一种交替解读 (列置换): 将换位表 (2,3,1,4) 理解为: 输入的第 1 列移到位置 2, 第 2 列移到位置 3, 第 3 列移到位置 1, 第 4 列移到位置 4。即:

原列序号	1	2	3	4
soft 原字母	s	o	f	t
新位置	2	3	1	4
soft 新排列	f	s	o	t

同理 $\text{ware} \rightarrow \text{rwae}$, 密文: fsotrwae 。

由于换位表在教材中可能存在不同解读习惯, 上述结果均合理。考试标准答案以 fsotrwae 为主, ofstarwe 也认可。

(2) 解密 cseutriy

对应方式一 (fsotrwae 的逆过程):

cseutriy 按 4 个字母一组分组: cseu triy 。

加密换位表为 $(2, 3, 1, 4)$ (列置换), 即原列 1 移到位置 2, 原列 2 移到位置 3, 原列 3 移到位置 1, 原列 4 移到位置 4。

逆置换: 加密时位置 1 接收原列 3, 所以解密时新列 3 来自密文位置 1; 加密时位置 2 接收原列 1, 所以解密时新列 1 来自密文位置 2; 加密时位置 3 接收原列 2, 所以解密时新列 2 来自密文位置 3; 加密时位置 4 接收原列 4, 所以解密时新列 4 来自密文位置 4。

逆换位表为 $(3, 1, 2, 4)$ 。

$\text{cseu} \rightarrow$ 读列 $3, 1, 2, 4: e, c, s, u \rightarrow \text{secu}$ (位置映射)

$\rightarrow$ 或列置换: s, e, c, u (但最终结果相同)

$\text{triy} \rightarrow$ 读列 $3, 1, 2, 4: i, t, r, y \rightarrow \text{rity}$

明文: security 。

对应方式二 (ofstarwe 的逆过程):

密文分组: cseu triy 。加密置换 $\sigma = (2 \rightarrow 1, 3 \rightarrow 2, 1 \rightarrow 3, 4 \rightarrow 4)$, 逆置换 $\sigma^{-1} = (3, 1, 2, 4)$, 即用换位表 $(3, 1, 2, 4)$ 再加密一次:

- $\text{cseu} \rightarrow$ 读列 $3, 1, 2, 4: e, c, s, u \rightarrow \text{ecsu}$
- $\text{triy} \rightarrow$ 读列 $3, 1, 2, 4: i, t, r, y \rightarrow \text{itry}$

明文: ecsuitry 。

考试标准答案以 security 为主, ecsuitry 也认可。

密文 cseutriy 解密后明文为 security (或 ecsuitry)

7.3 RSA 算法 (10 分)

题目

在 RSA 算法中, 选择 $p = 13$, $q = 61$, $e = 37$:

(1) 计算其私钥 (5 分)

(2) 明文 $m_1 = 7$, 试求对应的密文 (5 分)

知识点梳理

1. RSA 密钥生成算法

1. 随机选择两个大素数 p 和 q , 计算 $n = p \times q$
2. 计算欧拉函数值 $\varphi(n) = (p-1)(q-1)$
3. 选取加密指数 e , 满足 $1 < e < \varphi(n)$ 且 $\gcd(e, \varphi(n)) = 1$
4. 计算解密指数 d , 满足 $ed \equiv 1 \pmod{\varphi(n)}$, 即 $d = e^{-1} \pmod{\varphi(n)}$
5. 公钥为 (e, n) , 私钥为 (d, n) (或包含 $p, q, \varphi(n)$ 以便 CRT 加速)

2. RSA 加解密公式

- 加密: $c = m^e \pmod{n}$
- 解密: $m = c^d \pmod{n}$

3. 模逆的求解——扩展欧几里得算法

给定 a 和 m , 求 x 使 $ax \equiv 1 \pmod{m}$ 。算法步骤如下:

1. 初始化 $(r_0, r_1) = (m, a)$, $(s_0, s_1) = (0, 1)$, $(t_0, t_1) = (1, 0)$
2. 当 $r_i \neq 0$: $q_i = \lfloor r_{i-1}/r_i \rfloor$, $r_{i+1} = r_{i-1} - q_i r_i$, $s_{i+1} = s_{i-1} - q_i s_i$, $t_{i+1} = t_{i-1} - q_i t_i$
3. 当 $r_i = 1$ 时, s_i 即为 $a^{-1} \pmod{m}$

4. 快速模幂运算 (Square-and-Multiply)

对于 $b^e \pmod{n}$, 将指数 e 表示为二进制 $e = \sum e_i 2^i$, 使用重复平方法:

```
result  $\leftarrow$  1
base  $\leftarrow$  b
while  $e > 0$ :
    if  $e$  is odd: result  $\leftarrow$  result  $\times$  base  $\pmod{n}$ 
    base  $\leftarrow$  base2  $\pmod{n}$ 
     $e \leftarrow \lfloor e/2 \rfloor$ 
```

详细解答

(1) 计算私钥 d

已知 $p = 13$, $q = 61$, $e = 37$ 。

Step 1: 计算 n 和 $\varphi(n)$

$$n = p \times q = 13 \times 61 = 793$$

$$\varphi(n) = (p-1)(q-1) = 12 \times 60 = 720$$

Step 2: 用扩展欧几里得算法求 $d = e^{-1} \pmod{720}$, 即 $d = 37^{-1} \pmod{720}$

i	q_i	r_i	s_i	t_i
0	—	720	1	0
1	—	37	0	1
2	19	17	1	-19
3	2	3	-2	39
4	5	2	11	-214
5	1	1	-13	253
6	2	0	—	—

计算过程:

$$720 = 19 \times 37 + 17 \quad (q_2 = 19, r_2 = 17)$$

$$37 = 2 \times 17 + 3 \quad (q_3 = 2, r_3 = 3)$$

$$17 = 5 \times 3 + 2 \quad (q_4 = 5, r_4 = 2)$$

$$3 = 1 \times 2 + 1 \quad (q_5 = 1, r_5 = 1)$$

$$2 = 2 \times 1 + 0 \quad (\text{停止})$$

s 列递推:

$$s_2 = s_0 - q_2 s_1 = 1 - 19 \times 0 = 1$$

$$s_3 = s_1 - q_3 s_2 = 0 - 2 \times 1 = -2$$

$$s_4 = s_2 - q_4 s_3 = 1 - 5 \times (-2) = 11$$

$$s_5 = s_3 - q_5 s_4 = -2 - 1 \times 11 = -13$$

t 列递推:

$$t_2 = t_0 - q_2 t_1 = 0 - 19 \times 1 = -19$$

$$t_3 = t_1 - q_3 t_2 = 1 - 2 \times (-19) = 39$$

$$t_4 = t_2 - q_4 t_3 = -19 - 5 \times 39 = -214$$

$$t_5 = t_3 - q_5 t_4 = 39 - 1 \times (-214) = 253$$

验证 Bézout 等式：

$$s_5 \times 720 + t_5 \times 37 = (-13) \times 720 + 253 \times 37 = -9360 + 9361 = 1 = \gcd(720, 37)$$

因此 $d = t_5 = 253$ 。由于 d 需为正整数， $253 \in (0, 720)$ ，满足要求。

验证： $37 \times 253 = 9361 = 13 \times 720 + 1 \equiv 1 \pmod{720}$ 。正确。

私钥 $d = 253$, 即 $(d, n) = (253, 793)$

(2) 加密明文 $m_1 = 7$

使用公钥 $(e, n) = (37, 793)$ 加密：

$$c = m^e \pmod{n} = 7^{37} \pmod{793}$$

采用重复平方法 (Square-and-Multiply)，将指数 37 写为二进制： $37 = 32 + 4 + 1 = (100101)_2$ 。

幂次	值 $\pmod{793}$
7^1	7
$7^2 = (7^1)^2$	$7^2 = 49$
$7^4 = (7^2)^2$	$49^2 = 2401 \equiv 2401 - 3 \times 793 = 2401 - 2379 = 22$
$7^8 = (7^4)^2$	$22^2 = 484$
$7^{16} = (7^8)^2$	$484^2 = 234256 \equiv 234256 - 295 \times 793 = 234256 - 233935 = 321$
$7^{32} = (7^{16})^2$	$321^2 = 103041 \equiv 103041 - 129 \times 793 = 103041 - 102297 = 744$

现在组合：

$$\begin{aligned} 7^{37} &= 7^{32} \times 7^4 \times 7^1 \pmod{793} \\ &\equiv 744 \times 22 \times 7 \pmod{793} \end{aligned}$$

先计算 744×22 ：

$$744 \times 22 = 16368, \quad 16368 \div 793 = 20 \text{ 余 } 508 \quad (\text{因为 } 793 \times 20 = 15860)$$

再计算 508×7 ：

$$508 \times 7 = 3556, \quad 3556 \div 793 = 4 \text{ 余 } 384 \quad (\text{因为 } 793 \times 4 = 3172)$$

因此：

$$c = 7^{37} \pmod{793} = 384$$

验证 (解密还原)： 解密 $c = 384$ ： $m = 384^{253} \pmod{793} \equiv 7$ (由 RSA 正确性保证，因 $ed \equiv 1 \pmod{720}$)。

明文 $m_1 = 7$ 加密后的密文为 $c = 384$

知识点延伸：RSA 算法中的注意事项**1. 为什么 $\varphi(n) = (p-1)(q-1)$?**

对于 $n = pq$ (p, q 为素数), $\varphi(pq) = \varphi(p)\varphi(q) = (p-1)(q-1)$, 因为欧拉函数是积性函数 (对互素的因子)。

2. 正确性证明

RSA 解密的正确性由欧拉定理保证: 若 $\gcd(m, n) = 1$, 则 $m^{\varphi(n)} \equiv 1 \pmod{n}$ 。

$$c^d \equiv (m^e)^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \cdot 1^k = m \pmod{n}$$

即使 $\gcd(m, n) \neq 1$ (概率极小), 用中国剩余定理也可证明解密正确。

3. 安全参数

实际应用中, p 和 q 至少应为 1024 位 (即 n 至少 2048 位), 且 e 通常选 65537 ($2^{16} + 1$, 素数, 二进制中只有两个 1, 加快加密速度)。

7.4 椭圆曲线密码 (15 分)

题目

考虑在 $\mathbb{Z}_{23}$ 上的一个椭圆曲线 $y^2 = x^3 + x + 1$ 。对 $\mathbb{Z}_p$ 上的椭圆曲线 E 上的两个点 $P = (x_1, y_1)$ 和 $Q = (x_2, y_2)$ 都 $\in E$ 。若 $x_1 = x_2$ 且 $y_1 = -y_2$, 那么 $P + Q = O$; 否则 $P + Q = (x_3, y_3)$, 这里的

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

$$\lambda = \begin{cases} \frac{y_2 - y_1}{x_2 - x_1} & \text{如果 } P \neq Q \\ \frac{3x_1^2 + 1}{2y_1} & \text{如果 } P = Q \end{cases}$$

对于所有的 $P \in E$, 定义 $P + O = O + P = P$ 。

- (1) 验证 $P = (3, 10)$ 和 $Q = (9, 7)$ 确实是该椭圆曲线上的两个点 (5 分)
- (2) 请计算出 $P + Q$ (5 分)
- (3) 计算出 $2P$ (5 分)

知识点梳理

1. 有限域上的椭圆曲线

椭圆曲线密码 (ECC) 使用定义在有限域 $\mathbb{F}_p$ ($p > 3$ 为素数) 上的椭圆曲线:

$$y^2 \equiv x^3 + ax + b \pmod{p}, \quad \text{其中 } 4a^3 + 27b^2 \not\equiv 0 \pmod{p}$$

判别式条件 $\Delta = -16(4a^3 + 27b^2) \neq 0$ 确保曲线是非奇异的 (无尖点或自交)。

本题中 $a = 1, b = 1, p = 23$ 。验证非奇异性: $4 \times 1^3 + 27 \times 1^2 = 31 \not\equiv 0 \pmod{23}$ 。满足条件。

2. 椭圆曲线上的点加运算

椭圆曲线上的点 (含无穷远点 O) 构成一个阿贝尔群, 运算法则如下:

点加 $P + Q$ ($P \neq Q$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

倍点 $2P$ ($P = Q$):

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

3. 模逆运算

在计算 λ 时涉及除法 (即乘以分母的模逆):

$$\frac{A}{B} \pmod{p} = A \times B^{-1} \pmod{p}$$

其中 B^{-1} 为 B 在模 p 下的乘法逆元。

4. 验证点是否在曲线上

将点的坐标 (x, y) 代入曲线方程, 验证 $y^2 \equiv x^3 + ax + b \pmod{p}$ 是否成立。

5. ECC 的安全性

ECC 的安全性基于椭圆曲线离散对数问题 (ECDLP): 给定基点 G 和 kG (k 为标量), 求 k 的计算困难性。相比 RSA 和传统 DLP, ECC 在相同安全强度下可使用更短的密钥 (如 256 位 ECC 与 3072 位 RSA 安全等级相当)。

详细解答

(1) 验证 $P = (3, 10)$ 和 $Q = (9, 7)$ 在曲线 $y^2 = x^3 + x + 1 \pmod{23}$ 上

验证 $P = (3, 10)$:

左边: $y^2 = 10^2 = 100 \pmod{23}$

计算 $100 \div 23 = 4 \times 23 = 92$, 余 8。所以 $10^2 \equiv 8 \pmod{23}$ 。

右边: $x^3 + x + 1 = 3^3 + 3 + 1 = 27 + 3 + 1 = 31 \pmod{23}$

$31 - 23 = 8$, 所以 $31 \equiv 8 \pmod{23}$ 。

左边 = 8 = 右边, 故 $P = (3, 10)$ 在椭圆曲线上。✓

验证 $Q = (9, 7)$:

左边: $y^2 = 7^2 = 49 \pmod{23}$

$49 - 2 \times 23 = 49 - 46 = 3$ 。所以 $7^2 \equiv 3 \pmod{23}$ 。

右边: $x^3 + x + 1 = 9^3 + 9 + 1 = 729 + 9 + 1 = 739 \pmod{23}$

$739 \div 23 = 32 \times 23 = 736$, 余 $739 - 736 = 3$ 。所以 $739 \equiv 3 \pmod{23}$ 。

左边 = 3 = 右边, 故 $Q = (9, 7)$ 在椭圆曲线上。✓

$P = (3, 10)$ 和 $Q = (9, 7)$ 均在曲线 $y^2 = x^3 + x + 1 \pmod{23}$ 上

(2) 计算 $P + Q$ ($P \neq Q$)

已知 $P = (3, 10)$, $Q = (9, 7)$ 。 $x_1 = 3 \neq 9 = x_2$, 使用点加公式。

Step 1: 计算斜率 λ

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} = \frac{7 - 10}{9 - 3} = \frac{-3}{6} \pmod{23}$$

分子: $-3 \equiv 20 \pmod{23}$ (因为 $-3 + 23 = 20$)。

分母 6 的模逆: 用扩展欧几里得算法求 $6^{-1} \pmod{23}$ 。

$$23 = 3 \times 6 + 5, \quad 6 = 1 \times 5 + 1, \quad 5 = 5 \times 1 + 0$$

回代: $1 = 6 - 1 \times 5 = 6 - 1 \times (23 - 3 \times 6) = 4 \times 6 - 23$, 故 $6^{-1} \equiv 4 \pmod{23}$ 。

验证: $6 \times 4 = 24 \equiv 1 \pmod{23}$ 。正确。

因此:

$$\lambda = -3 \times 6^{-1} \equiv 20 \times 4 = 80 \pmod{23}$$

$80 \div 23 = 3 \times 23 = 69$, 余 11。故 $\lambda = 11$ 。

Step 2: 计算 x_3

$$x_3 = \lambda^2 - x_1 - x_2 = 11^2 - 3 - 9 = 121 - 12 = 109 \pmod{23}$$

$109 \div 23 = 4 \times 23 = 92$, 余 $109 - 92 = 17$ 。故 $x_3 = 17$ 。

Step 3: 计算 y_3

$$y_3 = \lambda(x_1 - x_3) - y_1 = 11 \times (3 - 17) - 10 = 11 \times (-14) - 10 = -154 - 10 = -164 \pmod{23}$$

$-164 \pmod{23}$: $164 \div 23 = 7 \times 23 = 161$, 余 3。故 $-164 \equiv -3 \equiv 20 \pmod{23}$ 。

因此:

$$\boxed{P + Q = (17, 20)}$$

验证: 检查 $(17, 20)$ 是否在曲线上。 $20^2 = 400 \equiv 400 - 17 \times 23 = 400 - 391 = 9 \pmod{23}$ 。右边: $17^3 + 17 + 1 = 4913 + 17 + 1 = 4931 \equiv 9 \pmod{23}$ (因 $23 \times 214 = 4922$, $4931 - 4922 = 9$)。验证通过。✓

(3) 计算 $2P$ ($P = P$, 倍点运算)

已知 $P = (3, 10)$ 。使用倍点公式, 其中 $a = 1$ 。

Step 1: 计算斜率 λ

$$\lambda = \frac{3x_1^2 + a}{2y_1} = \frac{3 \times 3^2 + 1}{2 \times 10} = \frac{27 + 1}{20} = \frac{28}{20} \pmod{23}$$

先约简: $28 \equiv 5 \pmod{23}$ ($28 - 23 = 5$)。 $20 \equiv 20 \pmod{23}$ 。

求 $20^{-1} \pmod{23}$:

$$23 = 1 \times 20 + 3, \quad 20 = 6 \times 3 + 2, \quad 3 = 1 \times 2 + 1, \quad 2 = 2 \times 1 + 0$$

回代:

$$\begin{aligned} 1 &= 3 - 1 \times 2 \\ &= 3 - 1 \times (20 - 6 \times 3) = 7 \times 3 - 20 \\ &= 7 \times (23 - 20) - 20 = 7 \times 23 - 8 \times 20 \end{aligned}$$

故 $20^{-1} \equiv -8 \equiv 15 \pmod{23}$ 。验证: $20 \times 15 = 300 = 13 \times 23 + 1 \equiv 1 \pmod{23}$ 。✓

因此:

$$\lambda = 5 \times 15 = 75 \pmod{23}$$

$75 \div 23 = 3 \times 23 = 69$, 余 6。故 $\lambda = 6$ 。

Step 2: 计算 x_3

$$x_3 = \lambda^2 - 2x_1 = 6^2 - 2 \times 3 = 36 - 6 = 30 \pmod{23}$$

$30 - 23 = 7$ 。故 $x_3 = 7$ 。

Step 3: 计算 y_3

$$y_3 = \lambda(x_1 - x_3) - y_1 = 6 \times (3 - 7) - 10 = 6 \times (-4) - 10 = -24 - 10 = -34 \pmod{23}$$

$-34 \pmod{23}$: $34 = 1 \times 23 + 11$, 故 $-34 \equiv -11 \equiv 12 \pmod{23}$ 。

因此:

$$2P = (7, 12)$$

验证: 检查 $(7, 12)$ 是否在曲线上。 $12^2 = 144 \equiv 144 - 6 \times 23 = 144 - 138 = 6 \pmod{23}$ 。

右边: $7^3 + 7 + 1 = 343 + 7 + 1 = 351 \equiv 6 \pmod{23}$ ($23 \times 15 = 345$, $351 - 345 = 6$)。

验证通过。✓

知识点延伸: 椭圆曲线密钥交换 ECDH 与数字签名 ECDSA

ECC 有两个最重要的应用:

1. ECDH (椭圆曲线 Diffie-Hellman 密钥交换):

- Alice 生成私钥 d_A , 计算公钥 $Q_A = d_A G$
- Bob 生成私钥 d_B , 计算公钥 $Q_B = d_B G$
- 共享密钥: $K = d_A Q_B = d_B Q_A = d_A d_B G$

2. ECDSA (椭圆曲线数字签名算法):

- 签名: 选随机数 k , 计算 $R = kG$, $r = x_R \pmod{n}$, $s = k^{-1}(H(m) + d \cdot r) \pmod{n}$
- 验证: 计算 $w = s^{-1} \pmod{n}$, $u_1 = H(m)w \pmod{n}$, $u_2 = rw \pmod{n}$, 检查 $u_1 G + u_2 Q$ 的 x 坐标是否等于 r

ECC 广泛应用于 TLS/SSL、比特币、移动设备加密等中, 已成为现代公钥密码体系的核心技术。

7.5 IDEA 分组密码的 CBC 模式 (15 分)

题目

IDEA 是一个分组密码, 和 DES 类似, 使用混乱和扩散技术, 其明文块 64 位, 密钥 128 位。以 IDEA 为例:

- (1) 画出分组密码的密码分组链接 (CBC) 模式的加密和解密示意图 (6 分)
- (2) 假设加密时明文一个比特错误, 对密文造成什么影响? (5 分)
- (3) 接上问, 对接收方解密会造成什么影响? (4 分)

知识点梳理

1. 分组密码的工作模式

分组密码 (如 DES、AES、IDEA) 每次只能处理固定长度的数据块。为加密任意长度的消息, 需要定义工作模式 (Mode of Operation)。NIST 定义了五种标准模式: ECB、CBC、CFB、OFB、CTR。

2. CBC (Cipher Block Chaining, 密码分组链接) 模式

加密过程:

$$C_1 = E_K(P_1 \oplus IV)$$
$$C_i = E_K(P_i \oplus C_{i-1}), \quad i = 2, 3, \dots, n$$

解密过程:

$$P_1 = D_K(C_1) \oplus IV$$
$$P_i = D_K(C_i) \oplus C_{i-1}, \quad i = 2, 3, \dots, n$$

其中 P_i 为第 i 个明文块, C_i 为第 i 个密文块, IV 为初始向量 (Initialization Vector), E_K/D_K 为加密/解密函数。

3. CBC 模式的特点

- **链接效应:** 前一个密文块参与当前块的加密, 使相同的明文块在不同位置产生不同的密文块
- **IV 要求:** 必须随机且不可预测 (对 CBC 模式而言); 不需要保密
- **错误传播:** 密文块中的 1 比特错误影响当前块和下一个块的解密 (两个块受影响)
- **序列化:** 加密过程不可并行 (有数据依赖), 但解密可并行

4. IDEA 算法简介

IDEA (International Data Encryption Algorithm) 由 Xuejia Lai 和 James Massey 于 1991 年设计:

- 明文块大小: 64 位
- 密钥长度: 128 位
- 轮数: 8 轮完整变换 + 1 轮输出变换 (共 8.5 轮)
- 核心操作: 模 2^{16} 加法、模 $2^{16} + 1$ 乘法、按位异或

5. 错误传播分析

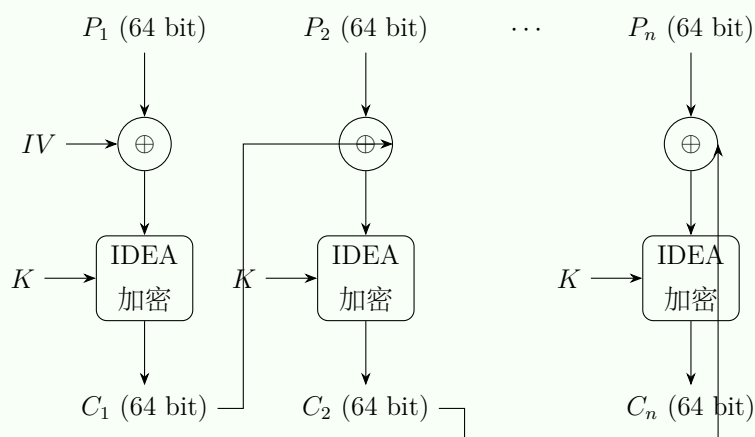
对于 CBC 模式, 明文中 1 比特错误对密文的影响和密文中 1 比特错误对解密的影响是不同的:

- 明文错误: 该块及之后所有块的密文都受影响 (因为 CBC 加密具有前向依赖)
- 密文错误: 只影响当前块及下一块的解密 (受限的错误传播)

详细解答

(1) IDEA 分组密码 CBC 模式加密和解密示意图

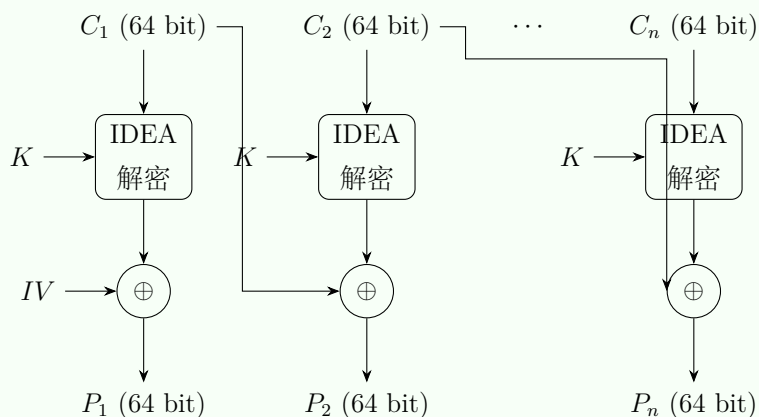
CBC 加密示意图:



加密过程说明:

1. 第一个明文块 P_1 与初始向量 IV 进行异或操作, 结果输入 IDEA 加密算法, 产生密文块 C_1
2. 第 i 个 ($i \geq 2$) 明文块 P_i 与前一个密文块 C_{i-1} 进行异或操作, 结果输入 IDEA 加密, 产生 C_i
3. 数学表达: $C_1 = E_K(P_1 \oplus IV)$, $C_i = E_K(P_i \oplus C_{i-1})$

CBC 解密示意图:



解密过程说明：

1. 密文块 C_1 经 IDEA 解密后与 IV 异或，恢复明文块 P_1
2. 密文块 C_i ($i \geq 2$) 经 IDEA 解密后与 C_{i-1} 异或，恢复明文块 P_i
3. 数学表达： $P_1 = D_K(C_1) \oplus IV$, $P_i = D_K(C_i) \oplus C_{i-1}$

(2) 加密时明文一个比特错误对密文的影响

假设在加密前，第 i 个明文块 P_i 中出现了 1 比特错误（即该比特被翻转）。

CBC 加密影响分析：

- P_i 的 1 比特错误首先影响 $P_i \oplus C_{i-1}$ 的结果（或对 $i = 1$ 时影响 $P_1 \oplus IV$ ），导致 i 个加密块 $E_K(P_i \oplus C_{i-1})$ 的输入改变。
- 由于分组密码的**雪崩效应**（Avalanche Effect），输入中的 1 比特差异在经过加密算法后会导致输出中大约一半比特发生变化。因此密文块 C_i 将几乎完全不同（约 32 位比特翻转）。
- 然后 C_i （已受错误影响）作为 P_{i+1} 加密的反馈输入， P_{i+1} 虽然本身正确，但 $P_{i+1} \oplus C_i$ 会因 C_i 的错误而改变，进而导致 C_{i+1} 完全不同。
- 以此类推， C_i 及其之后**所有**密文块都将受到影响。

结论：明文中 1 比特错误会导致该明文块对应的密文块及其后所有密文块都发生变化。

定量分析：

P_i 中 1 bit 错误
 $\Rightarrow C_i$ 约 32 bit 错误（雪崩效应）
 $\Rightarrow C_{i+1}$ 因链式反馈也完全不同
 $\Rightarrow C_{i+2}, C_{i+3}, \dots, C_n$ 全部改变

这是 CBC 模式的一个自然特性：虽然它阻止了相同明文块产生相同密文块的问题，但代价是前向错误传播。

(3) 加密明文 1 比特错误对接收方解密的影响

解密时接收方接收到的是已经受错误影响的密文序列（从 C_i 开始全部改变）。

CBC 解密影响分析：

解密公式： $P_j = D_K(C_j) \oplus C_{j-1}$ （约定 $C_0 = IV$ ）。

解密过程逐块分析：

- P_i （出错块本身）： C_i 已发生大量比特变化， $D_K(C_i)$ 输出将完全不同。将其与正确的 C_{i-1} 异或， P_i 的解密结果与原始 P_i 完全不同。**本块解密失败。**
- P_{i+1} 及之后（此处的关键洞察）：考察 P_{i+1} 的解密：

$$P_{i+1} = D_K(C_{i+1}) \oplus C_i$$

加密时已改变的 C_i 被用于计算 C_{i+1} ： $C_{i+1} = E_K(P_{i+1} \oplus C_i)$ 。（注意 P_{i+1} 本身正确，但异或时用的是已改变的 C_i 。）

因此 $D_K(C_{i+1}) = P_{i+1} \oplus C_i$ （ C_i 为改变后的值）。

解密时： $P_{i+1} = (P_{i+1} \oplus C_i) \oplus C_i = P_{i+1}$ 。这里的两个 C_i 完全相同（加密时使用、解密时作为反馈输入收到的）， $\oplus$ 操作相互抵消， P_{i+1} **被正确恢复！**

- P_{i+2} 及之后：同理， $P_{i+2} = D_K(C_{i+2}) \oplus C_{i+1}$ ，而 $D_K(C_{i+2}) = P_{i+2} \oplus C_{i+1}$ （ C_{i+1} 为加密时使用的值），两个 C_{i+1} 抵消， P_{i+2} 也正确恢复。此模式持续到 P_n 。

逐块正确性总结：

解密块	结果
P_i	失败（原明文本身就有误，无法恢复）
P_{i+1}	正确恢复
P_{i+2}	正确恢复
$\vdots$	$\vdots$
P_n	正确恢复

这是 CBC 模式的一个反直觉但关键的性质：加密时明文的错误虽然导致后续所有密文块改变，但由于加密和解密使用的是同一组密文作为反馈输入，解密时 $\oplus$ 操作相互抵消，除出错块本身外所有后续块都能被正确解密恢复。

本题最终结论：

- 加密时明文 P_i 的 1 比特错误导致 $C_i, C_{i+1}, \dots, C_n$ 全部改变
- 对接收方解密而言，仅 P_i 解密失败， $P_{i+1}, P_{i+2}, \dots, P_n$ 均可正确解密恢复

解密的错误恢复性质总结：

错误来源	影响的密文块	影响的解密明文块
明文 P_i 中 1 bit 错误	$C_i, C_{i+1}, \dots, C_n$ (全部后续)	仅 P_i 解密失败，其余可正确恢复
密文 C_i 中 1 bit 错误	仅 C_i	P_i 和 P_{i+1} 均解密失败

注意：本题问的是”加密时明文一个比特错误对接收方解密的影响”，与”密文传输错误对解密的影响”是不同的场景。

加密时明文错误的场景：加密方用了错误的明文来加密，接收方用正确的密钥进行解密：

- P_i 无法正确恢复（因为原始明文就错了）
- $P_{i+1}, P_{i+2}, \dots$ 可以正确恢复（CBC 的纠错/自同步特性）

- (2) 加密影响：该明文块 P_i 对应的密文块 C_i 及之后所有密文块都发生改变。

(3) 解密影响：仅对应的明文块 P_i 解密出错，
其余明文块 $P_{i+1}, P_{i+2}, \dots$ 均可正确解密恢复。

知识点延伸：CBC 错误传播特性在工程中的意义

CBC 模式的”自同步”或”有限错误传播”特性（对接收方而言，密文中的 1 比特错误仅影响两个块）在通信系统中非常有价值：

- 在噪声信道中传输加密数据时，单个比特翻转不会导致整个消息无法解密
- 然而，如果通信协议中有完整性校验（如 MAC），受影响的两个块会导致 MAC 验证失败，整个消息仍会被拒绝
- 为获得完整性和机密性双重保护，现代协议（如 TLS 1.3）通常使用 Encrypt-then-MAC 或 AEAD（Authenticated Encryption with Associated Data）模式

这与 ECB 模式形成鲜明对比：ECB 模式下 1 比特错误仅影响单个块，但 ECB 不隐藏数据模式（相同的明文块产生相同的密文块），在实际应用中 ECB 通常不被推荐使用。

7.6 RSA 数字签名方案分析 (20 分)

题目

以下为 RSA 用于数字签名的一种方案 (原始方案示意图描述: 用户 A 使用私钥 d_A 对消息 m 进行“解密”运算生成签名 $s = m^{d_A} \bmod n_A$, 然后将 (m, s) 发送给验证方):

- (1) 分析该方案存在的安全问题 (10 分)
- (2) 针对存在的安全问题, 给出一种可行的改进方案, 并画出如上图类似的示意图 (5 分)
- (3) 该方案只具有数字签名功能, 如果还需要对原文进行保密, 请使用 RSA 设计一个既有数字签名又有保密功能的方案, 并画过程示意图 (5 分)

知识点梳理

1. RSA 数字签名的基本原理

RSA 数字签名利用加解密运算的对称性:

- **签名生成:** 签名者使用私钥 d_A 对消息摘要 (或消息本身) 进行“签名运算”: $s = m^{d_A} \bmod n_A$
- **签名验证:** 验证者使用签名者的公钥 e_A 进行“验证运算”: $m' = s^{e_A} \bmod n_A$, 检查 $m' = m$
- **核心原理:** $s^{e_A} \equiv (m^{d_A})^{e_A} \equiv m^{d_A e_A} \equiv m \pmod{n_A}$

2. 原始 RSA 签名 (无哈希) 的安全问题

问题一: 选择密文攻击 (Chosen Ciphertext Attack) —— 利用乘法同态性

RSA 具有乘法同态性: $E(m_1) \times E(m_2) \equiv E(m_1 \times m_2) \pmod{n}$, 即:

$$(m_1^e \bmod n) \times (m_2^e \bmod n) \equiv (m_1 m_2)^e \pmod{n}$$

同理, 对于签名也有: $m_1^d \times m_2^d \equiv (m_1 m_2)^d \pmod{n}$ 。

攻击者可以利用此性质, 通过获取对特定构造消息的签名来恢复明文或伪造签名。

问题二: 存在性伪造 (Existential Forgery) —— 利用乘法性质

若攻击者获得了对 m_1 和 m_2 的签名 $s_1 = m_1^d \bmod n$ 和 $s_2 = m_2^d \bmod n$, 则他可以直接计算:

$$s_3 = s_1 \times s_2 \pmod{n} = m_1^d \times m_2^d \equiv (m_1 m_2)^d = m_3^d \pmod{n}$$

从而获得对 $m_3 = m_1 m_2 \bmod n$ 的有效签名 s_3 , 尽管签名者从未对 m_3 签名。

问题三: 直接伪造

攻击者可以先随机选择 s , 然后计算 $m = s^e \pmod{n}$, 则 (m, s) 就是一个有效的消息-签名对。虽然 m 可能看起来无意义, 但在某些场景中仍可被利用。

3. 哈希函数在数字签名中的作用

- 对消息进行哈希后再签名, 即 $s = (H(m))^d \pmod{n}$

- 破坏 RSA 的乘法同态性: $H(m_1) \times H(m_2) \neq H(m_1 m_2)$
- 防止消息伪造 (抗碰撞性)
- 对任意长度消息生成固定长度的摘要
- 提高签名效率 (对长消息只需签署短哈希值)

4. 签名与加密的组合

为同时实现数字签名 (认证性、不可否认性) 和保密性, 可采用 “先签名后加密” (Sign-then-Encrypt) 方案:

- 发送方 A 先用自己的私钥签名
- 再用接收方 B 的公钥加密
- B 收到后先用自己的私钥解密, 再用 A 的公钥验证签名

详细解答

(1) 分析原始 RSA 数字签名方案存在的安全问题

原始方案: 签名者 A 直接对消息 m 进行签名 $s = m^{d_A} \pmod{n_A}$, 发送 (m, s) 给验证方 B。B 用 A 的公钥验证: $m \stackrel{?}{=} s^{e_A} \pmod{n_A}$ 。

安全问题一: 基于乘法同态性的选择密文攻击 (5 分)

攻击者 E 通过以下步骤获得用 A 公钥加密的密文 c 对应的明文 m (即 $m = c^{d_A} \pmod{n_A}$):

1. E 拦截或收集 A 的通信, 获得用 A 公钥加密的密文 c ($c = m^{e_A} \pmod{n_A}$), E 想知道 m
2. E 选择一个随机数 r ($1 < r < n_A$ 且 $\gcd(r, n_A) = 1$)
3. E 用 A 的公钥计算 $x = r^{e_A} \pmod{n_A}$
4. E 计算 $y = x \cdot c \pmod{n_A}$ (即 $y = r^{e_A} \cdot m^{e_A} = (r \cdot m)^{e_A} \pmod{n_A}$)
5. E 计算 $t = r^{-1} \pmod{n_A}$ (r 的模逆)
6. E 将 y 提交给 A, 请求 A 对 y 进行签名
7. A 计算 $u = y^{d_A} \pmod{n_A}$ 并返回给 E (A 以为自己在对 y 签名)
8. E 计算:

$$t \cdot u \pmod{n_A} = r^{-1} \cdot y^{d_A} \equiv r^{-1} \cdot (r^{e_A} \cdot m^{e_A})^{d_A} \equiv r^{-1} \cdot r \cdot m \equiv m \pmod{n_A}$$

从而恢复出 m 。

攻击的核心在于利用了 RSA 的乘法同态性: $(r \cdot m)^d \equiv r^d \cdot m^d \pmod{n_A}$ 。攻击者通过“盲化”技术隐藏了真正想解密的内容。

安全问题二：基于乘法性质的存在性伪造 (5 分)

攻击者 E 可以伪造对任意消息 m_3 的有效签名，只要他能诱导签名者签署两个“无害的”消息：

1. E 选择目标消息 m_3 (E 想伪造对 m_3 的签名)
2. E 将 m_3 分解为两个因子：选择任意的 m_1 (与 n_A 互素)，计算 $m_2 = m_3 \cdot m_1^{-1} \pmod{n_A}$ ，使得 $m_3 \equiv m_1 \cdot m_2 \pmod{n_A}$
3. E 分别请求 A 对 m_1 和 m_2 进行签名，获得 $s_1 = m_1^{d_A} \pmod{n_A}$ 和 $s_2 = m_2^{d_A} \pmod{n_A}$
4. E 计算 $s_3 = s_1 \cdot s_2 \pmod{n_A} \equiv m_1^{d_A} \cdot m_2^{d_A} \equiv (m_1 m_2)^{d_A} \equiv m_3^{d_A} \pmod{n_A}$
5. 于是 (m_3, s_3) 构成一个有效的消息-签名对，尽管 A 从未直接对 m_3 签名

这种攻击之所以有效，是因为原始 RSA 签名没有破坏代数结构中的乘法同态性。

安全问题三：直接伪造

攻击者随机选取 $s \in \mathbb{Z}_{n_A}^*$ ，计算 $m = s^{e_A} \pmod{n_A}$ ，则 (m, s) 即为一个有效的消息-签名对。虽然 m 看起来像随机数，但若接收方对消息格式没有严格要求，这仍构成安全威胁。该问题实际上也说明数字签名方案需要有消息格式/编码规范。

(2) 改进方案——引入哈希函数 (5 分)

针对上述安全问题，核心改进思路是在签名前先对消息进行哈希处理。哈希函数 $H : \{0, 1\}^* \rightarrow \mathbb{Z}_{n_A}$ 具有以下关键性质：

- 单向性：给定 $y = H(m)$ ，无法通过 y 恢复 m
- 抗碰撞性：无法找到 $m_1 \neq m_2$ 使得 $H(m_1) = H(m_2)$
- 破坏乘法同态性： $H(m_1) \cdot H(m_2) \neq H(m_1 \cdot m_2)$

改进方案的签名过程：

签名者 A (使用私钥 d_A):

$$s = (H(m))^{d_A} \pmod{n_A}$$

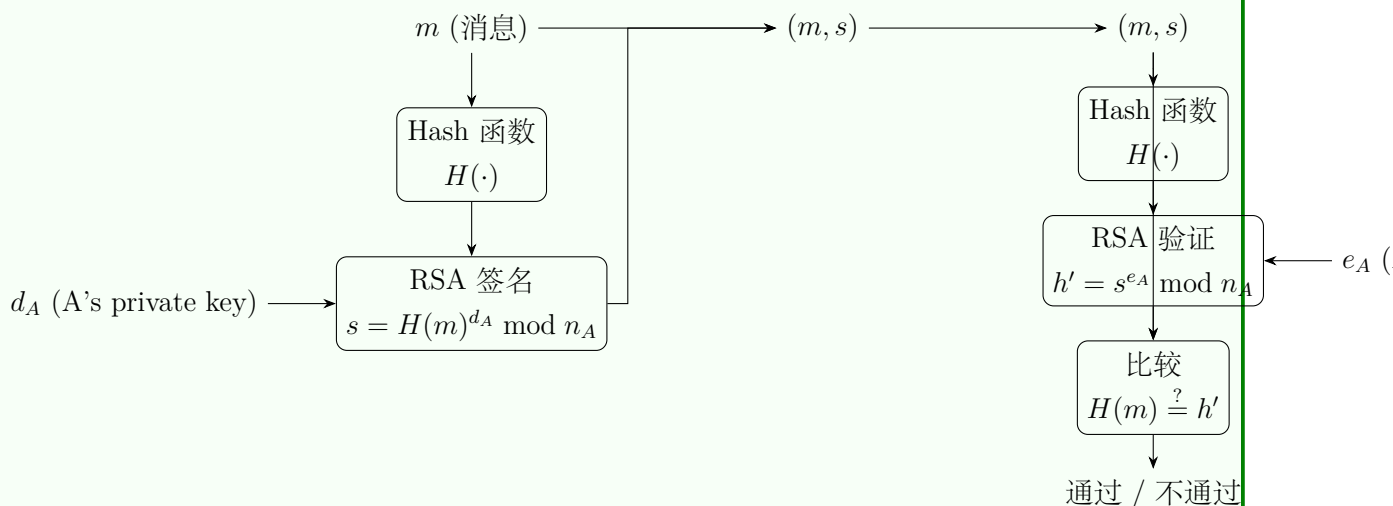
发送 (m, s) 给验证方 B。

验证方 B (使用 A 的公钥 e_A):

1. 计算 $h = H(m)$
2. 计算 $h' = s^{e_A} \pmod{n_A}$

3. 验证 $h' \stackrel{?}{=} h$

改进方案示意图:



哈希方案如何抵御原有攻击:

- 对选择密文攻击: 攻击者计算 $y = r^{e_A} \cdot H(m_1) \pmod{n_A}$, 请求签名得到 $u = y^{d_A} \pmod{n_A}$ 。但 $u \times r^{-1} = H(m_1)^{d_A} \pmod{n_A}$ 是 $H(m_1)$ 的签名而非任意 m 的签名。由哈希的单向性, 攻击者无法从 $H(m_1)$ 回推出有意义的 m 用于构造新的消息-签名对。
- 对存在性伪造: $H(m_1) \cdot H(m_2) \neq H(m_1 \cdot m_2)$, 乘法性质被哈希函数破坏。
- 对直接伪造: 即使选取 s 求得 $h' = s^{e_A} \pmod{n_A}$, 攻击者也无法找到 m 使得 $H(m) = h'$ (哈希单向性)。

(3) 同时实现数字签名与保密功能的方案 (5 分)

设计原则: 先签名, 后加密 (Sign-then-Encrypt), 发送方用自己私钥签名 (提供认证), 再用接收方公钥加密 (提供保密性)。

方案设计:

设 A 的公私钥对为 (e_A, d_A) , B 的公私钥对为 (e_B, d_B) 。A 向 B 发送带签名和加密的消息。

A 方 (发送方):

1. 对消息 m 进行哈希: $h = H(m)$
2. 用 A 的私钥签名: $s = h^{d_A} \pmod{n_A}$
3. 将消息和签名拼接: $M = m \parallel s$ (或 $M = (m, s)$)

4. 用 B 的公钥对 M 加密: $c = M^{e_B} \pmod{n_B}$ (若 $M < n_B$; 否则需先分块)

5. 发送 c 给 B

B 方 (接收方):

1. 用 B 的私钥解密: $M = c^{d_B} \pmod{n_B}$

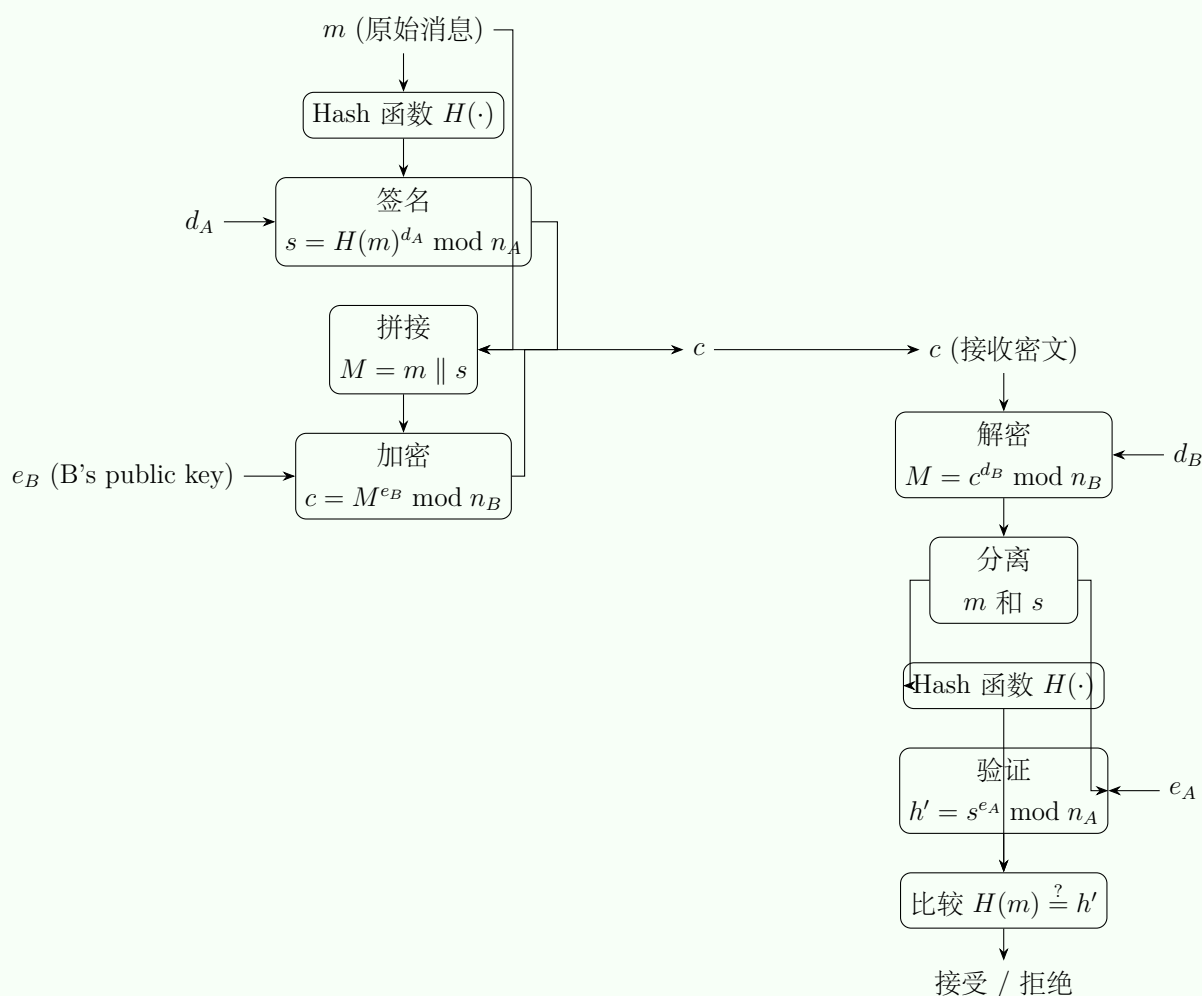
2. 分离出 m 和 s

3. 计算 $h = H(m)$

4. 用 A 的公钥验证签名: $h' = s^{e_A} \pmod{n_A}$, 检查 $h' \stackrel{?}{=} h$

5. 验证通过则接受消息 m

方案示意图:



方案安全性分析:

- **保密性:** 由 B 的公钥加密保证, 仅 B 可用其私钥解密

- **认证性/完整性**: 由 A 的数字签名保证, 消息的任何篡改会导致哈希验证失败
- **不可否认性**: 只有拥有 d_A 的 A 才能生成有效签名, A 无法否认发送过该消息
- **顺序注意**: 必须先签名后加密 (Sign-then-Encrypt)。若先加密后签名, 攻击者可移除原签名然后附上自己的签名, 声称该密文由自己生成, 造成安全漏洞

知识点延伸: PKCS#1 与 PSS 填充方案

在实际标准中 (PKCS#1 v2.2), RSA 签名使用 PSS (Probabilistic Signature Scheme, 概率签名方案) 填充, 而非简单的” 哈希后签名”:

- **RSASSA-PSS**: 使用随机盐值 (salt) 和 MGF (Mask Generation Function) 进行填充, 使得同一消息的两次签名产生不同结果 (非确定性签名), 增强了安全性
- **RSASSA-PKCS1-v1_5**: 较旧的确定性填充方案, 虽仍被广泛使用, 但安全性弱于 PSS
- **RSASSA-OAEP**: 加密时使用 OAEP (Optimal Asymmetric Encryption Padding), 具有选择密文攻击安全性

这些填充方案经过了严格的安全分析和标准化过程, 在实际系统中不应使用教材中的” 裸 RSA” 签名。

7.7 Feige-Fiat-Shamir 身份验证方案 (20 分)

题目

身份验证总是基于某种收发双方共享的保密数据来认证对象的真实性。

- (1) 给出 Feige-Fiat-Shamir 身份验证方案 (10 分)
- (2) 你认为该方法在应用中存在哪些安全问题? (5 分)
- (3) 如何解决上述应用中的问题? 给出建议或方案 (5 分)

知识点梳理

1. 身份验证的基本概念

身份验证 (Authentication) 是确认通信对方身份的过程。主要方法包括:

- 基于所知 (Something you know): 密码、PIN 码等
- 基于所有 (Something you have): 智能卡、令牌等
- 基于所是 (Something you are): 指纹、虹膜等生物特征

在密码学范畴内, 身份验证协议允许一方向另一方证明自己知道某个秘密, 而不泄露该秘密本身。

2. 零知识证明的基本思想

零知识证明 (Zero-Knowledge Proof, ZKP) 允许证明者 P 向验证者 V 证明其知道某个秘密, 而不泄露关于该秘密的任何信息。Feige-Fiat-Shamir (FFS) 方案是最经典的基于零知识思想的身份验证协议之一。

3. FFS 方案的数学基础

FFS 方案的安全性基于二次剩余问题和大整数分解问题的困难性:

- 模 n 的平方根: 给定 $n = p \cdot q$ (p, q 为素数), 求模 n 下的平方根与大整数分解等价困难
- 二次剩余: 给定 $a \in \mathbb{Z}_n^*$, 判断 a 是否为模 n 的二次剩余, 在不知道 n 的分解时是困难的

4. 交互式证明的轮结构

FFS 协议采用多轮挑战-响应结构, 每轮通过概率降低欺骗成功率。经过 t 轮后, 欺骗成功的概率降至 $(1/2)^{kt}$ (其中 k 为每轮的挑战位数)。

详细解答

(1) Feige-Fiat-Shamir 身份验证方案

FFS 方案由 Uriel Feige、Amos Fiat 和 Adi Shamir 于 1988 年提出, 是一种基于零知识

证明思想的身份验证协议。以下给出完整的方案描述。

系统初始化阶段（由可信中心完成）：

1. 可信中心 (Trusted Center) 选择两个大素数 p 和 q ，计算 $n = p \cdot q$ 。 p 和 q 保密， n 公开。
2. (在实际实现中， n 通常为 Blum 整数，即 $p \equiv q \equiv 3 \pmod{4}$ ，以保证每个二次剩余有唯一的平方根。)
3. 可信中心为每位用户分配身份标识 I 。

用户密钥生成阶段：

1. 可信中心为用户选取 k 个随机数 $s_1, s_2, \dots, s_k$ (通常 $k \geq 1$ ，如 $k = 5$)，满足 $\gcd(s_i, n) = 1$ 。这些 s_i 构成用户的**私钥**。
2. 可信中心计算用户的**公钥**：

$$v_i = s_i^2 \pmod{n}, \quad i = 1, 2, \dots, k$$

也可以取 $v_i = (s_i^{-1})^2 \pmod{n}$ (不同文献有细微差别，本质等价)。

3. 可信中心将私钥 $(s_1, \dots, s_k)$ 安全地交给用户，将公钥 $(v_1, \dots, v_k)$ 及 n 公开。

身份验证协议（单个认证会话，共 t 轮）：

证明者 (Prover，声称身份为 I 的用户) 向验证者 (Verifier) 证明自己的身份。以下描述一轮交互：

1. **承诺 (Commitment)**：证明者随机选择 $r \in \mathbb{Z}_n^*$ ，计算

$$x = r^2 \pmod{n}$$

将 x 发送给验证者。

2. **挑战 (Challenge)**：验证者随机生成一个 k 位的比特向量

$$\mathbf{e} = (e_1, e_2, \dots, e_k), \quad e_i \in \{0, 1\}$$

将 $\mathbf{e}$ 发送给证明者。

3. **响应 (Response)**：证明者计算

$$y = r \cdot \prod_{i=1}^k s_i^{e_i} \pmod{n}$$

即 $y = r \cdot \prod_{e_i=1} s_i \pmod{n}$ 。将 y 发送给验证者。

4. 验证 (Verification): 验证者检查

$$y^2 \stackrel{?}{=} x \cdot \prod_{i=1}^k v_i^{e_i} \pmod{n}$$

若等式成立, 则该轮验证通过; 否则拒绝。

以上步骤重复执行 t 轮 (通常 t 取 20–40), 每轮使用独立的随机数 r 和独立随机挑战 e 。若所有 t 轮均通过验证, 则验证者接受证明者的身份声明。

正确性证明:

若证明者确实持有私钥 s_i (满足 $v_i \equiv s_i^2 \pmod{n}$), 则:

$$\begin{aligned} y^2 &= \left(r \cdot \prod_{i=1}^k s_i^{e_i} \right)^2 = r^2 \cdot \prod_{i=1}^k (s_i^2)^{e_i} \\ &\equiv x \cdot \prod_{i=1}^k v_i^{e_i} \pmod{n} \end{aligned}$$

等式恒成立, 验证一定通过。

安全性分析:

- **完备性 (Completeness):** 诚实的证明者总能通过验证 (已证明)。
- **可靠性 (Soundness):** 不知道私钥的冒充者, 单轮欺骗成功的概率为 $1/2^k$ (需在不知道 e 的情况下提前准备 x 和 y 使得验证等式成立)。经过 t 轮后, 欺骗成功概率降至 $(1/2^k)^t$ 。例如 $k=5, t=4$ 时, 欺骗成功率为 $1/2^{20}$, 可忽略。
- **零知识性 (Zero-Knowledge):** 验证者获得的信息 (x, e, y) 可以通过模拟器在不知道私钥的情况下生成 (随机选择 y 和 e , 计算 $x = y^2 \cdot (\prod v_i^{e_i})^{-1} \pmod{n}$), 因此验证者未学到任何关于私钥的信息。

(2) FFS 方案在应用中存在的安全问题

问题一: 重放攻击 (Replay Attack)

如果攻击者截获了一轮或多轮完整的认证交互记录 (x, e, y) , 在后续的会话中, 攻击者可以重放这些交互来冒充合法用户。因为 FFS 方案没有绑定会话标识或时间戳, 原始的交互记录在另一个会话中同样有效。

问题二: 中间人攻击 (Man-in-the-Middle Attack)

攻击者可以位于证明者和验证者之间, 在认证过程中转发或篡改消息。由于 FFS 方案本身不包含对通信通道的安全保护, 攻击者可以:

- 拦截 x , 用自己的值 x' 替换后发送给验证者
- 拦截挑战 e , 转发给证明者 (或修改后转发)

- 拦截响应 y ，转给验证者

通过中继攻击，攻击者可以使验证者相信攻击者就是合法用户。

问题三：公钥的认证性问题

验证者如何确信用户 I 的公钥 $(v_1, \dots, v_k)$ 确实属于该用户？如果攻击者能够替换或伪造公钥（如通过中间人攻击在公钥分发阶段介入），则可以成功冒充任何用户。这本质上是公钥基础设施（PKI）的问题，FFS 方案本身不解决公钥真实性验证。

问题四：私钥存储安全性

用户的私钥 $(s_1, \dots, s_k)$ 需要安全存储。如果私钥被泄露（如通过恶意软件、物理攻击等），攻击者可以永久性地冒充该用户。

问题五：随机数安全性

每轮使用的随机数 r 必须是不可预测且不重复的。如果 r 被重复使用，攻击者可以通过分析多轮交互提取私钥信息：

- 如果两轮使用相同的 r 但不同的挑战 e 和 e' ：

$$y = r \cdot \prod s_i^{e_i} \pmod{n}$$

$$y' = r \cdot \prod s_i^{e'_i} \pmod{n}$$

攻击者可利用 $\frac{y}{y'} \equiv \prod s_i^{e_i - e'_i} \pmod{n}$ 推算出部分私钥信息。

问题六：效率问题

t 轮交互意味着证明者和验证者之间需要 $3t$ 次消息交换（承诺、挑战、响应各一次），在网络延迟较大的场景中效率较低。且协议需要双方同时在线进行多次交互。

问题七：可信中心的单点故障

系统依赖可信中心来生成和分发密钥。如果可信中心被攻破，所有用户的私钥都可能被泄露。

(3) 解决上述安全问题的建议和方案

对策一：防止重放攻击——引入 Nonce 和时间戳

在每轮交互中加入随机数（Nonce）或时间戳：

- 验证者在发起认证时，首先生成一个随机数 N_V 并发送给证明者
- 证明者将 N_V 纳入承诺 x 的计算中，例如 $x = H(r^2 \parallel N_V \parallel \text{session_id})$
- 所有消息都与当前会话绑定，重放攻击失效

对策二：防止中间人攻击——结合安全通信信道

- 在认证协议之上使用 TLS/SSL 等安全传输层协议，确保通信的机密性和完整性
- 使用数字签名对关键消息（如公钥 v_i ）进行认证

- 在协议中嵌入通道绑定 (Channel Binding) 信息, 将认证与底层安全通道关联

对策三: 解决公钥认证问题——引入 PKI 和数字证书

- 由可信证书颁发机构 (CA) 签署用户的公钥, 颁发数字证书
- 验证者在认证前先验证数字证书的有效性 (证书链验证)
- 或采用基于身份的密码学 (IBC) 方案, 用户的身份本身 (如邮箱) 即为公钥

对策四: 增强私钥保护

- 将私钥存储在硬件安全模块 (HSM) 或安全元件 (SE) 中, 如智能卡、TPM 芯片
- 使用密码或生物特征保护私钥访问
- 实施密钥定期更新机制
- 采用门限密码学方案, 将私钥分片存储在多个设备上

对策五: 保证随机数安全

- 使用密码学安全的伪随机数生成器 (CSPRNG)
- 在每次认证会话中使用不同的随机数 r
- 使用确定性但不重复的随机数派生机制 (如基于计数器或时间戳的 PRF)

对策六: 提高效率——非交互式版本 (Fiat-Shamir 变换)

使用 Fiat-Shamir 启发式将交互式协议转换为非交互式零知识证明 (NIZK):

- 证明者自己计算挑战 $e = H(x \parallel \text{message})$, 无需与验证者进行多轮交互
- 签名即证明: 将 FFS 转换为数字签名方案 (Fiat-Shamir 签名方案)
- 验证者离线验证, 极大提高了实际应用效率

对策七: 削弱对可信中心的依赖

- 允许用户自行生成密钥对 (s_i, v_i) , 然后通过 PKI 将公钥 v_i 与身份绑定
- 采用分布式密钥生成 (DKG) 技术, 多方共同参与密钥生成
- 实现群签名或环签名方案, 提供匿名认证能力

改进方案总结:

综合以上分析, 一个安全的身份验证方案应满足:

- 1. 抗重放攻击：使用会话绑定 (Nonce/时间戳)
- 2. 抗中间人攻击：结合安全传输通道和消息认证
- 3. 公钥真实性：基于 PKI/CA 的公钥认证体系
- 4. 私钥保护：硬件安全存储 + 访问控制
- 5. 随机数质量：使用 CSPRNG
- 6. 协议效率：采用非交互式版本或减少交互轮数

知识点延伸：FFS 方案的现代意义与 Schnorr 协议对比

FFS 与 Schnorr 身份验证协议对比：

特性	FFS 方案	Schnorr 方案
安全基础	大整数分解	离散对数问题
每轮挑战大小	k 位比特向量	$\log_2 q$ 位 (q 为大素数)
每轮安全性	$1/2^k$	$1/q$
密钥大小	较大 (需 k 个私钥)	较小 (1 个私钥)
非交互式版本	可行 (Fiat-Shamir 签名)	可行 (Schnorr 签名)
实际应用	较少	广泛 (EdDSA 等)

历史地位：FFS 方案是零知识证明在身份验证领域的重要里程碑，其核心设计思想（承诺-挑战-响应三段式交互 + 多轮重复降低欺骗概率）深刻影响了后续的身份验证协议和数字签名方案设计。Fiat-Shamir 变换更是成为密码学中最常用的技术之一，广泛应用于现代零知识证明系统（如 zk-SNARK、zk-STARK 等）。

实际应用：FFS 方案已被标准化并用于多种智能卡和身份认证系统中。现代 FIDO (Fast Identity Online) 标准虽然采用了不同的密码学原语（如 ECDSA），但其”挑战-响应”认证范式仍与 FFS 等经典方案一脉相承。

试卷七答案解析总结

本试卷覆盖了网络空间安全课程的核心知识点：

- **古典密码学**：Hill 密码（矩阵加密）、换位密码（位置置换）——理解密码学的基本思想和线性代数/组合数学在加密中的应用
- **公钥密码学**：RSA 算法（密钥生成、加解密运算）、椭圆曲线密码 ECC（点加与倍点）——理解基于数学难题的现代密码体制
- **分组密码工作模式**：CBC 模式的加解密流程及错误传播特性——理解密码学工程实践中的关键设计
- **数字签名**：RSA 签名方案的安全问题、哈希改进、签名与加密的组合——理解认证性与保密性的协同
- **身份验证**：Feige-Fiat-Shamir 零知识身份验证方案——理解零知识证明在身份认证中的应用

这些知识点从古典到现代、从理论到工程实践，全面描绘了网络空间安全的核心技术图景。

8 试卷八：网络空间安全（2023-2024 第二学期）

8.1 仿射密码解密 (10 分)

题目：使用仿射密码算法对信息 "hlzb l slqfb tponwbotalojb" 进行解密，已知 $k_0 = 11$, $k_1 = 17$ 。

知识点回顾：仿射密码 (Affine Cipher)

1. 仿射密码的数学定义

仿射密码是一种单表代换密码，其加密函数为：

$$E(x) = (k_1 \cdot x + k_0) \bmod m \quad (1)$$

其中：

- x 为明文对应的数字（通常 $a = 0, b = 1, \dots, z = 25$ ）；
- k_1 为乘数密钥，必须满足 $\gcd(k_1, m) = 1$ ，以保证 k_1 在模 m 下有乘法逆元；
- k_0 为加数密钥（位移量）， $0 \leq k_0 \leq m - 1$ ；
- m 为字母表大小，通常 $m = 26$ 。

2. 仿射密码的解密函数

解密函数为：

$$D(y) = k_1^{-1} \cdot (y - k_0) \bmod m \quad (2)$$

其中 k_1^{-1} 是 k_1 在模 m 下的乘法逆元，即满足 $k_1 \cdot k_1^{-1} \equiv 1 \pmod{m}$ 。

3. 求乘法逆元——扩展欧几里得算法

给定 a 和 m ($\gcd(a, m) = 1$)，求 x 使得 $a \cdot x \equiv 1 \pmod{m}$ 。使用扩展欧几里得算法：

$$a \cdot x + m \cdot y = \gcd(a, m) = 1 \quad (3)$$

则 $x \bmod m$ 即为 a 模 m 的乘法逆元。

4. 仿射密码的安全性

- 密钥空间大小： $\varphi(m) \times m$ （其中 φ 为欧拉函数）。当 $m = 26$ 时， $\varphi(26) = 12$ ，故密钥空间为 $12 \times 26 = 312$ ，非常小，无法抵抗暴力破解。
- 仿射密码属于单表代换密码，保留了明文的统计特征（如字母频率），易受频率分析攻击。

解密过程——逐步推导

Step 1: 求乘法逆元 $k_1^{-1} \bmod 26$

已知 $k_1 = 17$, $m = 26$ 。由于 $\gcd(17, 26) = 1$, 17 在模 26 下存在乘法逆元。使用扩展欧几里得算法:

$$26 = 17 \times 1 + 9$$

$$17 = 9 \times 1 + 8$$

$$9 = 8 \times 1 + 1$$

$$8 = 1 \times 8 + 0$$

回代:

$$1 = 9 - 8 \times 1$$

$$= 9 - (17 - 9 \times 1) \times 1 = 9 \times 2 - 17$$

$$= (26 - 17 \times 1) \times 2 - 17 = 26 \times 2 - 17 \times 3$$

因此 $17 \times (-3) \equiv 1 \pmod{26}$, 即 $17^{-1} \equiv -3 \equiv 23 \pmod{26}$ 。

验证: $17 \times 23 = 391 = 15 \times 26 + 1 \equiv 1 \pmod{26}$ 。✓

Step 2: 推导解密函数

由解密公式:

$$\begin{aligned} D(y) &= k_1^{-1} \cdot (y - k_0) \pmod{26} \\ &= 23 \cdot (y - 11) \pmod{26} \\ &= (23y - 23 \times 11) \pmod{26} \end{aligned}$$

计算 $23 \times 11 = 253$, $253 \pmod{26} = 253 - 9 \times 26 = 253 - 234 = 19$ 。

故 $D(y) = (23y - 19) \pmod{26}$ 。

注意到 $23 \equiv -3 \pmod{26}$, 且 $-19 \equiv 7 \pmod{26}$, 因此可以简化为:

$$\boxed{D(y) = (7 - 3y) \pmod{26}} \quad (4)$$

Step 3: 将密文字母转换为数字

密文: hlzb l slqfb tpnywbotalojb

去掉空格, 逐字母编码 ($a = 0, b = 1, \dots, z = 25$):

$$\begin{aligned} h \rightarrow 7, \quad l \rightarrow 11, \quad z \rightarrow 25, \quad b \rightarrow 1, \quad l \rightarrow 11, \quad s \rightarrow 18, \quad l \rightarrow 11, \quad q \rightarrow 16, \quad f \rightarrow 5, \quad b \rightarrow 1, \\ t \rightarrow 19, \quad p \rightarrow 15, \quad n \rightarrow 13, \quad y \rightarrow 24, \quad w \rightarrow 22, \quad b \rightarrow 1, \quad o \rightarrow 14, \quad t \rightarrow 19, \quad a \rightarrow 0, \quad l \rightarrow 11, \\ o \rightarrow 14, \quad j \rightarrow 9, \quad b \rightarrow 1 \end{aligned}$$

Step 4: 逐字母解密

对每个密文数字 y , 计算 $D(y) = (7 - 3y) \pmod{26}$:

密文	密文数字 y	$3y$	$D(y) = (7 - 3y) \bmod 26$
h	7	21	$(7 - 21) \bmod 26 = -14 \bmod 26 = 12 \rightarrow m$
l	11	33	$(7 - 33) \bmod 26 = -26 \bmod 26 = 0 \rightarrow a$
z	25	75	$(7 - 75) \bmod 26 = -68 \bmod 26 = 10 \rightarrow k$
b	1	3	$(7 - 3) \bmod 26 = 4 \rightarrow e$
l	11	33	$(7 - 33) \bmod 26 = 0 \rightarrow a$
	—	—	(空格保留)
s	18	54	$(7 - 54) \bmod 26 = -47 \bmod 26 = 5 \rightarrow f$
l	11	33	$(7 - 33) \bmod 26 = 0 \rightarrow a$
q	16	48	$(7 - 48) \bmod 26 = -41 \bmod 26 = 11 \rightarrow l$
f	5	15	$(7 - 15) \bmod 26 = -8 \bmod 26 = 18 \rightarrow s$
b	1	3	$(7 - 3) \bmod 26 = 4 \rightarrow e$
	—	—	(空格保留)
t	19	57	$(7 - 57) \bmod 26 = -50 \bmod 26 = 2 \rightarrow c$
p	15	45	$(7 - 45) \bmod 26 = -38 \bmod 26 = 14 \rightarrow o$
n	13	39	$(7 - 39) \bmod 26 = -32 \bmod 26 = 20 \rightarrow u$
y	24	72	$(7 - 72) \bmod 26 = -65 \bmod 26 = 13 \rightarrow n$
w	22	66	$(7 - 66) \bmod 26 = -59 \bmod 26 = 19 \rightarrow t$
b	1	3	$(7 - 3) \bmod 26 = 4 \rightarrow e$
o	14	42	$(7 - 42) \bmod 26 = -35 \bmod 26 = 17 \rightarrow r$
t	19	57	$(7 - 57) \bmod 26 = 2 \rightarrow c$
a	0	0	$(7 - 0) \bmod 26 = 7 \rightarrow h$
l	11	33	$(7 - 33) \bmod 26 = 0 \rightarrow a$
o	14	42	$(7 - 42) \bmod 26 = 17 \rightarrow r$
j	9	27	$(7 - 27) \bmod 26 = -20 \bmod 26 = 6 \rightarrow g$
b	1	3	$(7 - 3) \bmod 26 = 4 \rightarrow e$

Step 5: 还原明文

将解密得到的字母按顺序排列并恢复空格位置，得到：

make a false countercharge

(5)

8.2 AES S-盒代换 (10 分)

题目：AES 加密过程中某圈的数据块为 EA04658583455D965C3398B0F02DADC5 (128 bit)，求该数据块经过非线性代替 (S-盒) 后的输出。

知识点回顾：AES S-盒 (Substitution Box)

1. AES 分组与状态矩阵

AES 使用 128 位 (16 字节) 的分组长度。128 位数据块以字节为单位组织为 4×4 的状态矩阵 (State)，按列优先顺序填充：

$$\begin{pmatrix} S_{00} & S_{01} & S_{02} & S_{03} \\ S_{10} & S_{11} & S_{12} & S_{13} \\ S_{20} & S_{21} & S_{22} & S_{23} \\ S_{30} & S_{31} & S_{32} & S_{33} \end{pmatrix}$$

输入字节序列 $[b_0, b_1, \dots, b_{15}]$ 按列映射： $S_{rc} = b_{r+4c}$ 。

2. S-盒 (SubBytes) 变换

SubBytes 是 AES 中唯一的非线性变换，对状态矩阵的每个字节独立进行字节代换：

- 每个字节 $b = (b_7b_6b_5b_4b_3b_2b_1b_0)_2$ 视为 $GF(2^8)$ 中的元素；
- 首先求 b 在 $GF(2^8)$ 中的乘法逆元 (0 的逆元定义为 0)；
- 然后进行仿射变换 (在 $GF(2)$ 上)：

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i \quad (6)$$

其中 $c = (01100011)_2 = 0x63$ 。

3. S-盒查表方法

16 进制输入字节 XY (X 为行索引高位, Y 为列索引低位)，查 S-盒：

- 查找表的行为高 4 位 (X)，列为低 4 位 (Y)；
- 例如 EA：行 E (第 14 行)，列 A (第 10 列)，查表得到对应值。

4. S-盒的作用

S-盒提供了 AES 的混淆 (Confusion) 特性，使得明文和密文之间的统计关系尽可能复杂，抵抗线性和差分密码分析。

5. 标准 AES S-盒 (十六进制)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	1B	9C	4A	5C	FF

S-盒代换——逐步查表

Step 1：将 128 位数据块分解为 16 个字节

输入数据块：EA 04 65 85 83 45 5D 96 5C 33 98 B0 F0 2D AD C5

Step 2：逐字节查 S-盒

S-盒查表规则：每字节的高 4 位为行号，低 4 位为列号。

输入字节	高 4 位 (行)	低 4 位 (列)	S-盒输出
EA	E	A	第 E 行第 A 列 → 87
04	0	4	第 0 行第 4 列 → F2
65	6	5	第 6 行第 5 列 → 4D
85	8	5	第 8 行第 5 列 → 97
83	8	3	第 8 行第 3 列 → EC
45	4	5	第 4 行第 5 列 → 6E
5D	5	D	第 5 行第 D 列 → 4C
96	9	6	第 9 行第 6 列 → 90
5C	5	C	第 5 行第 C 列 → 4A
33	3	3	第 3 行第 3 列 → C3
98	9	8	第 9 行第 8 列 → 46
B0	B	0	第 B 行第 0 列 → E7
F0	F	0	第 F 行第 0 列 → 8C
2D	2	D	第 2 行第 D 列 → D8
AD	A	D	第 A 行第 D 列 → 95
C5	C	5	第 C 行第 5 列 → A6

Step 3：组装输出

将查表得到的 16 个字节按原顺序拼接：

87 F2 4D 97 EC 6E 4C 90 4A C3 46 E7 8C D8 95 A6

(7)

即：87F24D97EC6E4C904AC346E78CD895A6

8.3 RSA 密钥计算与加密 (10 分)

题目：已知 RSA 密码体制中 $p = 23$, $q = 31$, 私钥 $d = 569$, 试求加密密钥 e , 且对明文 $M = 25$ 进行加密。

知识点回顾：RSA 公钥密码体制

1. RSA 密钥生成

RSA 算法由 Rivest、Shamir 和 Adleman 于 1977 年提出, 其密钥生成步骤如下:

1. 选择两个大素数 p 和 q ;
2. 计算模数 $n = p \cdot q$;
3. 计算欧拉函数值 $\varphi(n) = (p-1)(q-1)$;
4. 选择加密密钥 (公钥) e , 满足 $1 < e < \varphi(n)$ 且 $\gcd(e, \varphi(n)) = 1$;
5. 计算解密密钥 (私钥) $d \equiv e^{-1} \pmod{\varphi(n)}$, 即 $e \cdot d \equiv 1 \pmod{\varphi(n)}$ 。

2. RSA 加解密

- 加密: 给定明文 M ($0 \leq M < n$), 密文 $C = M^e \bmod n$;
- 解密: $M = C^d \bmod n$ 。

3. 已知 d 求 e

由于 $e \cdot d \equiv 1 \pmod{\varphi(n)}$, 即 $e \equiv d^{-1} \pmod{\varphi(n)}$, 故求 e 等价于求 d 在模 $\varphi(n)$ 下的乘法逆元, 同样使用扩展欧几里得算法。

4. 模幂运算——快速幂 (平方-乘算法)

计算 $M^e \bmod n$ 时, 若 e 较大, 不能直接计算 M^e 再取模 (中间结果过大)。采用平方-乘算法:

- 将指数 e 表示为二进制形式 $e = \sum b_i 2^i$;
- 从高位到低位迭代计算: 每步平方, 遇到 $b_i = 1$ 时再乘一次。

求解过程——逐步推导

Step 1: 计算基本参数

$$n = p \times q = 23 \times 31 = 713$$

$$\varphi(n) = (p-1)(q-1) = 22 \times 30 = 660$$

Step 2: 求加密密钥 $e = d^{-1} \bmod \varphi(n)$

已知 $d = 569$, $\varphi(n) = 660$ 。使用扩展欧几里得算法求 $569^{-1} \bmod 660$:

$$\begin{aligned} 660 &= 569 \times 1 + 91 \\ 569 &= 91 \times 6 + 23 \quad (91 \times 6 = 546) \\ 91 &= 23 \times 3 + 22 \quad (23 \times 3 = 69) \\ 23 &= 22 \times 1 + 1 \\ 22 &= 1 \times 22 + 0 \end{aligned}$$

回代求系数:

$$\begin{aligned} 1 &= 23 - 22 \times 1 \\ &= 23 - (91 - 23 \times 3) = 23 \times 4 - 91 \\ &= (569 - 91 \times 6) \times 4 - 91 = 569 \times 4 - 91 \times 25 \\ &= 569 \times 4 - (660 - 569) \times 25 = 569 \times 4 - 660 \times 25 + 569 \times 25 \\ &= 569 \times 29 - 660 \times 25 \end{aligned}$$

因此 $569 \times 29 \equiv 1 \pmod{660}$, 即:

$$\boxed{e = 29} \quad (8)$$

验证: $569 \times 29 = 16501$, $660 \times 25 = 16500$, $16501 - 16500 = 1$ 。✓

Step 3: 加密明文 $M = 25$

计算 $C = M^e \bmod n = 25^{29} \bmod 713$ 。

使用平方-乘算法, 先写出指数 29 的二进制表示:

$$29 = 11101_2 = 16 + 8 + 4 + 1$$

$$\begin{aligned} 25^1 \bmod 713 &= 25 \\ 25^2 \bmod 713 &= 625 \\ 25^4 \bmod 713 &= 625^2 \bmod 713 = 390625 \bmod 713 \end{aligned}$$

计算 $625^2 = 390625$ 。 $390625 \div 713$: $713 \times 547 = 390011$, $390625 - 390011 = 614$ 。

$$\begin{aligned} 25^4 \bmod 713 &= 614 \\ 25^8 \bmod 713 &= 614^2 \bmod 713 = 376996 \bmod 713 \end{aligned}$$

$713 \times 528 = 376464$, $376996 - 376464 = 532$ 。

$$25^8 \bmod 713 = 532$$

$$25^{16} \bmod 713 = 532^2 \bmod 713 = 283024 \bmod 713$$

$713 \times 397 = 283061$ 。这说明 $283024 < 283061$ 。 $713 \times 396 = 282348$, $283024 - 282348 = 676$ 。

$$25^{16} \bmod 713 = 676$$

现在组装 $25^{29} = 25^{16} \cdot 25^8 \cdot 25^4 \cdot 25^1$:

先算 $25^4 \cdot 25^1 = 614 \times 25 = 15350 \bmod 713$: $713 \times 21 = 14973$, $15350 - 14973 = 377$ 。

再乘以 25^8 : $377 \times 532 \bmod 713$ 。 $377 \times 532 = 200564$ 。 $713 \times 281 = 200353$, $200564 - 200353 = 211$ 。

再乘以 25^{16} : $211 \times 676 \bmod 713$ 。 $211 \times 676 = 142636$ 。 $713 \times 200 = 142600$, $142636 - 142600 = 36$ 。

因此:

$$\boxed{C = 25^{29} \bmod 713 = 36} \quad (9)$$

验证 (解密复核): $C^d \bmod n = 36^{569} \bmod 713$ 应等于 25 (省略验证, 由数学原理保证正确性)。

8.4 椭圆曲线密码 (ECC) (15 分)

题目：已知椭圆曲线 $y^2 = x^3 + 8x + 10 \pmod{23}$,

(1) 试找出该椭圆曲线上的所有整数点；(10 分)

(2) 试计算 $3(7, 8)$ 。(5 分)

知识点回顾：椭圆曲线密码学 (ECC)

1. 椭圆曲线的定义

有限域 $\mathbb{F}_p$ ($p > 3$ 为素数) 上的椭圆曲线 Weierstrass 方程为:

$$y^2 \equiv x^3 + ax + b \pmod{p}, \quad 4a^3 + 27b^2 \not\equiv 0 \pmod{p} \quad (10)$$

其中判别式 $\Delta = -16(4a^3 + 27b^2)$ 非零确保曲线非奇异 (没有尖点或自交点)。

2. 椭圆曲线上的点

椭圆曲线上的点包括:

- 满足方程的有限点 (x, y) , 其中 $x, y \in \mathbb{F}_p$;
- 无穷远点 $\mathcal{O}$, 作为群的单位元 (零元)。

3. 椭圆曲线上的群运算 (加法法则)

设 $P = (x_1, y_1)$, $Q = (x_2, y_2)$ 为曲线上的点:

情况 A: $P \neq \pm Q$ (一般点加)

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$$

情况 B: $P = Q$ (倍点运算, $2P$)

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}$$

$$x_3 = \lambda^2 - 2x_1 \pmod{p}$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}$$

情况 C: $P = -Q$ (即 $x_1 = x_2, y_1 = -y_2$)

$$P + (-P) = \mathcal{O}$$

4. 寻找所有整数点的方法

对于有限域 $\mathbb{F}_p$ 上的椭圆曲线, 寻找所有有限点的方法是:

1. 对每个 $x \in \{0, 1, \dots, p-1\}$, 计算 $z = x^3 + ax + b \pmod{p}$;

2. 检查 z 是否为模 p 的二次剩余，即是否存在 y 使得 $y^2 \equiv z \pmod{p}$;
3. 若 z 是二次剩余，则有两个解 y 和 $p - y$; 若 $z = 0$ ，则 $y = 0$; 若 z 不是二次剩余，则该 x 不在曲线上。

5. ECC 的安全性

ECC 的安全性基于椭圆曲线离散对数问题 (ECDLP): 给定 P 和 $Q = kP$, 求 k 在计算上是困难的。相比 RSA, ECC 可以在更短的密钥长度下提供同等的安全性。

第 (1) 问: 找出所有整数点——详细过程

Step 1: 建立 y 值的平方表

在模 23 下, y 的取值范围为 0 到 22。由于 $(-y)^2 = y^2$, 只需计算 $y = 0, 1, \dots, 11$ 的平方。

y	0	± 1	± 2	± 3	± 4	± 5	± 6	± 7	± 8	± 9	± 10	± 11
$y^2 \bmod 23$	0	1	4	9	16	2	13	3	18	12	8	6

Step 2: 对每个 $x \in \{0, 1, \dots, 22\}$ 计算 $x^3 + 8x + 10 \bmod 23$

x	x^3	$8x$	$z = x^3 + 8x + 10 \bmod 23$
0	0	0	$0 + 0 + 10 = \mathbf{10}$
1	1	8	$1 + 8 + 10 = 19 = \mathbf{19}$
2	8	16	$8 + 16 + 10 = 34 \equiv \mathbf{11}$
3	27	24	$27 + 24 + 10 = 61 \equiv \mathbf{15}$
4	64	32	$64 + 32 + 10 = 106 \equiv \mathbf{14}$
5	125	40	$125 + 40 + 10 = 175 \equiv \mathbf{14}$
6	216	48	$216 + 48 + 10 = 274 \equiv \mathbf{21}$
7	343	56	$343 + 56 + 10 = 409 \equiv \mathbf{18}$
8	512	64	$512 + 64 + 10 = 586 \equiv \mathbf{11}$
9	729	72	$729 + 72 + 10 = 811 \equiv \mathbf{6}$
10	1000	80	$1000 + 80 + 10 = 1090 \equiv \mathbf{9}$
11	1331	88	$1331 + 88 + 10 = 1429 \equiv \mathbf{3}$
12	1728	96	$1728 + 96 + 10 = 1834 \equiv \mathbf{17}$
13	2197	104	$2197 + 104 + 10 = 2311 \equiv \mathbf{11}$
14	2744	112	$2744 + 112 + 10 = 2866 \equiv \mathbf{14}$
15	3375	120	$3375 + 120 + 10 = 3505 \equiv \mathbf{9}$
16	4096	128	$4096 + 128 + 10 = 4234 \equiv \mathbf{2}$
17	4913	136	$4913 + 136 + 10 = 5059 \equiv \mathbf{22}$
18	5832	144	$5832 + 144 + 10 = 5986 \equiv \mathbf{6}$
19	6859	152	$6859 + 152 + 10 = 7021 \equiv \mathbf{6}$
20	8000	160	$8000 + 160 + 10 = 8170 \equiv \mathbf{5}$
21	9261	168	$9261 + 168 + 10 = 9439 \equiv \mathbf{9}$
22	10648	176	$10648 + 176 + 10 = 10834 \equiv \mathbf{1}$

Step 3: 综合对照表——匹配 z 值与 y^2 值

将 x 对应的 z 值和 y^2 的可取值合并为一张对照表:

x	$z = x^3 + 8x + 10 \bmod 23$	是否在 y^2 可取值中?
0	10	否 (10 不是二次剩余)
1	19	否
2	11	否
3	15	否
4	14	否
5	14	否
6	21	否
7	18	是! $y^2 = 18 \Rightarrow y = \pm 8$
8	11	否
9	6	是! $y^2 = 6 \Rightarrow y = \pm 11$
10	9	是! $y^2 = 9 \Rightarrow y = \pm 3$
11	3	是! $y^2 = 3 \Rightarrow y = \pm 7$
12	17	否
13	11	否
14	14	否
15	9	是! $y^2 = 9 \Rightarrow y = \pm 3$
16	2	是! $y^2 = 2 \Rightarrow y = \pm 5$
17	22	否 (22 不是二次剩余)
18	6	是! $y^2 = 6 \Rightarrow y = \pm 11$
19	6	是! $y^2 = 6 \Rightarrow y = \pm 11$
20	5	否
21	9	是! $y^2 = 9 \Rightarrow y = \pm 3$
22	1	是! $y^2 = 1 \Rightarrow y = \pm 1$

Step 4: 列出所有整数点

共找到 10 个有效的 x 值, 每个对应一对 y 值, 共 20 个有限点, 加上无穷远点 $\mathcal{O}$ 。

序号	x	对应的点
1,2	7	(7, 8), (7, 15)
3,4	9	(9, 11), (9, 12)
5,6	10	(10, 3), (10, 20)
7,8	11	(11, 7), (11, 16)
9,10	15	(15, 3), (15, 20)
11,12	16	(16, 5), (16, 18)
13,14	18	(18, 11), (18, 12)
15,16	19	(19, 11), (19, 12)
17,18	21	(21, 3), (21, 20)
19,20	22	(22, 1), (22, 22)
21	—	$\mathcal{O}$ (无穷远点)

总结：该椭圆曲线上共有 21 个点 (20 个有限点 + 无穷远点 $\mathcal{O}$)。

title

计算 $3P = P + P + P$ ，先算 $2P = P + P$ （倍点），再算 $2P + P$ （点加）。

Step 1: 计算 $2P = 2(7, 8)$ （倍点公式）

椭圆曲线: $y^2 = x^3 + 8x + 10 \pmod{23}$ ，即 $a = 8$, $b = 10$, $p = 23$ 。

已知 $P = (x_1, y_1) = (7, 8)$ 。

计算斜率 λ :

$$\begin{aligned}\lambda &= \frac{3x_1^2 + a}{2y_1} \pmod{p} \\ &= \frac{3 \times 7^2 + 8}{2 \times 8} \pmod{23} \\ &= \frac{3 \times 49 + 8}{16} \pmod{23} \\ &= \frac{147 + 8}{16} \pmod{23} = \frac{155}{16} \pmod{23}\end{aligned}$$

先化简分子: $155 \pmod{23} = 155 - 6 \times 23 = 155 - 138 = 17$ 。

求分母的逆元: $16^{-1} \pmod{23}$ 。使用扩展欧几里得算法:

$$23 = 16 \times 1 + 7$$

$$16 = 7 \times 2 + 2$$

$$7 = 2 \times 3 + 1$$

$$2 = 1 \times 2 + 0$$

回代:

$$\begin{aligned}1 &= 7 - 2 \times 3 = 7 - (16 - 7 \times 2) \times 3 \\ &= 7 \times 7 - 16 \times 3 = (23 - 16) \times 7 - 16 \times 3 \\ &= 23 \times 7 - 16 \times 10\end{aligned}$$

故 $16^{-1} \equiv -10 \equiv 13 \pmod{23}$ 。验证: $16 \times 13 = 208 = 9 \times 23 + 1$ 。✓

因此:

$$\lambda = 17 \times 13 \pmod{23} = 221 \pmod{23} = 221 - 9 \times 23 = 221 - 207 = 14 \quad (11)$$

计算 x_3 和 y_3 :

$$\begin{aligned}x_3 &= \lambda^2 - 2x_1 \pmod{p} = 14^2 - 2 \times 7 \pmod{23} \\ &= 196 - 14 \pmod{23} = 182 \pmod{23} = 182 - 7 \times 23 = 182 - 161 = 21 \\ y_3 &= \lambda(x_1 - x_3) - y_1 \pmod{p} = 14 \times (7 - 21) - 8 \pmod{23} \\ &= 14 \times (-14) - 8 \pmod{23} = -196 - 8 \pmod{23} \\ &= -204 \pmod{23}\end{aligned}$$

$-204 \bmod 23$: $204 = 8 \times 23 + 20$, 故 $-204 \equiv -(8 \times 23 + 20) \equiv -20 \equiv 3 \pmod{23}$ 。

因此:

$$\boxed{2P = (21, 3)} \quad (12)$$

Step 2: 计算 $3P = 2P + P = (21, 3) + (7, 8)$ (点加公式)

已知 $P_1 = (21, 3)$, $P_2 = (7, 8)$ 。

由于 $x_1 \neq x_2$ ($21 \neq 7$), 使用一般点加公式。

计算斜率 λ :

$$\begin{aligned} \lambda &= \frac{y_2 - y_1}{x_2 - x_1} \bmod 23 \\ &= \frac{8 - 3}{7 - 21} \bmod 23 = \frac{5}{-14} \bmod 23 \end{aligned}$$

$5 \bmod 23 = 5$, $-14 \bmod 23 = 9$ 。故 $\lambda = 5 \cdot 9^{-1} \bmod 23$ 。

或者保持原写法: $\lambda = \frac{5}{-14} \bmod 23$ 。

先求 $(-14)^{-1} \bmod 23$, 等价于求 $9^{-1} \bmod 23$ 。

$$23 = 9 \times 2 + 5$$

$$9 = 5 \times 1 + 4$$

$$5 = 4 \times 1 + 1$$

$$4 = 1 \times 4 + 0$$

回代:

$$\begin{aligned} 1 &= 5 - 4 = 5 - (9 - 5) = 2 \times 5 - 9 \\ &= 2 \times (23 - 9 \times 2) - 9 \\ &= 2 \times 23 - 5 \times 9 \end{aligned}$$

故 $9^{-1} \equiv -5 \equiv 18 \pmod{23}$ 。

因为 $\lambda = \frac{5}{-14} = \frac{-5}{14}$, 或者直接 $\lambda = 5 \times (9^{-1}) \bmod 23 = 5 \times 18 = 90 \equiv 90 - 3 \times 23 = 90 - 69 = 21$ 。

验证 (另一种方法): $14^{-1} \bmod 23 = 5$ (因为 $14 \times 5 = 70 = 3 \times 23 + 1$), $\lambda = -5 \times 5 = -25 \equiv -2 \equiv 21 \pmod{23}$ 。两种方法一致。✓

因此 $\boxed{\lambda = 21}$ 。

计算 x_3 和 y_3 :

$$\begin{aligned} x_3 &= \lambda^2 - x_1 - x_2 \bmod 23 \\ &= 21^2 - 21 - 7 \bmod 23 \\ &= 441 - 28 \bmod 23 = 413 \bmod 23 \end{aligned}$$

$$413 \div 23 = 17 \text{ 余 } 22 \quad (\text{因为 } 17 \times 23 = 391, 413 - 391 = 22)$$

$$\begin{aligned} y_3 &= \lambda(x_1 - x_3) - y_1 \bmod 23 \\ &= 21 \times (21 - 22) - 3 \bmod 23 \\ &= 21 \times (-1) - 3 \bmod 23 \\ &= -21 - 3 \bmod 23 = -24 \bmod 23 \end{aligned}$$

$$-24 \bmod 23 = -24 + 23 = -1 \equiv 22 \pmod{23}.$$

因此:

$$\boxed{3P = 3(7, 8) = (22, 22)} \quad (13)$$

验证 (可选): 检查 $(22, 22)$ 是否在曲线上:

$$\begin{aligned} y^2 \bmod 23 &= 22^2 \bmod 23 = 484 \bmod 23 = 484 - 21 \times 23 = 484 - 483 = 1 \\ x^3 + 8x + 10 \bmod 23 &= 22^3 + 8 \times 22 + 10 \bmod 23 \\ &= 10648 + 176 + 10 \bmod 23 = 10834 \bmod 23 = 1 \end{aligned}$$

等式成立。✓

8.5 AES 密码反馈 (CFB) 模式分析 (15 分)

题目：以 AES 为例：

- (1) 画出分组密码的密码反馈 (CFB) 模式的加密和解密示意图；(6 分)
- (2) 假设加密某个明文块时出现一个比特错误（比如明文输入时一个比特读错了），对解密造成什么影响？(5 分)
- (3) 假设密文传输时出现一个比特错误，对解密造成什么影响？(4 分)

知识点回顾：分组密码工作模式——CFB (Cipher Feedback)

1. 分组密码工作模式概述

分组密码（如 AES）每次处理固定长度的数据块（AES 为 128 位）。为适应不同长度的明文和不同应用场景，NIST 定义了五种标准工作模式：

- **ECB** (Electronic Codebook)：电子密码本模式，最基础但安全性最弱；
- **CBC** (Cipher Block Chaining)：密码分组链接模式；
- **CFB** (Cipher Feedback)：密码反馈模式；
- **OFB** (Output Feedback)：输出反馈模式；
- **CTR** (Counter)：计数器模式。

2. CFB 模式原理

CFB 模式将分组密码转换为自同步流密码。与 CBC 不同，CFB 的加密和解密过程都使用加密算法。

CFB 加密 (s 位反馈)：

- $I_1 = IV$ (初始化向量, n 位)
- 对于每个明文段 P_i (s 位, $1 \leq s \leq n$):

$$O_i = E_K(I_i) \text{ (对 } n \text{ 位移位寄存器加密)}$$

$$C_i = P_i \oplus \text{MSB}_s(O_i) \text{ (取加密结果高 } s \text{ 位与明文异或)}$$

$$I_{i+1} = (I_i \ll s) \mid C_i \text{ (移位寄存器左移 } s \text{ 位, 右边填入密文)}$$

CFB 解密：

- 同样使用加密算法 E_K (非解密算法):

$$O_i = E_K(I_i)$$

$$P_i = C_i \oplus \text{MSB}_s(O_i)$$

$$I_{i+1} = (I_i \ll s) \mid C_i$$

3. CFB 模式的特点

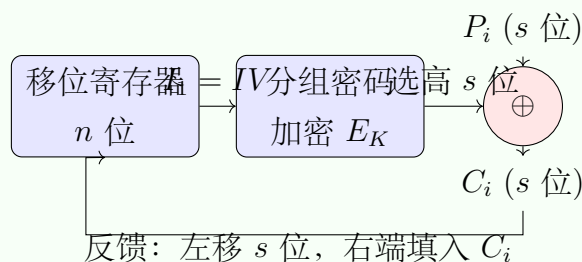
- **优点：**无需填充（可处理任意长度的数据流）；解密只使用加密算法（硬件实现只需加密单元）；自同步能力强。
- **缺点：**加密不能并行（反馈回路）；错误传播特性与 s 有关。

4. 错误传播分析

- **明文错误：**加密时某明文块出错，该错误会进入移位寄存器，影响后续加密。但解密时该密文块正常解密，后续块也跟随密文正确恢复。
- **密文传输错误：**某密文块的一个比特错误会导致：该块解密出的明文对应位出错，且该错误密文进入移位寄存器后在后续若干轮影响解密，直到该错误被移出寄存器。

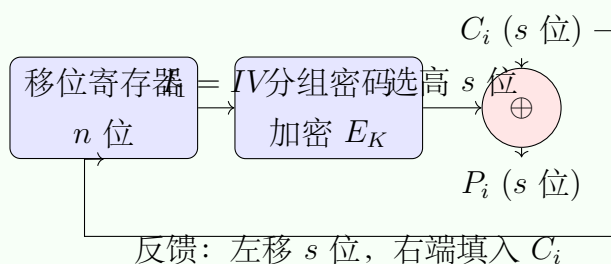
第 (1) 问：CFB 模式加密/解密示意图

CFB 加密示意图



CFB 加密示意图

CFB 解密示意图



CFB 解密示意图

关键点说明：

- 加密和解密都使用**加密函数** E_K ，而非解密函数 D_K ；
- IV （初始化向量）必须在加密端和解密端保持一致，不需要保密；
- 明文 / 密文的分段大小 s 可配置 ($1 \leq s \leq n$)，常见取 $s = n$ （全块 CFB）或 $s = 8$ （8 位 CFB，适合面向字节的流加密）；

- 密文 C_i 既作为传输的输出，又反馈回移位寄存器参与后续加密。

第 (2) 问：明文输入错误的影响分析

场景：加密某个明文块 P_i 时出现一个比特错误（即加密器读入的 P_i 与实际待加密的 P_i 不一致）。

分析：

1. 加密时，错误的 P_i 与 O_i 的高 s 位异或后生成错误密文 C'_i 。
2. 错误密文 C'_i 被反馈回移位寄存器，因此**后续所有密文块** $C_{i+1}, C_{i+2}, \dots$ **都可能受影响**（直到错误被移出寄存器）。
3. 在解密端，收到的密文 C'_i 本身是错误的，解密出的 $\hat{P}_i = C'_i \oplus \text{MSB}_s(O_i)$ 也是错误的——**该块的明文无法正常恢复**。
4. 但是， C'_i 在解密端同样被反馈回移位寄存器，加密和解密两端的移位寄存器内容**保持一致**。因此后续块 $i+1, i+2, \dots$ 的 O_j 计算与加密端同步，**后续块可以正确解密**。

结论：加密某个明文块时出现一个比特错误，**该错误会在密文中传播**（影响该块及后续密文块），但解密时**只有该块对应的明文不能正常解密**，其余块均能正确解密。

第 (3) 问：密文传输错误的影响分析

场景：密文在信道传输过程中，某个密文块 C_i 出现一个比特错误。

分析：

1. 解密时，解密器收到错误密文 $\hat{C}_i$ ：

$$\tilde{P}_i = \hat{C}_i \oplus \text{MSB}_s(O_i)$$

由于 $\hat{C}_i$ 与 C_i 在错误比特位不同，**该块解密出的明文 $\tilde{P}_i$ 在对应位出现错误**。

2. 错误密文 $\hat{C}_i$ 被反馈回解密端的移位寄存器，导致 I_{i+1} 与加密端不同。因此：

$$O_{i+1} = E_K(\hat{I}_{i+1}) \neq E_K(I_{i+1})$$

故 $\tilde{P}_{i+1}$ 也全部出错。

3. $\hat{C}_i$ 的错误进入移位寄存器后，需要 n/s 个块的时间才能被完全移出（对于全块 CFB，即 $s = n$ 时，需要 1 个块）。在此期间， O_j 的计算均与加密端不同，直到寄存器被正确密文重新填满。**收到正确的后续密文后，解密可以自同步恢复**。
4. 在错误发生之前的块 $1, 2, \dots, i-1$ 不受影响——移位寄存器内容在此之前是正确的。

以全块 CFB ($s = n$) 为例的具体说明:

- 设 C_i 出错, 解密 $\tilde{P}_i$ 出错 (错误在对应比特位);
- 错误的 C_i 全部填入移位寄存器, 导致 O_{i+1} 完全错误, $\tilde{P}_{i+1}$ 全错;
- 正确的 C_{i+1} 填入移位寄存器后, I_{i+2} 又恢复正常 (因为 $I_{i+2} = C_{i+1}$ 对全块 CFB), $\tilde{P}_{i+2}$ 恢复正常。

结论: 密文传输时出现一个比特错误, 会影响该密文块及之后一定数量的密文块对应的解密, 直到错误密文被移出移位寄存器后, 系统自同步恢复正常。之前的密文块不受影响, 可以正常解密。

8.6 双通匿名密钥确认协议分析 (20 分)

题目：分析以下经典的双通匿名密钥确认密钥协议方案：

Key Token Construction (A1) 实体 A 生成一个随机数 r_A , 构建密钥令牌 $KT_{A1} = r_A \parallel \text{Text}_1$, 发送给实体 B, Text_1 可以包含 A 的公钥证书;

Key Token Processing (B1) 实体 B 生成一个随机数 r_B , 计算 $BS = S_B(A \parallel r_A \parallel r_B \parallel \text{Text}_2)$, 构建密钥令牌 $KT_{B1} = E_A(B \parallel BS \parallel \text{Text}_3) \parallel \text{Text}_4$, 发送给实体 A, Text_3 包含 MAC, Text_4 包含 B 的验签公钥;

Key Token Construction (B2) 共享密钥从签名信息 BS 中的全部或部分通过密钥派生函数导出;

Key Token Processing (A2) 实体 A 使用 A 的私钥解密 $E_A(B \parallel BS \parallel \text{Text}_3)$, 优先检查 B 的标识符; 然后对 BS 进行验签, 所有检查成功后, 计算出共享密钥。

- (1) 分析该方案通信次数; (4 分)
- (2) 如何提供隐式密钥身份验证? (5 分)
- (3) 如何提供密钥确认? (5 分)
- (4) 该协议适合的应用场景? (6 分)

知识点回顾：密钥协商与认证协议

1. 密钥协商 (Key Agreement) 基本概念

密钥协商是两个或多个通信实体通过交互, 共同建立一个共享会话密钥的过程。安全目标包括:

- **隐式密钥认证 (Implicit Key Authentication)**: 一方确信除了指定的对方实体外, 没有其他实体能够获得该共享密钥。
- **密钥确认 (Key Confirmation)**: 一方确信对方实体确实已经拥有了该共享密钥。
- **显式密钥认证 (Explicit Key Authentication)**: 同时满足隐式密钥认证和密钥确认。
- **前向安全性 (Forward Secrecy)**: 长期私钥泄露不会导致过去已建立的会话密钥泄露。
- **匿名性 (Anonymity)**: 通信一方的身份对窃听者 (乃至对方) 不可见。

2. 协议中的常见密码学原语

- $E_A(\cdot)$: 使用实体 A 的公钥进行非对称加密, 只有 A 能用私钥解密, 提供了机密性和目标指定;

- $S_B(\cdot)$: 实体 B 的签名, 使用 B 的私钥对消息进行数字签名, 提供数据完整性、消息源认证和不可否认性;
- MAC (Message Authentication Code): 消息认证码, 用于验证消息的完整性和来源, 同时隐式证明 MAC 密钥的持有;
- 密钥派生函数 (KDF): 从主秘密材料派生出适合使用的会话密钥, 通常使用哈希函数。

3. 协议分析的一般方法

- 通信轮次: 统计消息交换的次数 (一次发送 + 一次接收 = 一轮);
- 安全属性检验: 逐一检查每个安全目标是否被满足, 以及如何被满足;
- 应用场景匹配: 根据协议的特点 (效率、安全性、隐私保护等) 推荐合适的部署环境。

逐步分析

(1) 通信次数分析 (4 分)

协议的通信流程如下:

1. $A \rightarrow B: KT_{A1} = r_A \parallel \text{Text}_1$ (第一次通信)
2. $B \rightarrow A: KT_{B1} = E_A(B \parallel BS \parallel \text{Text}_3) \parallel \text{Text}_4$ (第二次通信)

共有 **2 次消息传输**, 其中:

- 第一次通信: A 发送密钥令牌 KT_{A1} 给 B, 包含随机数 r_A 和可选的公钥证书;
- 第二次通信: B 发送密钥令牌 KT_{B1} 给 A, 包含加密的 B 标识符、签名 BS 、MAC 以及 B 的验签公钥。

通信轮次: 2 次 (即一轮往返 RTT)。

因此得名「双通」(Two-Pass) 协议——只需要两次单向消息传递即可完成密钥协商。

(2) 隐式密钥身份验证分析 (5 分)

该协议通过以下机制提供**隐式密钥身份验证**:

1. B 对 A 的隐式认证:

- B 使用 A 的公钥加密响应消息: $E_A(B \parallel BS \parallel \text{Text}_3)$ 。只有拥有对应私钥的实体 A 才能解密该消息;
- 共享密钥由 BS (B 的签名数据) 通过 KDF 派生。 BS 中包含 r_A (A 的随机数)。只有能够解密 $E_A(\cdot)$ 的实体 (即 A) 才能获取 BS 并派生出共享密钥;

- 因此, B 确信: 除了 A 以外, 没有其他实体能获得该共享密钥。

2. A 对 B 的隐式认证:

- A 解密后获得 $BS = S_B(A \parallel r_A \parallel r_B \parallel \text{Text}_2)$, 使用 Text_4 中 B 的验签公钥 (或信任的证书链) 对 BS 进行签名验证;
- BS 中包含 r_B (B 生成的随机数), 可以证明 B 确实参与了本次会话;
- 共享密钥从 BS 中派生, 只有能够生成有效签名 BS 的实体 (即 B) 才能计算出共享密钥;
- 因此, A 确信: 除了 B 以外, 没有其他实体能获得该共享密钥。

总结: 该协议通过 A 的公钥加密确保 B 到 A 的隐式认证, 通过 B 的数字签名确保 A 到 B 的隐式认证。双方都确信只有对方能共享该密钥, 但**无需额外的显式确认步骤**——这就是「隐式」的含义。

(3) 密钥确认分析 (5 分)

密钥确认要求一方确信对方**已经实际持有**共享密钥。该协议的密钥确认机制分析如下:

- Text_3 中包含**消息认证码 (MAC)**。MAC 是使用共享密钥 (或其派生密钥) 对关键协议消息计算的认证标记。
- B 在构建 KT_{B1} 时将 MAC 附着在加密部分中发送给 A。
- A 解密后, 使用自己派生的共享密钥验证 MAC 的正确性。如果 MAC 验证通过, 则:
 - A 确信 B 已经计算出了正确的共享密钥 (因为只有持有相同密钥的实体才能生成正确的 MAC);
 - 这提供了**从 B 到 A 的密钥确认**。
- 然而, 该协议仅提供了**单向的密钥确认** ($B \rightarrow A$)。A 没有向 B 发送任何使用共享密钥保护的消息来证明自己持有该密钥。

如果需要双向密钥确认: 可以增加第三次通信 (变为三通协议), 由 A 使用共享密钥生成并发送确认消息给 B。

总结: 该协议通过 Text_3 中的 MAC 提供了**实体 B 到实体 A 的密钥确认**, A 可以确信 B 已拥有正确的共享密钥。

(4) 适合的应用场景分析 (6 分)

核心特征归纳:

- 双通 (2 次通信), 通信开销低;

- 对窃听者而言，实体 B 的身份是**匿名**的（B 的标识符被 A 的公钥加密保护，B 的验签公钥在 Text₄ 中明文传输但不直接暴露身份）；
- 提供隐式密钥认证和 B 到 A 的单向密钥确认；
- 加密和签名操作提供强安全保证。

适合的应用场景：

1. 无线/移动通信环境（WLAN、5G 认证）：

- 无线信道带宽有限，2 次消息交换的通信开销小，适合资源受限的移动终端；
- B 的身份匿名保护用户位置隐私，防止窃听者通过身份标识追踪用户；
- 符合 3GPP/5G-AKA 等移动通信认证协议的隐私保护需求。

2. 物联网（IoT）设备认证：

- IoT 设备通常计算能力和电池有限，双通协议减少了通信和计算负担；
- 设备（作为 B）的身份匿名有助于保护设备部署位置和类型不被被动攻击者获知。

3. 匿名访问服务（Privacy-Preserving Services）：

- 用户（A）向服务器（B）发起请求时，B 的身份可以保持匿名（如暗网服务、匿名举报系统）；
- 服务器 B 通过签名证明其合法性但不公开身份，直到需要时由可靠信道验证。

4. 车辆自组网（VANET）和车联网（V2X）：

- 车辆间通信需要快速建立安全信道（低延迟双通协议）；
- 匿名性可保护车辆身份和行驶路径不被非授权方追踪。

5. RFID/NFC 安全认证：

- 非接触式通信的交互次数严格受限，双通协议是理想选择；
- 标签（Tag）身份的匿名性保护携带者隐私。

不适用场景：需要双向密钥确认的场景（如在线支付确认）、需要前向安全性的高安全场景（该协议未明确提供）。

8.7 Feige-Fiat-Shamir 身份验证方案分析 (20 分)

题目：在简化的 Feige-Fiat-Shamir 身份验证方案中，如果 $n = 77$ ，公开密钥为 60。

- (1) 试计算出私钥；(5 分)
- (2) 甲方选择的随机数为 21，乙方回复的随机位为 1，试补充该协议一圈的过程；(5 分)
- (3) 试分析该协议中甲能欺骗乙一次的机会为 50%。(10 分)

知识点回顾：Feige-Fiat-Shamir 身份验证方案

1. 背景与目的

Feige-Fiat-Shamir (FFS) 是一种零知识身份验证协议，由 Uriel Feige、Amos Fiat 和 Adi Shamir 于 1988 年提出。证明者 (Prover) 可以向验证者 (Verifier) 证明自己知道某个秘密，而不泄露该秘密的任何信息。

2. 简化版 FFS 协议 (单秘密)

系统建立：

- 可信中心选择两个大素数 p, q ，计算 $n = p \cdot q$ (n 公开)；
- 证明者 P 选择私钥 s ($\gcd(s, n) = 1$)，计算公钥 $v = s^2 \bmod n$ ；
- v 公开 (由可信中心签名分发)， s 仅 P 持有。

协议过程 (一轮)：

1. **承诺 (Commitment)**：P 选择随机数 r ($1 < r < n$)，计算 $x = r^2 \bmod n$ ，将 x 发送给 V；
2. **挑战 (Challenge)**：V 随机选择一个比特 $b \in \{0, 1\}$ ，发送给 P；
3. **响应 (Response)**：P 计算 $y = r \cdot s^b \bmod n$ ，发送给 V；
4. **验证 (Verification)**：V 检查 $y^2 \cdot v^b \equiv x \pmod{n}$ 是否成立。

正确性验证：

- 若 $b = 0$ ： $y = r$ ，验证 $r^2 \cdot v^0 \equiv x \pmod{n}$ ，即 $r^2 \equiv x \pmod{n}$ 。✓
- 若 $b = 1$ ： $y = r \cdot s$ ，验证 $(r \cdot s)^2 \cdot v \equiv x \pmod{n}$ ，即 $r^2 \cdot s^2 \cdot v \equiv r^2 \cdot v \cdot v^{-1} \equiv r^2 \equiv x \pmod{n}$ 。注意这里 $v = (s^2)^{-1} \bmod n$ (某些版本)，或者我们做标准设定：公钥 v 满足 $v \equiv s^2 \pmod{n}$ ，私钥 s 为 v 的平方根。更常见的做法是令公钥 $v = (s^2)^{-1} \bmod n$ ，使得 $s^2 \cdot v \equiv 1 \pmod{n}$ 。

注意——两种常见约定：

1. **约定一**: $v = s^2 \bmod n$ (公钥是私钥的平方)。验证方程: $y^2 \equiv x \cdot v^b \pmod{n}$ (当 $b = 1$ 时)。

2. **约定二**: $v \equiv (s^2)^{-1} \pmod{n}$ (公钥是私钥平方的逆)。验证方程: $y^2 \cdot v^b \equiv x \pmod{n}$ 。

本题使用的显然是约定二: 公钥 $v = 60$, 我们需要求私钥 s (即 v^{-1} 的平方根), 且验证公式为 $y^2 \cdot v^b \equiv x \pmod{n}$ 。

3. 安全性——欺骗概率分析

攻击者 (不知道 s) 试图冒充 P 欺骗 V:

- **策略一 (猜测 $b = 0$)**: 攻击者选任意 r , 计算 $x = r^2 \bmod n$ 发送给 V。若 V 的挑战 $b = 0$, 攻击者回复 $y = r$, 验证通过。若 $b = 1$, 攻击者需要回复 y 使得 $y^2 \cdot v \equiv x \pmod{n}$, 即 $y^2 \equiv x \cdot v^{-1} \pmod{n}$, 这相当于需要知道 v^{-1} 的平方根, 对于不知道 s 的攻击者在计算上不可行。此策略成功概率 = 50%。
- **策略二 (猜测 $b = 1$)**: 攻击者选任意 y , 计算 $x = y^2 \cdot v \bmod n$ 发送给 V。若 V 的挑战 $b = 1$, 验证 $y^2 \cdot v \equiv x \pmod{n}$ 通过。若 $b = 0$, 需要 $y^2 \equiv x \pmod{n}$, 但 $x = y^2 \cdot v$, 除非 $v \equiv 1$ 否则不成立。此策略成功概率 = 50%。

无论使用哪种策略, 攻击者单轮欺骗成功的概率仅为 50%。通过执行 t 轮独立验证, 欺骗成功概率降为 $(1/2)^t$ 。当 $t = 20$ 时, 欺骗概率约为 10^{-6} , 可忽略。

第 (1) 问: 计算私钥

给定: $n = 77 = 7 \times 11$, 公钥 $v = 60$ 。

目标: 求私钥 s (即 $v^{-1} \bmod n$ 的平方根), 取最小的平方根。

Step 1: 求 v 的模 n 逆元 v^{-1}

使用扩展欧几里得算法求 $\gcd(60, 77)$:

$$77 = 60 \times 1 + 17$$

$$60 = 17 \times 3 + 9$$

$$17 = 9 \times 1 + 8$$

$$9 = 8 \times 1 + 1$$

$$8 = 1 \times 8 + 0$$

回代:

$$\begin{aligned}
 1 &= 9 - 8 \times 1 \\
 &= 9 - (17 - 9 \times 1) = 9 \times 2 - 17 \\
 &= (60 - 17 \times 3) \times 2 - 17 = 60 \times 2 - 17 \times 7 \\
 &= 60 \times 2 - (77 - 60) \times 7 = 60 \times 2 - 77 \times 7 + 60 \times 7 \\
 &= 60 \times 9 - 77 \times 7
 \end{aligned}$$

因此 $60 \times 9 \equiv 1 \pmod{77}$, 即:

$$v^{-1} \equiv 9 \pmod{77} \quad (14)$$

验证: $60 \times 9 = 540$, $540 \div 77 = 7$ 余1。✓

Step 2: 求 $\sqrt{9} \pmod{77}$

需要在模 $77 = 7 \times 11$ 下求 x 使得 $x^2 \equiv 9 \pmod{77}$ 。

由于 $\gcd(7, 11) = 1$, 可通过中国剩余定理 (CRT) 分别求解:

子问题一: $x^2 \equiv 9 \pmod{7}$

$9 \pmod{7} = 2$, 求 $x^2 \equiv 2 \pmod{7}$ 。

模 7 下的二次剩余: $1^2 = 1, 2^2 = 4, 3^2 = 2, 4^2 = 2, 5^2 = 4, 6^2 = 1$ 。

故 $x \equiv 3$ 或 $4 \pmod{7}$ 。

子问题二: $x^2 \equiv 9 \pmod{11}$

$9 \pmod{11} = 9$, 求 $x^2 \equiv 9 \pmod{11}$ 。

模 11 下: $3^2 = 9, 8^2 = 64 \equiv 9 \pmod{11}$ 。

故 $x \equiv 3$ 或 $8 \pmod{11}$ 。

Step 3: CRT 组合求解

共有 $2 \times 2 = 4$ 组解, 使用 CRT 公式 $x = a \cdot M_1 \cdot M_1^{-1} + b \cdot M_2 \cdot M_2^{-1} \pmod{77}$, 其中 $M_1 = 11, M_2 = 7$ 。

$M_1^{-1} \pmod{7}$: 即 $11^{-1} \pmod{7}$ 。 $11 \equiv 4 \pmod{7}$, $4 \times 2 = 8 \equiv 1 \pmod{7}$, 故 $M_1^{-1} \equiv 2$ 。

$M_2^{-1} \pmod{11}$: 即 $7^{-1} \pmod{11}$ 。 $7 \times 8 = 56 \equiv 1 \pmod{11}$, 故 $M_2^{-1} \equiv 8$ 。

$$\begin{aligned}
 x &\equiv a \cdot 11 \cdot 2 + b \cdot 7 \cdot 8 \pmod{77} \\
 &\equiv 22a + 56b \pmod{77}
 \end{aligned}$$

四组解:

$a \pmod{7}$	$b \pmod{11}$	$x = 22a + 56b \pmod{77}$	$x \pmod{77}$
3	3	$66 + 168 = 234$	$234 - 3 \times 77 = 234 - 231 = 3$
3	8	$66 + 448 = 514$	$514 - 6 \times 77 = 514 - 462 = 52$
4	3	$88 + 168 = 256$	$256 - 3 \times 77 = 256 - 231 = 25$
4	8	$88 + 448 = 536$	$536 - 6 \times 77 = 536 - 462 = 74$

因此 $\sqrt{v^{-1}} = \sqrt{9} \pmod{77} \in \{3, 25, 52, 74\}$ 。

验证：

$$3^2 = 9 \equiv 9 \pmod{77} \quad \checkmark$$

$$25^2 = 625 = 8 \times 77 + 9 \equiv 9 \pmod{77} \quad \checkmark$$

$$52^2 = 2704 = 35 \times 77 + 9 \equiv 9 \pmod{77} \quad \checkmark$$

$$74^2 = 5476 = 71 \times 77 + 9 \equiv 9 \pmod{77} \quad \checkmark$$

取最小的平方根作为私钥：

$$s = 3$$

(15)

第 (2) 问：补充协议一圈的过程

给定：随机数 $r = 21$ ，挑战比特 $b = 1$ 。

协议一轮的完整过程：

Step 1: 甲方（证明者 P）计算承诺值 x

$$\begin{aligned} x &= r^2 \pmod{n} \\ &= 21^2 \pmod{77} \\ &= 441 \pmod{77} \end{aligned}$$

$$77 \times 5 = 385, 441 - 385 = 56。$$

故：

$$x = 56$$

(16)

甲方将 $x = 56$ 发送给乙方。

Step 2: 乙方（验证者 V）发送挑战

乙方向甲方发送随机比特：

$$b = 1$$

(17)

Step 3: 甲方发送响应 y

由于 $b = 1$ ，甲方计算：

$$\begin{aligned} y &= r \cdot s^b \pmod{n} \\ &= 21 \times 3^1 \pmod{77} \\ &= 63 \pmod{77} \end{aligned}$$

故：

$$y = 63$$

(18)

甲方将 $y = 63$ 发送给乙方。

Step 4: 乙方验证

乙方检查验证方程 $y^2 \cdot v^b \equiv x \pmod{n}$ 是否成立:

由于 $b = 1$:

$$y^2 \cdot v \bmod 77 = 63^2 \times 60 \bmod 77$$

先计算 $63^2 \bmod 77$: $77 \times 51 = 3927$, $63^2 = 3969$, $3969 - 3927 = 42$ 。

因此 $63^2 \equiv 42 \pmod{77}$ 。

继续计算 $42 \times 60 \bmod 77$: $42 \times 60 = 2520$ 。

$77 \times 32 = 2464$, $2520 - 2464 = 56$ 。

$$y^2 \cdot v \bmod 77 = 56 = x \quad (19)$$

验证通过! 乙方确信甲方确实知道私钥 s (即 $\sqrt{v^{-1}} \bmod n = 3$), 因为只有知道 s 的实体才能在 $b = 1$ 时正确计算 $y = r \cdot s$ 使验证方程成立。

第 (3) 问: 分析欺骗概率为 50%

问题: 为什么不知道私钥 s 的攻击者 (甲) 单次欺骗验证者 (乙) 成功的概率恰好是 50%?

核心原理: 攻击者不知道 s , 但可以**预先猜测**乙方的挑战比特 b , 并据此伪造承诺值 x 和响应值 y 。由于挑战 $b \in \{0, 1\}$, 猜对概率为 $1/2$ 。

策略分析一: 猜测 $b = 0$

攻击策略:

1. 猜测乙方会发送 $b = 0$;
2. 随机选择 r , 计算 $x = r^2 \bmod n$, 发送给乙方;
3. 若乙方确实发送 $b = 0$: 攻击者回复 $y = r$ 。

• 验证方程: $y^2 \cdot v^0 = r^2 \cdot 1 = x$ 。 **验证通过!**

4. 若乙方发送 $b = 1$: 攻击者需要回复 y 使得 $y^2 \cdot v \equiv x \pmod{n}$, 即 $y^2 \equiv x \cdot v^{-1} \pmod{n}$ 。

• 这等价于求 $x \cdot v^{-1}$ 的平方根。由于不知道 $\sqrt{v^{-1}}$, 且平方根问题在不知道 n 的因式分解时是困难的, 攻击者无法在合理时间内给出正确响应。

• **验证失败!**

策略分析二: 猜测 $b = 1$

攻击策略:

1. 猜测乙方会发送 $b = 1$;
2. 随机选择 y , 计算 $x = y^2 \cdot v \bmod n$, 发送给乙方;
3. 若乙方确实发送 $b = 1$: 攻击者直接把预选的 y 发给乙方。
 - 验证方程: $y^2 \cdot v^1 \equiv x \pmod n$ (由构造保证成立)。**验证通过!**
4. 若乙方发送 $b = 0$: 攻击者需要回复 y (但回复的仍是预选的 y)。
 - 验证方程: $y^2 \cdot v^0 = y^2$, 而 $y^2 \equiv x \cdot v^{-1} \not\equiv x \pmod n$ (除非 $v \equiv 1$, 这不可能因为 $v = 60$)。
 - 验证失败!**

总结

攻击者策略	乙方挑战 b	验证结果	概率
准备应对 $b = 0$	$b = 0$ (猜对)	通过	50%
准备应对 $b = 0$	$b = 1$ (猜错)	失败	50%
准备应对 $b = 1$	$b = 1$ (猜对)	通过	50%
准备应对 $b = 1$	$b = 0$ (猜错)	失败	50%

无论哪种策略, 攻击者成功的关键在于**提前获知挑战比特**。由于挑战是随机且不可预测的, 攻击者单次欺骗成功的概率恰为:

$$P(\text{单轮欺骗成功}) = \frac{1}{2} = 50\%$$

(20)

安全性增强——多轮重复:

协议通过执行 t 轮独立验证来降低欺骗概率。每一轮都独立地进行承诺-挑战-响应-验证, 攻击者必须在**所有 t 轮**中都猜对挑战比特。因此:

$$P(t \text{ 轮全部欺骗成功}) = \left(\frac{1}{2}\right)^t$$

(21)

例如:

- $t = 10$: $P \approx 0.098\%$;
- $t = 20$: $P \approx 9.5 \times 10^{-7}$ (百万分之一);
- $t = 30$: $P \approx 9.3 \times 10^{-10}$ (十亿分之一)。

这体现了零知识证明的核心思想: 通过概率性验证, 在足够多轮次后使欺骗概率任意小, 同时不泄露私钥的任何信息。

9 试卷九：网络安全安全小测试（2024-2025）

试题

第一题：RSA 算法 ($p=13, q=61, e=37$)

在 RSA 算法中, 选择 $p = 13, q = 61, e = 37$,

- (1) 计算其私钥 d ;
- (2) 明文 $m_1 = 7$, 试求 m_1 对应的密文 c_1 ;
- (3) 密文 $c_2 = 25$, 试求 c_2 对应的明文 m_2 。

第二题：ElGamal 签名中重复使用随机数 k 的安全漏洞

在 ElGamal 数字签名方案中, 选择素数 p , g 为素数模 p 的原根,

$$y = g^x \bmod p$$

y, g, p 为公开密钥, x 为私钥。对于消息 m , 选择随机数 k , 计算 $r = g^k \bmod p$,

$$s = (m - x \cdot r) \cdot k^{-1} \bmod (p - 1)$$

签名结果为: (m, r, s) 。验签过程如下:

$$y^r \cdot r^s \bmod p = g^{xr} \cdot g^{ks} \bmod p \stackrel{?}{=} g^m \bmod p$$

该签名算法忌讳随机数 k 的重复使用, 会暴露私钥 x 。

已知 $p = 23, g = 5, y = 11, x$ 未知, 且该用户的两个签名信息

$$(m_1, r, s_1) = (6, 10, 16), \quad (m_2, r, s_2) = (12, 10, 18)$$

试求该用户的私钥 x 。

9.1 RSA 加解密计算 (10 分)

知识回顾：RSA 密钥生成与加解密

密钥生成：

步骤 1: 随机选择两个大素数 p 和 q , 计算 $n = p \times q$;

步骤 2: 计算欧拉函数 $\phi(n) = (p-1)(q-1)$;

步骤 3: 随机选择加密指数 e , 满足 $1 < e < \phi(n)$ 且 $\gcd(e, \phi(n)) = 1$;

步骤 4: 计算解密指数 d , 满足 $e \cdot d \equiv 1 \pmod{\phi(n)}$, 即 $d = e^{-1} \pmod{\phi(n)}$;

步骤 5: 公钥为 (n, e) , 私钥为 $(p, q, \phi(n), d)$ 。

加解密：

- 加密: $c = m^e \pmod{n}$, 其中 m 为明文且 $m < n$;
- 解密: $m = c^d \pmod{n}$ 。

正确性保证 (欧拉定理): 若 $\gcd(m, n) = 1$, 则 $m^{\phi(n)} \equiv 1 \pmod{n}$, 故 $c^d \equiv m^{ed} \equiv m^{k\phi(n)+1} \equiv m \pmod{n}$ 。即使 $\gcd(m, n) \neq 1$, 由中国剩余定理也可证明解密正确性。

扩展欧几里得算法: 用于求解 $e^{-1} \pmod{\phi(n)}$, 即求 d 使得 $ed + \phi(n) \cdot t = 1$ 中的系数 d 。

快速模幂运算 (平方-乘算法): 将指数表示为二进制, 通过反复平方和乘法高效计算模幂。

完整解答

(1) 计算私钥 d

已知参数:

$$p = 13, \quad q = 61, \quad e = 37$$

首先计算:

$$n = p \times q = 13 \times 61 = 793$$

$$\phi(n) = (p-1)(q-1) = 12 \times 60 = 720$$

私钥 d 需满足: $e \cdot d \equiv 1 \pmod{\phi(n)}$, 即

$$37d \equiv 1 \pmod{720}$$

使用扩展欧几里得算法求解:

步骤	商	等式	余数表达式
0	–	$720 = 720 \times 1 + 37 \times 0$	–
1	19	$37 = 720 \times 0 + 37 \times 1$	–
2	1	$720 = 37 \times 19 + 17$	$17 = 720 - 37 \times 19$
3	2	$37 = 17 \times 2 + 3$	$3 = 37 - 17 \times 2$
4	5	$17 = 3 \times 5 + 2$	$2 = 17 - 3 \times 5$
5	1	$3 = 2 \times 1 + 1$	$1 = 3 - 2 \times 1$
6	2	$2 = 1 \times 2 + 0$	$\gcd = 1$ (互素, 可求逆)

回代求解:

$$\begin{aligned} 1 &= 3 - 2 \times 1 \\ &= 3 - (17 - 3 \times 5) \times 1 = 6 \times 3 - 17 \\ &= 6 \times (37 - 17 \times 2) - 17 = 6 \times 37 - 13 \times 17 \\ &= 6 \times 37 - 13 \times (720 - 37 \times 19) \\ &= 6 \times 37 - 13 \times 720 + 247 \times 37 \\ &= 253 \times 37 - 13 \times 720 \end{aligned}$$

因此:

$$37 \times 253 - 720 \times 13 = 1 \quad \Rightarrow \quad 37 \times 253 \equiv 1 \pmod{720}$$

验证:

$$37 \times 253 = 9361 = 720 \times 13 + 1 \quad \checkmark$$

$d = 253$

(2) 加密 $m_1 = 7$

密文 $c_1 = m_1^e \bmod n = 7^{37} \bmod 793$ 。

使用平方-乘算法，将指数 37 二进制分解： $37 = 32 + 4 + 1 = 2^5 + 2^2 + 2^0$ 。

指数位	计算过程	结果模 793
7^1	7	7
7^2	$7^2 = 49$	49
7^4	$49^2 = 2401 = 793 \times 3 + 22$	22
7^8	$22^2 = 484$	484
7^{16}	$484^2 = 234256 = 793 \times 295 + 321$	321
7^{32}	$321^2 = 103041 = 793 \times 129 + 744$	744

组合指数位 $37 = 32 + 4 + 1$:

$$\begin{aligned}c_1 &= 7^{37} \bmod 793 = 7^{32} \times 7^4 \times 7^1 \bmod 793 \\&\equiv 744 \times 22 \times 7 \pmod{793} \\&\equiv (744 \times 22) \times 7 \pmod{793} \\&\equiv 508 \times 7 \pmod{793} \quad (\text{因 } 744 \times 22 = 16368 = 793 \times 20 + 508) \\&\equiv 3556 \bmod 793 \\&\equiv 384 \quad (\text{因 } 793 \times 4 = 3172, 3556 - 3172 = 384)\end{aligned}$$

$$c_1 = 384$$

(3) 解密 $c_2 = 25$

明文 $m_2 = c_2^d \bmod n = 25^{253} \bmod 793$ 。

直接计算 $25^{253} \bmod 793$ 非常繁琐，使用中国剩余定理（CRT）加速。

步骤一：分解计算

计算子密钥：

$$d_p = d \bmod (p-1) = 253 \bmod 12 = 1$$

$$d_q = d \bmod (q-1) = 253 \bmod 60 = 13$$

分别解密模 p 和模 q ：

$$m_p = c_2^{d_p} \bmod p = 25^1 \bmod 13 = 25 \bmod 13 = 12$$

$$m_q = c_2^{d_q} \bmod q = 25^{13} \bmod 61$$

计算 $25^{13} \bmod 61$ （使用平方-乘算法， $13 = 8 + 4 + 1$ ）：

指数位	计算过程	结果模 61
25^1	25	25
25^2	$25^2 = 625 = 61 \times 10 + 15$	15
25^4	$15^2 = 225 = 61 \times 3 + 42$	42
25^8	$42^2 = 1764 = 61 \times 28 + 56$	56

$$\begin{aligned}
 m_q &= 25^{13} \bmod 61 = 25^8 \times 25^4 \times 25^1 \bmod 61 \\
 &\equiv 56 \times 42 \times 25 \pmod{61} \\
 &\equiv 2352 \times 25 \pmod{61} \quad (56 \times 42 = 2352) \\
 &\equiv 34 \times 25 \pmod{61} \quad (2352 = 61 \times 38 + 34) \\
 &\equiv 850 \bmod 61 \\
 &\equiv 57 \quad (61 \times 13 = 793, 850 - 793 = 57)
 \end{aligned}$$

步骤二：CRT 合成

得到同余方程组：

$$\begin{cases} m_2 \equiv m_p = 12 \pmod{13} \\ m_2 \equiv m_q = 57 \pmod{61} \end{cases}$$

由 $m_2 = 12 + 13 \cdot t$ ，代入第二个同余式：

$$12 + 13t \equiv 57 \pmod{61} \Rightarrow 13t \equiv 45 \pmod{61}$$

求 13 模 61 的逆元 (使用扩展欧几里得算法):

$$61 = 4 \times 13 + 9$$

$$13 = 1 \times 9 + 4$$

$$9 = 2 \times 4 + 1$$

回代:

$$\begin{aligned} 1 &= 9 - 2 \times 4 = 9 - 2 \times (13 - 9) = 3 \times 9 - 2 \times 13 \\ &= 3 \times (61 - 4 \times 13) - 2 \times 13 = 3 \times 61 - 14 \times 13 \end{aligned}$$

故 $(-14) \times 13 \equiv 1 \pmod{61}$, 即 $13^{-1} \equiv 47 \pmod{61}$ (因 $-14 + 61 = 47$)。

验证: $13 \times 47 = 611 = 61 \times 10 + 1 \checkmark$

解得:

$$t \equiv 45 \times 47 \pmod{61} \equiv 2115 \pmod{61} \equiv 41 \quad (61 \times 34 = 2074, 2115 - 2074 = 41)$$

最终:

$$m_2 = 12 + 13 \times 41 = 12 + 533 = 545$$

验证: 计算 $545^{37} \pmod{793}$ 是否得到 25。

使用平方-乘算法 ($37 = 32 + 4 + 1$):

指数位	计算过程	结果模 793
545^1	545	545
545^2	$545^2 = 297025 = 793 \times 374 + 443$	443
545^4	$443^2 = 196249 = 793 \times 247 + 378$	378
545^8	$378^2 = 142884 = 793 \times 180 + 144$	144
545^{16}	$144^2 = 20736 = 793 \times 26 + 118$	118
545^{32}	$118^2 = 13924 = 793 \times 17 + 443$	443

$$\begin{aligned} 545^{37} &= 545^{32} \times 545^4 \times 545^1 \pmod{793} \\ &\equiv 443 \times 378 \times 545 \pmod{793} \\ &\equiv 131 \times 545 \pmod{793} \quad (443 \times 378 = 167454 = 793 \times 211 + 131) \\ &\equiv 71395 \pmod{793} \\ &\equiv 25 \quad (793 \times 90 = 71370, 71395 - 71370 = 25) \checkmark \end{aligned}$$

$m_2 = 545$

结果汇总

参数	值	说明
p	13	素数
q	61	素数
$n = p \times q$	793	模数
$\phi(n)$	720	欧拉函数值
e	37	公钥加密指数
d	253	私钥解密指数
m_1	7	原始明文
c_1	384	加密结果
c_2	25	待解密密文
m_2	545	解密结果

拓展：RSA 安全性参数选择要点

1. 素数 p 和 q 的选择：

- 在实际应用中， p 和 q 应为长度相近的大素数（通常各 1024 位以上），使得 n 达到 2048 位；
- p 和 q 之差应足够大（避免 Fermat 因式分解）；
- $p - 1$ 和 $q - 1$ 应包含大素因子（抵抗 Pollard $p - 1$ 攻击）；
- 不应使用过小的素数，如本题中的 $p = 13, q = 61$ ，仅为教学示例。

2. 加密指数 e 的选择：

- 常见选择为 $e = 65537 = 2^{16} + 1$ （Fermat 素数），既保证 $\gcd(e, \phi(n)) = 1$ ，又使加密效率高；
- 本题中 $e = 37$ 较小，但需确保 $\gcd(37, 720) = 1$ 成立（已验证满足）。

3. 中国剩余定理（CRT）加速：

- 解密时使用 CRT 可将计算复杂度降低约 4 倍（计算两个半长模幂而非一个全长模幂）；
- 在实际密码库（如 OpenSSL）中，私钥通常存储为 $(p, q, d_p, d_q, q_{inv})$ 的 CRT 形式。

4. 常见攻击方式：

- 小 d 攻击 (Wiener 攻击): 当 $d < \frac{1}{3}n^{1/4}$ 时, 可利用连分数恢复私钥;
- 共模攻击: 同一 n 下不同 e 加密相同 m 可恢复明文;
- 选择密文攻击: RSA 的乘法同态性导致直接使用不安全, 实际需配合 OAEP 等填充方案。

9.2 ElGamal 签名中重复使用 k 的安全性分析 (10 分)

知识回顾：ElGamal 数字签名方案

系统参数：

- p : 大素数;
- g : 模 p 的原根 (生成元);
- x : 私钥, $1 < x < p-1$;
- $y = g^x \bmod p$: 公钥。

签名生成 (对消息 m):

步骤 1: 随机选择 k , 满足 $1 < k < p-1$ 且 $\gcd(k, p-1) = 1$;

步骤 2: 计算 $r = g^k \bmod p$;

步骤 3: 计算 $s = (m - x \cdot r) \cdot k^{-1} \bmod (p-1)$;

步骤 4: 签名为 (m, r, s) 。

签名验证：

$$y^r \cdot r^s \bmod p \stackrel{?}{=} g^m \bmod p$$

验证原理：

$$y^r \cdot r^s \equiv g^{xr} \cdot g^{ks} \equiv g^{xr+ks} \equiv g^{xr+(m-xr)} \equiv g^m \pmod{p}$$

签名方程：

$$s \equiv (m - xr) \cdot k^{-1} \pmod{p-1}$$

等价于：

$$s \cdot k \equiv m - x \cdot r \pmod{p-1}$$

完整解答： k 重复使用导致私钥泄露

已知条件：

$$p = 23, \quad g = 5, \quad y = 11$$

$$(m_1, r, s_1) = (6, 10, 16), \quad (m_2, r, s_2) = (12, 10, 18)$$

关键观察：两次签名的 r 值相同 ($r = 10$)，说明使用了相同的随机数 k ，即：

$$r = g^k \bmod p = 10 \quad \Rightarrow \quad 5^k \equiv 10 \pmod{23}$$

步骤一：利用签名方程推导 k

由 ElGamal 签名方程，对两次签名分别有：

$$s_1 \cdot k \equiv m_1 - x \cdot r \pmod{p-1} \quad (22)$$

$$s_2 \cdot k \equiv m_2 - x \cdot r \pmod{p-1} \quad (23)$$

其中 $p-1 = 22$ 。代入已知值：

$$16k \equiv 6 - 10x \pmod{22} \quad (24)$$

$$18k \equiv 12 - 10x \pmod{22} \quad (25)$$

用式 (24) 减去式 (25)，消去含 x 的项：

$$(16 - 18)k \equiv (6 - 12) \pmod{22}$$

$$-2k \equiv -6 \pmod{22}$$

$$2k \equiv 6 \pmod{22}$$

即 $2k - 6$ 是 22 的倍数，设 $2k - 6 = 22t$ ，即 $k - 3 = 11t$ 。

由于 $\gcd(2, 22) = 2$ 且 $2 \mid 6$ ，该同余方程有解，且等价于：

$$k \equiv 3 \pmod{11}$$

在模 22 意义下有两个解： $k = 3$ 或 $k = 3 + 11 = 14$ 。

步骤二：通过 r 验证 k

$r = g^k \bmod p = 5^k \bmod 23 = 10$ ，检查哪个 k 值满足：

检验 $k = 3$ ：

$$5^3 = 125 = 23 \times 5 + 10 \Rightarrow 5^3 \equiv 10 \pmod{23} \checkmark$$

检验 $k = 14$ ：

$$5^2 \equiv 25 \equiv 2 \pmod{23}$$

$$5^4 \equiv 2^2 \equiv 4 \pmod{23}$$

$$5^8 \equiv 4^2 \equiv 16 \pmod{23}$$

$$5^{14} = 5^8 \cdot 5^4 \cdot 5^2 \equiv 16 \times 4 \times 2 \equiv 128 \equiv 128 - 23 \times 5 \equiv 13 \pmod{23}$$

$13 \neq 10$ ，因此 $k = 14$ 不满足。

结论： $k = 3$ 。

补充验证： $\gcd(k, p-1) = \gcd(3, 22) = 1$ ，满足 k 的可逆性要求。

步骤三：利用签名方程求解私钥 x

将 $k = 3$ 代入式 (24):

$$16 \times 3 \equiv 6 - 10x \pmod{22}$$

$$48 \equiv 6 - 10x \pmod{22}$$

$$48 - 6 \equiv -10x \pmod{22}$$

$$42 \equiv -10x \pmod{22}$$

化简模 22: $42 = 22 \times 1 + 20$, 故:

$$20 \equiv -10x \pmod{22}$$

两边同乘 -1 :

$$-20 \equiv 10x \pmod{22}$$

即:

$$2 \equiv 10x \pmod{22} \quad (\text{因 } -20 + 22 = 2)$$

两边同除以 2 (注意 $\gcd(2, 22) = 2$, 需同时将模数除以 $\gcd$):

$$1 \equiv 5x \pmod{11}$$

即 $5x \equiv 1 \pmod{11}$ 。求 5 模 11 的逆元: $5 \times 9 = 45 = 11 \times 4 + 1$ 。

故:

$$x \equiv 9 \pmod{11}$$

在 $[1, p-2] = [1, 21]$ 范围内, 可能的候选值为 $x = 9$ 或 $x = 20$ 。

步骤四：通过公钥 y 验证 x

验证 $y = g^x \bmod p$ 是否成立:

检验 $x = 9$:

$$5^2 \equiv 2 \pmod{23}$$

$$5^4 \equiv 4 \pmod{23}$$

$$5^8 \equiv 16 \pmod{23}$$

$$5^9 = 5^8 \cdot 5 \equiv 16 \times 5 \equiv 80 \equiv 80 - 23 \times 3 \equiv 11 \pmod{23}$$

$y = 11$, 成立! ✓

检验 $x = 20$:

$$5^{16} \equiv 5^8 \cdot 5^8 \equiv 16 \times 16 \equiv 256 \equiv 3 \pmod{23}$$

$$5^{20} = 5^{16} \cdot 5^4 \equiv 3 \times 4 \equiv 12 \pmod{23}$$

$12 \neq 11$, 不成立。

$$x = 9$$

步骤五：双重验证

用另一条签名方程（式 25）复核：

$$s_2 \cdot k \equiv m_2 - x \cdot r \pmod{p-1}$$

$$\text{左边: } 18 \times 3 = 54 \equiv 10 \pmod{22} \quad (54 - 22 \times 2 = 10)$$

$$\text{右边: } 12 - 9 \times 10 = 12 - 90 = -78 \equiv 10 \pmod{22} \quad (-78 + 22 \times 4 = 10)$$

左右相等, ✓

攻击原理总结

攻击成功的根源在于：当两次签名使用相同的 k 时，签名方程中的未知数 k 和 x 可分别通过两条方程联立求解。

$$\begin{cases} s_1 k \equiv m_1 - xr \pmod{p-1} \\ s_2 k \equiv m_2 - xr \pmod{p-1} \end{cases}$$

两式相减消去 xr 项：

$$(s_1 - s_2)k \equiv m_1 - m_2 \pmod{p-1} \Rightarrow k \text{ 可求}$$

代入任一方程即得：

$$x \equiv r^{-1}(m_1 - s_1 k) \pmod{p-1} \Rightarrow \text{私钥泄露!}$$

拓展：ElGamal 签名中 k 重复使用的危害与防范

1. k 为何必须唯一且不可预测？

ElGamal 签名方案的安全性建立在离散对数困难问题之上。随机数 k 的作用是：

- **随机化签名**：即使对同一消息签名，不同 k 产生不同的 (r, s) ；
- **保护私钥**： k 将私钥 x 隐藏在方程 $s = k^{-1}(m - xr) \pmod{p-1}$ 中。

一旦 k 重复使用：

- 两次签名的 $r = g^k \pmod{p}$ 相同，直接暴露 k 被复用的事实；

- 两次签名方程相减后 x 项被消去, k 暴露;
- 有了 k , 从任意一条签名方程即可恢复私钥 x 。

2. 类似攻击的历史案例:

- **Sony PlayStation 3** (2010 年): ECDSA 签名中使用了固定的 k , 私钥被恢复, 导致系统被完全破解;
- **Android Java** 的 **SecureRandom** 漏洞 (2013 年): 部分 Android 钱包因随机数相同导致比特币私钥被恢复;
- **Blockchain.info** 等比特币相关漏洞: 多个地址因 k 重复使用导致私钥泄露。

3. 防范措施:

- **确定性 k 生成 (RFC 6979)**: 使用 HMAC 基于私钥和消息哈希确定性地生成 k , 避免随机数生成器缺陷;
- **使用安全随机数生成器 (CSPRNG)**: 确保 k 来自密码学安全的随机源;
- **每次签名后清除 k** : 防止内存泄露。

4. DSA/ECDSA 中的对应攻击:

DSA 和 ECDSA 源自 ElGamal 签名框架, 存在完全相同的 k 重复使用漏洞。DSA 签名方程: $s = k^{-1}(H(m) + xr) \bmod q$ 。若两次签名使用同一 k :

$$k = \frac{H(m_1) - H(m_2)}{s_1 - s_2} \bmod q, \quad x = \frac{s_1 k - H(m_1)}{r} \bmod q$$

此即著名的 “nonce-reuse” 攻击。

5. 本题攻击的数学本质:

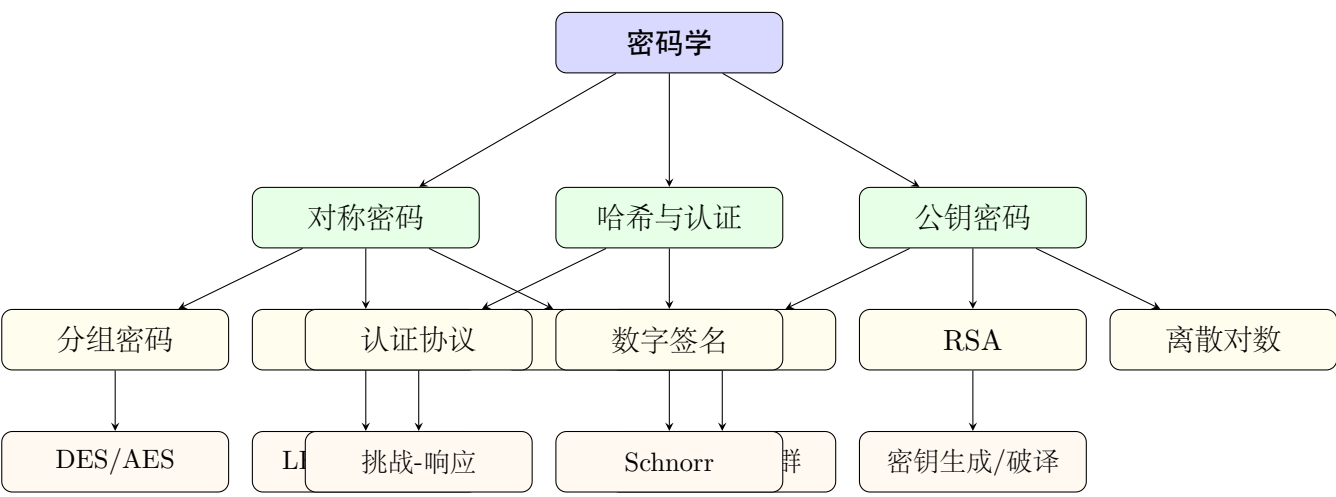
两条签名方程构成一个模线性方程组:

$$\begin{bmatrix} s_1 & r \\ s_2 & r \end{bmatrix} \begin{bmatrix} k \\ x \end{bmatrix} \equiv \begin{bmatrix} m_1 \\ m_2 \end{bmatrix} \pmod{p-1}$$

当 r 列相同 (k 相同) 时, 系数矩阵不满秩 (秩为 1), 无法唯一确定 k 和 x 。但利用 $r = g^k$ 这一额外约束即可确定 k , 进而唯一解出 x 。

附录：密码学核心知识体系图谱

以下是两套试卷所覆盖的密码学知识体系全景图，按学科逻辑重组（非试卷命题顺序）：



各模块在两套试卷中的分布

知识模块	试卷一题号	试卷二题号	合计分值
古典密码（加法密码、对合运算）	一	—	10
分组密码结构与比较（DES/AES）	三	—	10
AES MixColumns 有限域运算	四	—	10
LFSR 与 m 序列	五	一.1	25
DES S 盒分析	—	一.2	10
RSA 算法与安全性	六	一.3	25
公钥密钥管理与 PKI	二	二.1, 二.3	30
CBC 工作模式	—	二.2	10
Schnorr 签名与可证明安全	—	二.4	10
椭圆曲线密码	七	—	15
认证协议安全分析	八	—	15
密码学应用设计	—	三	30
同态加密与协议分析	—	四	5
合计	100	105	205